



(12) **United States Patent**
Skarphedinsson et al.

(10) **Patent No.:** **US 12,418,555 B1**
(45) **Date of Patent:** **Sep. 16, 2025**

(54) **GUIDING QUERY CREATION FOR A
GENERATIVE ARTIFICIAL INTELLIGENCE
(AI)-ENABLED ASSISTANT**

(71) Applicant: **LACEWORK, INC.**, Mountain View,
CA (US)

(72) Inventors: **Njall Skarphedinsson**, Redwood City,
CA (US); **Theodore M. Reed**, Berkeley
Heights, NJ (US); **Joshua Vertes**,
Venice, CA (US); **Pamela**
Bhattacharya, Redmond, WA (US);
Yijou Chen, Cupertino, CA (US)

(73) Assignee: **Fortinet Inc.**, Sunnyvale, CA (US)

(*) Notice: Subject to any disclaimer, the term of this
patent is extended or adjusted under 35
U.S.C. 154(b) by 0 days.

(21) Appl. No.: **18/623,796**

(22) Filed: **Apr. 1, 2024**

Related U.S. Application Data

(63) Continuation-in-part of application No. 18/415,879,
filed on Jan. 18, 2024, now Pat. No. 12,309,185,
(Continued)

(51) **Int. Cl.**
H04L 67/306 (2022.01)
G06F 9/455 (2018.01)
(Continued)

(52) **U.S. Cl.**
CPC **H04L 63/1425** (2013.01); **G06F 9/455**
(2013.01); **G06F 9/545** (2013.01);
(Continued)

(58) **Field of Classification Search**
CPC . H04L 63/1425; H04L 67/535; H04L 43/045;
H04L 43/06; H04L 63/10;
(Continued)

(56) **References Cited**

U.S. PATENT DOCUMENTS

5,584,024 A 12/1996 Shwartz
5,806,062 A 9/1998 Chen et al.
(Continued)

FOREIGN PATENT DOCUMENTS

CN 110999250 B 11/2021
CN 114598840 A 6/2022
WO 2016138067 A1 9/2016

OTHER PUBLICATIONS

Ai-Yaseen et al., "Real-Time Intrusion Detection System Using
Multi-Agent System", IAENG International Journal of Computer
Science, vol. 43, No. 1, Feb. 2016, pp. 80-90, International Asso-
ciation of Engineers (IAENG), Hong Kong.

(Continued)

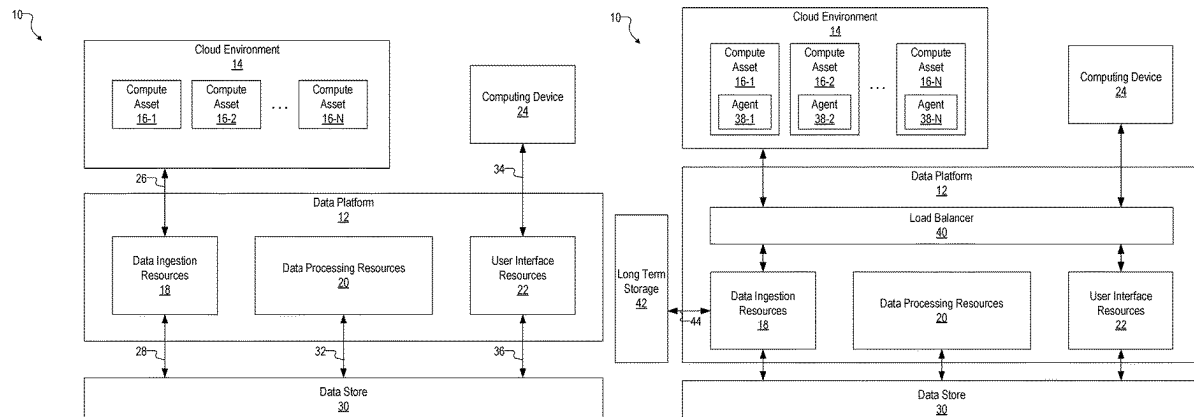
Primary Examiner — Minh Chau Nguyen

(74) *Attorney, Agent, or Firm* — Jaffery Watson Hamilton
& DeSanctis LLP

(57) **ABSTRACT**

Guiding query creation for a generative artificial intelligence
(AI)-enabled assistant, including: receiving, via a natural
language interface for a security framework monitoring a
cloud deployment, a natural language input comprising one
or more entity identifiers; gathering information based on
the one or more entity identifiers; providing, to a generative
artificial intelligence (AI) model, a prompt based on the
natural language input and comprising the gathered infor-
mation; and receiving, from the generative AI model, a
response to the prompt.

20 Claims, 64 Drawing Sheets



Related U.S. Application Data

which is a continuation-in-part of application No. 18/469,212, filed on Sep. 18, 2023, now Pat. No. 12,126,643, which is a continuation-in-part of application No. 18/153,270, filed on Jan. 11, 2023, now Pat. No. 11,770,398, which is a continuation-in-part of application No. 17/671,199, filed on Feb. 14, 2022, now Pat. No. 11,785,104, which is a continuation-in-part of application No. 17/504,311, filed on Oct. 18, 2021, now Pat. No. 11,677,772, which is a continuation of application No. 16/665,961, filed on Oct. 28, 2019, now Pat. No. 11,153,339, which is a continuation of application No. 16/134,794, filed on Sep. 18, 2018, now Pat. No. 10,581,891.

- (60) Provisional application No. 63/515,566, filed on Jul. 25, 2023, provisional application No. 63/426,936, filed on Nov. 21, 2022, provisional application No. 63/243,013, filed on Sep. 10, 2021, provisional application No. 62/650,971, filed on Mar. 30, 2018, provisional application No. 62/590,986, filed on Nov. 27, 2017.

(51) Int. Cl.

G06F 9/54 (2006.01)
G06F 16/2455 (2019.01)
G06F 16/901 (2019.01)
G06F 16/9038 (2019.01)
G06F 16/9535 (2019.01)
G06F 16/9537 (2019.01)
G06F 21/57 (2013.01)
H04L 9/40 (2022.01)
H04L 43/045 (2022.01)
H04L 43/06 (2022.01)
H04L 67/50 (2022.01)

(52) U.S. Cl.

CPC **G06F 16/9024** (2019.01); **G06F 16/9038** (2019.01); **G06F 16/9535** (2019.01); **G06F 16/9537** (2019.01); **G06F 21/57** (2013.01); **H04L 43/045** (2013.01); **H04L 43/06** (2013.01); **H04L 63/10** (2013.01); **H04L 67/306** (2013.01); **H04L 67/535** (2022.05); **G06F 16/2456** (2019.01)

(58) Field of Classification Search

CPC H04L 67/306; G06F 16/9038; G06F 16/9535; G06F 16/9537; G06F 16/9024; G06F 16/2456

See application file for complete search history.

(56) References Cited**U.S. PATENT DOCUMENTS**

6,347,339 B1 2/2002 Morris et al.
6,363,411 B1 3/2002 Dugan et al.
6,434,663 B1 8/2002 Grimsrud et al.
6,938,084 B2 8/2005 Gamache et al.
7,054,873 B2 5/2006 Nordström et al.
7,233,333 B2 6/2007 Lomask
7,310,733 B1 12/2007 Pearson et al.
7,478,246 B2 1/2009 Arndt et al.
7,484,091 B2 1/2009 Bade et al.
7,526,501 B2 4/2009 Albahari et al.
7,529,801 B2 5/2009 Moore et al.
7,562,045 B2 7/2009 Beadle et al.
7,707,411 B2 4/2010 Bade et al.
7,739,211 B2 6/2010 Coffman et al.
7,743,153 B2 6/2010 Hall et al.

7,747,559 B2 6/2010 Leitner et al.
7,765,431 B2 7/2010 Agha et al.
7,797,548 B2 9/2010 Pearson et al.
7,856,544 B2 12/2010 Schenfeld et al.
7,926,026 B2 4/2011 Klein et al.
7,962,635 B2 6/2011 Naidu et al.
7,996,885 B2 8/2011 Jaiswal et al.
8,032,925 B2 10/2011 Cho
8,037,284 B2 10/2011 Schenfeld et al.
8,037,521 B2 10/2011 Minato
8,050,907 B2 11/2011 Baisley et al.
8,086,852 B2 12/2011 Bade et al.
8,103,906 B1 1/2012 Alibakhsh et al.
8,122,122 B1 2/2012 Clingenpeel et al.
8,140,977 B2 3/2012 Kriss et al.
8,151,107 B2 4/2012 Song et al.
8,160,999 B2 4/2012 Jin et al.
8,209,204 B2 6/2012 Adler et al.
8,276,197 B1 9/2012 Mangal et al.
8,291,233 B2 10/2012 Pearson et al.
8,301,660 B2 10/2012 Yalamanchi
8,341,711 B1 12/2012 Pennington et al.
8,351,456 B2 1/2013 Kadous et al.
8,352,589 B2 1/2013 Ridel et al.
8,359,584 B2 1/2013 Rao et al.
8,443,442 B2 5/2013 Wang et al.
8,490,055 B2 7/2013 Basak
8,497,863 B2 7/2013 Xie et al.
8,549,002 B2 10/2013 Herter et al.
8,561,157 B2 10/2013 Ge
8,595,262 B1 11/2013 Hayden
8,607,306 B1 12/2013 Bridge et al.
8,655,989 B2 2/2014 Ritter et al.
8,671,453 B2 3/2014 Underwood et al.
8,725,587 B2 5/2014 Beadle et al.
8,826,403 B2 9/2014 Bhaskaran et al.
8,843,646 B2 9/2014 Kuzin et al.
8,862,524 B2 10/2014 Zheng et al.
8,959,608 B2 2/2015 Ahmed et al.
9,021,583 B2 4/2015 Wittenstein et al.
9,037,273 B2 5/2015 Mikkelsen
9,043,764 B2 5/2015 Ranganathan et al.
9,053,306 B2 6/2015 Yoshigaki et al.
9,053,437 B2 6/2015 Adler et al.
9,064,210 B1 6/2015 Hart
9,075,618 B2 7/2015 Winternitz et al.
9,110,873 B2 8/2015 Woodall et al.
9,159,024 B2 10/2015 Bhanot et al.
9,189,623 B1 11/2015 Lin et al.
9,225,730 B1 12/2015 Brezinski
9,231,935 B1 1/2016 Bridge et al.
9,239,873 B2 1/2016 Branch et al.
9,246,897 B2 1/2016 He
9,323,806 B2 4/2016 Sadikov et al.
9,332,020 B2 5/2016 Thomas et al.
9,369,450 B1 6/2016 Barak et al.
9,391,978 B2 7/2016 Burch et al.
9,400,882 B2 7/2016 Pearson et al.
9,430,830 B2 8/2016 Madabhushi et al.
9,495,522 B2 11/2016 Singh et al.
9,497,224 B2 11/2016 Sweet et al.
9,515,999 B2 12/2016 Ylonen
9,516,053 B1 12/2016 Muddu et al.
9,537,851 B2 1/2017 Gordon et al.
9,558,265 B1 1/2017 Tacchi et al.
9,569,869 B2 2/2017 Hesse et al.
9,582,766 B2 2/2017 Sadikov et al.
9,589,069 B2 3/2017 Yang et al.
9,591,010 B1 3/2017 Muddu et al.
9,596,253 B2 3/2017 Chauhan et al.
9,596,254 B1 3/2017 Muddu et al.
9,596,295 B2 3/2017 Banadaki et al.
9,600,915 B2 3/2017 Winternitz et al.
9,602,506 B2 3/2017 Kang et al.
9,602,526 B2 3/2017 Liu et al.
9,639,676 B2 5/2017 Betz et al.
9,652,875 B2 5/2017 Vassilvitskii et al.
9,654,503 B1 5/2017 Kowalyshyn
9,659,337 B2 5/2017 Lee et al.

(56)

References Cited

U.S. PATENT DOCUMENTS

9,665,660 B2	5/2017	Wensel	10,459,979 B2	10/2019	Piechowicz et al.
9,667,641 B2	5/2017	Muddu et al.	10,462,169 B2	10/2019	Durairaj et al.
9,679,243 B2	6/2017	Zou et al.	10,491,705 B2	11/2019	Oetting et al.
9,699,205 B2	7/2017	Muddu et al.	10,496,263 B2	12/2019	So et al.
9,710,332 B1	7/2017	Fan et al.	10,496,468 B2	12/2019	Gefen et al.
9,720,703 B2	8/2017	Reick et al.	10,496,678 B1	12/2019	Tang
9,720,704 B2	8/2017	Reick et al.	10,505,818 B1	12/2019	Yona et al.
9,727,441 B2	8/2017	Agarwal et al.	10,510,007 B2	12/2019	Singhal et al.
9,727,604 B2	8/2017	Jin et al.	10,515,095 B2	12/2019	Childress et al.
9,729,416 B1	8/2017	Khanal et al.	10,521,584 B1	12/2019	Mehr
9,734,040 B2	8/2017	Gounares	10,534,633 B2	1/2020	Hilemon et al.
9,740,744 B2	8/2017	Stetson et al.	10,565,373 B1	2/2020	Rao et al.
9,741,138 B2	8/2017	Friedlander et al.	10,581,891 B1	3/2020	Kapoor et al.
9,749,339 B2	8/2017	Kadambe et al.	10,587,609 B2	3/2020	Ebrahimi et al.
9,753,960 B1	9/2017	Trojanovsky	10,592,535 B2	3/2020	Ahn et al.
9,760,619 B1	9/2017	Lattanzi et al.	10,594,718 B1	3/2020	Deaguero et al.
9,781,115 B2	10/2017	Heise	10,599,718 B2	3/2020	Kumar et al.
9,787,705 B1	10/2017	Love et al.	RE47,937 E	4/2020	Ramachandran et al.
9,805,080 B2	10/2017	Joshi et al.	RE47,952 E	4/2020	Ramachandran et al.
9,805,140 B2	10/2017	Chakrabarti et al.	10,614,200 B2	4/2020	Betz et al.
9,811,790 B2	11/2017	Ahern et al.	10,642,867 B2	5/2020	Palanciuc
9,813,435 B2	11/2017	Muddu et al.	10,656,979 B2	5/2020	Ishakian et al.
9,819,671 B2	11/2017	Ji	10,664,757 B2	5/2020	Lastras-Montano et al.
9,824,473 B2	11/2017	Winternitz et al.	10,666,668 B2	5/2020	Muddu et al.
9,830,435 B2	11/2017	Haven	10,673,880 B1	6/2020	Pratt et al.
9,836,183 B1	12/2017	Love et al.	10,685,295 B1	6/2020	Ross et al.
9,838,410 B2	12/2017	Muddu et al.	10,693,900 B2	6/2020	Zadeh et al.
9,843,837 B2	12/2017	Gopalan	10,698,954 B2	6/2020	Piechowicz et al.
9,852,230 B2	12/2017	Fleury et al.	10,701,051 B2	6/2020	Ohsumi
9,853,968 B2	12/2017	Shen et al.	10,708,082 B1	7/2020	Bakiraj et al.
9,864,672 B2	1/2018	Seto et al.	10,735,329 B2	8/2020	Wang et al.
9,887,999 B2	2/2018	Dong et al.	10,754,940 B2	8/2020	Ohsumi
9,923,911 B2	3/2018	Vasseur et al.	10,756,982 B2	8/2020	Bai et al.
9,942,220 B2	4/2018	Bajenov et al.	10,768,002 B2	9/2020	Epperlein et al.
9,946,800 B2	4/2018	Qian et al.	10,771,488 B2	9/2020	Verma et al.
9,953,014 B1	4/2018	Reshadi et al.	10,775,183 B2	9/2020	Ho et al.
9,954,842 B2	4/2018	Huang	10,776,191 B2	9/2020	Zheng et al.
9,985,827 B2	5/2018	Li et al.	10,788,570 B2	9/2020	Wilson
10,003,605 B2	6/2018	Muddu et al.	10,791,131 B2	9/2020	Nor et al.
10,033,611 B1	7/2018	Linkous et al.	10,797,974 B2	10/2020	Giura et al.
10,104,071 B2	10/2018	Gordon et al.	10,812,497 B2	10/2020	Venkatramani et al.
10,115,111 B2	10/2018	Miltonberger	10,824,675 B2	11/2020	Alonso et al.
10,116,670 B2	10/2018	Muddu et al.	10,824,813 B2	11/2020	Smith et al.
10,121,000 B1	11/2018	Rivlin et al.	10,885,452 B1	1/2021	Garg
10,122,740 B1	11/2018	Finkelshtein et al.	10,904,007 B2	1/2021	Kim et al.
10,127,273 B2	11/2018	Dickey	10,904,270 B2	1/2021	Muddu et al.
10,142,357 B1	11/2018	Tamersoy et al.	10,911,470 B2	2/2021	Muddu et al.
10,148,677 B2	12/2018	Muddu et al.	10,951,648 B2	3/2021	Doron et al.
10,149,148 B2	12/2018	Zha et al.	10,986,114 B1	4/2021	Singh et al.
10,158,652 B2	12/2018	Muddu et al.	11,036,716 B2	6/2021	Griffith et al.
10,182,058 B2	1/2019	Xu	11,036,800 B1	6/2021	Kayyoor et al.
10,205,735 B2	2/2019	Apostolopoulos	11,044,264 B2	6/2021	Durairaj et al.
10,205,736 B2	2/2019	Rieke et al.	11,048,492 B2	6/2021	Jain et al.
10,237,254 B2	3/2019	McDowell et al.	11,080,392 B2	8/2021	Bennett et al.
10,237,294 B1	3/2019	Zadeh et al.	11,082,289 B2	8/2021	Dang et al.
10,243,970 B2	3/2019	Muddu et al.	11,120,343 B2	9/2021	Das et al.
10,249,266 B2	4/2019	Zamir	11,126,533 B2	9/2021	Knowles et al.
10,254,848 B2	4/2019	Winternitz et al.	11,153,339 B1	10/2021	Kapoor et al.
10,331,659 B2	6/2019	Ahuja et al.	11,194,849 B2	12/2021	Lassoued et al.
10,338,895 B2	7/2019	Zhang et al.	11,212,299 B2	12/2021	Gamble et al.
10,339,309 B1	7/2019	Kling et al.	11,233,821 B2	1/2022	Yadav et al.
10,367,704 B2	7/2019	Giura et al.	11,258,807 B2	2/2022	Muddu et al.
10,382,303 B2	8/2019	Khanal et al.	11,281,519 B2	3/2022	Krishnaswamy et al.
10,382,529 B2	8/2019	Wan et al.	11,314,789 B2	4/2022	Goldfarb
10,389,738 B2	8/2019	Muddu et al.	11,411,966 B2	8/2022	Muddu et al.
10,389,742 B2	8/2019	Reddy et al.	11,431,735 B2	8/2022	Shua
10,419,463 B2	9/2019	Muddu et al.	11,463,464 B2	10/2022	Zadeh et al.
10,419,465 B2	9/2019	Muddu et al.	11,489,863 B1	11/2022	Shua et al.
10,419,468 B2	9/2019	Glatfelter et al.	11,494,787 B2	11/2022	Erickson et al.
10,419,469 B1	9/2019	Singh et al.	11,544,138 B2	1/2023	Kapish et al.
10,425,437 B1	9/2019	Bog et al.	11,575,693 B1	2/2023	Muddu et al.
10,432,639 B1	10/2019	Bebbe et al.	11,606,272 B1	3/2023	Popelka et al.
10,447,526 B2	10/2019	Tucker et al.	11,636,090 B2	4/2023	Li et al.
10,454,753 B2	10/2019	Sasturkar et al.	11,640,388 B2	5/2023	Yang et al.
10,454,889 B2	10/2019	Huang	11,647,034 B2	5/2023	Levin et al.
			11,658,990 B2	5/2023	Shapoury
			11,669,571 B2	6/2023	Binkley et al.
			11,677,772 B1	6/2023	Kapoor et al.
			11,693,958 B1	7/2023	Steiman

(56)

References Cited

U.S. PATENT DOCUMENTS

11,722,554	B2	8/2023	Keren et al.	2010/0329162	A1	12/2010	Kadous et al.
11,734,351	B2	8/2023	Binkley et al.	2011/0023098	A1	1/2011	Pearson et al.
11,734,419	B1	8/2023	Mackle	2011/0029952	A1	2/2011	Harrington
11,748,473	B2	9/2023	Araujo et al.	2011/0055138	A1	3/2011	Khanduja et al.
11,755,576	B1	9/2023	Jiang et al.	2011/0119100	A1	5/2011	Ruhl et al.
11,755,602	B2	9/2023	Smith et al.	2011/0154287	A1	6/2011	Mukkamala et al.
11,769,098	B2	9/2023	Adinarayan et al.	2011/0302631	A1	12/2011	Sureshchandra et al.
11,770,387	B1	9/2023	Shivamoggi et al.	2012/0005243	A1	1/2012	Merwe et al.
11,770,398	B1	9/2023	Erlingsson et al.	2012/0054732	A1	3/2012	Jain et al.
11,785,104	B2	10/2023	Erlingsson et al.	2012/0089875	A1	4/2012	Faust et al.
2002/0059531	A1	5/2002	Lai On	2012/0102029	A1	4/2012	Larson et al.
2002/0161889	A1	10/2002	Gamache et al.	2012/0143898	A1	6/2012	Bruno et al.
2002/0184225	A1	12/2002	Ghukasyan	2012/0158858	A1	6/2012	Gkantsidis et al.
2003/0037136	A1	2/2003	Labovitz et al.	2012/0159333	A1	6/2012	Mital et al.
2003/0179227	A1	9/2003	Ahmad et al.	2012/0173541	A1	7/2012	Venkataramani
2003/0233361	A1	12/2003	Cady	2012/0317149	A1	12/2012	Jagota et al.
2004/0015470	A1	1/2004	Smith et al.	2012/0317151	A1	12/2012	Ruf et al.
2004/0225929	A1	11/2004	Agha et al.	2012/0323956	A1	12/2012	Dumitru et al.
2005/0060287	A1	3/2005	Hellman et al.	2013/0024412	A1	1/2013	Gong et al.
2005/0102284	A1	5/2005	Srinivasan et al.	2013/0067100	A1	3/2013	Kuzin et al.
2005/0102365	A1	5/2005	Moore et al.	2013/0081118	A1	3/2013	Ge
2005/0108142	A1	5/2005	Beadle et al.	2013/0086667	A1	4/2013	Haven
2005/0188222	A1	8/2005	Motsinger et al.	2013/0097320	A1	4/2013	Ritter et al.
2005/0231760	A1	10/2005	Minato	2013/0151453	A1	6/2013	Bhanot et al.
2005/0246288	A1	11/2005	Kimura et al.	2013/0173915	A1	7/2013	Haulund
2005/0246521	A1	11/2005	Bade et al.	2013/0205357	A1	8/2013	Bahnck et al.
2006/0025987	A1	2/2006	Baisley et al.	2013/0219295	A1	8/2013	Feldman et al.
2006/0026419	A1	2/2006	Arndt et al.	2013/0269007	A1	10/2013	Yoshigaki et al.
2006/0036896	A1	2/2006	Gamache et al.	2013/0304915	A1	11/2013	Kawai
2006/0085437	A1	4/2006	Brodhun et al.	2014/0041005	A1	2/2014	He
2006/0090095	A1	4/2006	Massa et al.	2014/0067750	A1	3/2014	Ranganathan et al.
2006/0109271	A1	5/2006	Lomask	2014/0098101	A1	4/2014	Friedlander et al.
2006/0259470	A1	11/2006	Chandrasekharan et al.	2014/0115001	A1	4/2014	Arroyo et al.
2006/0288415	A1	12/2006	Wong	2014/0115011	A1	4/2014	Buerner et al.
2007/0050497	A1	3/2007	Haley et al.	2014/0125672	A1	5/2014	Winternitz et al.
2007/0118909	A1	5/2007	Hertzog et al.	2014/0181944	A1	6/2014	Ahmed et al.
2007/0130330	A1	6/2007	Ridel et al.	2014/0208191	A1	7/2014	Zaric et al.
2007/0162605	A1	7/2007	Chalasani et al.	2014/0229607	A1	8/2014	Jung et al.
2007/0162963	A1	7/2007	Penet et al.	2014/0245443	A1	8/2014	Chakraborty
2007/0168696	A1	7/2007	Ridel et al.	2014/0279779	A1	9/2014	Zou et al.
2007/0169175	A1	7/2007	Hall et al.	2014/0280068	A1	9/2014	Dhoopar et al.
2007/0214111	A1	9/2007	Jin et al.	2014/0325631	A1	10/2014	Pearson et al.
2007/0225956	A1	9/2007	Pratt et al.	2014/0359558	A1	12/2014	Chamberlain
2007/0266425	A1	11/2007	Cho	2014/0379716	A1	12/2014	Branch et al.
2007/0282916	A1	12/2007	Albahari et al.	2015/0058619	A1	2/2015	Sweet et al.
2008/0034411	A1	2/2008	Aoyama	2015/0135312	A1	5/2015	Wada et al.
2008/0065879	A1	3/2008	Song et al.	2015/0161201	A1	6/2015	Sadikov et al.
2008/0072062	A1	3/2008	Pearson et al.	2015/0172321	A1	6/2015	Kirti et al.
2008/0109730	A1	5/2008	Coffman et al.	2015/0188751	A1	7/2015	Vasseur et al.
2008/0147707	A1	6/2008	Jin et al.	2015/0213598	A1	7/2015	Madabhushi et al.
2008/0148180	A1	6/2008	Liu et al.	2015/0302440	A1	10/2015	Monden et al.
2008/0151893	A1	6/2008	Nordmark et al.	2015/0310649	A1	10/2015	Winternitz et al.
2008/0155335	A1	6/2008	Klein et al.	2015/0319185	A1	11/2015	Kirti et al.
2008/0244718	A1	10/2008	Frost et al.	2015/0341379	A1	11/2015	Lefebvre et al.
2008/0263643	A1	10/2008	Jaiswal et al.	2015/0356144	A1	12/2015	Chawla et al.
2008/0270451	A1	10/2008	Thomsen et al.	2016/0063226	A1	3/2016	Singh et al.
2009/0006843	A1	1/2009	Bade et al.	2016/0078365	A1	3/2016	Baumard
2009/0007010	A1	1/2009	Kriss et al.	2016/0080204	A1	3/2016	Mishra et al.
2009/0019160	A1	1/2009	Schuler	2016/0080404	A1	3/2016	Kohout et al.
2009/0063857	A1	3/2009	Bade et al.	2016/0110434	A1	4/2016	Kakaraddi et al.
2009/0165109	A1	6/2009	Hird	2016/0120070	A1	4/2016	Myrah et al.
2009/0177573	A1	7/2009	Beadle et al.	2016/0149937	A1	5/2016	Katmor et al.
2009/0222740	A1	9/2009	Yuan	2016/0203411	A1	7/2016	Sadikov et al.
2009/0228474	A1	9/2009	Chiu et al.	2016/0205125	A1	7/2016	Kim et al.
2009/0271504	A1	10/2009	Ginter et al.	2016/0218911	A1	7/2016	Wessels et al.
2009/0287720	A1	11/2009	Herter et al.	2016/0261544	A1	9/2016	Conover
2009/0307651	A1	12/2009	Senthil et al.	2016/0330183	A1	11/2016	McDowell et al.
2009/0327328	A1	12/2009	Woodall et al.	2016/0330206	A1	11/2016	Xu
2010/0042823	A1	2/2010	Arndt et al.	2016/0357521	A1	12/2016	Zhang et al.
2010/0094767	A1	4/2010	Miltonberger	2016/0359592	A1	12/2016	Kulshreshtha et al.
2010/0114931	A1	5/2010	Xie et al.	2016/0359872	A1	12/2016	Yadav et al.
2010/0172261	A1	7/2010	Shinbo et al.	2016/0373428	A1	12/2016	Shi
2010/0217860	A1	8/2010	Naidu et al.	2017/0063830	A1	3/2017	Huang
2010/0274785	A1	10/2010	Procopiuc et al.	2017/0063888	A1	3/2017	Muddu et al.
2010/0309206	A1	12/2010	Xie et al.	2017/0063903	A1	3/2017	Muddu et al.
				2017/0063905	A1	3/2017	Muddu et al.
				2017/0063906	A1	3/2017	Muddu et al.
				2017/0063908	A1	3/2017	Muddu et al.
				2017/0063909	A1	3/2017	Muddu et al.

(56)

References Cited

U.S. PATENT DOCUMENTS

2017/0063910 A1	3/2017	Muddu et al.	2019/0028327 A1	1/2019	Silva et al.
2017/0063911 A1	3/2017	Muddu et al.	2019/0042879 A1	2/2019	Munoz
2017/0063912 A1	3/2017	Muddu et al.	2019/0042950 A1	2/2019	Lin et al.
2017/0070594 A1	3/2017	Oetting et al.	2019/0050445 A1	2/2019	Griffith et al.
2017/0076206 A1	3/2017	Lastras-Montano et al.	2019/0058626 A1	2/2019	Knowles et al.
2017/0085553 A1	3/2017	Gordon et al.	2019/0068627 A1	2/2019	Thampy
2017/0086069 A1	3/2017	Liu	2019/0075126 A1	3/2019	Muddu et al.
2017/0102961 A1	4/2017	Hilemon et al.	2019/0087480 A1	3/2019	Palanciuc
2017/0111245 A1	4/2017	Ishakian et al.	2019/0101622 A1	4/2019	Wilson
2017/0116315 A1	4/2017	Xiong et al.	2019/0109870 A1	4/2019	Bedhapudi et al.
2017/0118099 A1	4/2017	Huang	2019/0123973 A1	4/2019	Jeuk et al.
2017/0118240 A1	4/2017	Reddy et al.	2019/0132224 A1	5/2019	Verma et al.
2017/0134240 A1	5/2017	Hévizi et al.	2019/0149553 A1	5/2019	Xu
2017/0142140 A1	5/2017	Muddu et al.	2019/0149565 A1 *	5/2019	Hagi G06N 3/042 726/23
2017/0147646 A1	5/2017	Lee et al.	2019/0158524 A1	5/2019	Zadeh et al.
2017/0148197 A1	5/2017	Winternitz et al.	2019/0163555 A1	5/2019	Zheng et al.
2017/0155570 A1	6/2017	Maheshwari et al.	2019/0222597 A1	7/2019	Crabtree et al.
2017/0155672 A1	6/2017	Muthukrishnan et al.	2019/0227860 A1	7/2019	Gefen et al.
2017/0163666 A1	6/2017	Venkatramani et al.	2019/0236204 A1	8/2019	Canim et al.
2017/0223036 A1	8/2017	Muddu et al.	2019/0259033 A1	8/2019	Reddy et al.
2017/0230183 A1	8/2017	Sweet et al.	2019/0312796 A1	10/2019	Giura et al.
2017/0249069 A1	8/2017	Zamir	2019/0312898 A1	10/2019	Verma et al.
2017/0251013 A1	8/2017	Kirti et al.	2019/0327251 A1	10/2019	Muddu et al.
2017/0257358 A1	9/2017	Ebrahimi et al.	2019/0339965 A1	11/2019	Garvey et al.
2017/0262521 A1	9/2017	Cho et al.	2019/0342282 A1	11/2019	Carbune et al.
2017/0272344 A1	9/2017	Tang et al.	2019/0342307 A1	11/2019	Gamble et al.
2017/0277553 A1	9/2017	Zada et al.	2019/0342311 A1	11/2019	Muddu et al.
2017/0277997 A1	9/2017	Zong et al.	2019/0349305 A1	11/2019	Wang et al.
2017/0279827 A1	9/2017	Savalle et al.	2019/0354554 A1	11/2019	Piechowicz et al.
2017/0286190 A1	10/2017	Ishakian et al.	2019/0356555 A1	11/2019	Bai et al.
2017/0288974 A1	10/2017	Yoshihira et al.	2019/0364067 A1	11/2019	Yona et al.
2017/0330096 A1	11/2017	Gupta et al.	2019/0378050 A1	12/2019	Edkin et al.
2017/0337262 A1	11/2017	Smith et al.	2019/0384784 A1	12/2019	Canim et al.
2017/0346683 A1	11/2017	Li et al.	2020/0014718 A1	1/2020	Durairaj et al.
2017/0353853 A1	12/2017	Zha et al.	2020/0021607 A1	1/2020	Muddu et al.
2017/0359361 A1	12/2017	Modani et al.	2020/0065857 A1	2/2020	Lagi et al.
2017/0366492 A1	12/2017	Ho et al.	2020/0074341 A1	3/2020	He et al.
2018/0004835 A1	1/2018	Piechowicz et al.	2020/0076685 A1	3/2020	Vaidya et al.
2018/0004859 A1	1/2018	Piechowicz et al.	2020/0080856 A1	3/2020	Ho et al.
2018/0007145 A1	1/2018	Piechowicz et al.	2020/0175042 A1	6/2020	Batrani
2018/0013650 A1	1/2018	Khanal et al.	2020/0175361 A1	6/2020	Che et al.
2018/0019932 A1	1/2018	Giura et al.	2020/0192690 A1	6/2020	Gupta et al.
2018/0020015 A1	1/2018	Munro et al.	2020/0218579 A1	7/2020	et al.
2018/0025361 A1	1/2018	Llagostera et al.	2020/0228555 A1	7/2020	Wittenschlaeger
2018/0034840 A1	2/2018	Marquardt et al.	2020/0252376 A1	8/2020	Feng et al.
2018/0039688 A1	2/2018	Ahn et al.	2020/0259852 A1	8/2020	Wolff et al.
2018/0063178 A1	3/2018	Jadhav et al.	2020/0272740 A1	8/2020	Obee et al.
2018/0067981 A1	3/2018	Ahuja et al.	2020/0278892 A1	9/2020	Nainar et al.
2018/0069885 A1	3/2018	Patterson et al.	2020/0280592 A1	9/2020	Ithal et al.
2018/0084069 A1	3/2018	Be'ery et al.	2020/0287923 A1	9/2020	Raghavendra et al.
2018/0089132 A1	3/2018	Atta et al.	2020/0287927 A1	9/2020	Zadeh et al.
2018/0096047 A1	4/2018	Childress et al.	2020/0311644 A1	10/2020	Willard, III et al.
2018/0097793 A1	4/2018	Agarwal et al.	2020/0314159 A1	10/2020	Gerson-Golan et al.
2018/0103052 A1	4/2018	Choudhury et al.	2020/0320106 A1	10/2020	Goldfarb
2018/0115578 A1	4/2018	Subbarayan et al.	2020/0322346 A1	10/2020	Rensburg et al.
2018/0123864 A1	5/2018	Tucker et al.	2020/0334293 A1	10/2020	Piechowicz et al.
2018/0124189 A1	5/2018	Edgington et al.	2020/0349049 A1	11/2020	Krebs et al.
2018/0137858 A1	5/2018	Saxena et al.	2020/0351151 A1	11/2020	Dang et al.
2018/0139200 A1	5/2018	Gordon et al.	2020/0396231 A1	12/2020	Krebs et al.
2018/0173789 A1	6/2018	Llagostera et al.	2020/0403860 A1	12/2020	Lewis et al.
2018/0174062 A1	6/2018	Simo et al.	2020/0404008 A1	12/2020	Venkatramani et al.
2018/0181750 A1	6/2018	Lamothe-Brassard	2020/0412752 A1	12/2020	Shapoury
2018/0191781 A1	7/2018	Palani et al.	2021/0019209 A1	1/2021	Krishnaswamy et al.
2018/0211425 A1	7/2018	Winternitz et al.	2021/0064666 A1 *	3/2021	Wang G06N 5/04
2018/0219888 A1	8/2018	Apostolopoulos	2021/0097052 A1	4/2021	Hans et al.
2018/0219897 A1	8/2018	Muddu et al.	2021/0182248 A1	6/2021	Jayanthi
2018/0227286 A1	8/2018	Ohsumi	2021/0200612 A1	7/2021	Martyanov
2018/0248901 A1	8/2018	Rieke	2021/0232420 A1	7/2021	Dhruvakumar et al.
2018/0268078 A1	9/2018	Gianetto et al.	2021/0286798 A1	9/2021	Li et al.
2018/0288063 A1	10/2018	Koottayi et al.	2021/0294798 A1	9/2021	Binkley et al.
2018/0329958 A1	11/2018	Choudhury et al.	2021/0329019 A1	10/2021	Shua
2018/0336353 A1	11/2018	Manadhata et al.	2021/0336976 A1	10/2021	Shua
2018/0357422 A1	12/2018	Telang et al.	2021/0365643 A1	11/2021	Agrawal et al.
2018/0359162 A1	12/2018	Savov et al.	2021/0377287 A1	12/2021	Shua
2018/0367548 A1	12/2018	Stokes, III et al.	2021/0406689 A1	12/2021	Zhang et al.
			2021/0406917 A1	12/2021	Erickson et al.
			2022/0004718 A1	1/2022	Quamar et al.
			2022/0050840 A1	2/2022	Parravicini et al.

(56)

References Cited**U.S. PATENT DOCUMENTS**

2022/0058193	A1	2/2022	Smith et al.
2022/0067186	A1	3/2022	Thakur et al.
2022/0086179	A1	3/2022	Levin et al.
2022/0092481	A1	3/2022	Neithalath et al.
2022/0092668	A1	3/2022	Lu et al.
2022/0121741	A1	4/2022	Araujo et al.
2022/0124108	A1	4/2022	Gamble et al.
2022/0129803	A1	4/2022	Bikumala et al.
2022/0179730	A1	6/2022	Chan et al.
2022/0191226	A1	6/2022	Chan et al.
2022/0247769	A1	8/2022	Erlingsson et al.
2022/0291840	A1	9/2022	Bhide et al.
2022/0292006	A1	9/2022	Ramachandran et al.
2022/0327119	A1	10/2022	Gasper et al.
2022/0342690	A1	10/2022	Shua
2022/0345480	A1	10/2022	Shua
2022/0345481	A1	10/2022	Shua
2022/0345483	A1	10/2022	Shua
2022/0350789	A1	11/2022	Yang et al.
2022/0350931	A1	11/2022	Shua
2022/0366352	A1	11/2022	Matsuoka et al.
2022/0374800	A1	11/2022	Adinarayan et al.
2022/0376970	A1	11/2022	Chawathe et al.
2022/0382611	A1	12/2022	Kapish et al.
2022/0382736	A1	12/2022	Beilis et al.
2022/0394082	A1	12/2022	Keren et al.
2022/0414072	A1	12/2022	Tandon et al.
2022/0414105	A1	12/2022	Umay et al.
2022/0417273	A1	12/2022	Levin et al.
2023/0025252	A1	1/2023	Erickson et al.
2023/0039566	A1	2/2023	Ghag et al.
2023/0052827	A1	2/2023	Araujo et al.
2023/0077030	A1	3/2023	Makkar et al.
2023/0088960	A1	3/2023	Popelka et al.
2023/0096930	A1	3/2023	Dasdan
2023/0101339	A1	3/2023	Li
2023/0101773	A1	3/2023	Katahanas et al.
2023/0138371	A1	5/2023	Bandukwala et al.
2023/0237570	A1	7/2023	Li et al.
2023/0244523	A1	8/2023	Gorantla et al.
2023/0251960	A1	8/2023	Sharma et al.
2023/0274095	A1	8/2023	Kelkar et al.
2023/0275909	A1	8/2023	Shivamoggi et al.
2023/0291755	A1	9/2023	Siebel et al.
2024/0070495	A1*	2/2024	Satish G06F 40/56
2024/0201983	A1*	6/2024	Groenewegen G06F 8/20

OTHER PUBLICATIONS

Akoglu et al., “Graph-based Anomaly Detection and Description: A Survey”, Apr. 28, 2014.

Amidon et al., “Program Fracture and Recombination for Efficient Automatic Code Reuse”, in 2015 IEEE High Performance Extreme Computing Conference (HPEC), Sep. 2015, pp. 1-6, IEEE.org (online), DOI: 10.1109/HPEC.2015.7396314.

Ammar et al., “Query Optimization Techniques in Graph Databases”, International Journal of Database Management Systems (IIDMS), vol. 8, No. 4, Aug. 2016, pp. 1-14 (Year: 2016).

Balasubramanian et al., “An Architecture for Intrusion Detection Using Autonomous Agents”, in Proceedings 14th Annual Computer Security Applications Conference (Cat. No. 98EX217), 19 pages, Jun. 1998, IEEE, DOI: 10.1109/CSAC.1998.738563.

Beutel et al., “User Behavior Modeling with Large-Scale Graph Analysis”, Computer Science Department, Carnegie Mellon University, May 2016.

Bugiel et al., “Towards Taming Privilege-Escalation Attacks on Android”, in NOSS (vol. 17, p. 19), Feb. 2012.

Chang et al., “Reality Bites—Progressive Querying and Result Visualization in Logical and VR Spaces”, Proceedings of 1994 IEEE Symposium on Visual Languages, pp. 100-109, Oct. 1994, IEEE, DOI: 10.1109/NL.1994.363635.

Chesson, “Communication and Control in a Cluster Network”, ACM '74: Proceedings of the 1974 annual ACM conference—vol. 2, Jan. 1974, pp. 509-514, <http://doi.org/10.1145/1408839> (Year 1974).

Crosbie et al., “Defending a Computer System using Autonomous Agents”, docs.lib.purdue.edu (online), Mar. 1995, 11 pages.

Hautamaki et al., “Outlier Detection Using k-Nearest Neighbour Graph”, Proceedings of the 17th International Conference on Pattern Recognition (ICPR 2004), vol. 3, Aug. 2004, IEEE, DOI: 10.1109/ICPR.2004.1334558.

Hooper et al., “Medusa: a simple tool for interaction graph analysis”, Bioinformatics, vol. 21 No. 24, Sep. 2005, pp. 4432-4433, Oxford University Press (online), URL: <https://academic.oup.com/bioinformatics/article/21/24/4432/179694>.

Koutra et al., “Exploring and Making Sense of Large Graphs”, Computer Science Department, Carnegie Mellon University, Aug. 2015.

Leopold et al., “A Visual Query System for the Specification and Scientific Analysis of Continual Queries”, Proceedings, IEEE Symposium on Human-Centric Computing Languages and Environments (Cat. No. 01TH8587), Sep. 2001, pp. 203-211, IEEE, doi: 10.1109/HCC.2001.995260.

Liao et al., “Visualizing Graph Dynamics and Similarity for Enterprise Network Security and Management”, VizSec '10: Proceedings of the Seventh International Symposium on Visualization for Cyber Security, Sep. 2010, pp. 34-45, URL: <https://doi.org/10.1145/1850795.1850799>.

Long et al., “Automatic Input Rectification”, 2012 34th International Conference on Software Engineering (ICSE), Jun. 2012, pp. 80-90, IEEE.org (online), DOI: 10.1109/ICSE.2012.6227204.

Mateescu et al., “Join-Graph Propagation Algorithms”, Journal of Artificial Intelligence Research, vol. 37, Mar. 2010, pp. 279-328, AI Access Foundation, Inc. (online), URL: <https://doi.org/10.1613/jair.2842>.

Moriano et al., “Insider Threat Event Detection in User-System Interactions”, MIST '17: Proceedings of the 2017 International Workshop on Managing Insider Security Threats, Oct. 2017, pp. 1-12, ACM Digital Library (online), URL: <https://doi.org/10.1145/3139923.3139928>.

Perkins et al., “Automatically Patching Errors in Deployed Software”, Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles, Oct. 2009, pp. 82-102, acm.org (online), URL: <https://doi.org/10.1145/1629575.1629585>.

Ranshous et al., “Anomaly detection in dynamic networks: a survey”, WIREs Computational Statistics, vol. 7, May/Jun. 2015, pp. 223-247, Wiley Periodicals, Inc, United States.

Rinard, “Living in the Comfort Zone”, ACM SIGPLAN Notices, vol. 42, Issue 10, Oct. 2007, pp. 611-622, acm.org (online), URL: <https://doi.org/10.1145/1297105.1297072>.

Rinard, “Manipulating Program Functionality to Eliminate Security Vulnerabilities”, in Moving Target Defense, Jan. 2011, pp. 109-115. Springer, New York, NY.

Samuel et al., “Let's Parse to Prevent Pwnage”, Proceedings of the 5th USENIX conference on Large-Scale Exploits and Emergent Threats (LEET'12), Apr. 2012, 3 pages, acm.org (online), URL: <https://www.usenix.org/conference/leet12/workshop-program/presentation/samuel>.

Shen et al., “Active Learning for Inference and Regeneration of Applications that Access Databases”, ACM Transactions on Programming Languages and Systems (TOPLAS), vol. 42, Issue 4, Article 18, Jan. 2021, pp. 1-119, acm.org (online), URL: <https://doi.org/10.1145/3430952>.

Tamassia et al., “Graph Drawing for Security Visualization”, In: Graph Drawing (GD 2008), Lecture Notes in Computer Science, vol. 5417, Springer, Berlin, Heidelberg (online), URL: https://doi.org/10.1007/978-3-642-00219-9_2.

Vaas et al., “Detecting disguised processes using Application-Behavior Profiling”, in 2017 IEEE International Symposium on Technologies for Homeland Security (HST), pp. 1-6, Jun. 2017, IEEE, DOI: 10.1109/THS.2017.7943508.

Vasilakis et al., “Supply-Chain Vulnerability Elimination via Active Learning and Regeneration”, Proceedings of the 2021 ACM SIGSAC

(56)

References Cited

OTHER PUBLICATIONS

Conference on Computer and Communications Security, Nov. 2021, pp. 1755-1770, acm.org (online), URL: <https://doi.org/10.1145/3460120.3484736>.

Yu et al., "Recommending Join Queries Based on Path Frequency", 2015 12th Web Information System and Application Conference (WISA), Sep. 2015, pp. 21-26, IEEE, DOI: 10.1109/WVISA.2015.52.

* cited by examiner

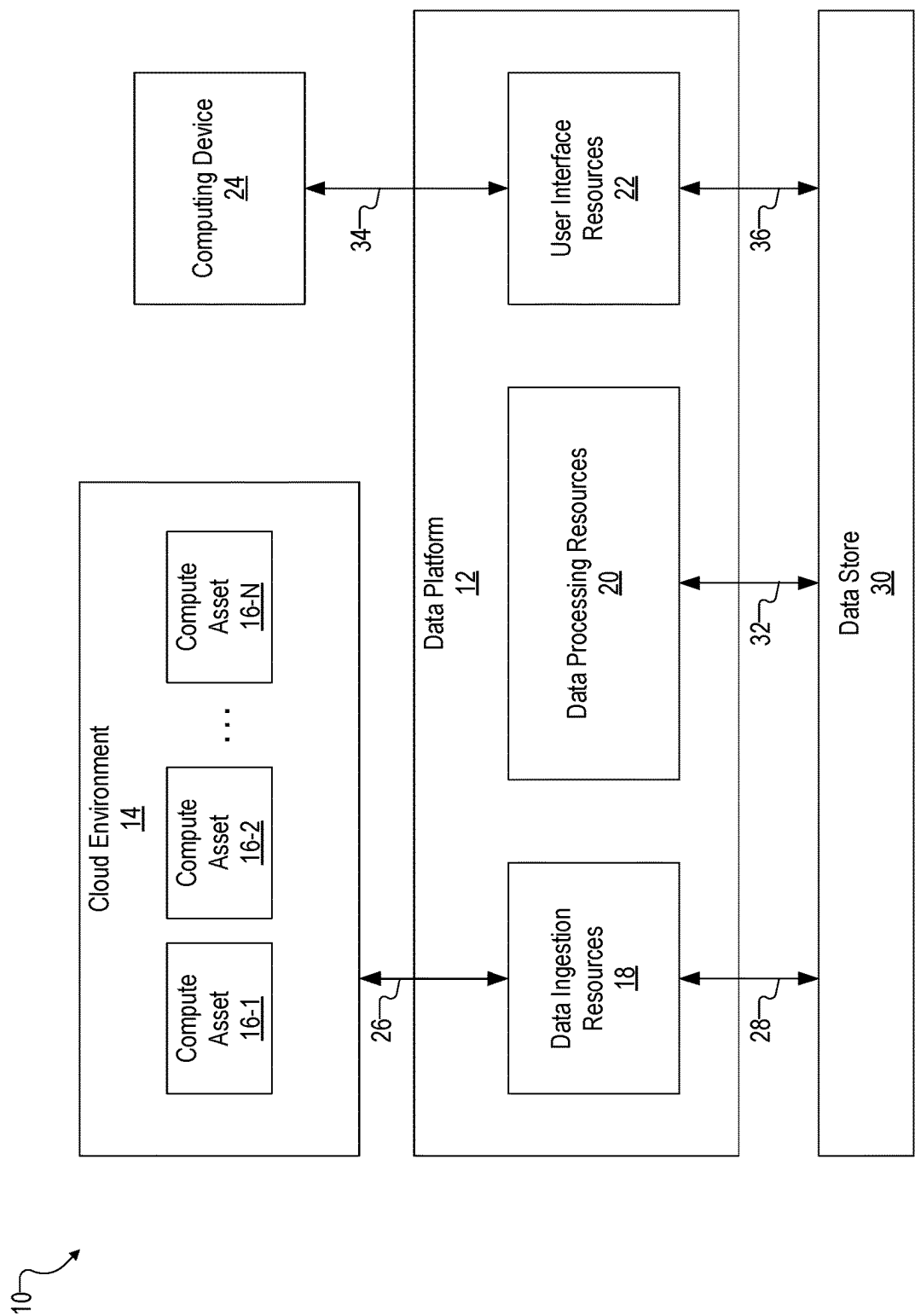


Fig. 1A

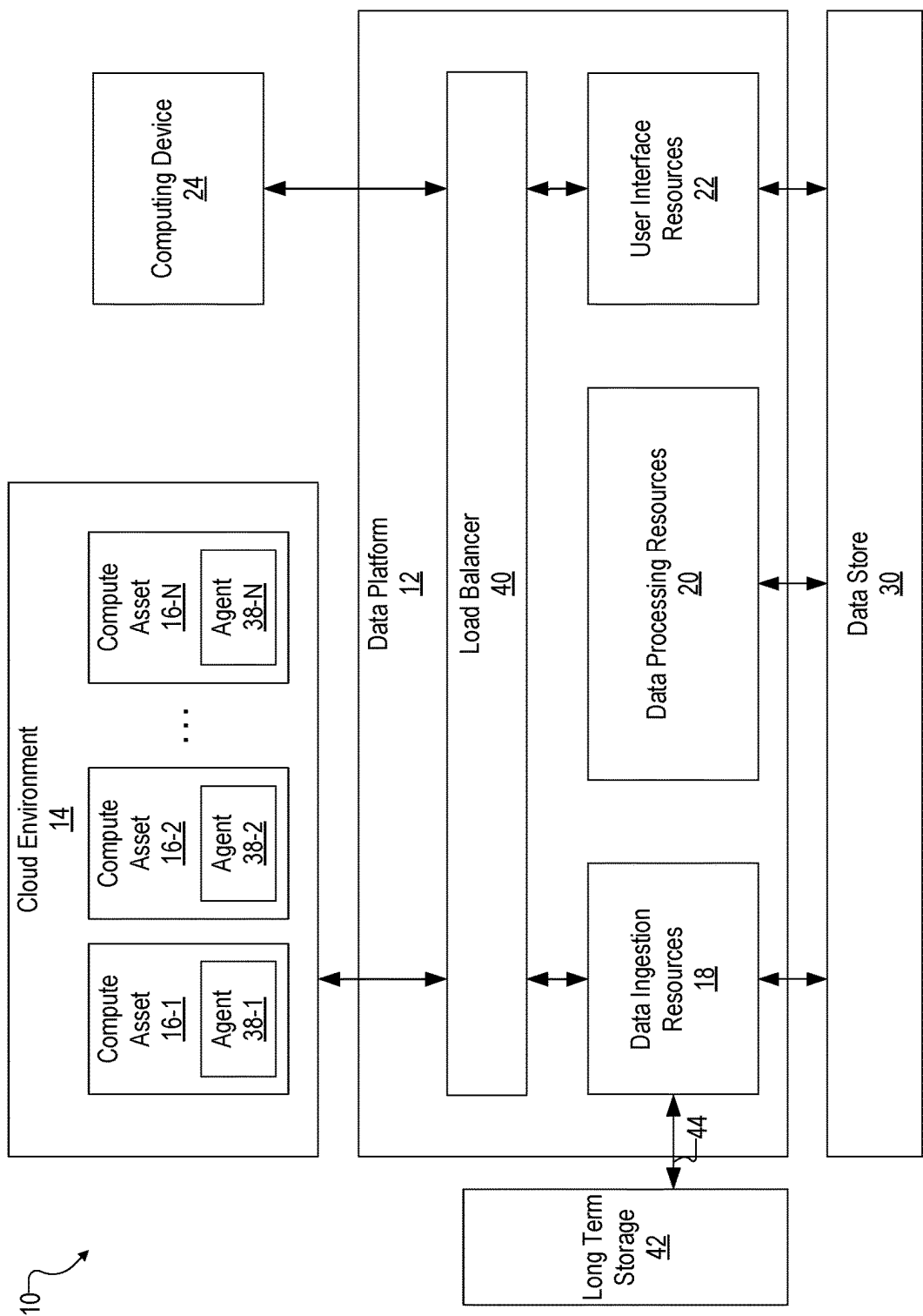


Fig. 1B

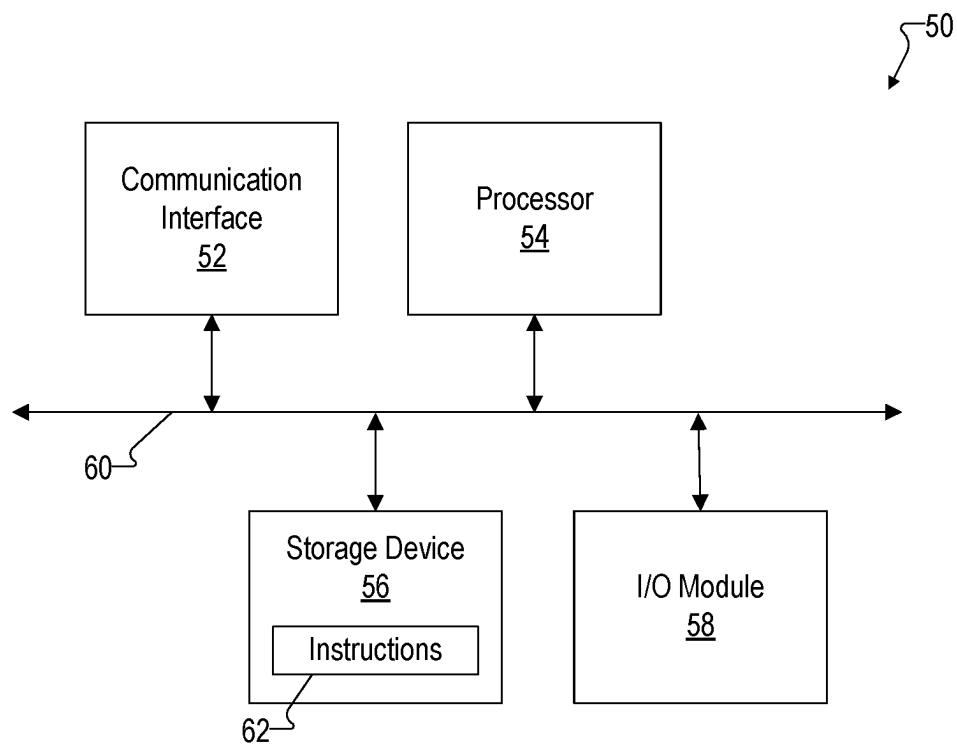


Fig. 1C

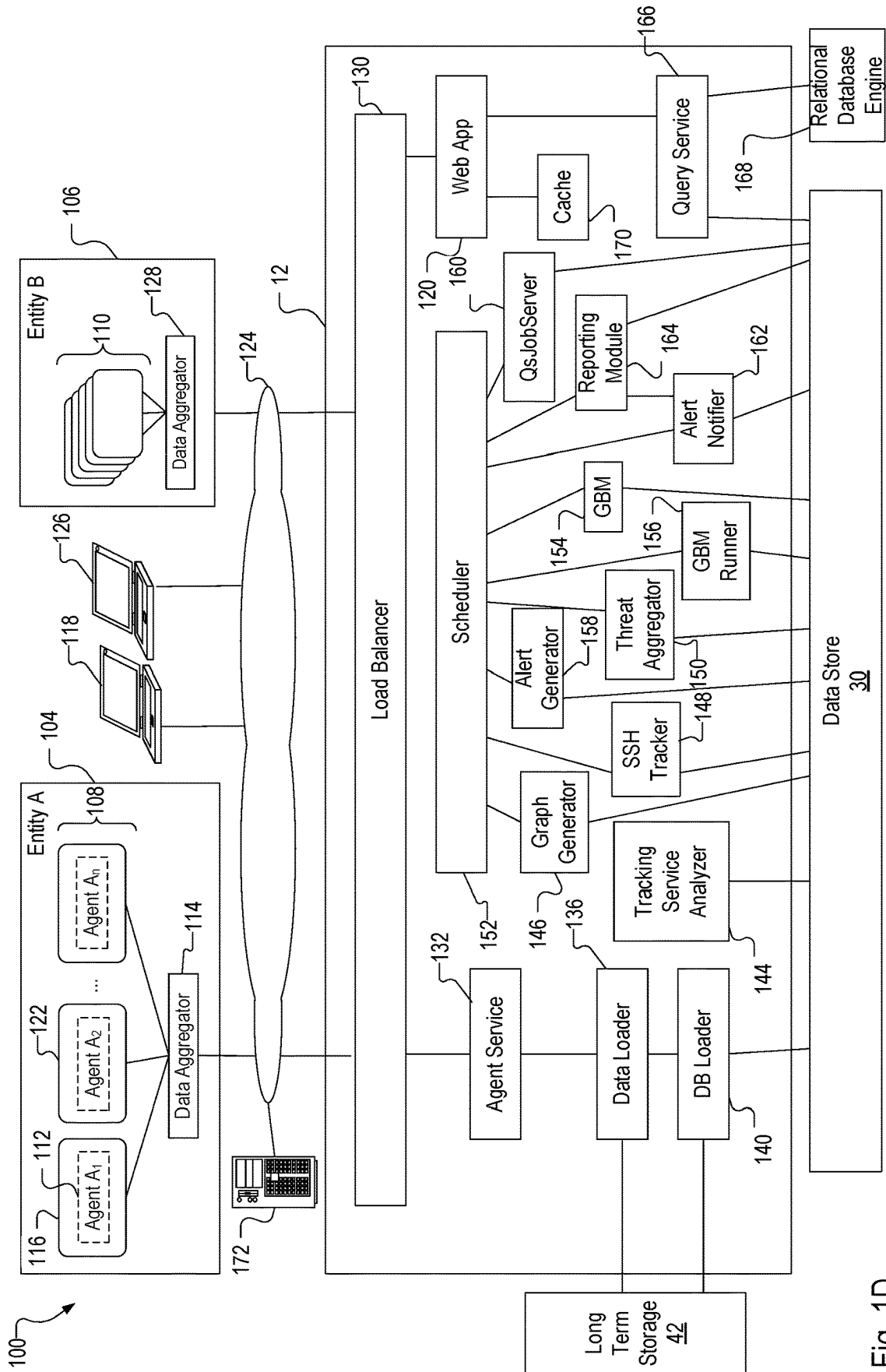


Fig. 1D

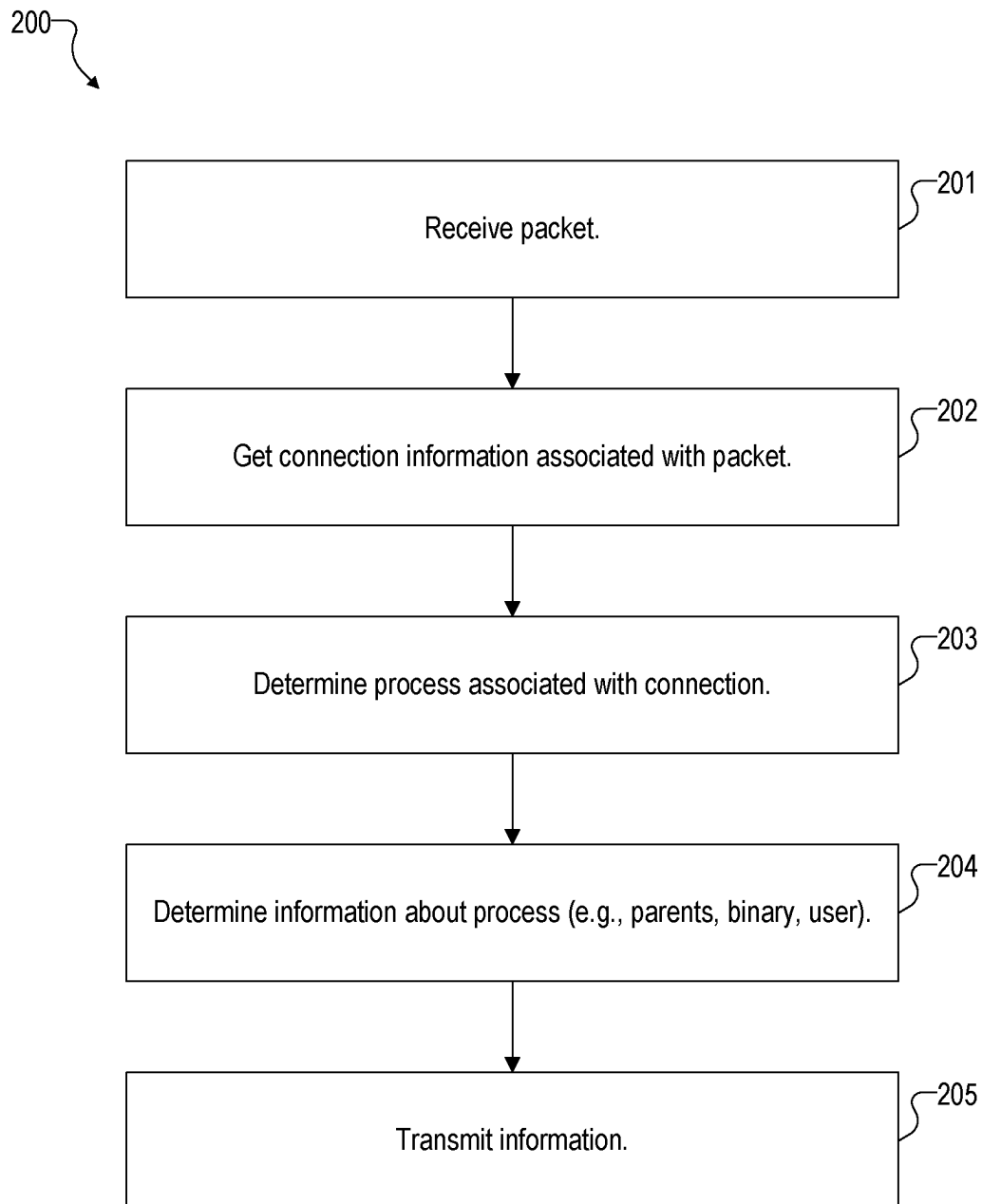


Fig. 2A

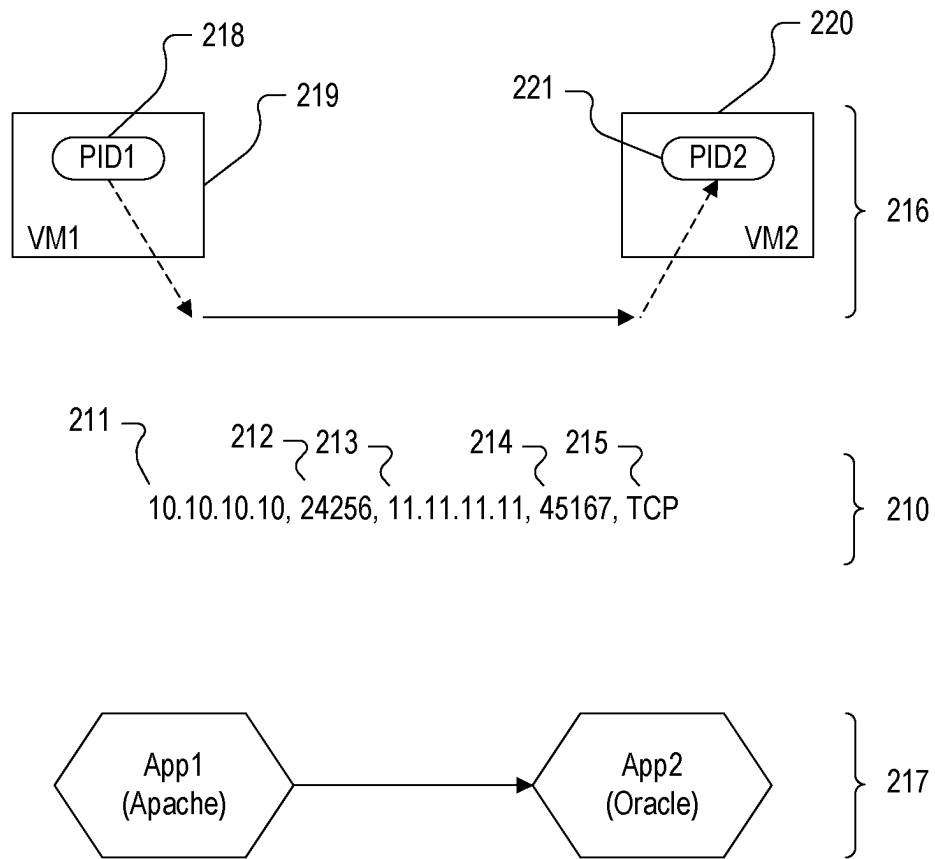


Fig. 2B

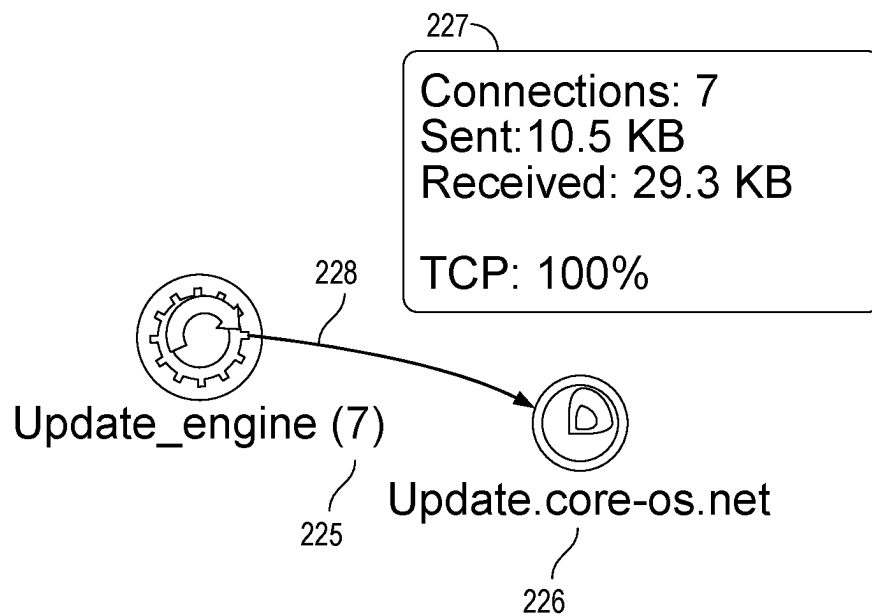


Fig. 2C

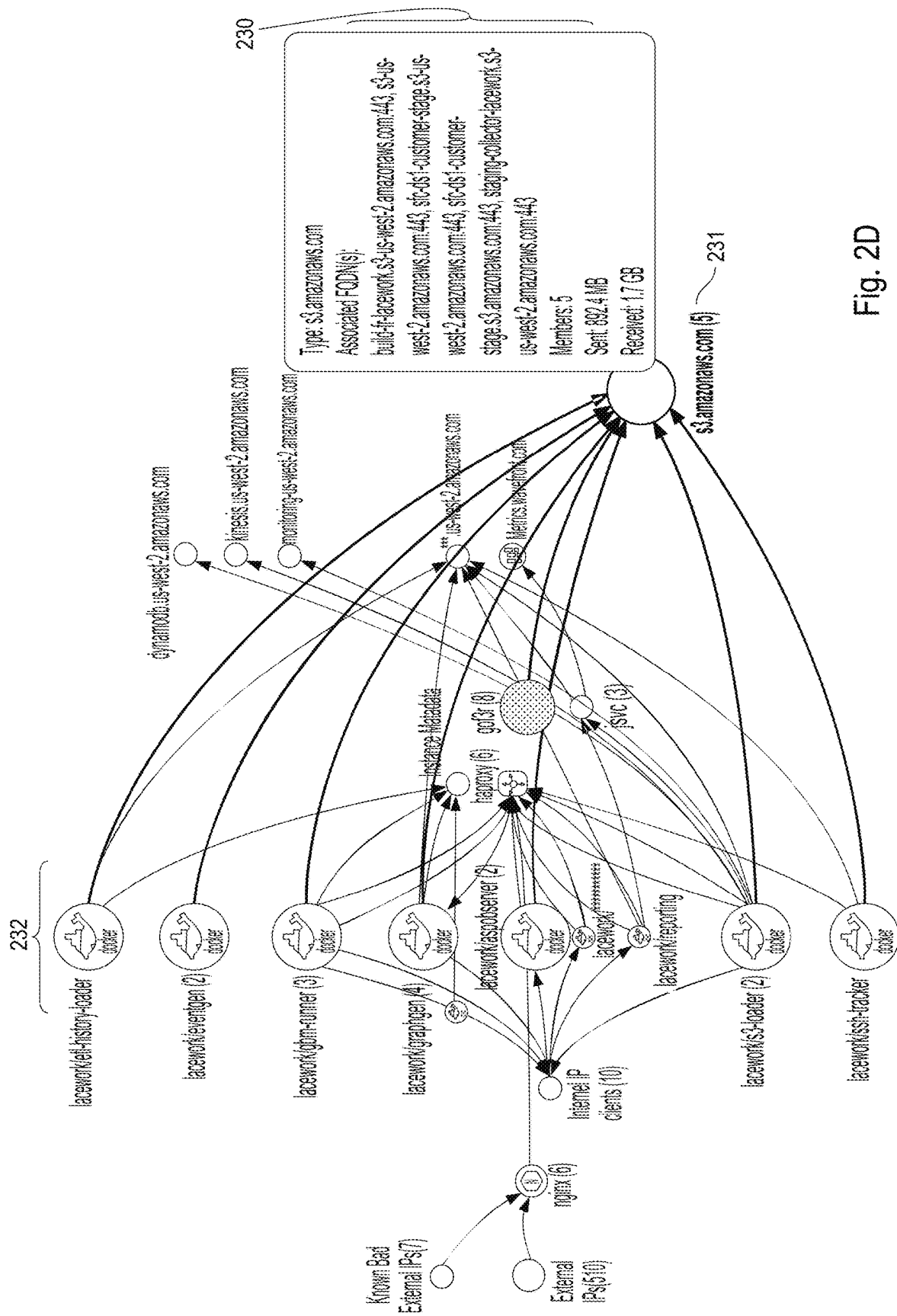
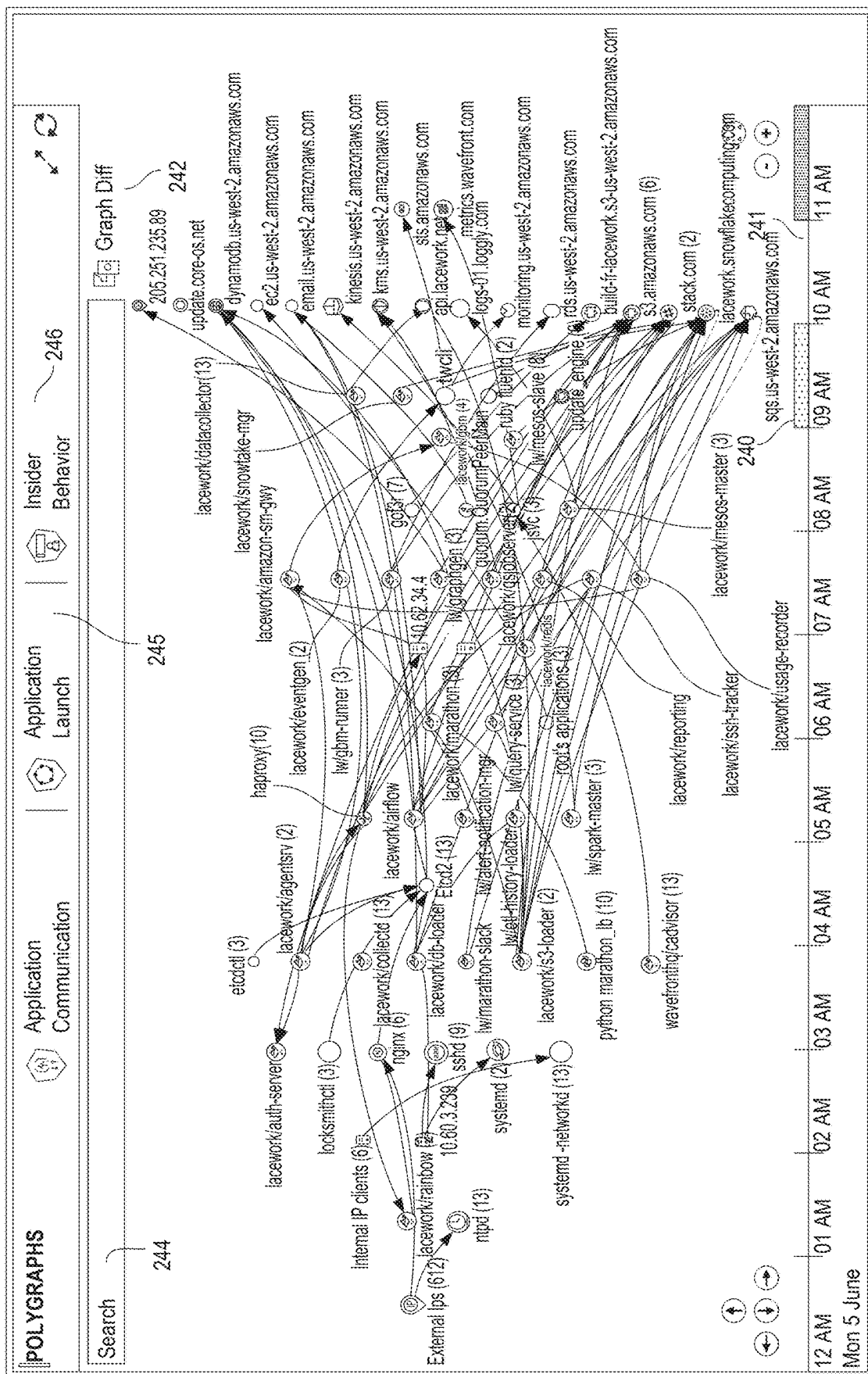
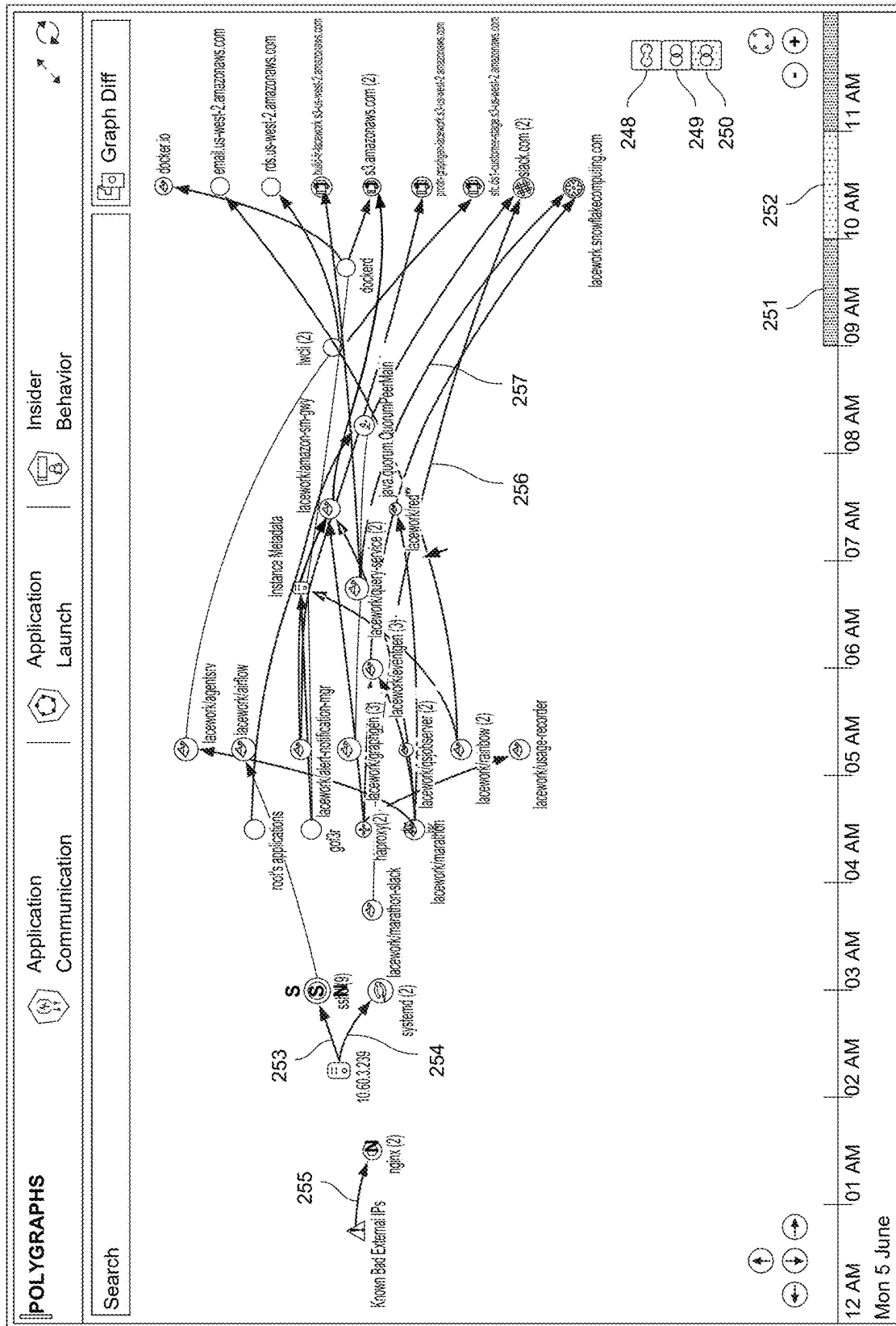


Fig. 2D





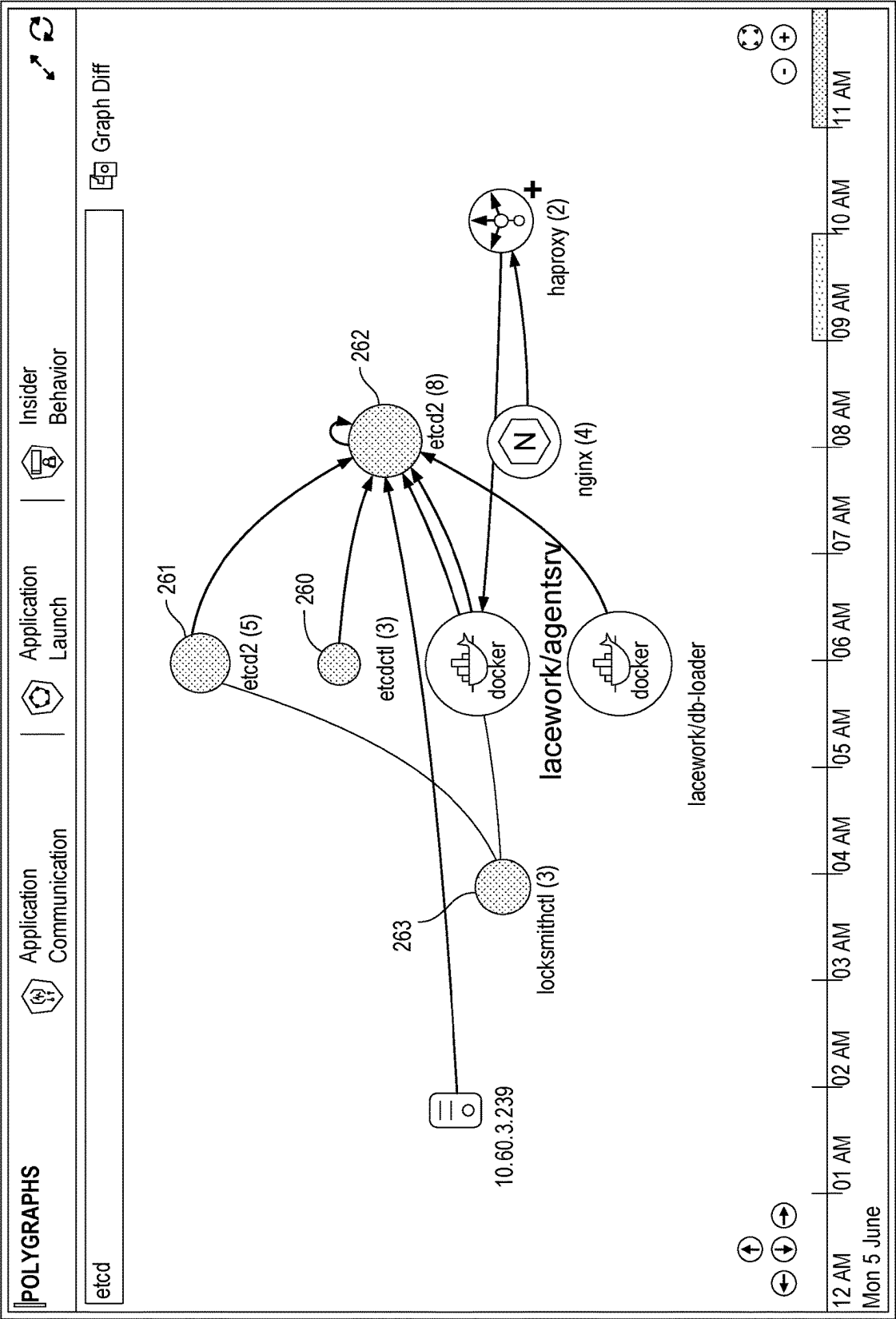
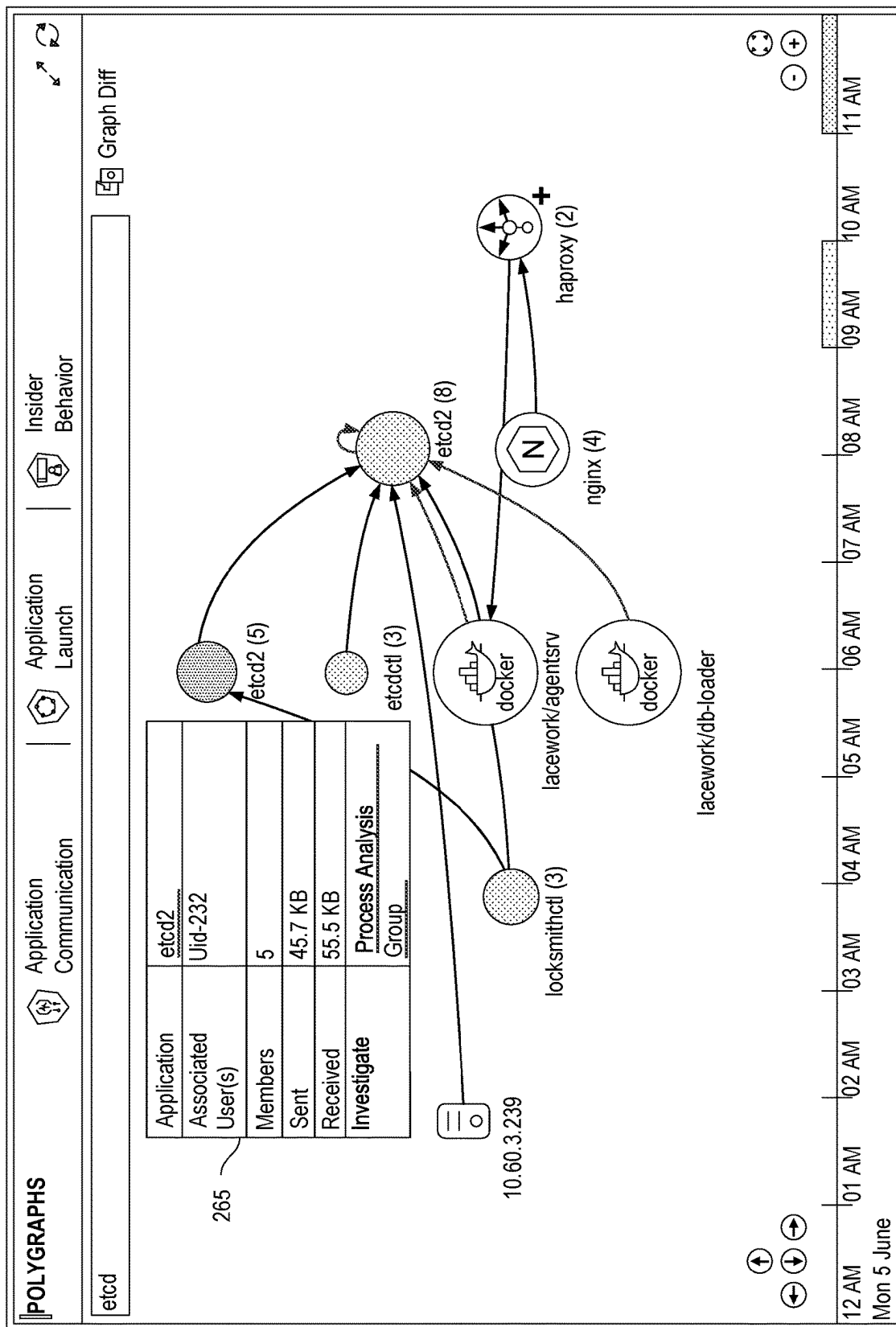


Fig. 2H



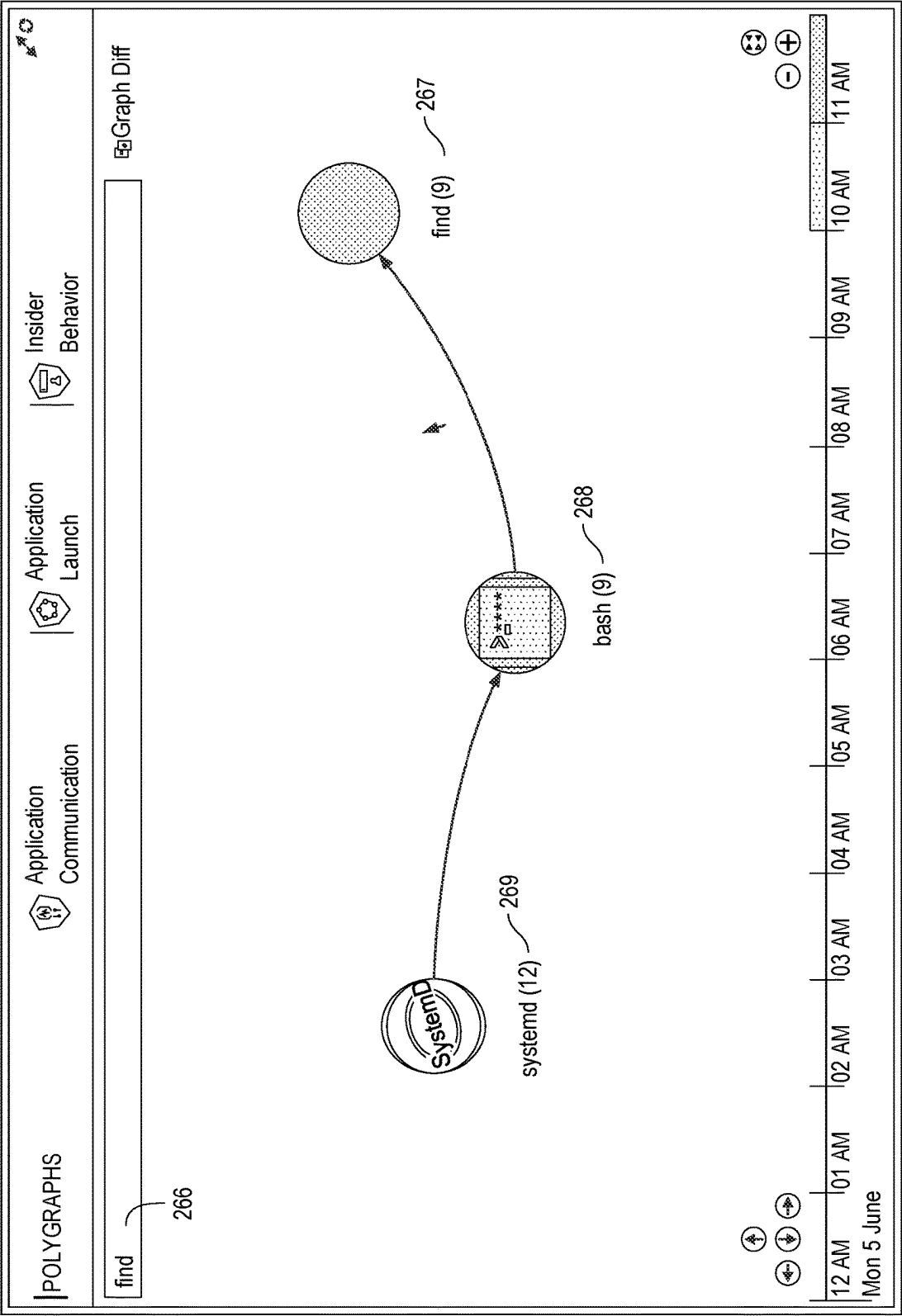


Fig. 2J

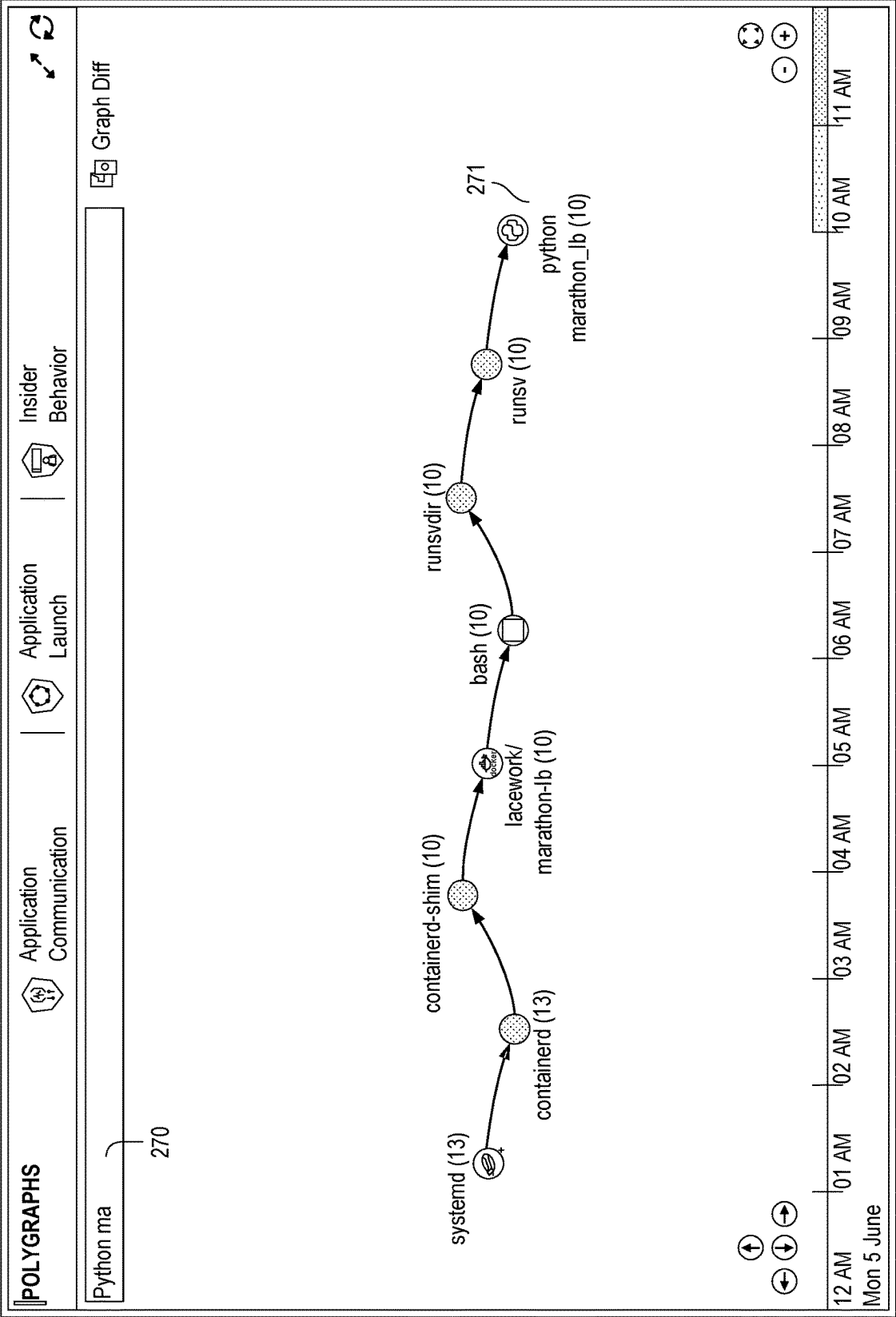


Fig. 2K

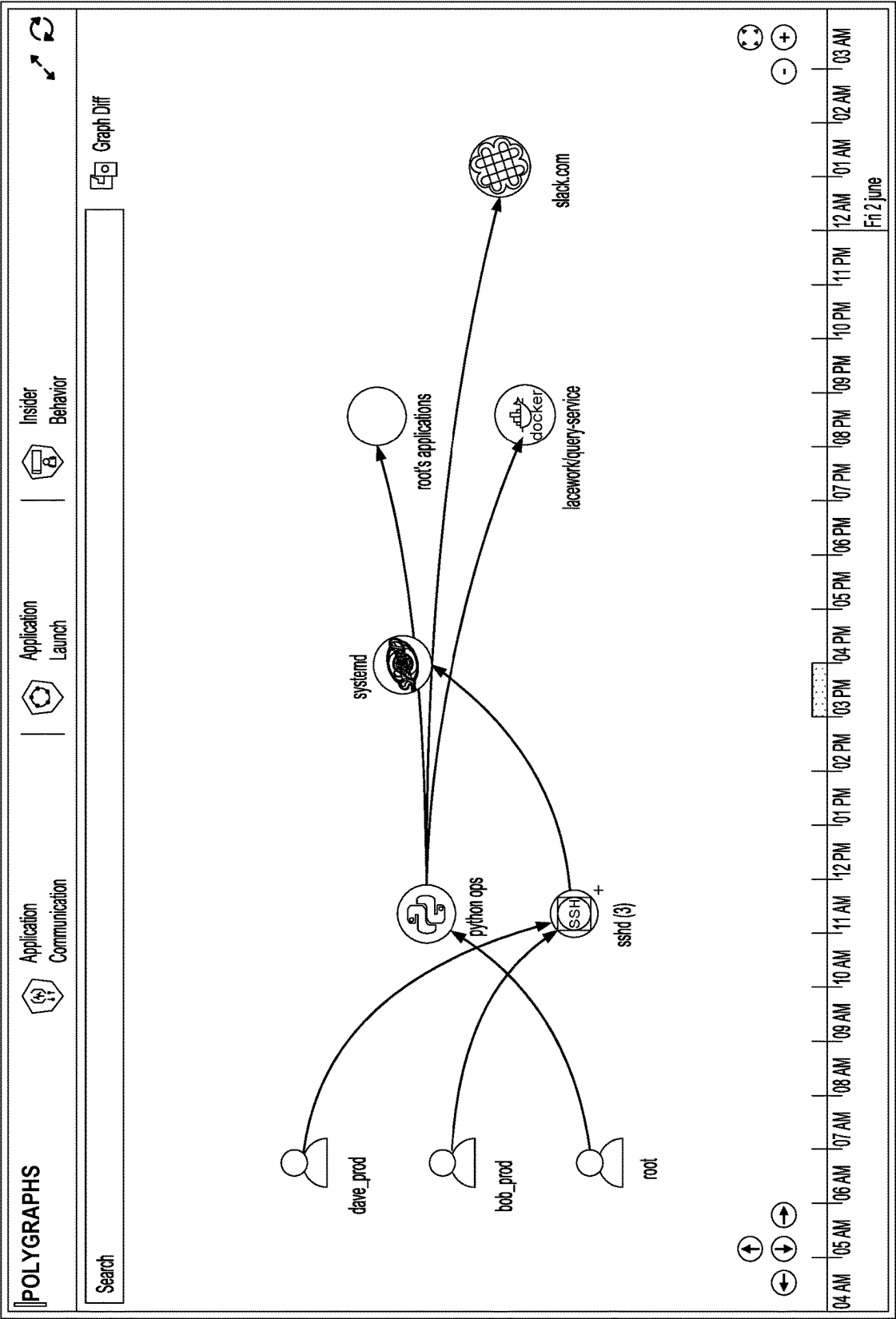


Fig. 2L

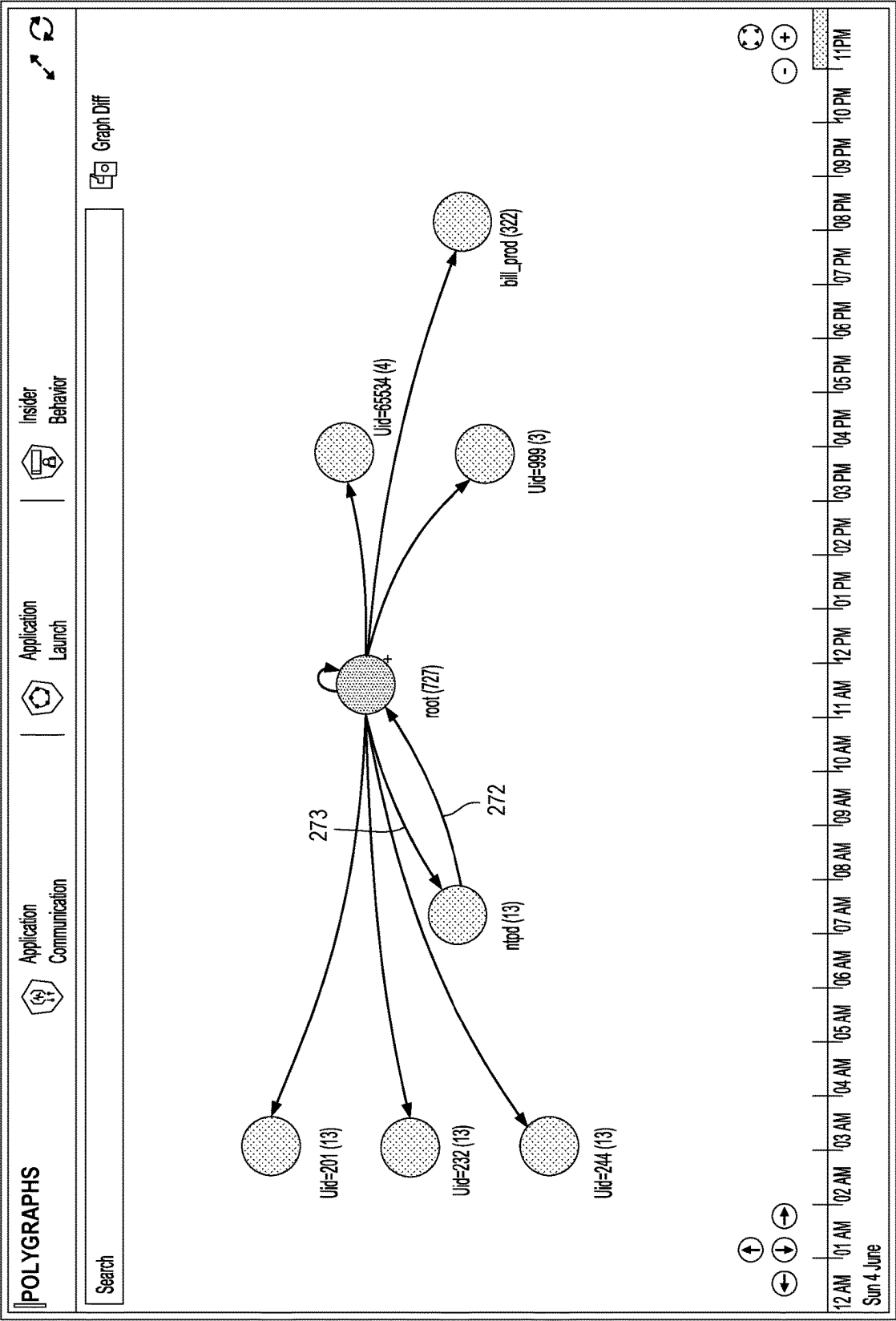


Fig. 2M

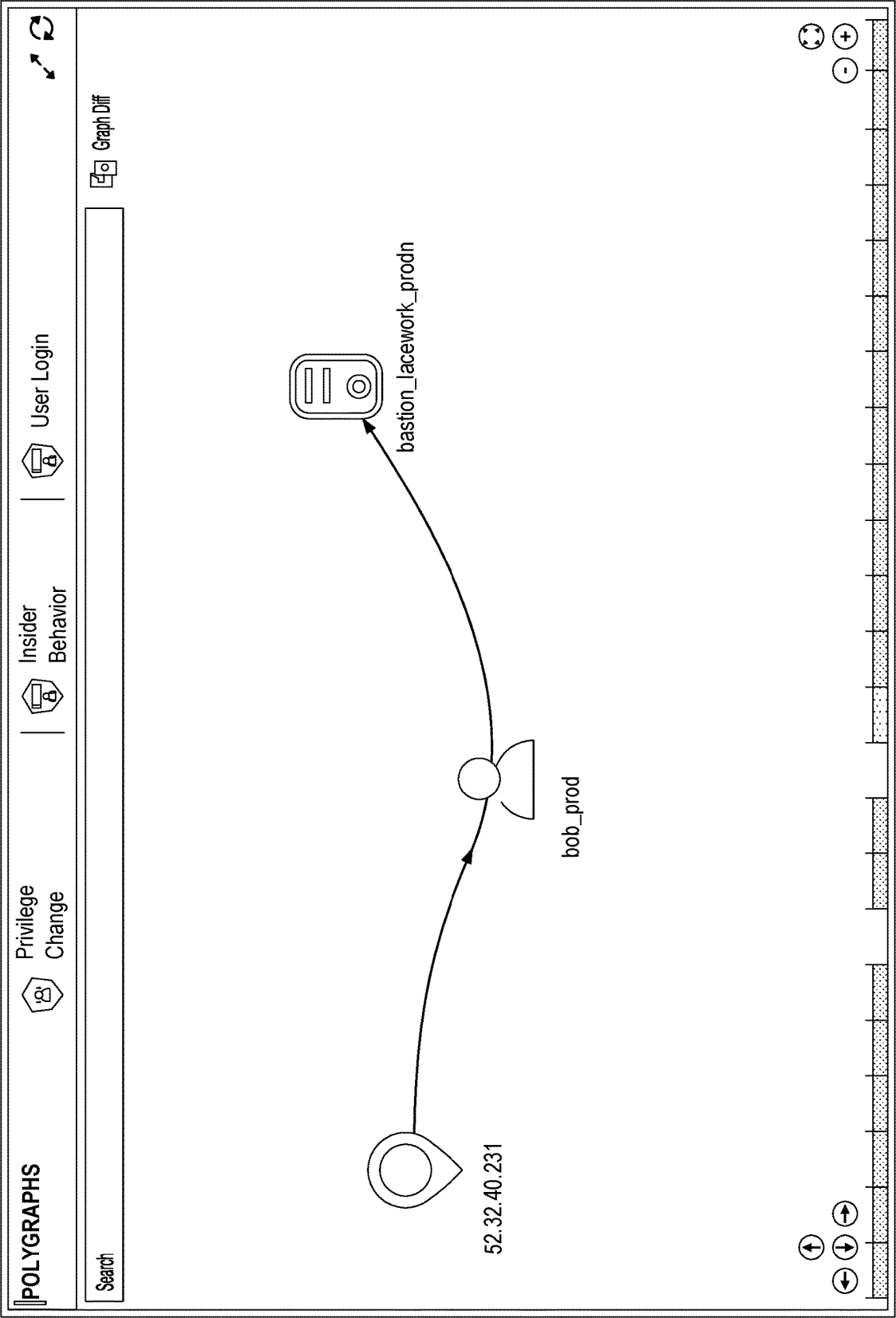


Fig. 2N

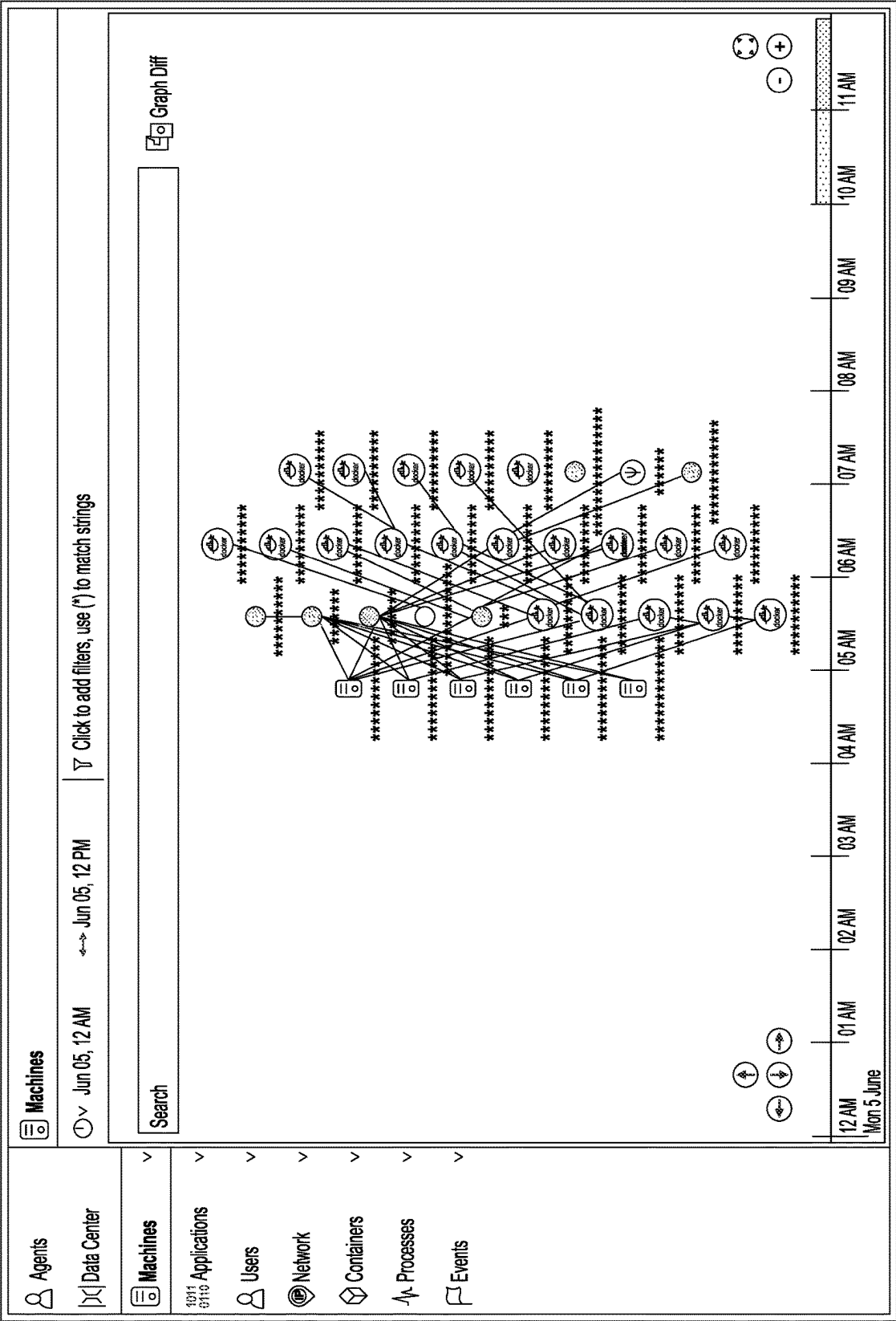


Fig. 20

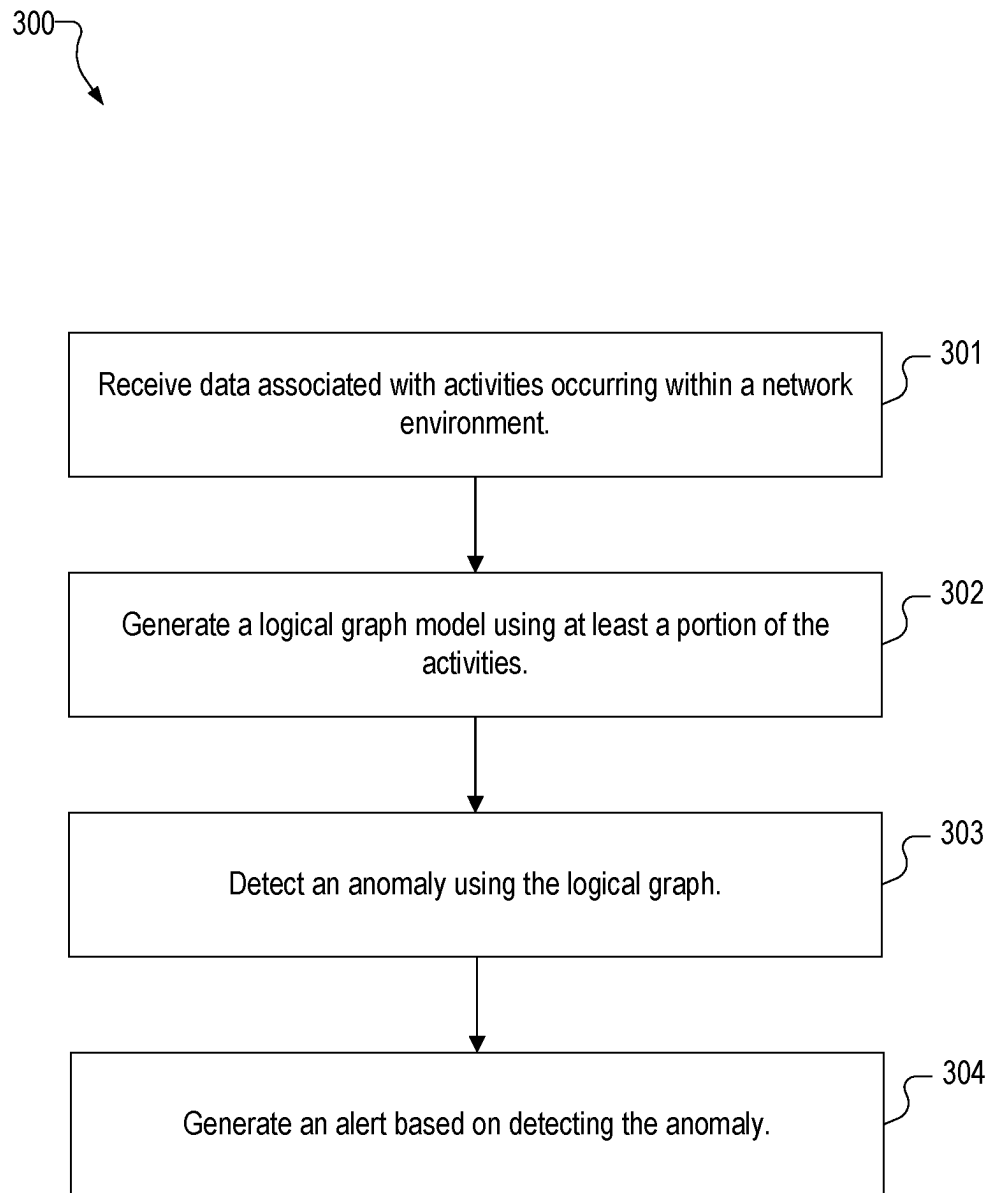
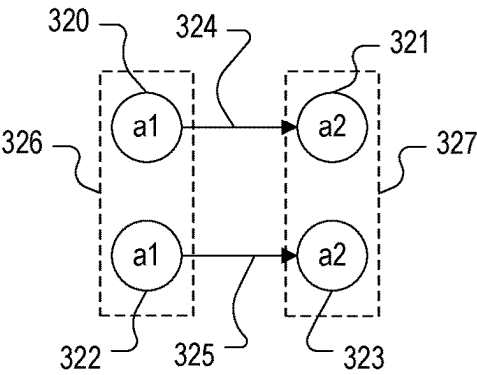
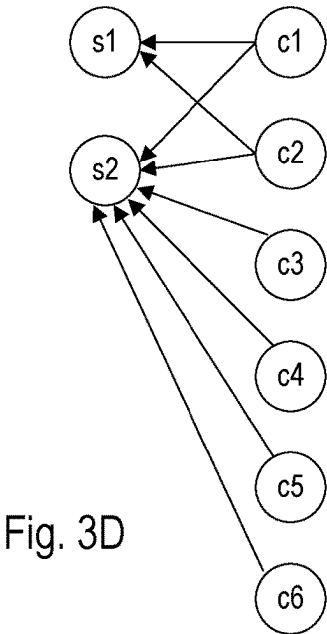
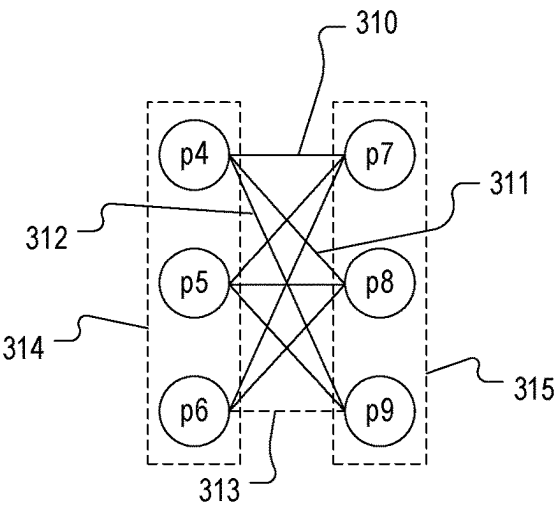
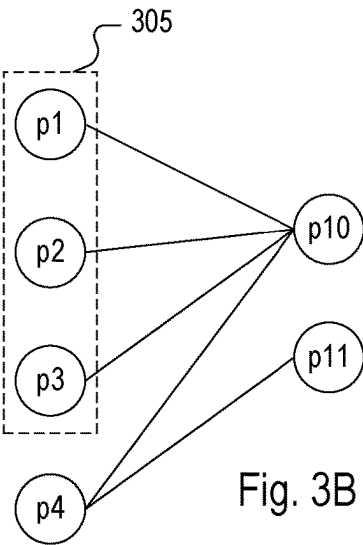


Fig. 3A



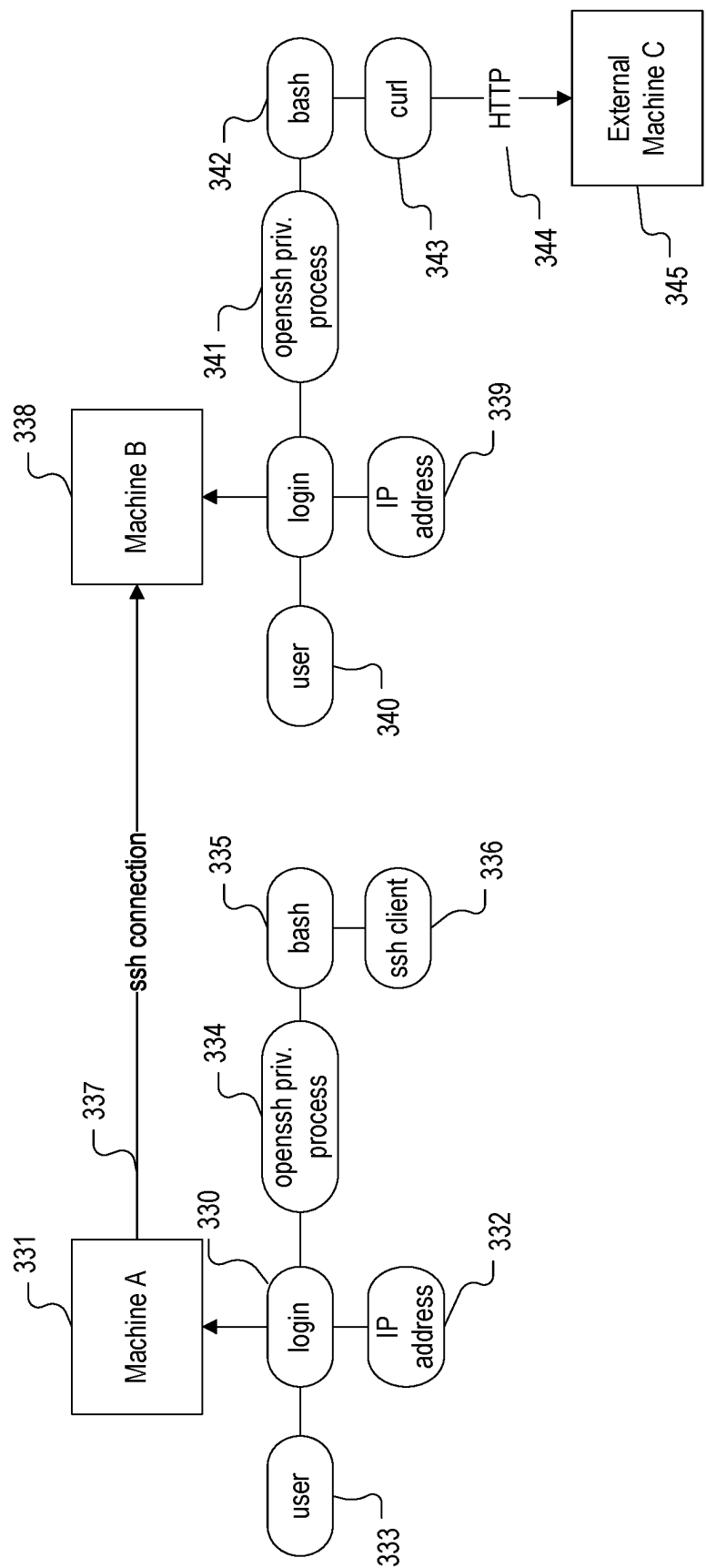


Fig. 3F

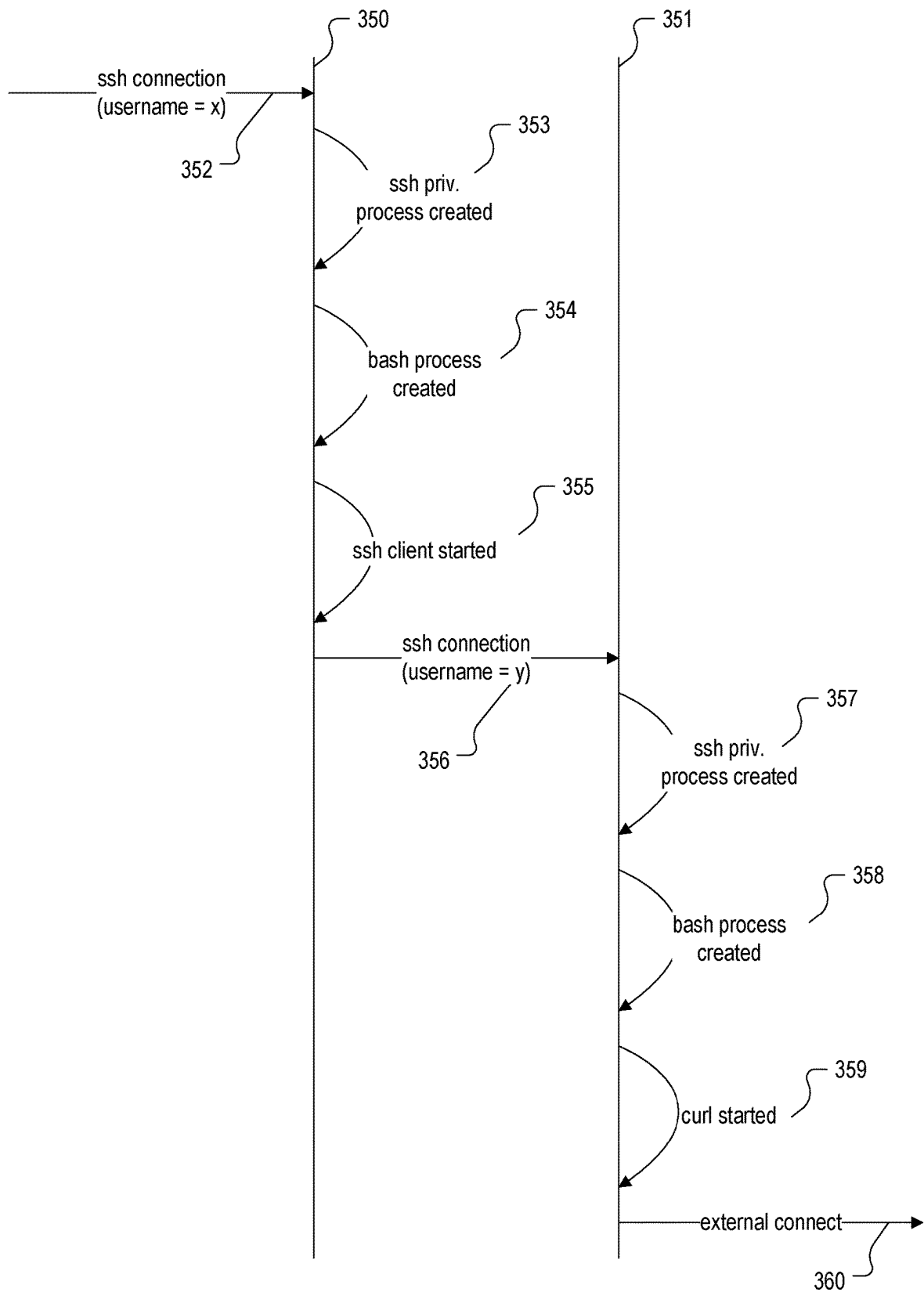


Fig. 3G

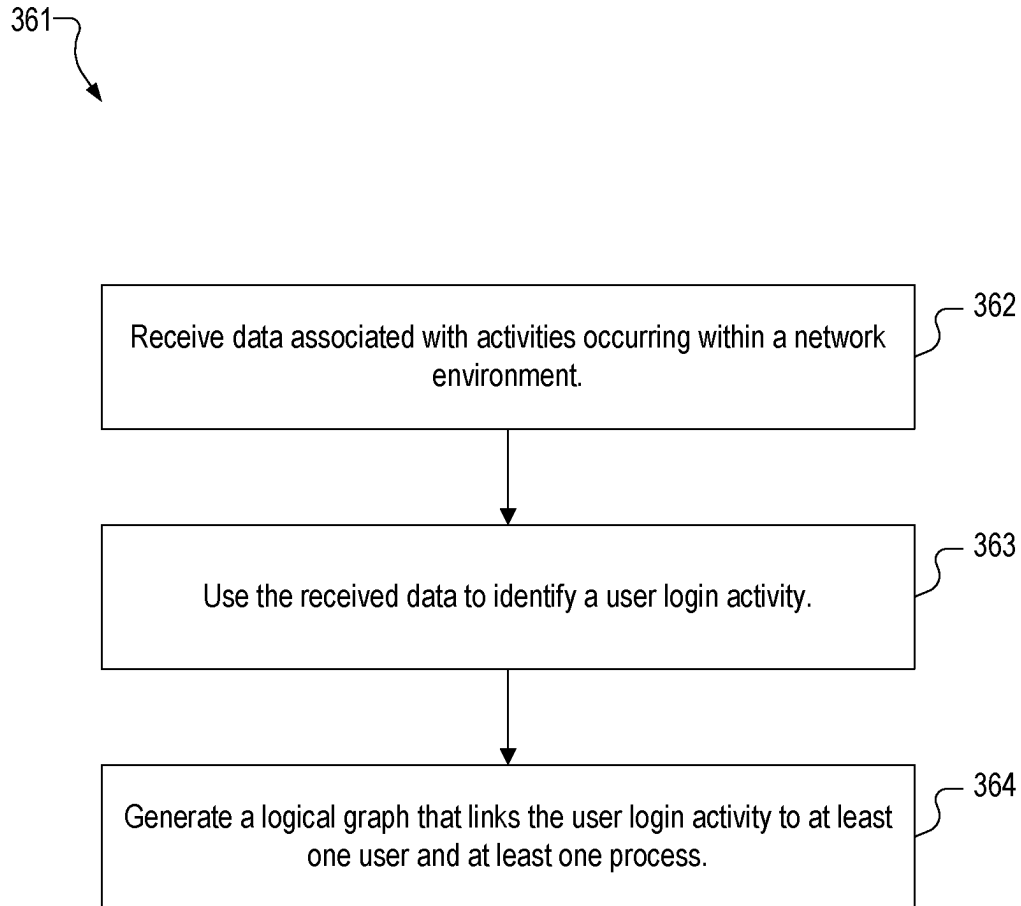


Fig. 3H

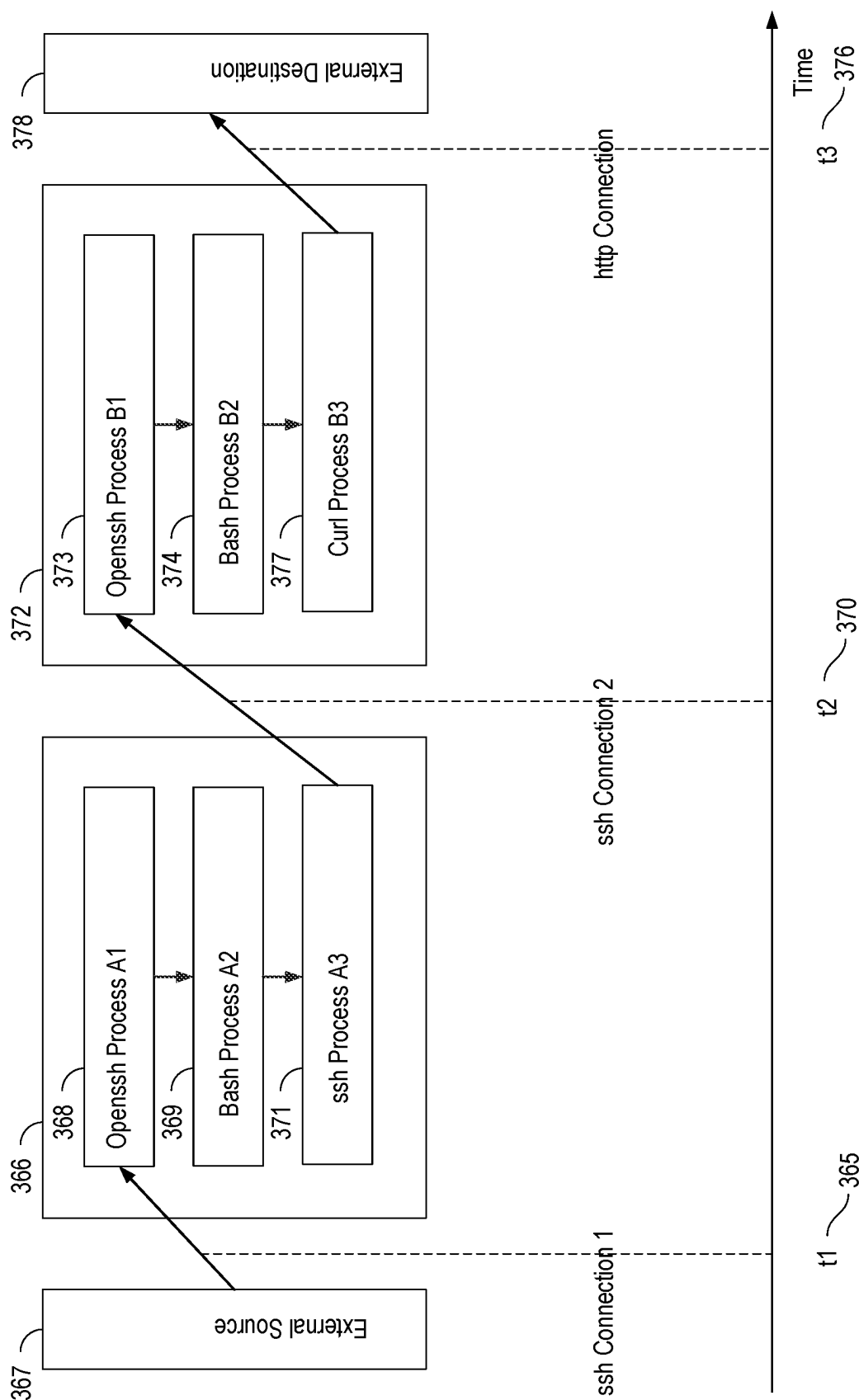


Fig. 3I

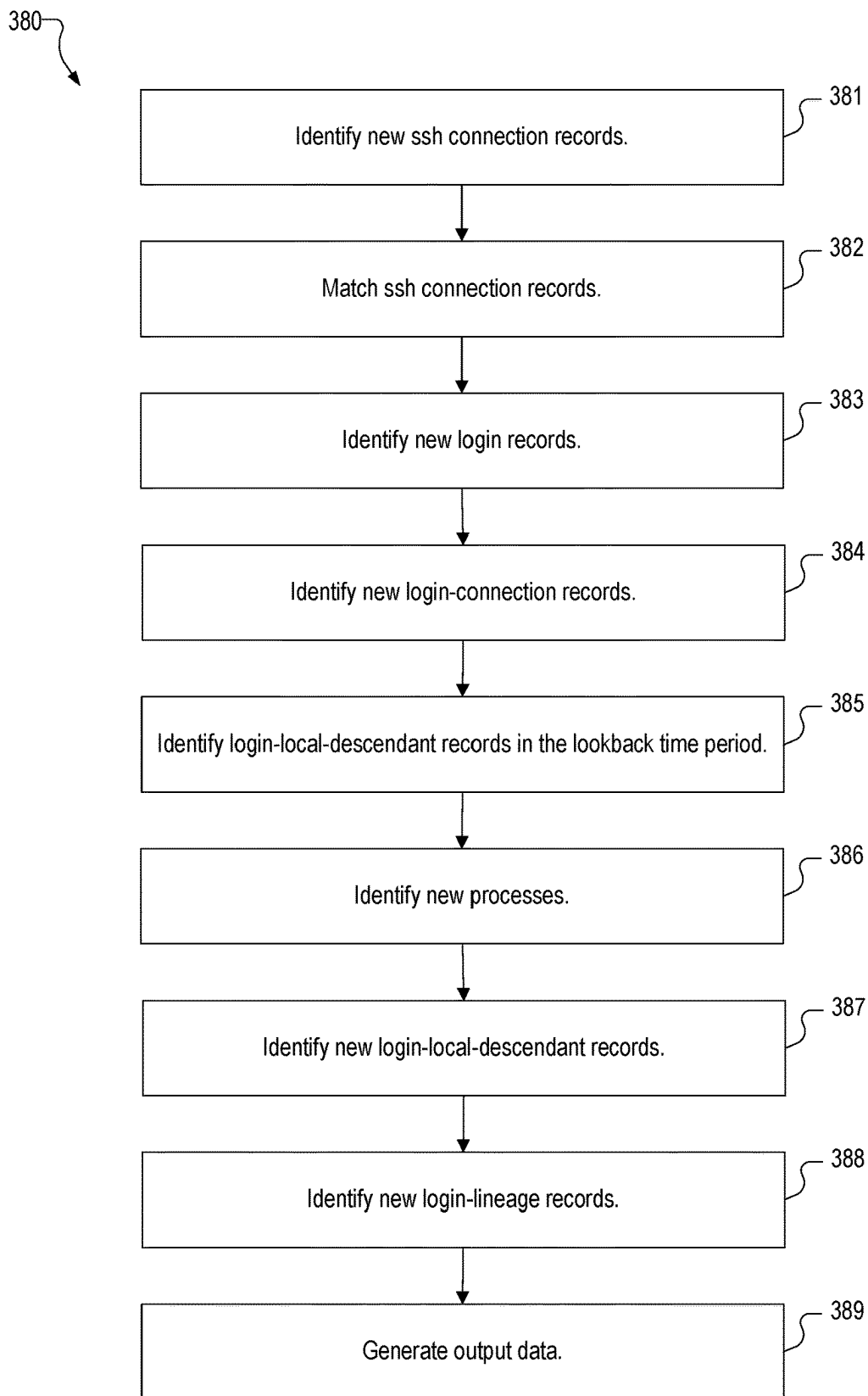


Fig. 3J

MID	start_time	PID_hash	src_IP_addr	src_port	dst_IP_addr	dst_port	prot	dir
A	t1	A1	1.1.1.10	10000	2.2.2.20	22	TCP	Incoming
A	t2	A3	2.2.2.20	10001	2.2.2.21	22	TCP	Outgoing
B	t2	B1	2.2.2.20	10001	2.2.2.21	22	TCP	Incoming

Fig. 3K

src_MID	src_PID_hash	dst_MID	dst_PID_hash	dst_start_time	src_IP_addr	src_port	dst_IP_addr	dst_port
null	null	A	A1	t1	1.1.1.10	10000	2.2.2.20	22
A	A3	B	B1	t2	2.2.2.20	10001	2.2.2.21	22

Fig. 3L

MID	login_time	sshd_PID_hash
A	t1	A1
B	t2	B1

Fig. 3M

MID	sshd_PID_hash	login_time	login_username	src_IP_addr	src_port	dst_IP_addr	dst_port
A	A1	t1	X	1.1.1.10	10000	2.2.2.20	22
B	B1	t2	Y	2.2.2.20	10001	2.2.2.21	22

Fig. 3N

MID	start_time	PID_hash	exe_path	parent_PID_hash
A	t1	A1	/usr/sbin/sshd	A0
A	t1	A2	/bin/bash	A1
A	t2	A3	/usr/bin/ssh	A2
B	t2	B1	/usr/sbin/sshd	B0
B	t2	B2	/bin/bash	B1
B	t3	B3	/usr/bin/curl	B2

Fig. 3O

MID	sshd_PID_hash	PID_hash
A	A1	A1
A	A1	A2
A	A1	A3
B	B1	B1
B	B1	B2
B	B1	B3

Fig. 3P

parent_MID	parent_sshd_PID_hash	child_MID	origin_sshd_PID_hash
A	A1	B	B1

Fig. 3Q

MID	sshd_PID_hash	parent_MID	parent_sshd_PID_hash	origin_MID	origin_sshd_PID_hash
A	A1	null	null	A	A1
B	B1	A	A1	A	A1

Fig. 3R

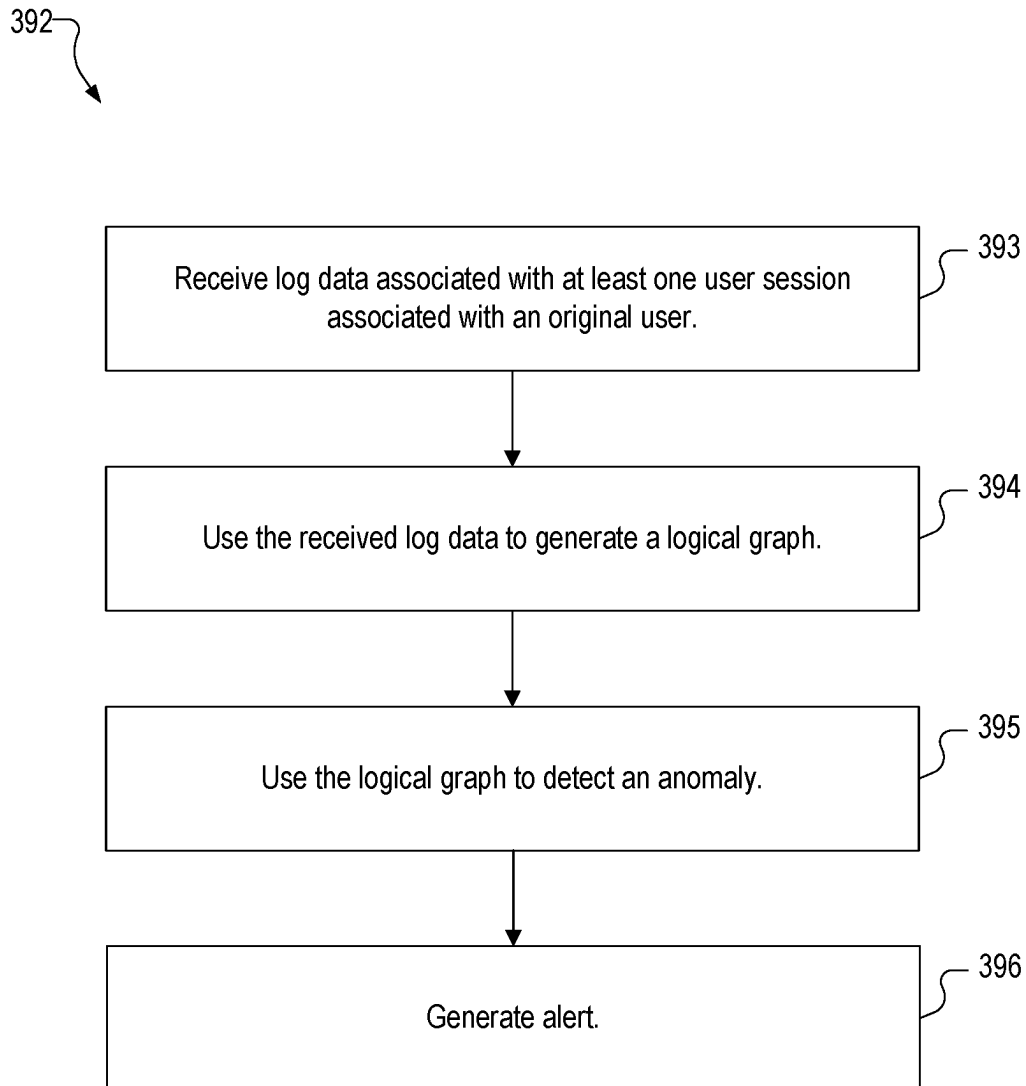


Fig. 3S

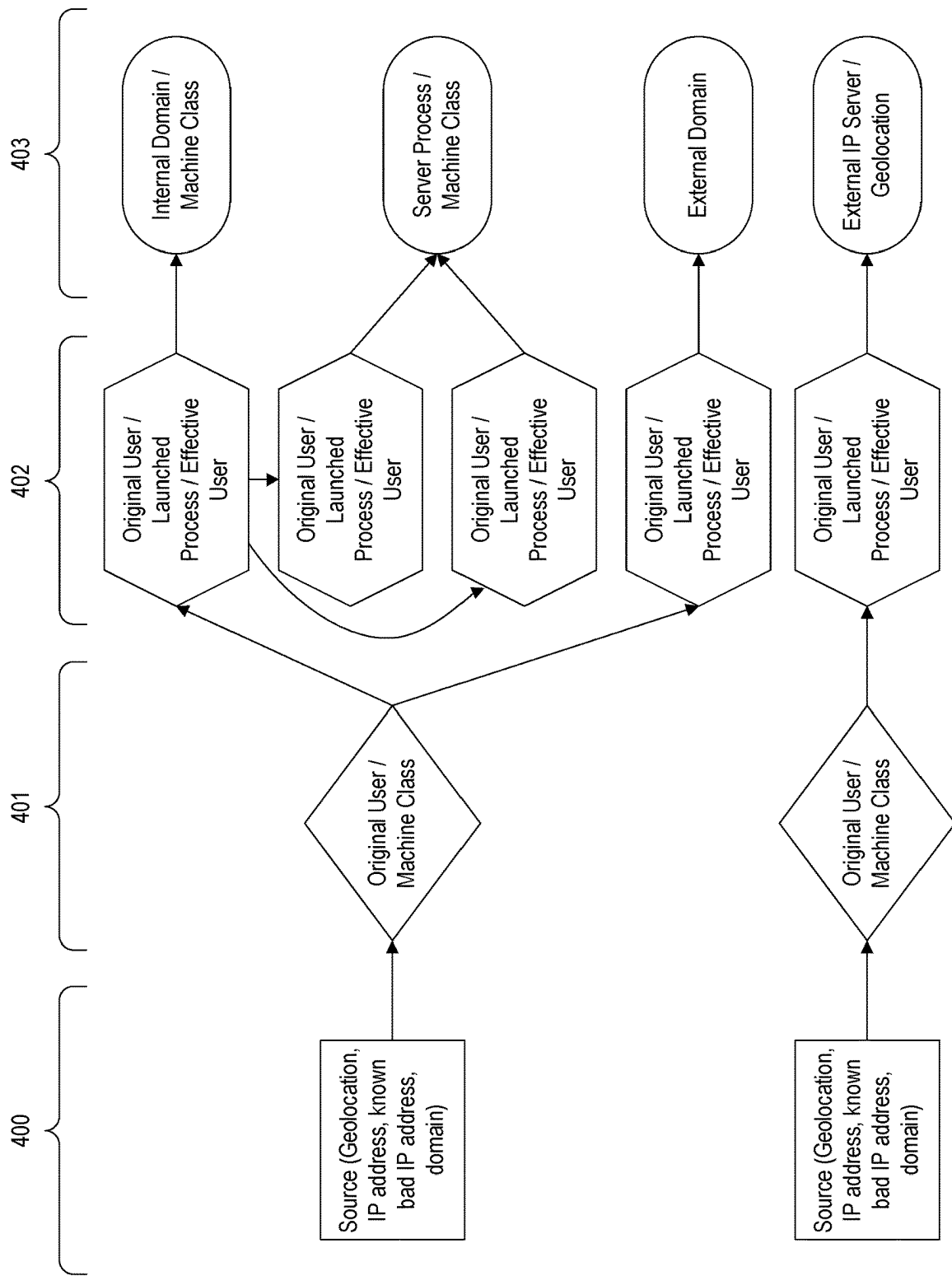


Fig. 4A

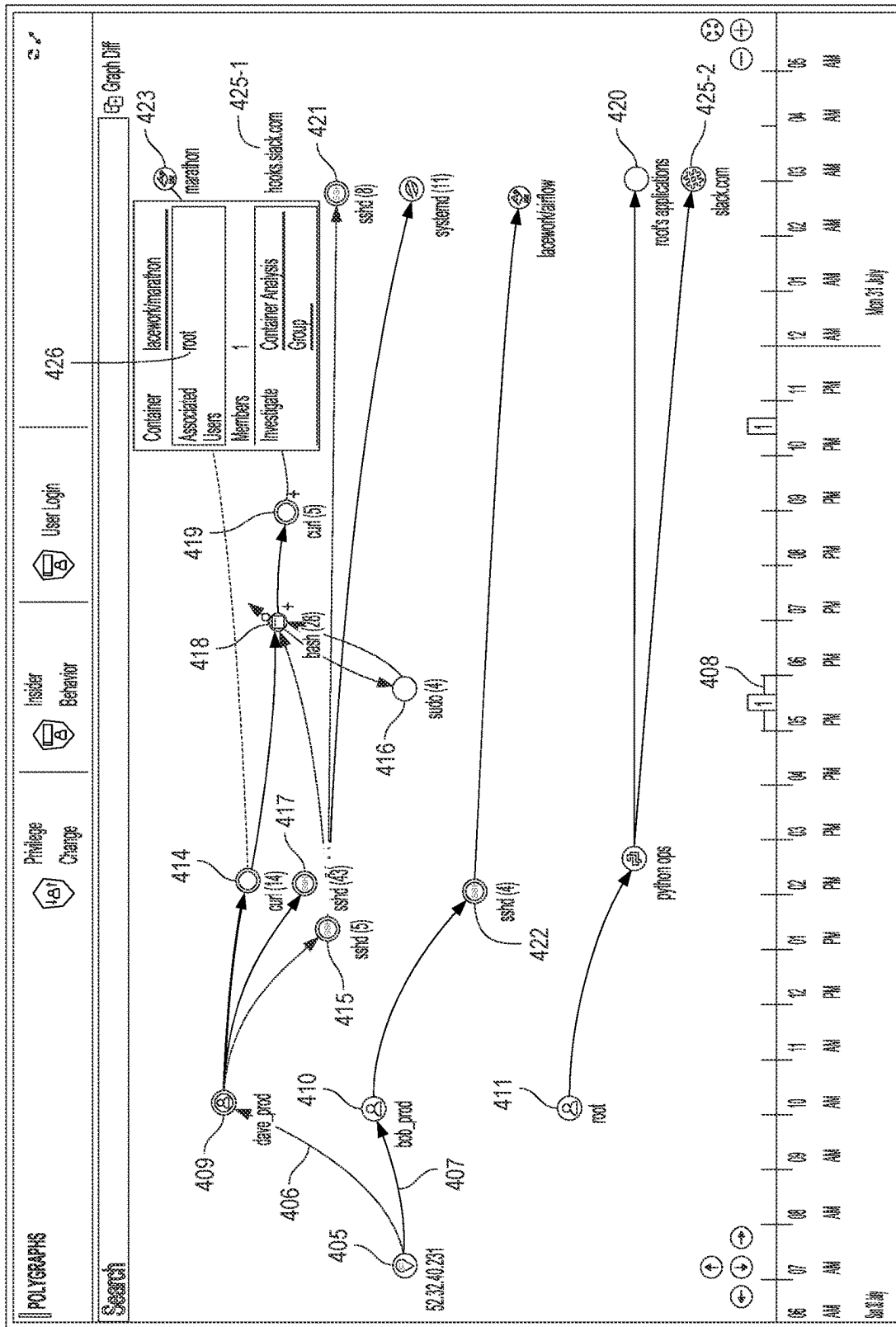


Fig. 4B

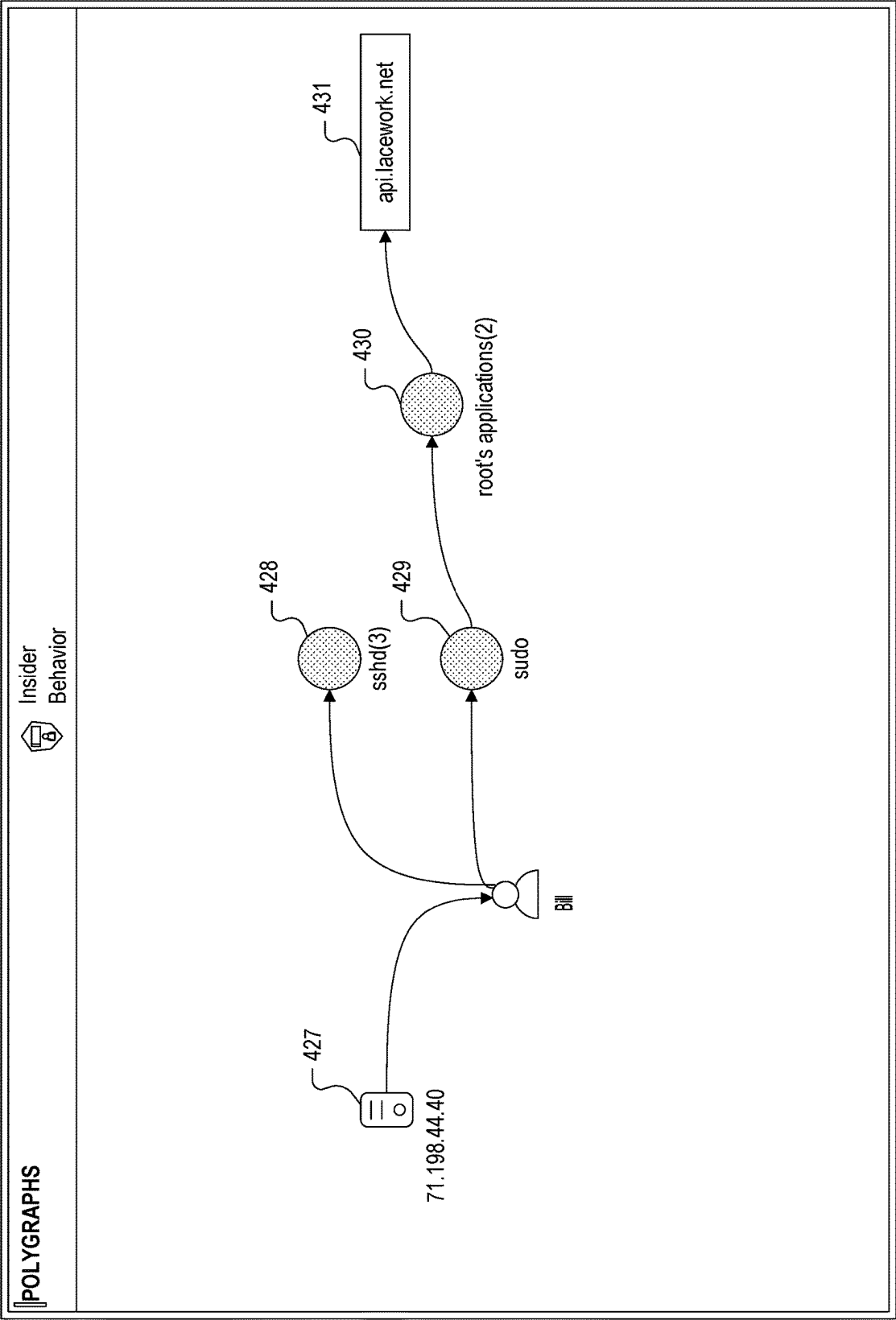


Fig. 4C

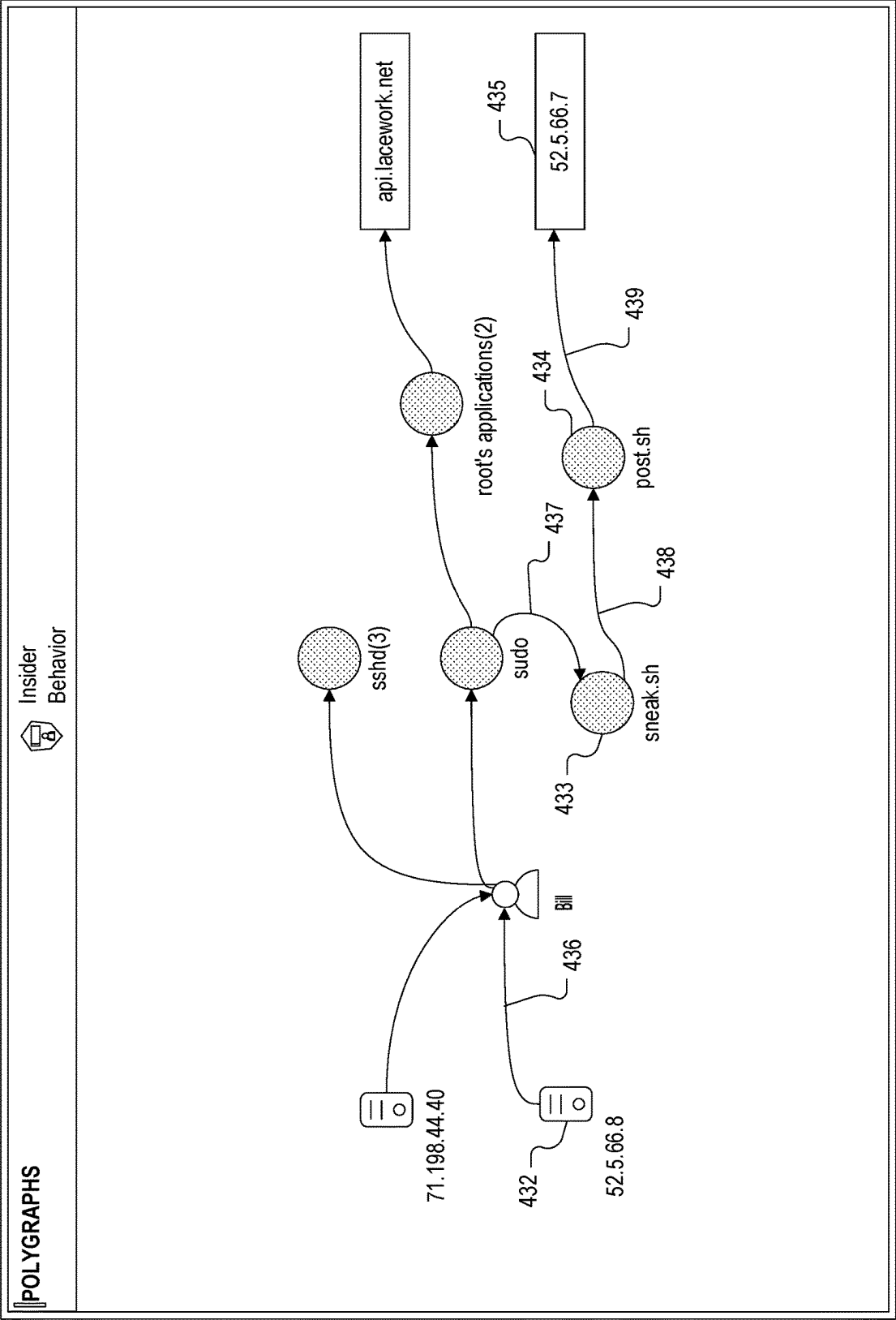


Fig. 4D

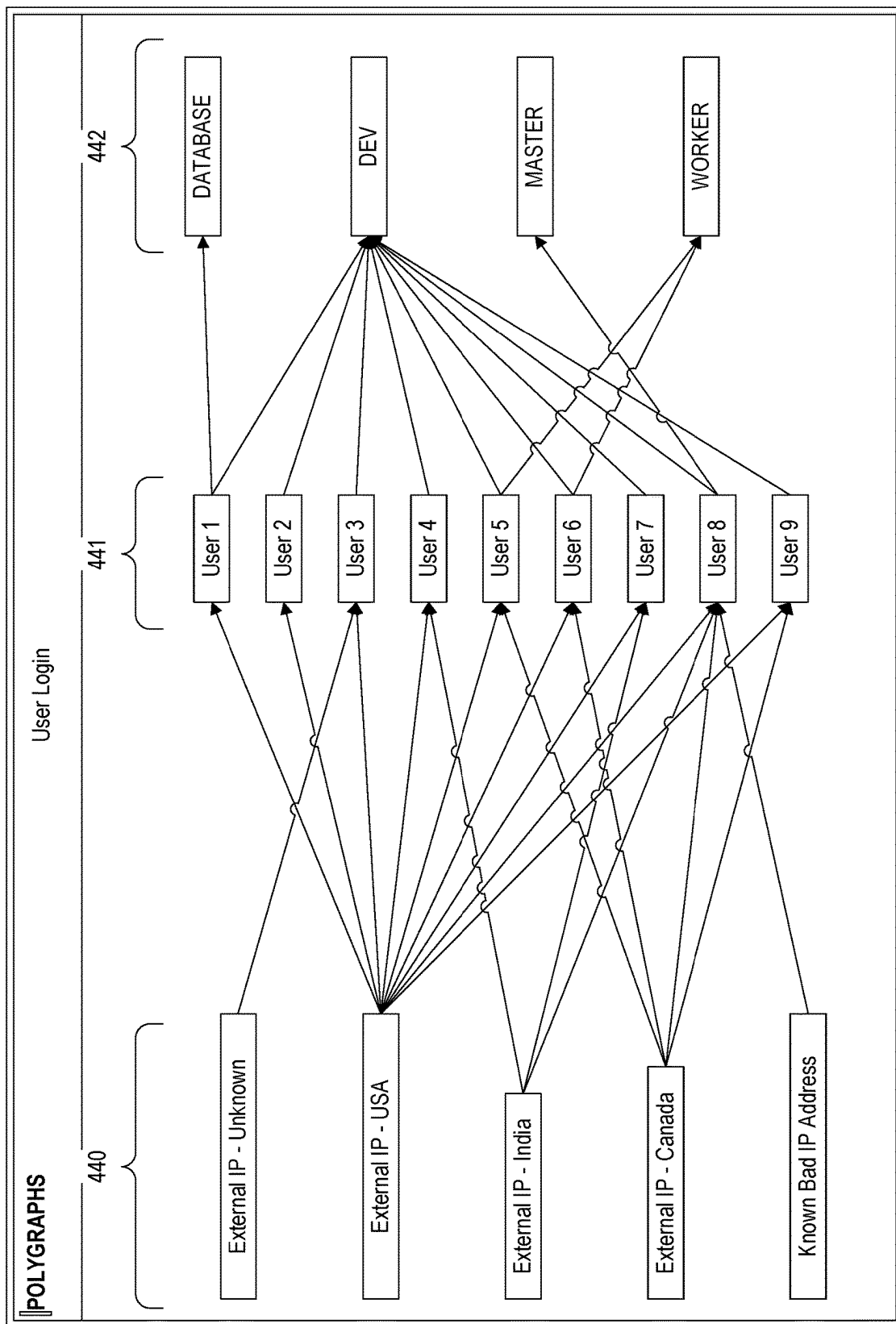


Fig. 4E

443

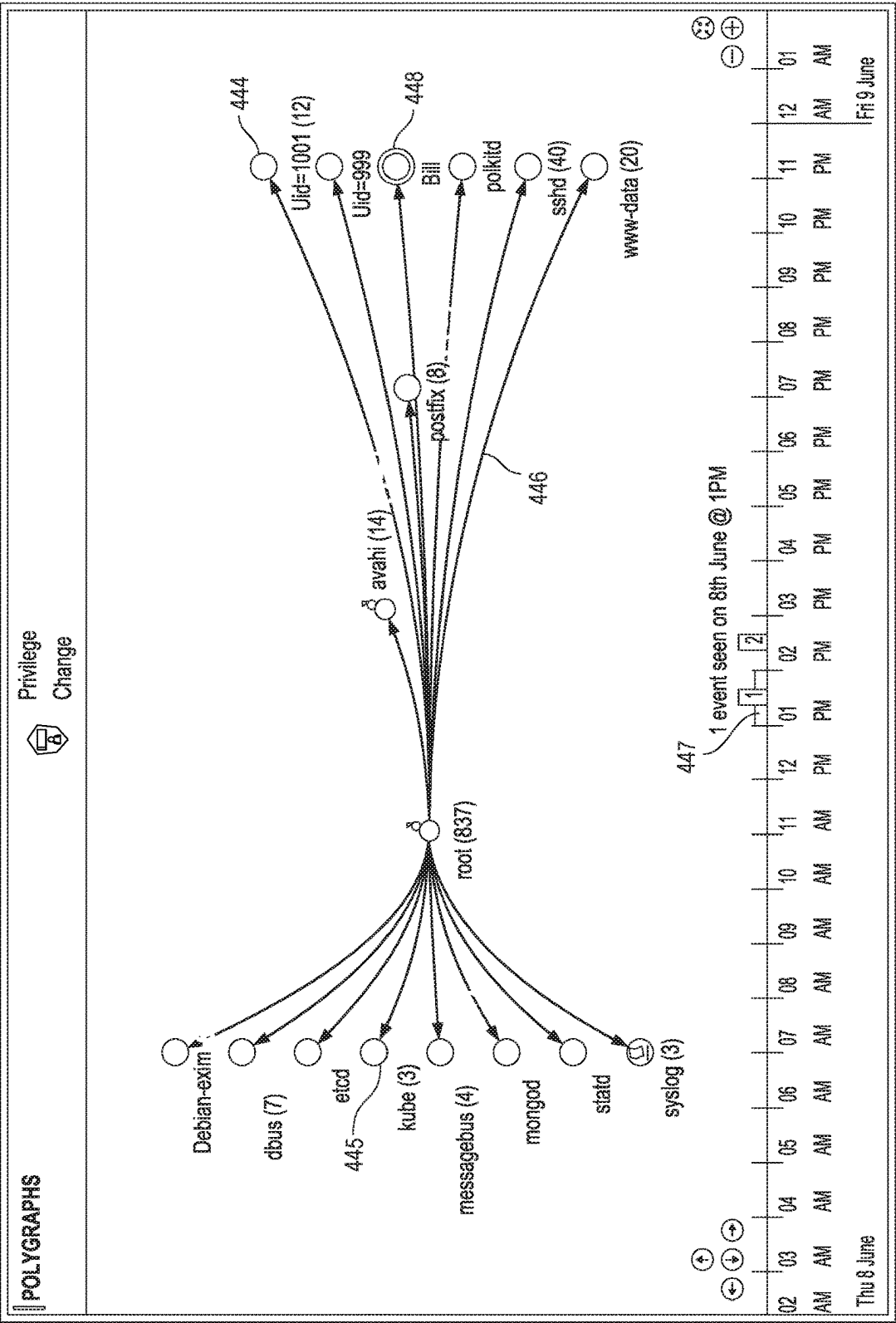


Fig. 4F

450

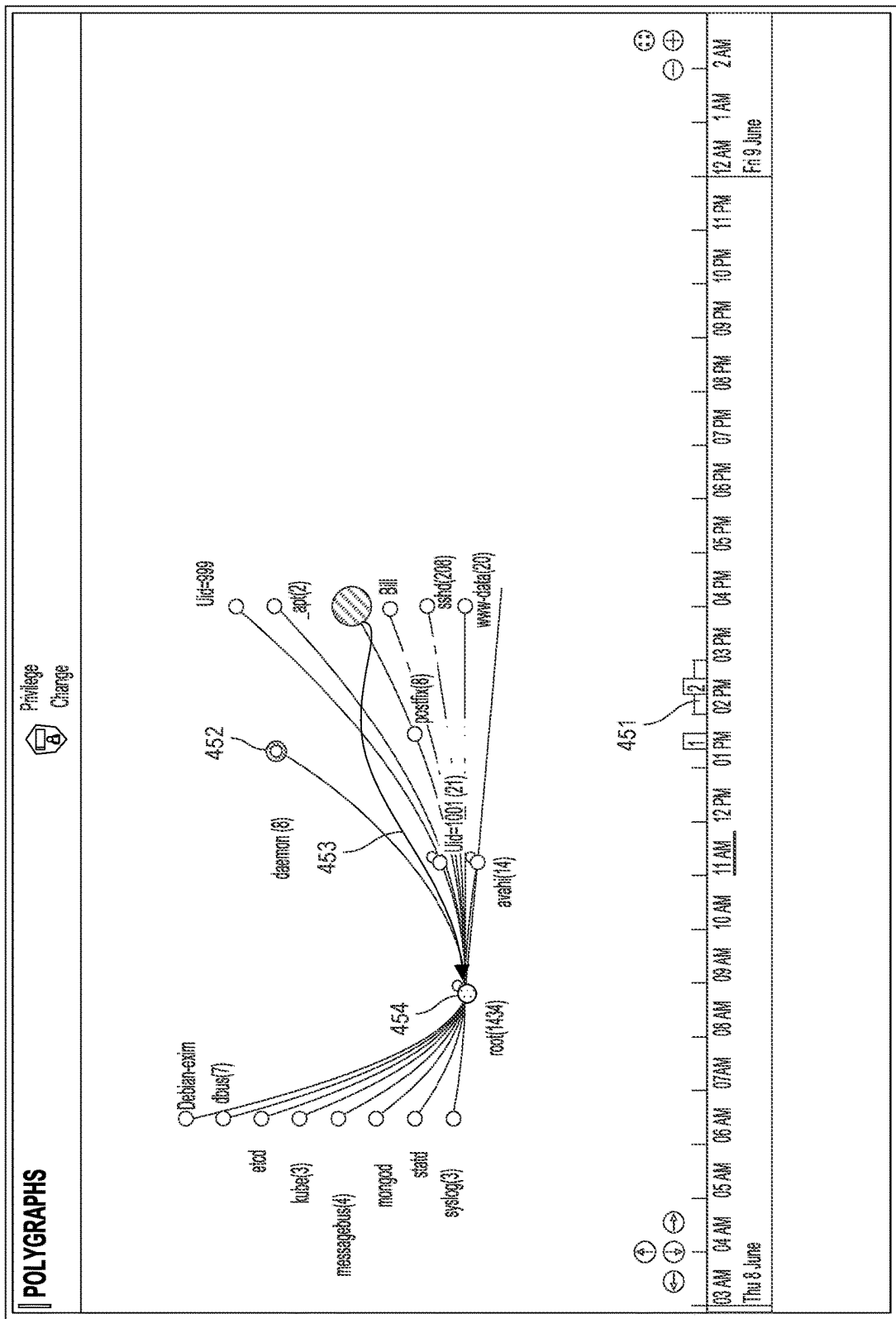
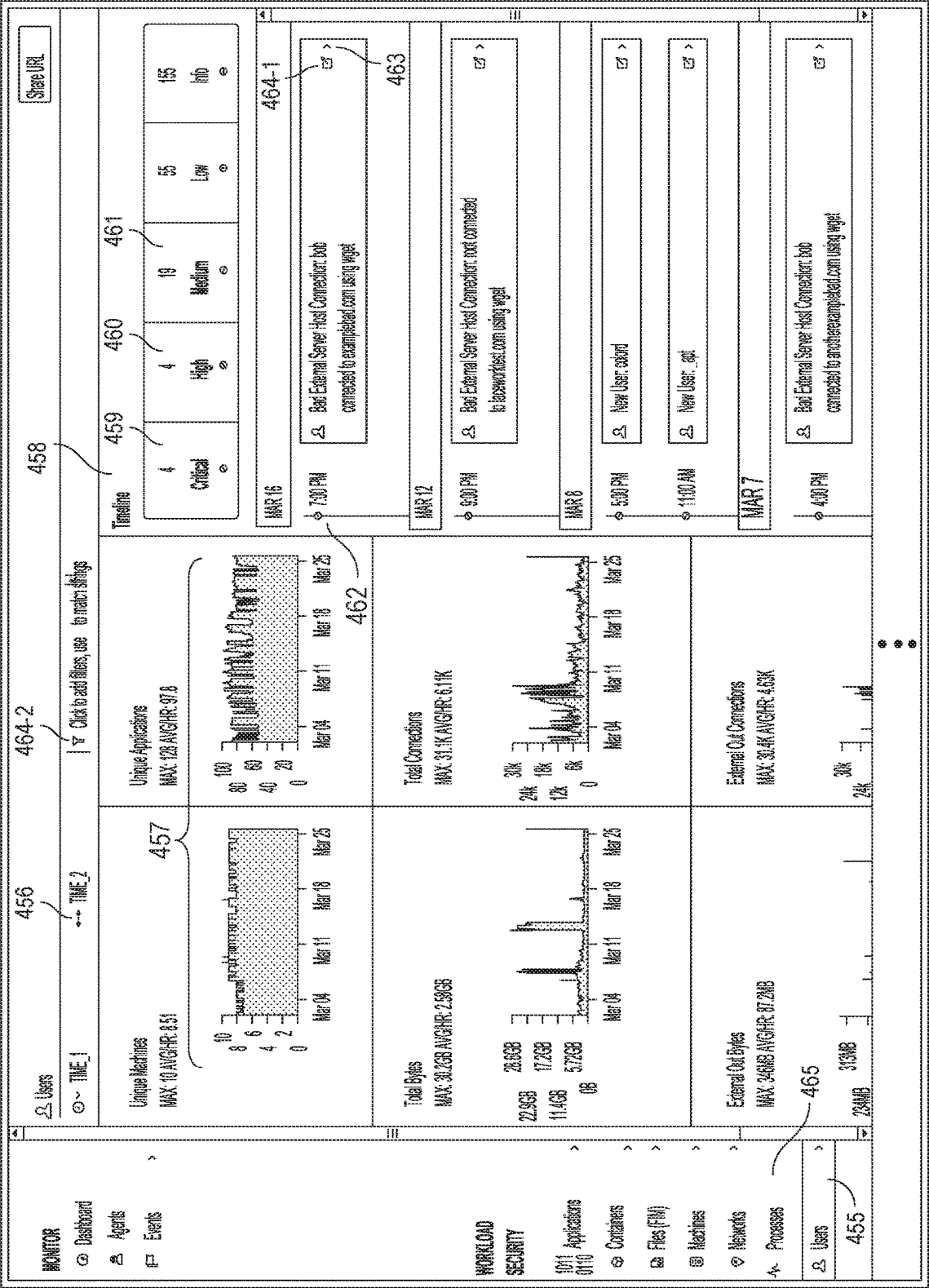


Fig. 4G

Fig 4H



466

Agents

Events

9291

468

WORKLOAD

SECURITY

Applications

Containers

Files (FIM)

Machines

Networks

Processes

Users

Event: 9291

TIME_1

TIME_2

Click to add filters. Use * to match strings

472

467

470

466

Share URL

Related Events Timeline

Showing events from Mar 09 12:AM to Mar 23 11 PM

Compliance

Application

CloudTrail

File

Machine

User

12

Critical

0

0

1

Medium

0

Low

0

Info

Event Summary

469

Title

Description

First Seen Time

3/16 7PM

Event Time Range

3/16 7PM - 3/16 8 PM

Severity

1

Event ID

9291

Event Type

NewExternalServerBadDNSConn

Description

External connection made to known bad host

475

examplebad.com

at port

80

from application

Investigation

473

'wget' running on host

dev1-lacework.internal

as user

harish

Question

Answer

Did the Hash(SHA256) of the application involved in event change in...

No

Was the DNS involved in the event seen in any other event in last 30...

No

Is the application involved in the event from a Packaged Software?

No

Did the application transfer more than median amount of data comp...

Yes

Is the application involved in the event running as Root?

No

Fig. 4I

477

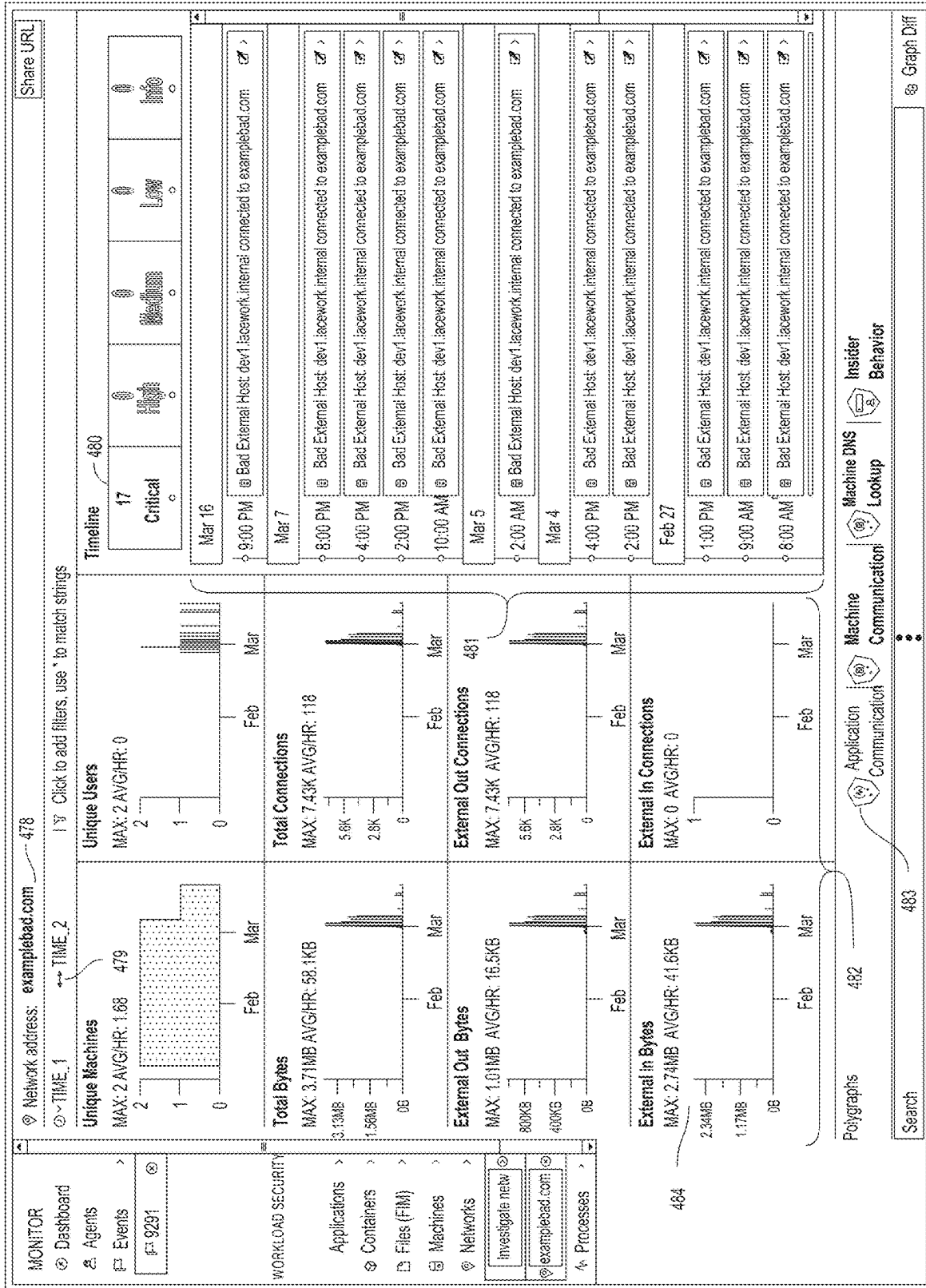


Fig. 4J

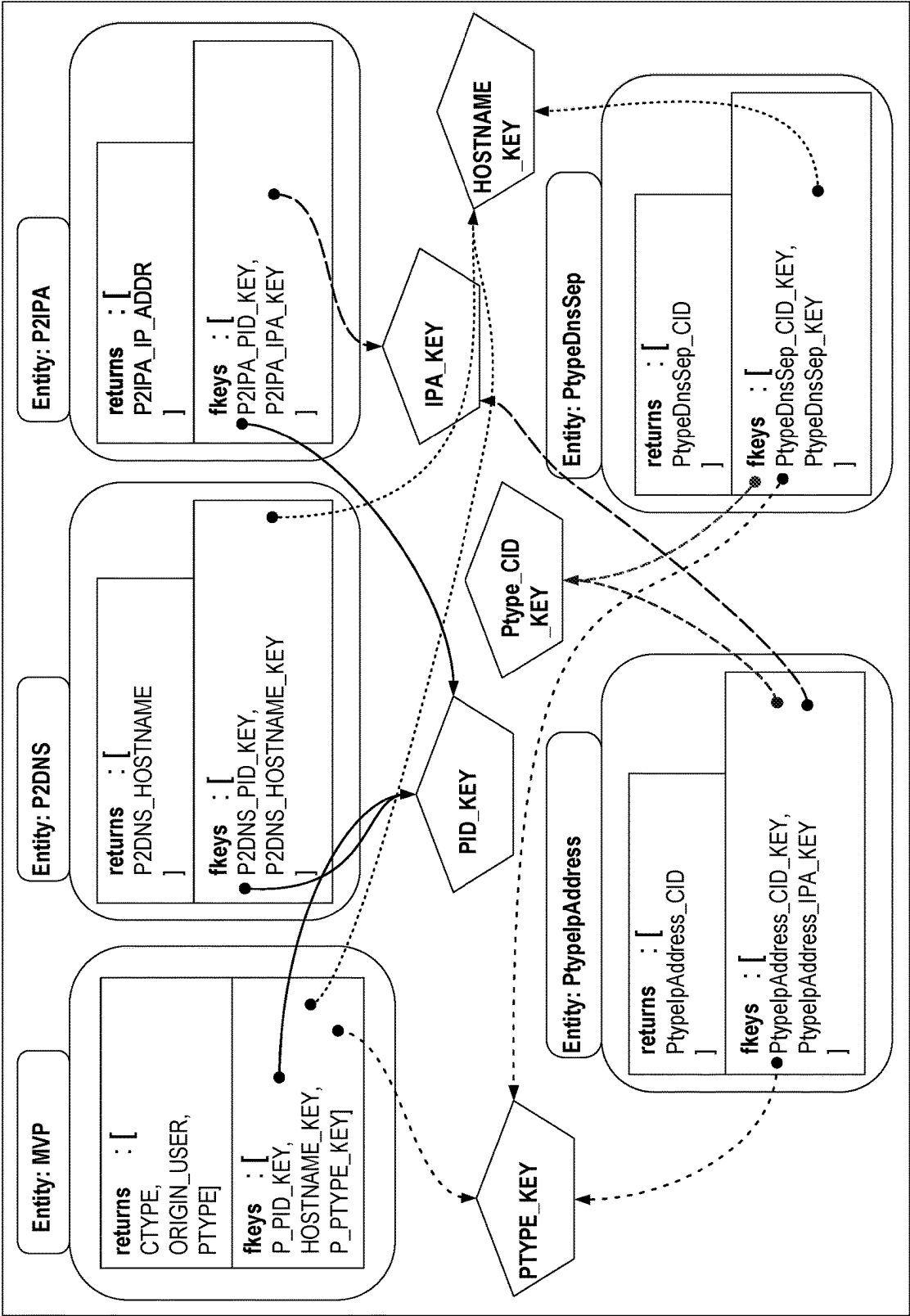


Fig. 4K

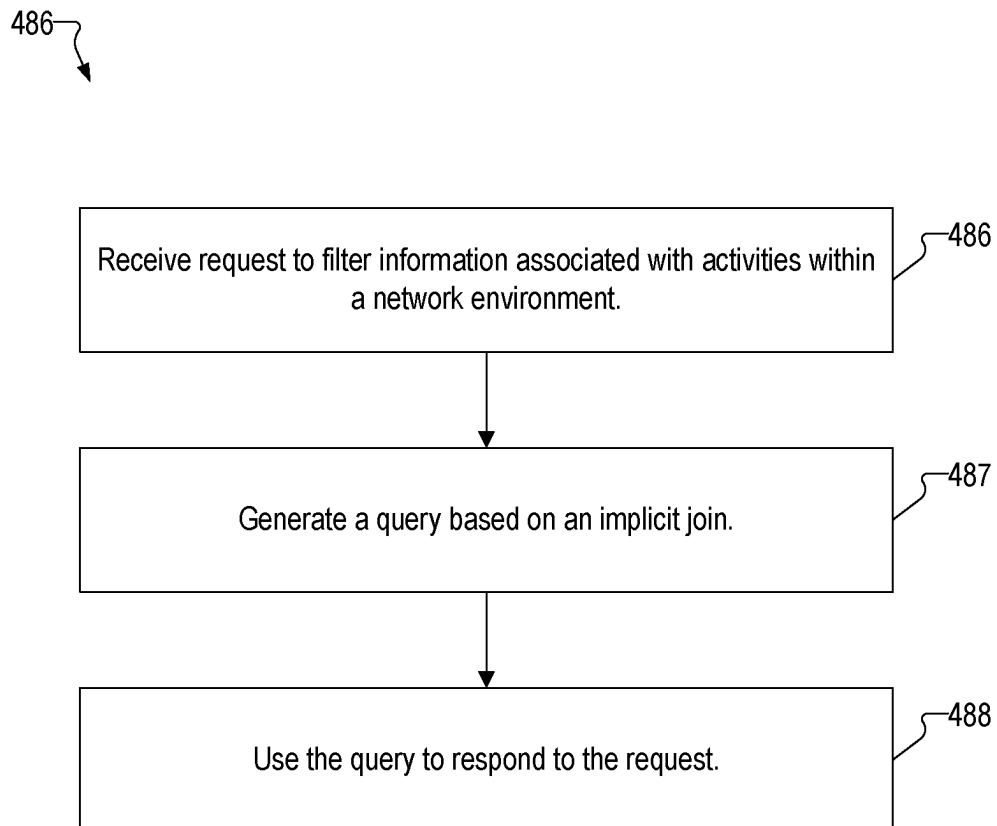


Fig. 4L

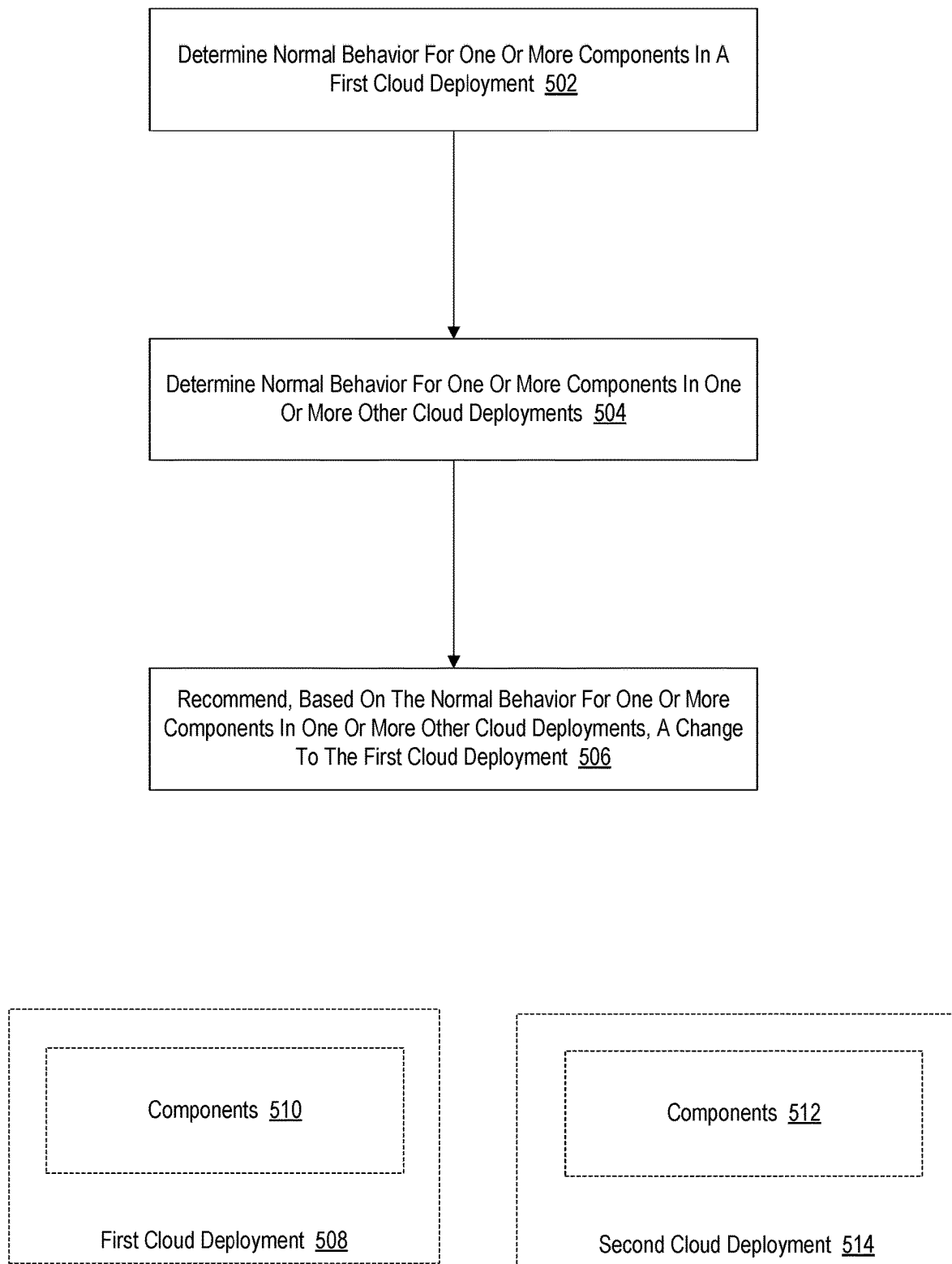


Fig. 5

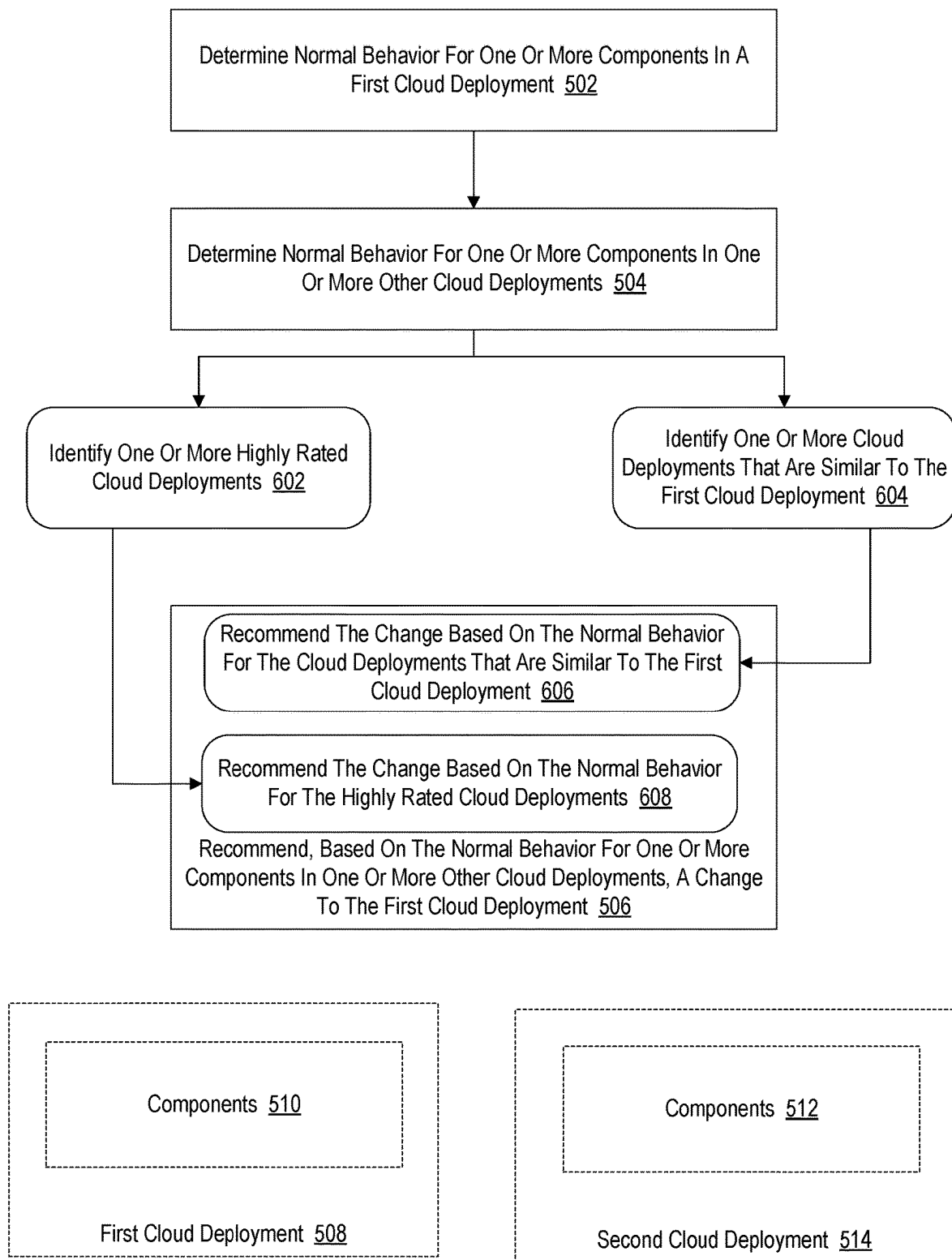


Fig. 6

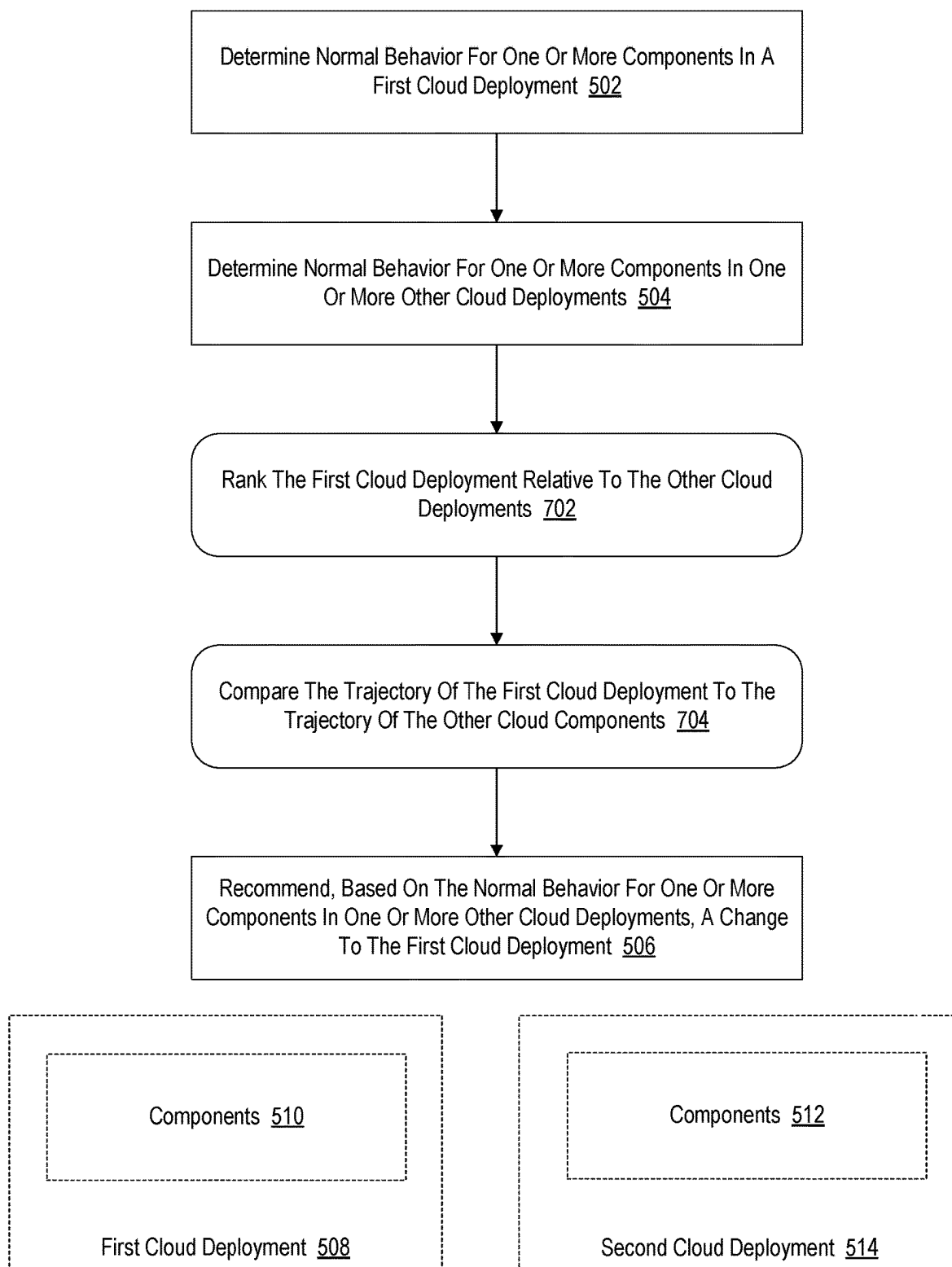


Fig. 7

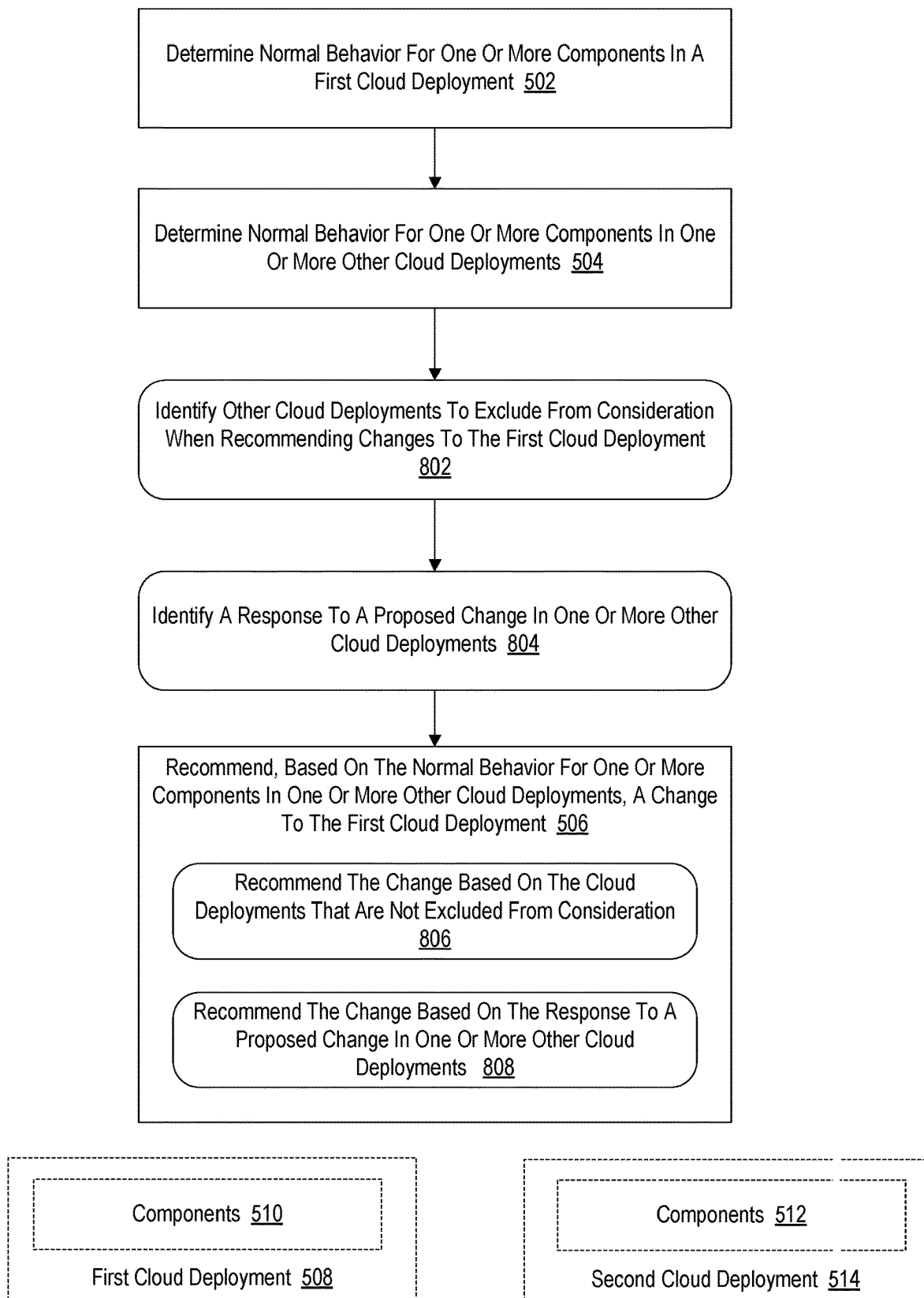


Fig. 8

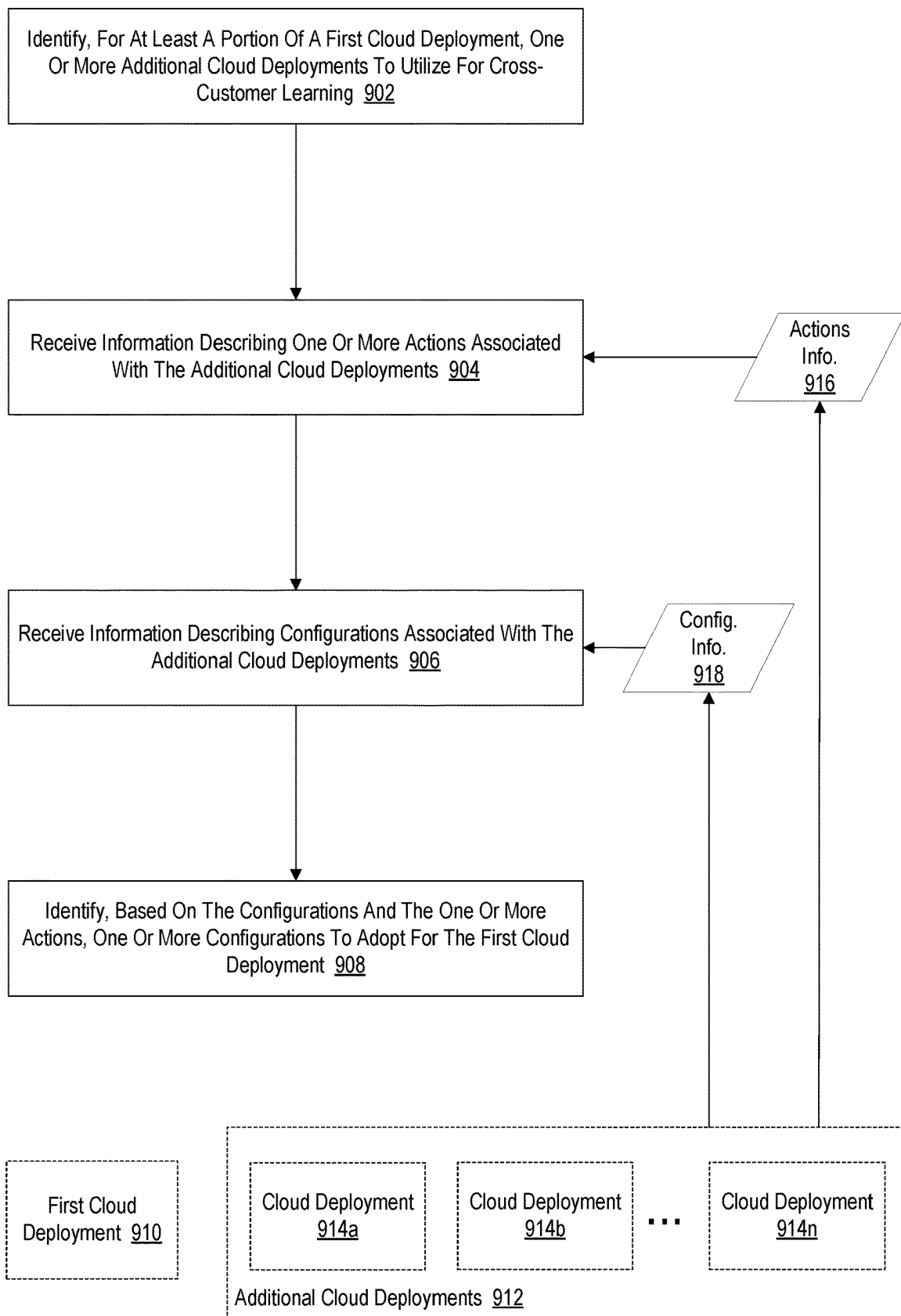


Fig. 9

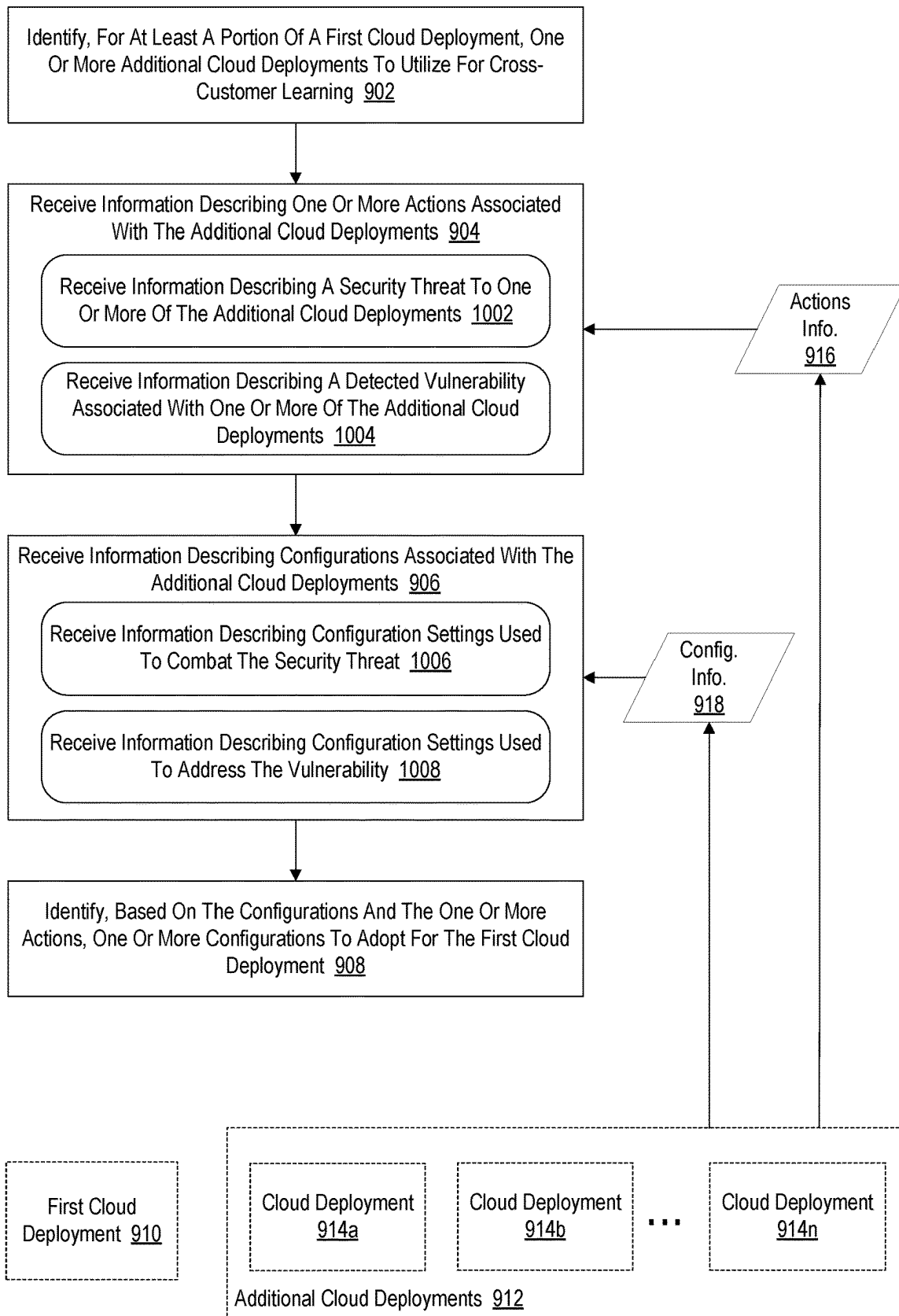


Fig. 10

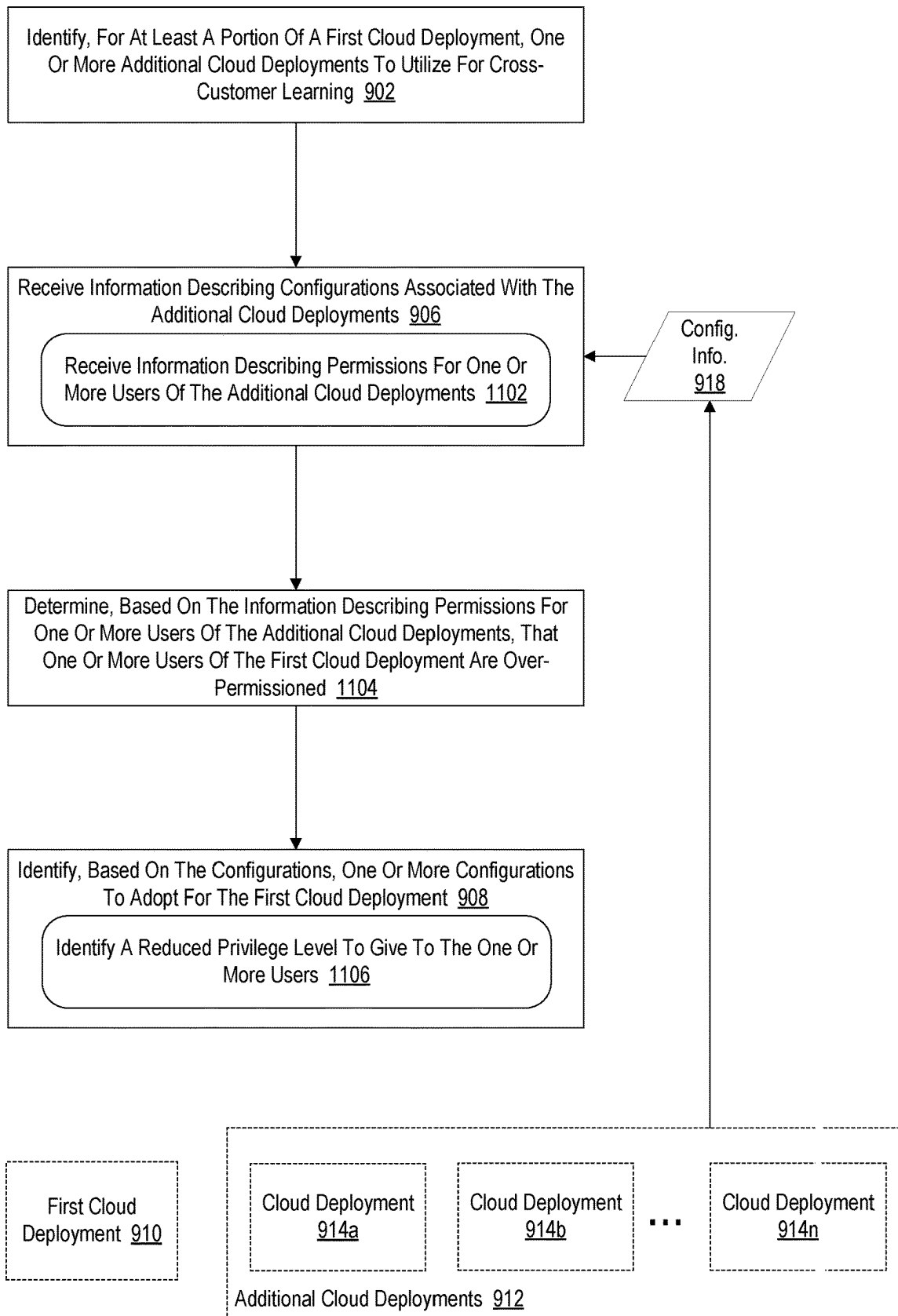


Fig. 11

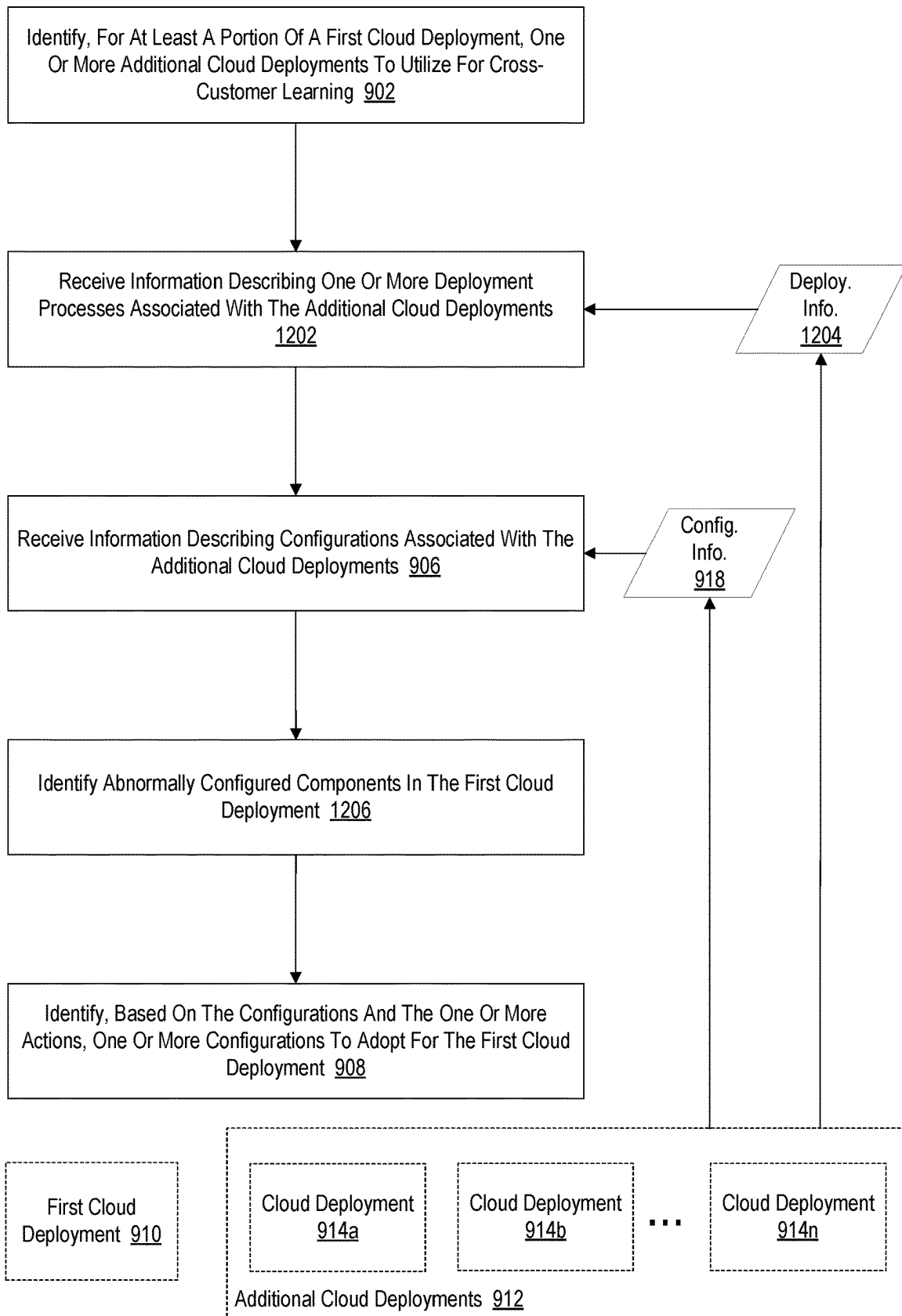


Fig. 12

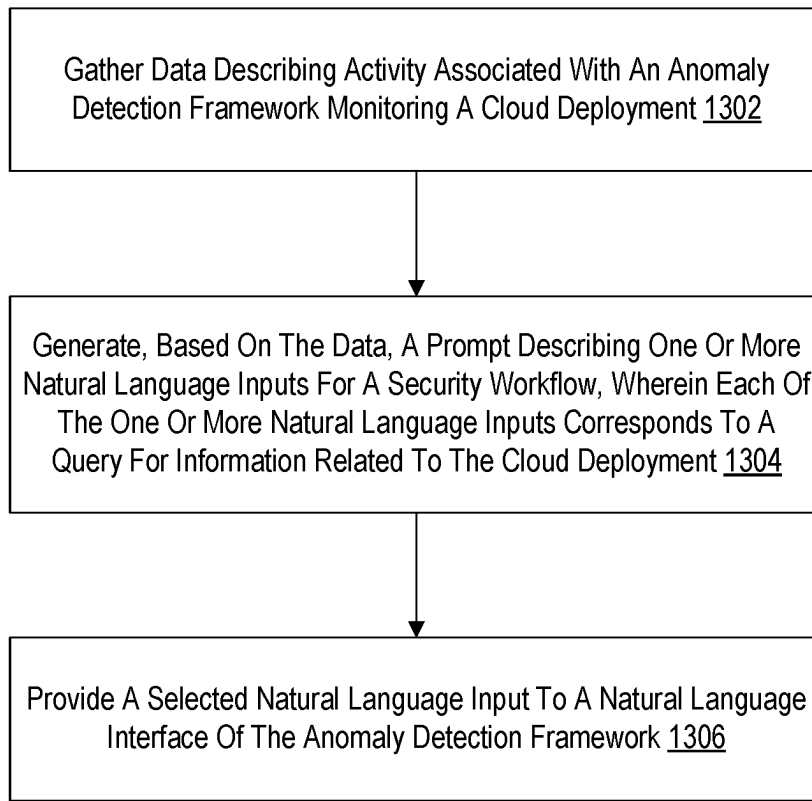


Fig. 13

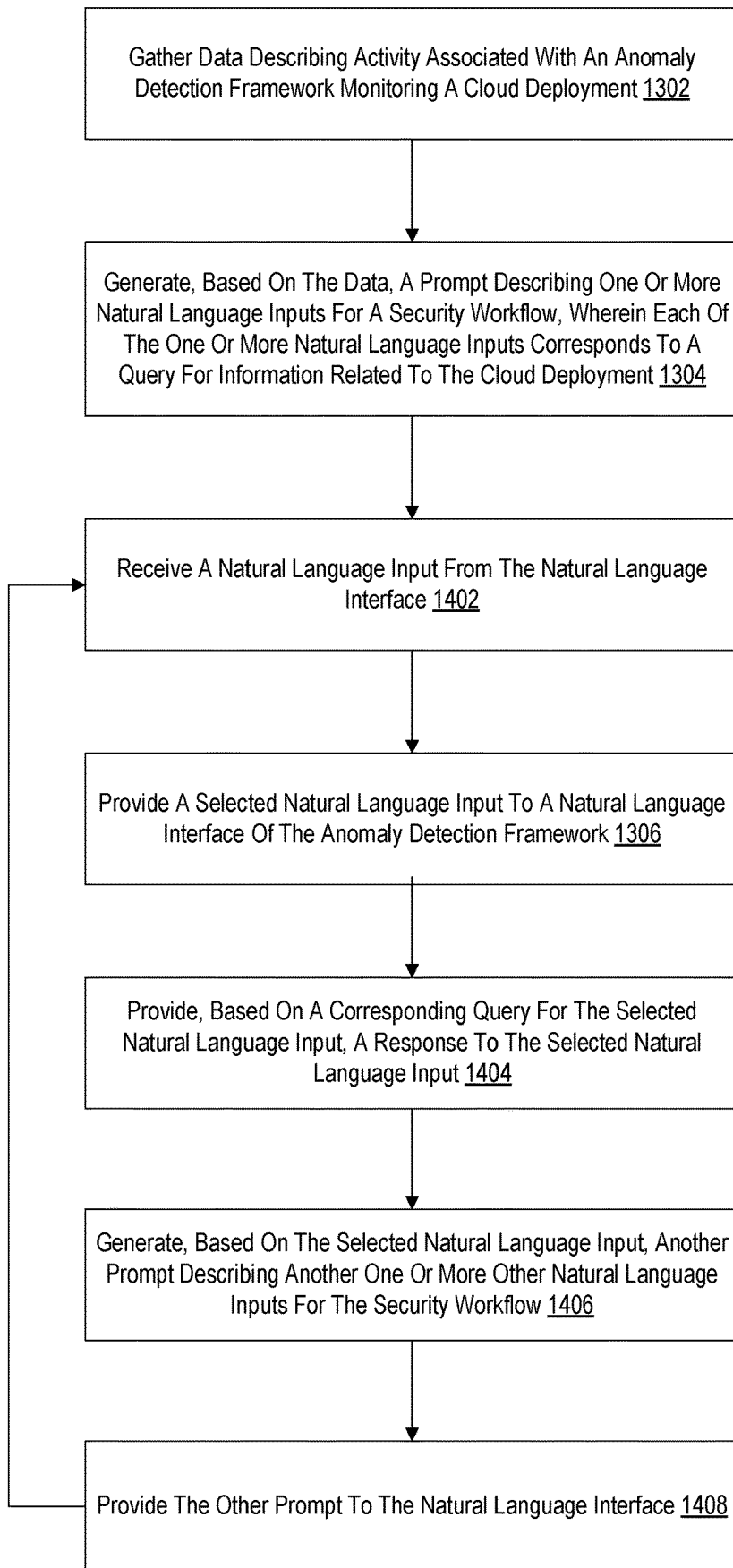


Fig. 14

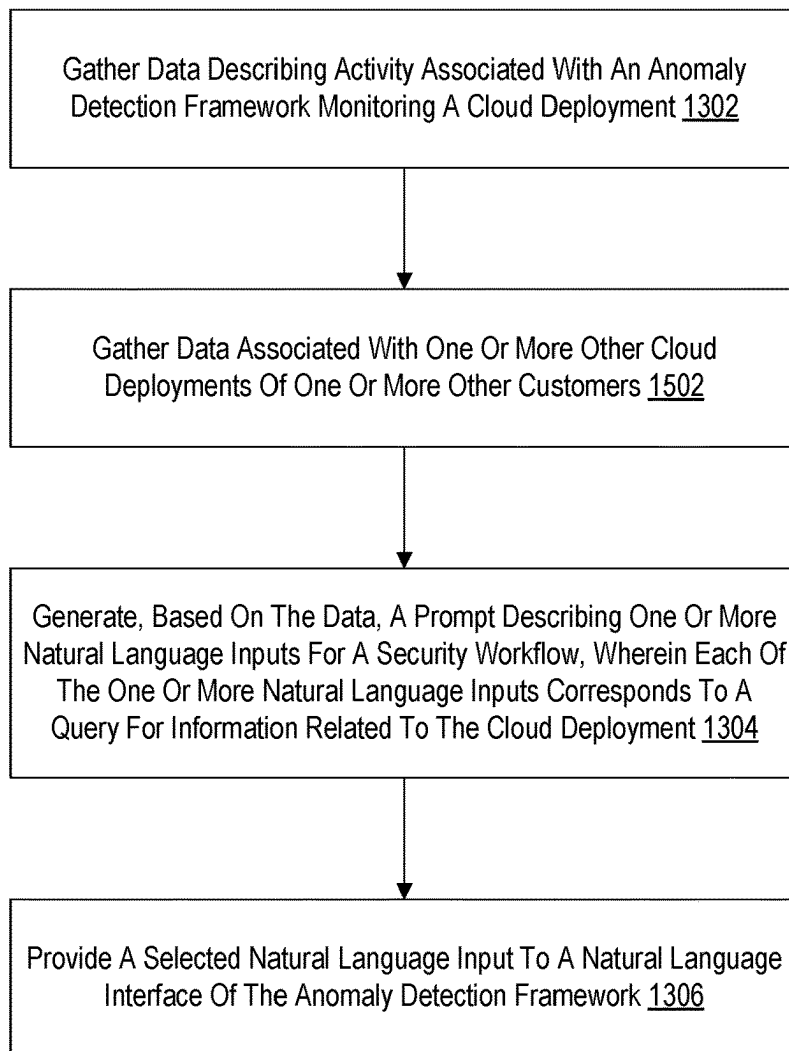


Fig. 15

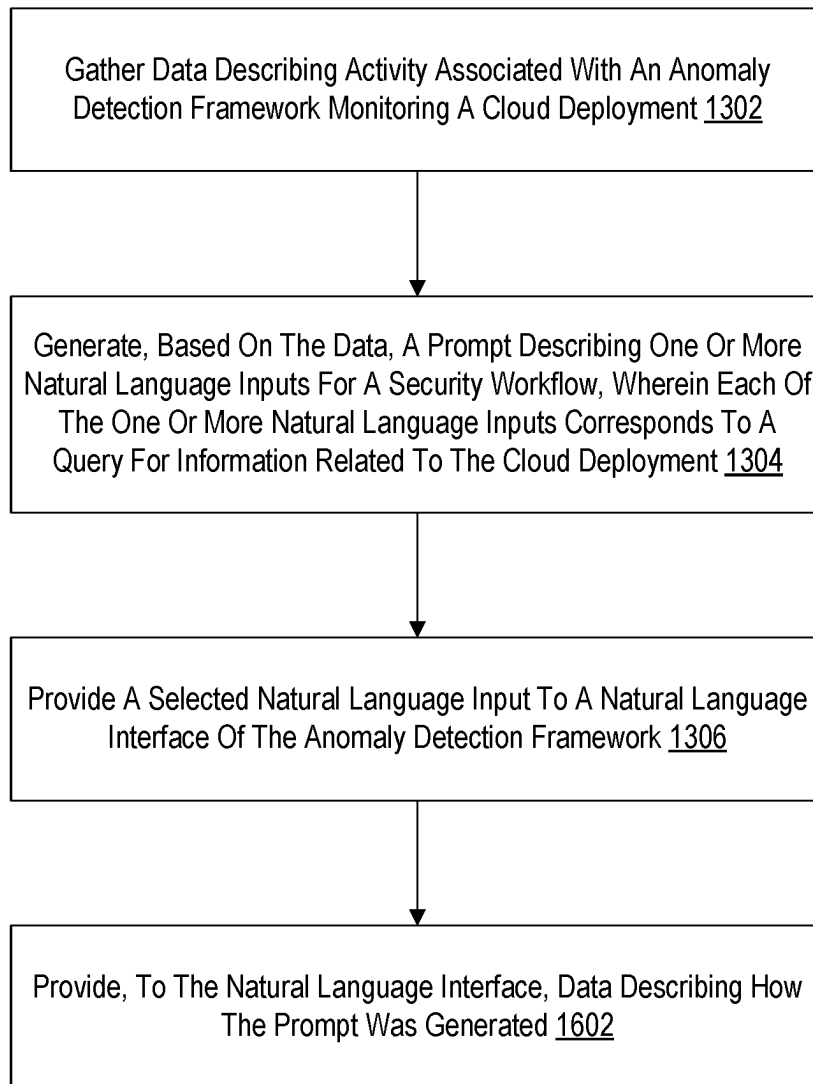


Fig. 16

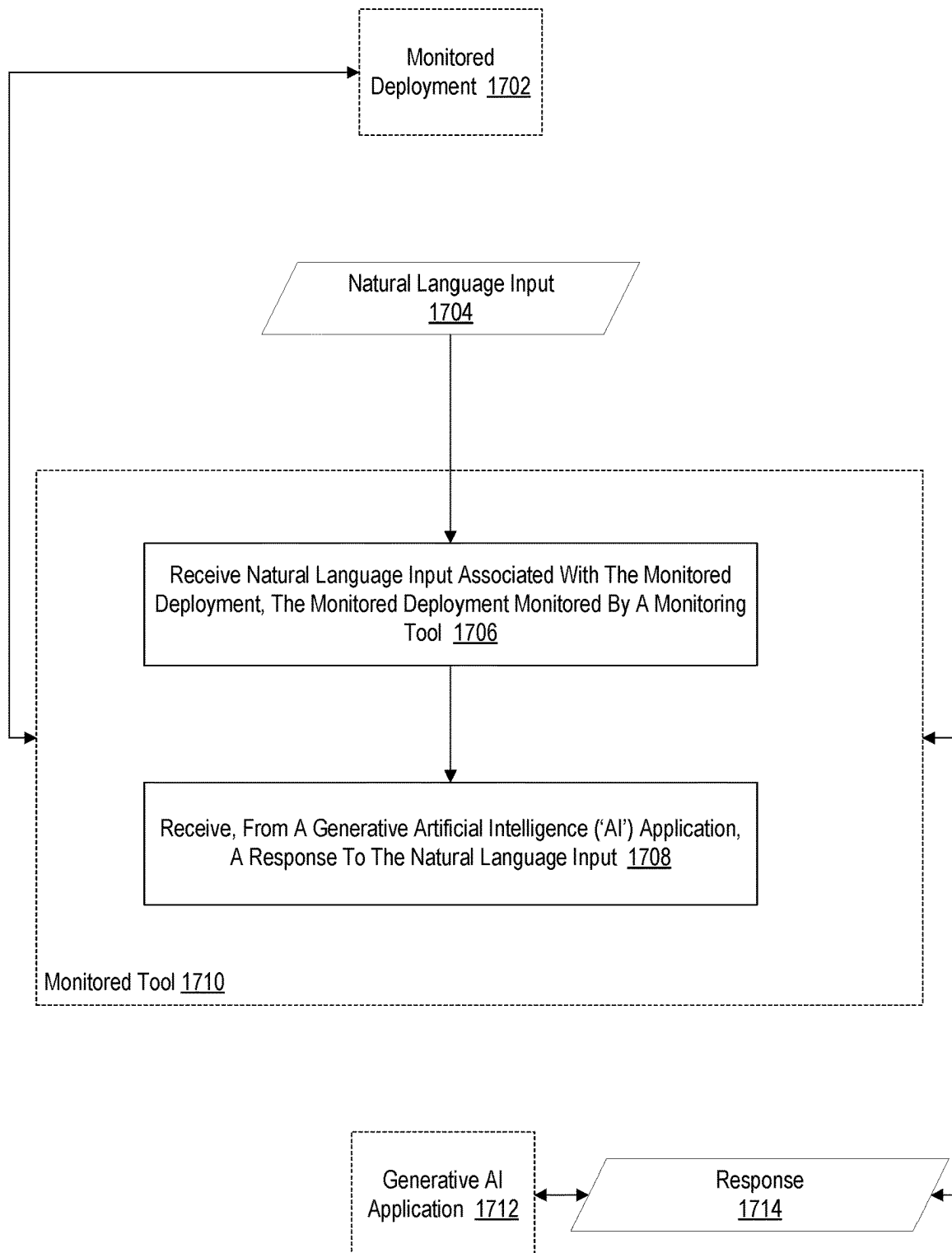


Fig. 17

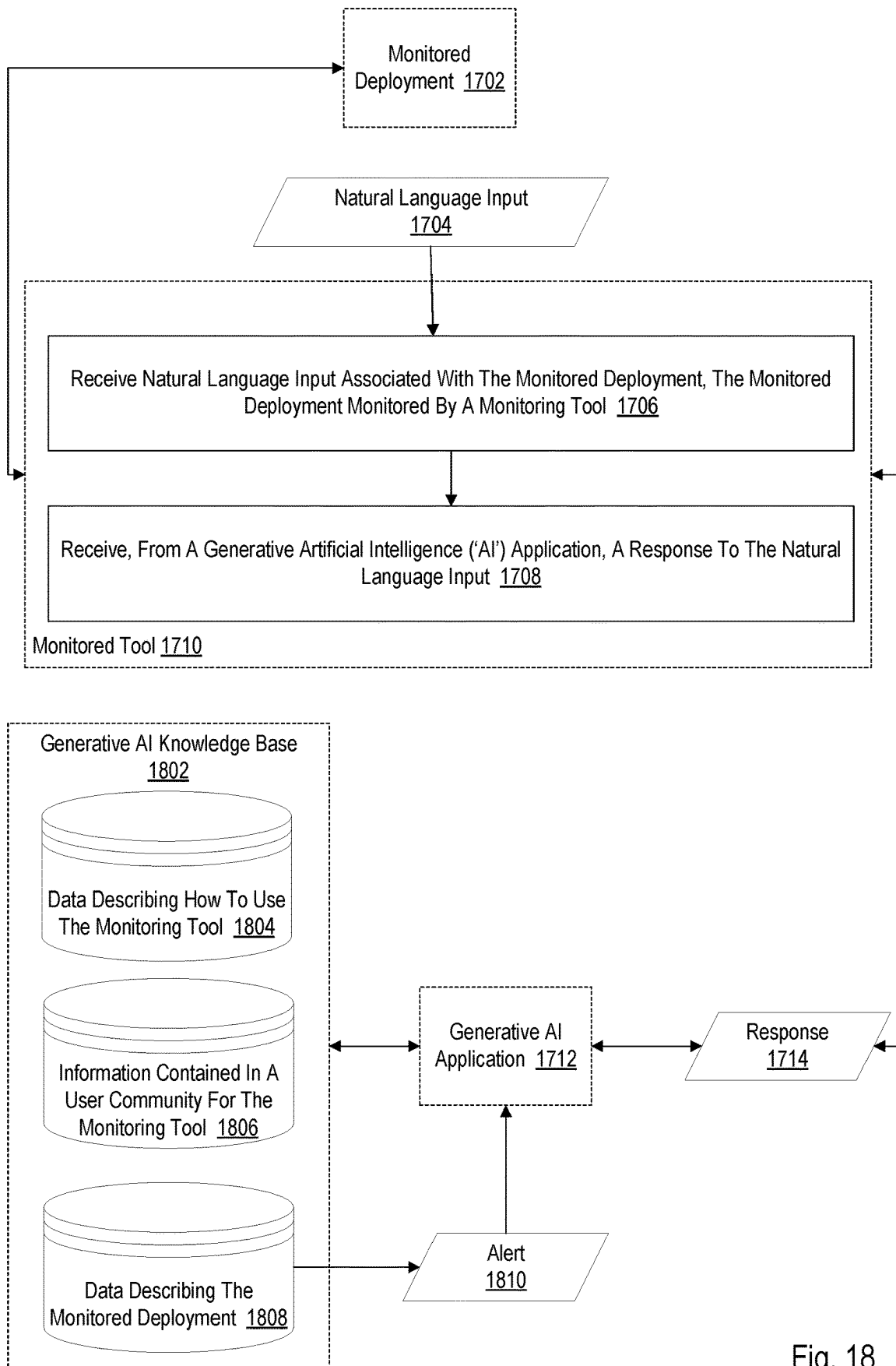


Fig. 18

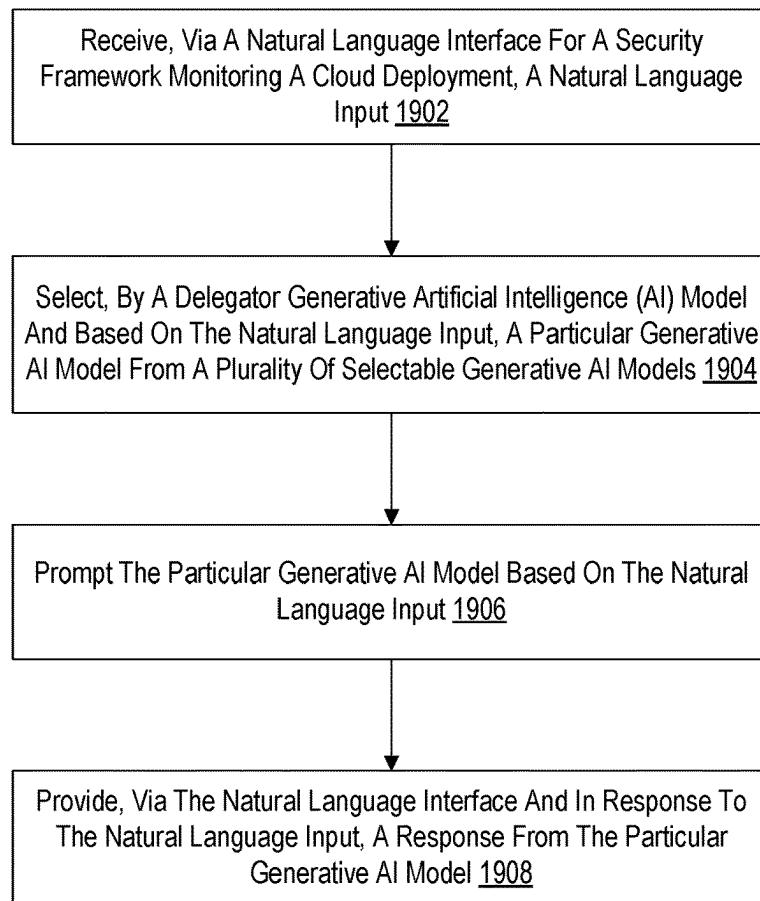


Fig. 19

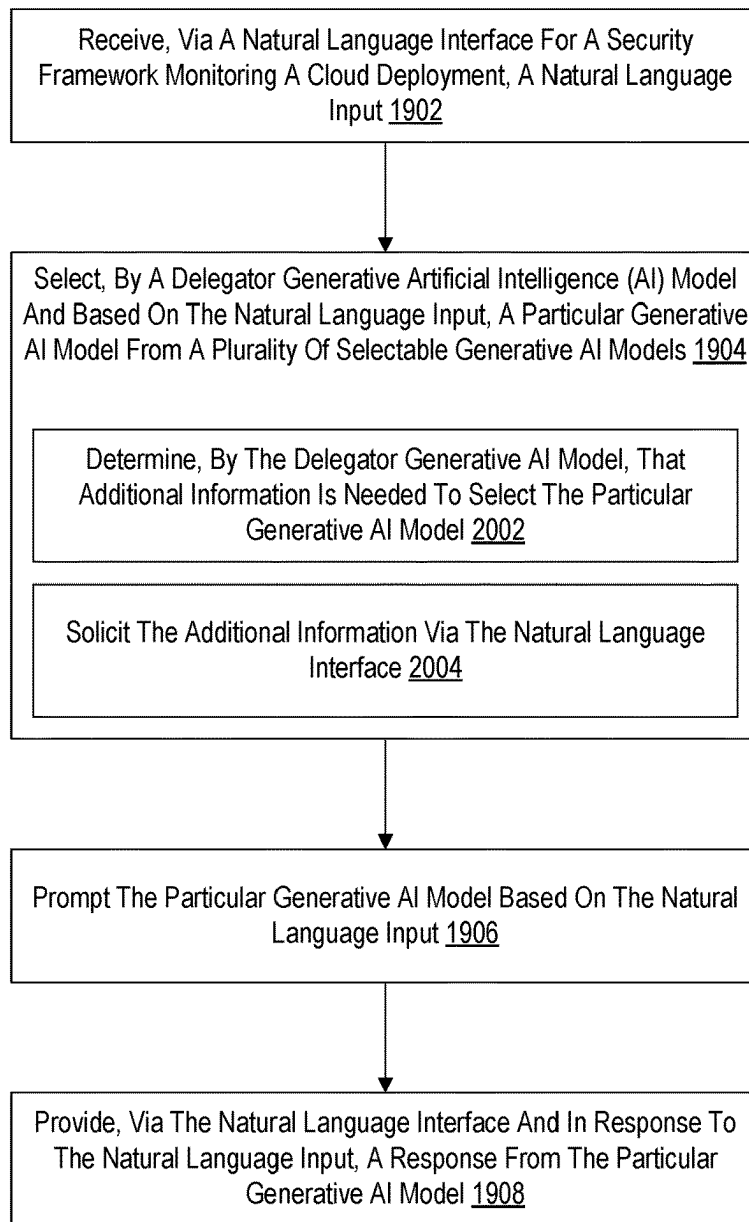


Fig. 20

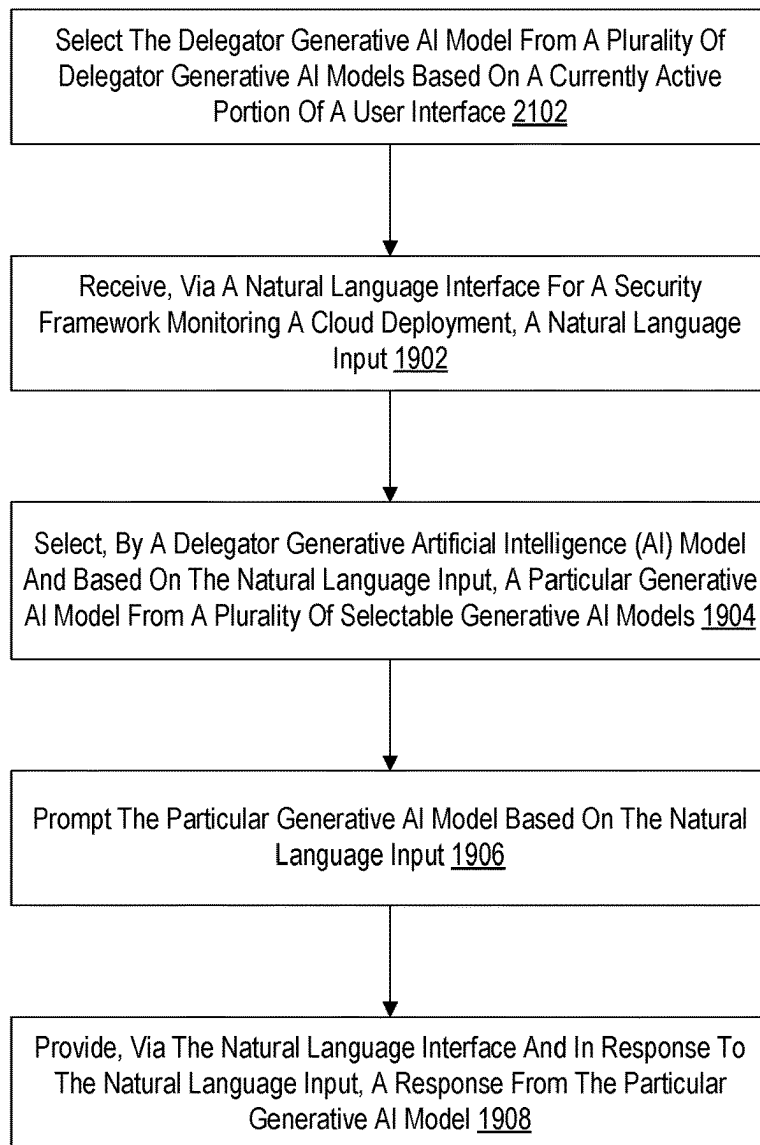


Fig. 21

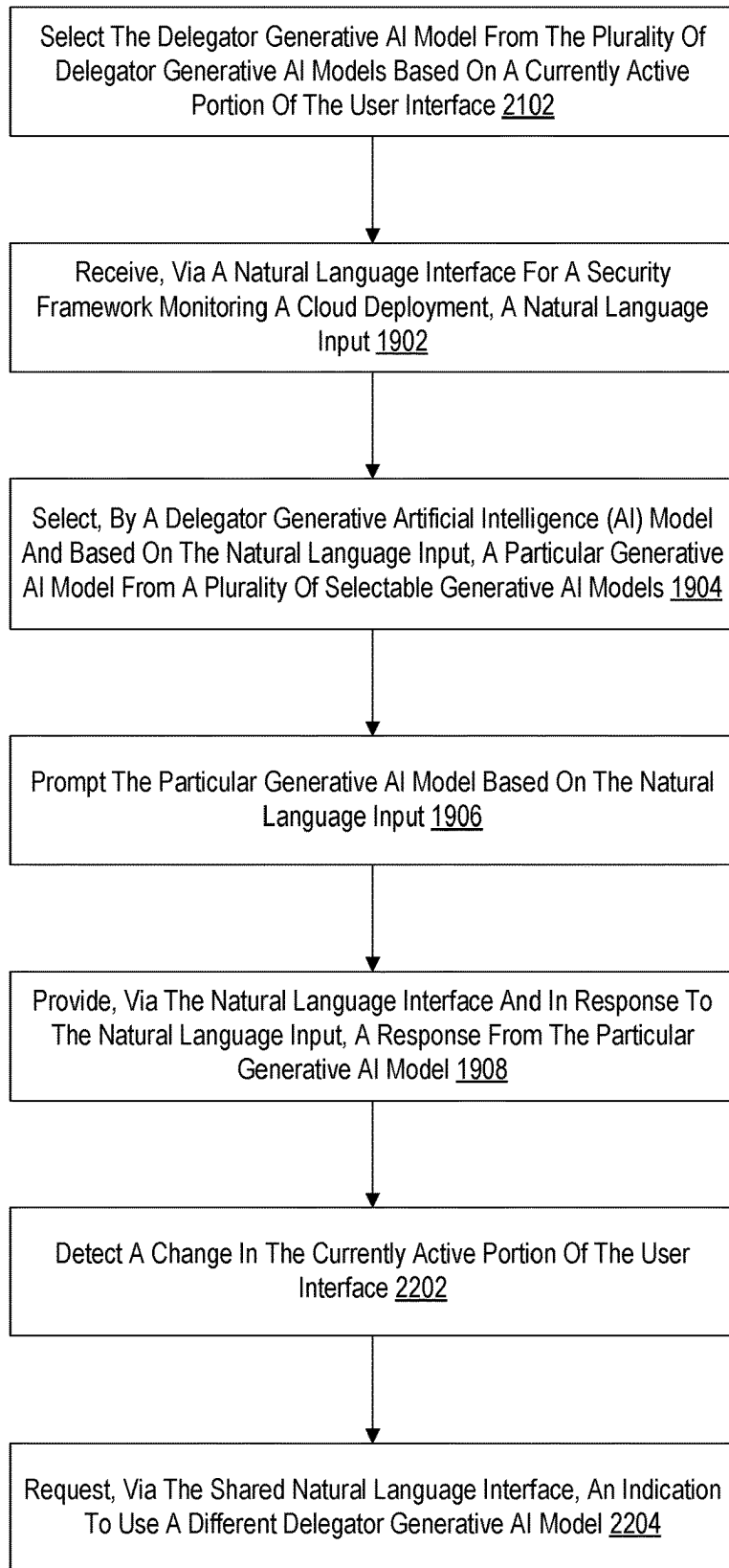


Fig. 22

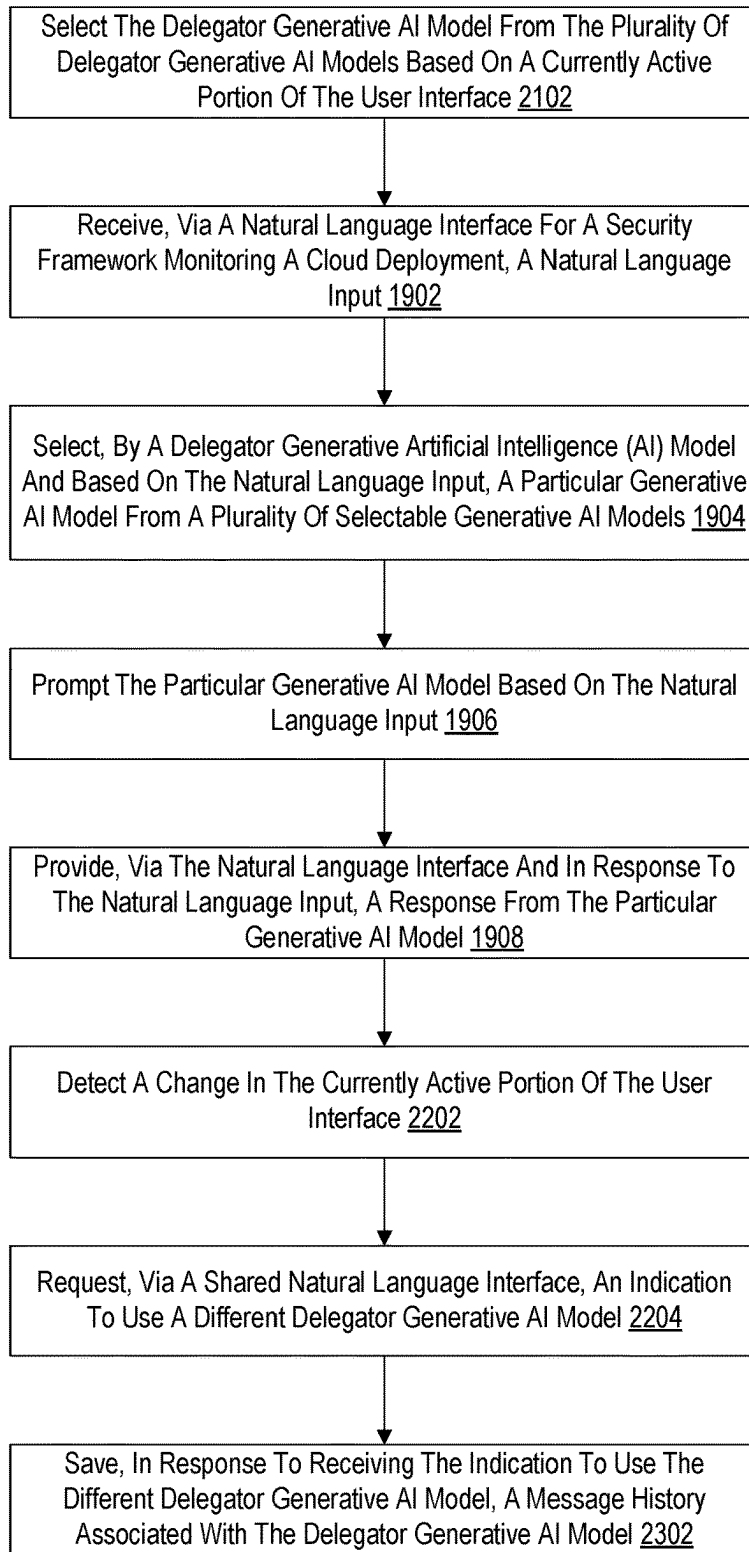


Fig. 23

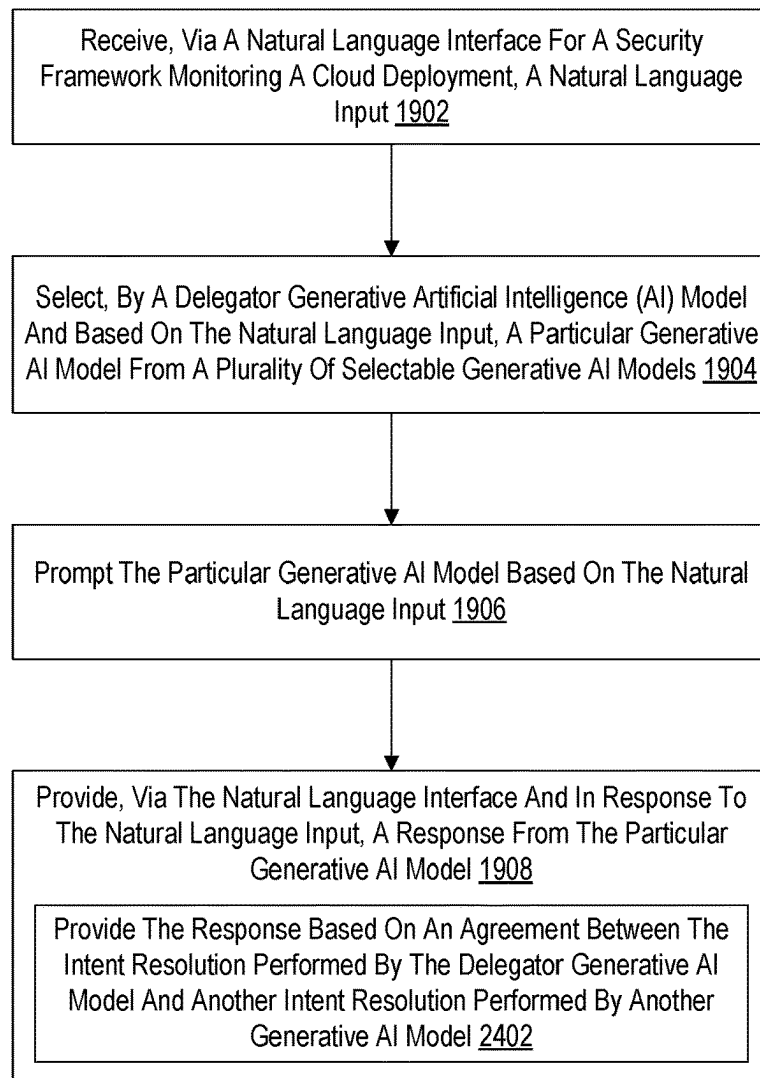


Fig. 24

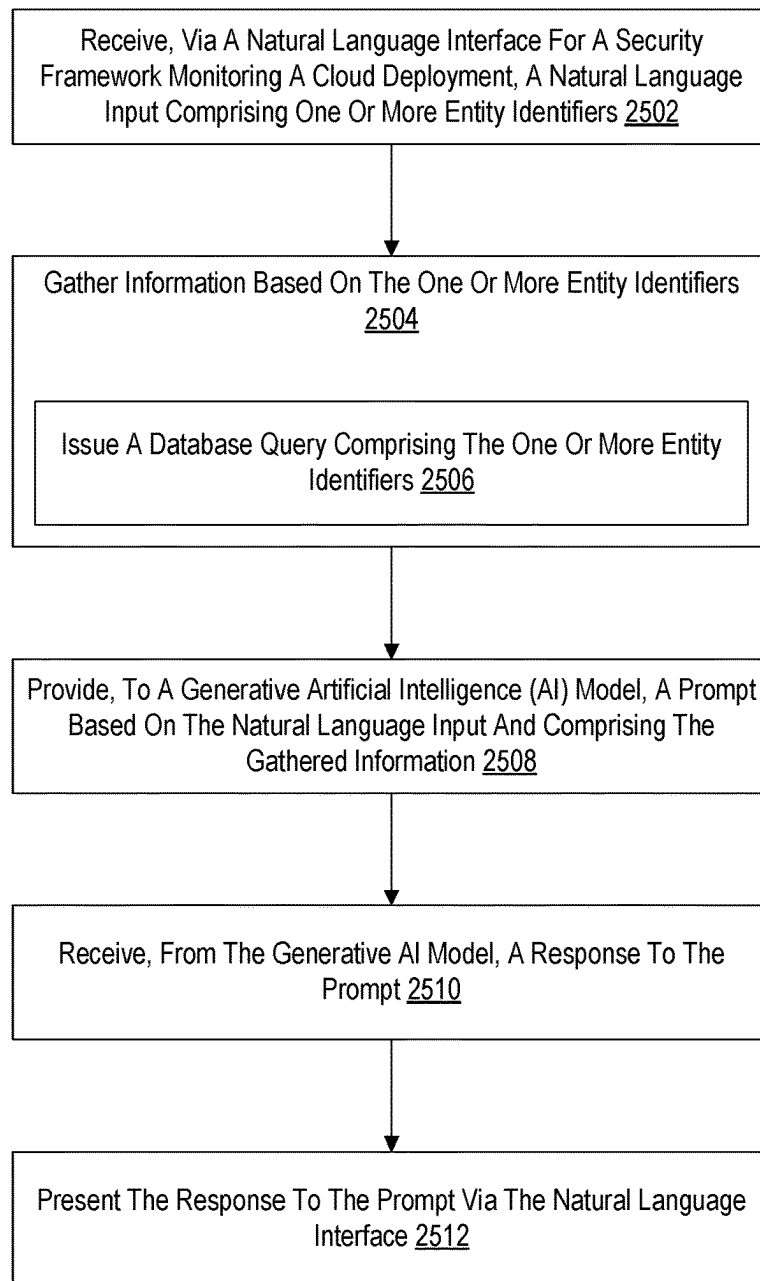


Fig. 25

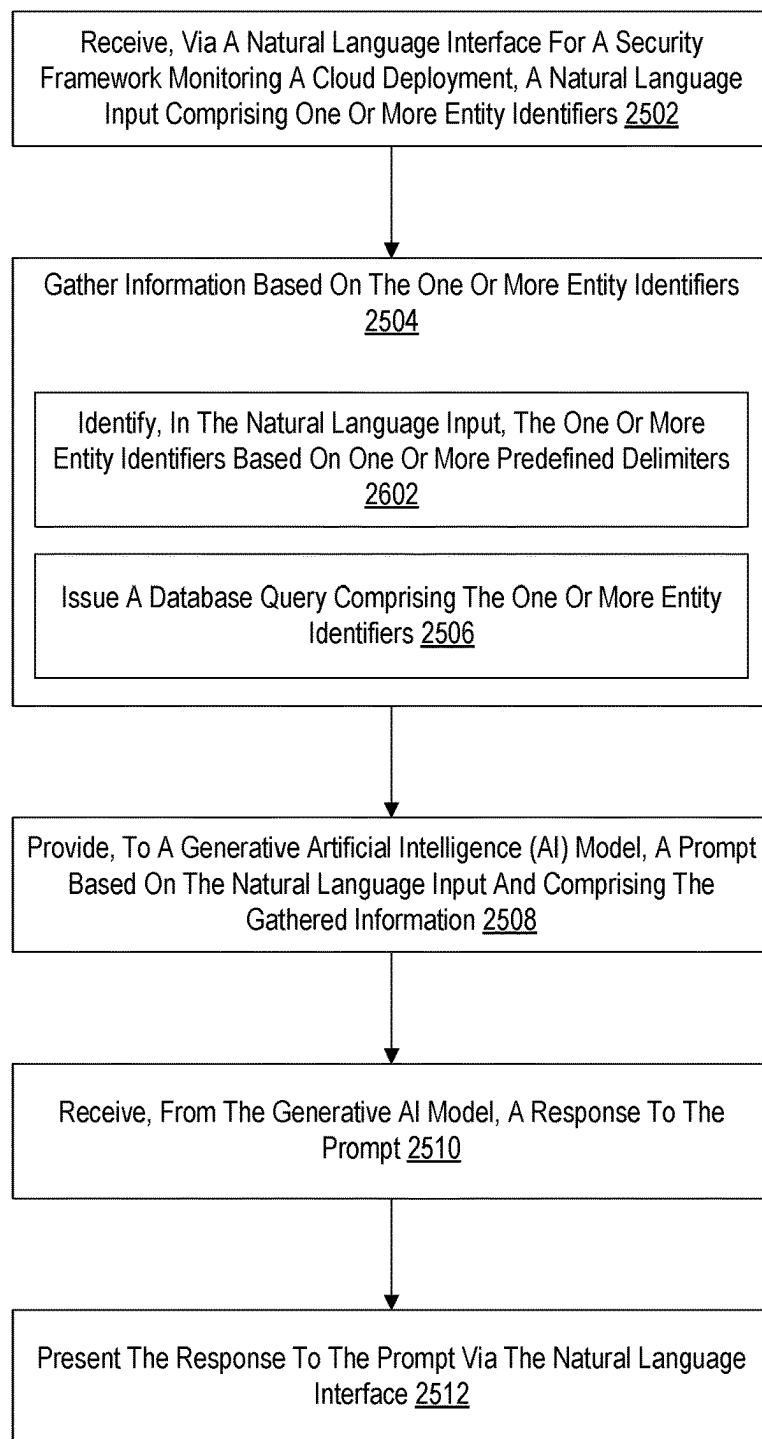


Fig. 26

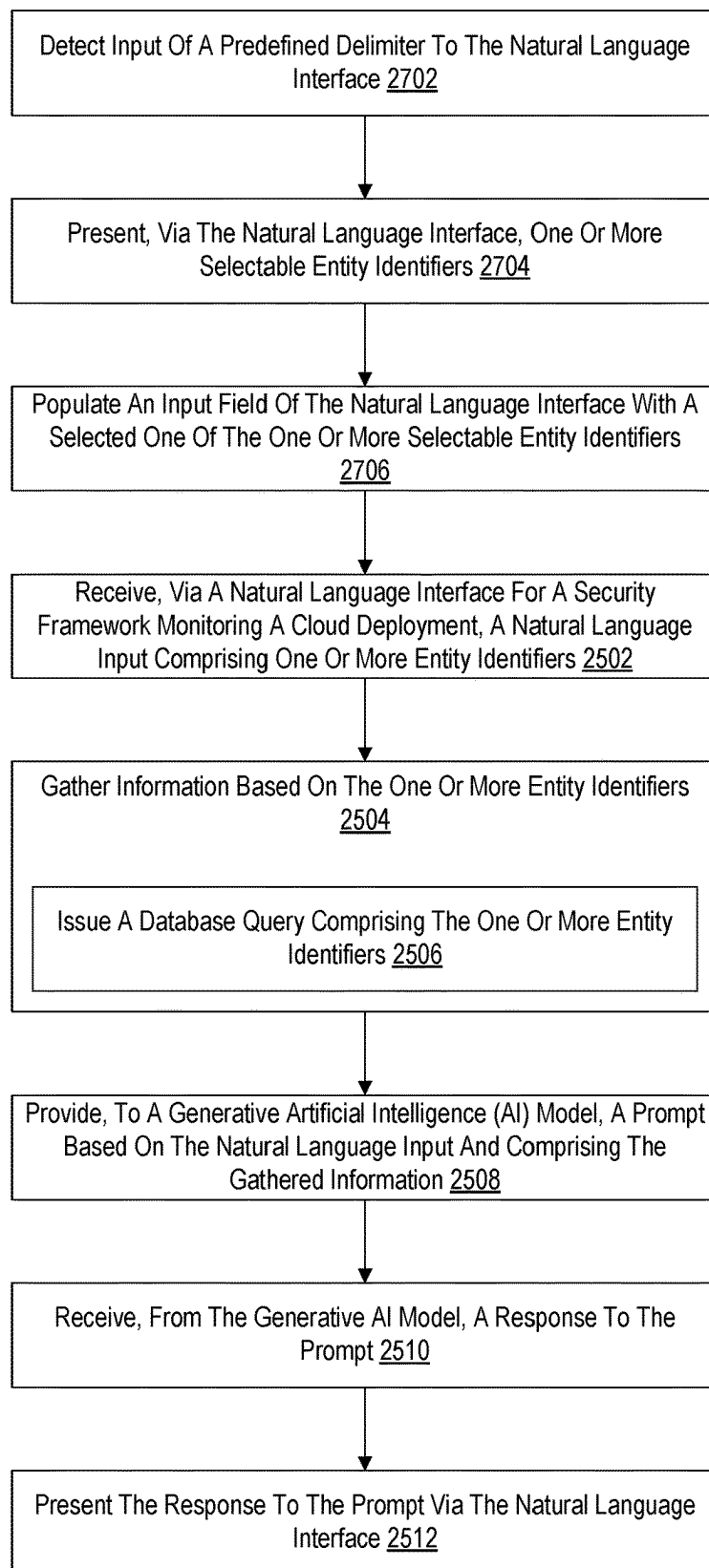


Fig. 27

GUIDING QUERY CREATION FOR A GENERATIVE ARTIFICIAL INTELLIGENCE (AI)-ENABLED ASSISTANT

BRIEF DESCRIPTION OF THE DRAWINGS

The accompanying drawings illustrate various embodiments and are a part of the specification. The illustrated embodiments are merely examples and do not limit the scope of the disclosure. Throughout the drawings, identical or similar reference numbers designate identical or similar elements.

FIG. 1A shows an illustrative configuration in which a data platform is configured to perform various operations with respect to a cloud environment that includes a plurality of compute assets.

FIG. 1B shows an illustrative implementation of the configuration of FIG. 1A.

FIG. 1C illustrates an example computing device.

FIG. 1D illustrates an example of an environment in which activities that occur within datacenters are modeled.

FIG. 2A illustrates an example of a process, used by an agent, to collect and report information about a client.

FIG. 2B illustrates a 5-tuple of data collected by an agent, physically and logically.

FIG. 2C illustrates a portion of a polygraph.

FIG. 2D illustrates a portion of a polygraph.

FIG. 2E illustrates an example of a communication polygraph.

FIG. 2F illustrates an example of a polygraph.

FIG. 2G illustrates an example of a polygraph as rendered in an interface.

FIG. 2H illustrates an example of a portion of a polygraph as rendered in an interface.

FIG. 2I illustrates an example of a portion of a polygraph as rendered in an interface.

FIG. 2J illustrates an example of a portion of a polygraph as rendered in an interface.

FIG. 2K illustrates an example of a portion of a polygraph as rendered in an interface.

FIG. 2L illustrates an example of an insider behavior graph as rendered in an interface.

FIG. 2M illustrates an example of a privilege change graph as rendered in an interface.

FIG. 2N illustrates an example of a user login graph as rendered in an interface.

FIG. 2O illustrates an example of a machine server graph as rendered in an interface.

FIG. 3A illustrates an example of a process for detecting anomalies in a network environment.

FIG. 3B depicts a set of example processes communicating with other processes.

FIG. 3C depicts a set of example processes communicating with other processes.

FIG. 3D depicts a set of example processes communicating with other processes.

FIG. 3E depicts two pairs of clusters.

FIG. 3F is a representation of a user logging into a first machine, then into a second machine from the first machine, and then making an external connection.

FIG. 3G is an alternate representation of actions occurring in FIG. 3F.

FIG. 3H illustrates an example of a process for performing extended user tracking.

FIG. 3I is a representation of a user logging into a first machine, then into a second machine from the first machine, and then making an external connection.

FIG. 3J illustrates an example of a process for performing extended user tracking.

FIG. 3K illustrates example records.

FIG. 3L illustrates example output from performing an ssh connection match.

FIG. 3M illustrates example records.

FIG. 3N illustrates example records.

FIG. 3O illustrates example records.

FIG. 3P illustrates example records.

FIG. 3Q illustrates an adjacency relationship between two login sessions.

FIG. 3R illustrates example records.

FIG. 3S illustrates an example of a process for detecting anomalies.

FIG. 4A illustrates a representation of an embodiment of an insider behavior graph.

FIG. 4B illustrates an embodiment of a portion of an insider behavior graph.

FIG. 4C illustrates an embodiment of a portion of an insider behavior graph.

FIG. 4D illustrates an embodiment of a portion of an insider behavior graph.

FIG. 4E illustrates a representation of an embodiment of a user login graph.

FIG. 4F illustrates an example of a privilege change graph.

FIG. 4G illustrates an example of a privilege change graph.

FIG. 4H illustrates an example of a user interacting with a portion of an interface.

FIG. 4I illustrates an example of a dossier for an event.

FIG. 4J illustrates an example of a dossier for a domain.

FIG. 4K depicts an example of an Entity Join graph by FilterKey and FilterKey Group (implicit join).

FIG. 4L illustrates an example process for dynamically generating and executing a query.

FIG. 5 sets forth a flowchart illustrating an example method of improving developer efficiency and application quality in accordance with some embodiments.

FIG. 6 sets forth a flowchart illustrating an additional example method of improving developer efficiency and application quality in accordance with some embodiments.

FIG. 7 sets forth a flowchart illustrating an additional example method of improving developer efficiency and application quality in accordance with some embodiments.

FIG. 8 sets forth a flowchart illustrating an additional example method of improving developer efficiency and application quality in accordance with some embodiments.

FIG. 9 sets forth a flowchart illustrating an example method of learning from similar cloud deployments in accordance with some embodiments of the present disclosure.

FIG. 10 sets forth a flowchart illustrating an additional example method of learning from similar cloud deployments in accordance with some embodiments of the present disclosure.

FIG. 11 sets forth a flowchart illustrating an additional example method of learning from similar cloud deployments in accordance with some embodiments of the present disclosure.

FIG. 12 sets forth a flowchart illustrating an additional example method of learning from similar cloud deployments in accordance with some embodiments of the present disclosure.

FIG. 13 sets forth a flowchart illustrating an example method of a guided anomaly detection framework in accordance with some embodiments of the present disclosure.

3

FIG. 14 sets forth a flowchart illustrating an additional example method of a guided anomaly detection framework in accordance with some embodiments of the present disclosure.

FIG. 15 sets forth a flowchart illustrating an additional example method of a guided anomaly detection framework in accordance with some embodiments of the present disclosure.

FIG. 16 sets forth a flowchart illustrating an additional example method of a guided anomaly detection framework in accordance with some embodiments of the present disclosure.

FIG. 17 sets forth a flowchart illustrating an example method of leveraging generative artificial intelligence ('AI') for securing a monitored deployment in accordance with some embodiments of the present disclosure.

FIG. 18 sets forth a flowchart illustrating an additional example method of leveraging generative artificial intelligence ('AI') for securing a monitored deployment in accordance with some embodiments of the present disclosure.

FIG. 19 sets forth a flowchart illustrating an example method of an architecture for a generative artificial intelligence (AI)-enabled assistant in accordance with some embodiments of the present disclosure.

FIG. 20 sets forth a flowchart illustrating an additional example method of an architecture for a generative artificial intelligence (AI)-enabled assistant in accordance with some embodiments of the present disclosure.

FIG. 21 sets forth a flowchart illustrating an additional example method of an architecture for a generative artificial intelligence (AI)-enabled assistant in accordance with some embodiments of the present disclosure.

FIG. 22 sets forth a flowchart illustrating an additional example method of an architecture for a generative artificial intelligence (AI)-enabled assistant in accordance with some embodiments of the present disclosure.

FIG. 23 sets forth a flowchart illustrating an additional example method of an architecture for a generative artificial intelligence (AI)-enabled assistant in accordance with some embodiments of the present disclosure.

FIG. 24 sets forth a flowchart illustrating an additional example method of an architecture for a generative artificial intelligence (AI)-enabled assistant in accordance with some embodiments of the present disclosure.

FIG. 25 sets forth a flowchart illustrating an example method of guiding query creation for a generative artificial intelligence (AI)-enabled assistant in accordance with some embodiments of the present disclosure.

FIG. 26 sets forth a flowchart illustrating an additional example method of guiding query creation for a generative artificial intelligence (AI)-enabled assistant in accordance with some embodiments of the present disclosure.

FIG. 27 sets forth a flowchart illustrating an additional example method of guiding query creation for a generative artificial intelligence (AI)-enabled assistant in accordance with some embodiments of the present disclosure.

DETAILED DESCRIPTION

Various illustrative embodiments are described herein with reference to the accompanying drawings. It will, however, be evident that various modifications and changes may be made thereto, and additional embodiments may be implemented, without departing from the scope of the invention as set forth in the claims. For example, certain features of one embodiment described herein may be combined with or substituted for features of another embodiment described

4

herein. The description and drawings are accordingly to be regarded in an illustrative rather than a restrictive sense.

FIG. 1A shows an illustrative configuration 10 in which a data platform 12 is configured to perform various operations with respect to a cloud environment 14 that includes a plurality of compute assets 16-1 through 16-N (collectively "compute assets 16"). For example, data platform 12 may include data ingestion resources 18 configured to ingest data from cloud environment 14 into data platform 12, data processing resources 20 configured to perform data processing operations with respect to the data, and user interface resources 22 configured to provide one or more external users and/or compute resources (e.g., computing device 24) with access to an output of data processing resources 20. Each of these resources are described in detail herein.

Cloud environment 14 may include any suitable network-based computing environment as may serve a particular application. For example, cloud environment 14 may be implemented by one or more compute resources provided and/or otherwise managed by one or more cloud service providers, such as Amazon Web Services (AWS), Google Cloud Platform (GCP), Microsoft Azure, and/or any other cloud service provider configured to provide public and/or private access to network-based compute resources. While FIG. 1A shows that compute assets 16 are included in a cloud environment, compute assets 16 may be deployed in any compute environment such as cloud environment 14 and/or a non-cloud environment (e.g., a local datacenter).

Compute assets 16 may include, but are not limited to, containers (e.g., container images, deployed and executing container instances, etc.), virtual machines, workloads, applications, processes, physical machines, compute nodes, clusters of compute nodes, software runtime environments (e.g., container runtime environments), and/or any other virtual and/or physical compute resource that may reside in and/or be executed by one or more computer resources in cloud environment 14. In some examples, one or more compute assets 16 may reside in one or more datacenters.

A compute asset 16 may be associated with (e.g., owned, deployed, or managed by) a particular entity, such as a customer or client of cloud environment 14 and/or data platform 12. Accordingly, for purposes of the discussion herein, cloud environment 14 may be used by one or more entities.

Data platform 12 may be configured to perform one or more data security monitoring and/or remediation services, compliance monitoring services, anomaly detection services, DevOps services, compute asset management services, and/or any other type of data analytics service as may serve a particular implementation. Data platform 12 may be managed or otherwise associated with any suitable data platform provider, such as a provider of any of the data analytics services described herein. The various resources included in data platform 12 may reside in the cloud and/or be located on-premises and be implemented by any suitable combination of physical and/or virtual compute resources, such as one or more computing devices, microservices, applications, etc.

Data ingestion resources 18 may be configured to ingest data from cloud environment 14 into data platform 12. This may be performed in various ways, some of which are described in detail herein. For example, as illustrated by arrow 26, data ingestion resources 18 may be configured to receive the data from one or more agents deployed within cloud environment 14, utilize an event streaming platform (e.g., Kafka) to obtain the data, and/or pull data (e.g., configuration data) from cloud environment 14. In some

examples, data ingestion resources **18** may obtain the data using one or more agentless configurations.

The data ingested by data ingestion resources **18** from cloud environment **14** may include any type of data as may serve a particular implementation. For example, the data may include data representative of configuration information associated with compute assets **16**, information about one or more processes running on compute assets **16**, network activity information, information about events (creation events, modification events, communication events, user-initiated events, etc.) that occur with respect to compute assets **16**, etc. In some examples, the data may or may not include actual customer data processed or otherwise generated by compute assets **16**.

As illustrated by arrow **28**, data ingestion resources **18** may be configured to load the data ingested from cloud environment **14** into a data store **30**. Data store **30** is illustrated in FIG. **1A** as being separate from and communicatively coupled to data platform **12**. However, in some alternative embodiments, data store **30** is included within data platform **12**.

Data store **30** may be implemented by any suitable data warehouse, data lake, data mart, and/or other type of database structure as may serve a particular implementation. Such data stores may be proprietary or may be embodied as vendor provided products or services such as, for example, Snowflake, Google BigQuery, Druid, Amazon Redshift, IBM Db2, Dremio, Databricks Lakehouse Platform, Cloudera, Azure Synapse Analytics, and others.

Although the examples described herein largely relate to embodiments where data is collected from agents and ultimately stored in a data store such as those provided by Snowflake, in other embodiments data that is collected from agents and other sources may be stored in different ways. For example, data that is collected from agents and other sources may be stored in a data warehouse, data lake, data mart, and/or any other data store.

A data warehouse may be embodied as an analytic database (e.g., a relational database) that is created from two or more data sources. Such a data warehouse may be leveraged to store historical data, often on the scale of petabytes. Data warehouses may have compute and memory resources for running complicated queries and generating reports. Data warehouses may be the data sources for business intelligence ('BI') systems, machine learning applications, and/or other applications. By leveraging a data warehouse, data that has been copied into the data warehouse may be indexed for good analytic query performance, without affecting the write performance of a database (e.g., an Online Transaction Processing ('OLTP') database). Data warehouses also enable joining data from multiple sources for analysis. For example, a sales OLTP application probably has no need to know about the weather at various sales locations, but sales predictions could take advantage of that data. By adding historical weather data to a data warehouse, it would be possible to factor it into models of historical sales data.

Data lakes, which store files of data in their native format, may be considered as "schema on read" resources. As such, any application that reads data from the lake may impose its own types and relationships on the data. Data warehouses, on the other hand, are "schema on write," meaning that data types, indexes, and relationships are imposed on the data as it is stored in an enterprise data warehouse (EDW). "Schema on read" resources may be beneficial for data that may be used in several contexts and poses little risk of losing data. "Schema on write" resources may be beneficial for data that has a specific purpose, and good for data that must relate

properly to data from other sources. Such data stores may include data that is encrypted using homomorphic encryption, data encrypted using privacy-preserving encryption, smart contracts, non-fungible tokens, decentralized finance, and other techniques.

Data marts may contain data oriented towards a specific business line whereas data warehouses contain enterprise-wide data. Data marts may be dependent on a data warehouse, independent of the data warehouse (e.g., drawn from an operational database or external source), or a hybrid of the two. In embodiments described herein, different types of data stores (including combinations thereof) may be leveraged.

Data processing resources **20** may be configured to perform various data processing operations with respect to data ingested by data ingestion resources **18**, including data ingested and stored in data store **30**. For example, data processing resources **20** may be configured to perform one or more data security monitoring and/or remediation operations, compliance monitoring operations, anomaly detection operations, DevOps operations, compute asset management operations, and/or any other type of data analytics operation as may serve a particular implementation. Various examples of operations performed by data processing resources **20** are described herein.

As illustrated by arrow **32**, data processing resources **20** may be configured to access data in data store **30** to perform the various operations described herein. In some examples, this may include performing one or more queries with respect to the data stored in data store **30**. Such queries may be generated using any suitable query language.

In some examples, the queries provided by data processing resources **20** may be configured to direct data store **30** to perform one or more data analytics operations with respect to the data stored within data store **30**. These data analytics operations may be with respect to data specific to a particular entity (e.g., data residing in one or more silos within data store **30** that are associated with a particular customer) and/or data associated with multiple entities. For example, data processing resources **20** may be configured to analyze data associated with a first entity and use the results of the analysis to perform one or more operations with respect to a second entity.

One or more operations performed by data processing resources **20** may be performed periodically according to a predetermined schedule. For example, one or more operations may be performed by data processing resources **20** every hour or any other suitable time interval. Additionally or alternatively, one or more operations performed by data processing resources **20** may be performed in substantially real-time (or near real-time) as data is ingested into data platform **12**. In this manner, the results of such operations (e.g., one or more detected anomalies in the data) may be provided to one or more external entities (e.g., computing device **24** and/or one or more users) in substantially real-time and/or in near real-time.

User interface resources **22** may be configured to perform one or more user interface operations, examples of which are described herein. For example, user interface resources **22** may be configured to present one or more results of the data processing performed by data processing resources **20** to one or more external entities (e.g., computing device **24** and/or one or more users), as illustrated by arrow **34**. As illustrated by arrow **36**, user interface resources **22** may access data in data store **30** to perform the one or more user interface operations.

FIG. 1B illustrates an implementation of configuration 10 in which an agent 38 (e.g., agent 38-1 through agent 38-N) is installed on each of compute assets 16. As used herein, an agent may include a self-contained binary and/or other type of code or application that can be run on any appropriate platforms, including within containers and/or other virtual compute assets. Agents 38 may monitor the nodes on which they execute for a variety of different activities, including but not limited to, connection, process, user, machine, and file activities. In some examples, agents 38 can be executed in user space, and can use a variety of kernel modules (e.g., auditd, iptables, netfilter, pcap, etc.) to collect data. Agents can be implemented in any appropriate programming language, such as C or Golang, using applicable kernel APIs.

Agents 38 may be deployed in any suitable manner. For example, an agent 38 may be deployed as a containerized application or as part of a containerized application. As described herein, agents 38 may selectively report information to data platform 12 in varying amounts of detail and/or with variable frequency.

Also shown in FIG. 1B is a load balancer 40 configured to perform one or more load balancing operations with respect to data ingestion operations performed by data ingestion resources 18 and/or user interface operations performed by user interface resources 22. Load balancer 40 is shown to be included in data platform 12. However, load balancer 40 may alternatively be located external to data platform 12. Load balancer 40 may be implemented by any suitable microservice, application, and/or other computing resources. In some alternative examples, data platform 12 may not utilize a load balancer such as load balancer 40.

Also shown in FIG. 1B is long term storage 42 with which data ingestion resources 18 may interface, as illustrated by arrow 44. Long term storage 42 may be implemented by any suitable type of storage resources, such as cloud-based storage (e.g., AWS S3, etc.) and/or on-premises storage and may be used by data ingestion resources 18 as part of the data ingestion process. Examples of this are described herein. In some examples, data platform 12 may not utilize long term storage 42.

The embodiments described herein can be implemented in numerous ways, including as a process; an apparatus; a system; a composition of matter; a computer program product embodied on a computer readable storage medium; and/or a processor, such as a processor configured to execute instructions stored on and/or provided by a memory coupled to the processor. In this specification, these implementations, or any other form that the invention may take, may be referred to as techniques. In general, the order of the steps of disclosed processes may be altered within the scope of the principles described herein. Unless stated otherwise, a component such as a processor or a memory described as being configured to perform a task may be implemented as a general component that is temporarily configured to perform the task at a given time or a specific component that is manufactured to perform the task. As used herein, the term 'processor' refers to one or more devices, circuits, and/or processing cores configured to process data, such as computer program instructions.

In some examples, a non-transitory computer-readable medium storing computer-readable instructions may be provided in accordance with the principles described herein. The instructions, when executed by a processor of a computing device, may direct the processor and/or computing device to perform one or more operations, including one or more of the operations described herein. Such instructions

may be stored and/or transmitted using any of a variety of known computer-readable media.

A non-transitory computer-readable medium as referred to herein may include any non-transitory storage medium that participates in providing data (e.g., instructions) that may be read and/or executed by a computing device (e.g., by a processor of a computing device). For example, a non-transitory computer-readable medium may include, but is not limited to, any combination of non-volatile storage media and/or volatile storage media. Exemplary non-volatile storage media include, but are not limited to, read-only memory, flash memory, a solid-state drive, a magnetic storage device (e.g., a hard disk, a floppy disk, magnetic tape, etc.), ferroelectric random-access memory ("RAM"), and an optical disc (e.g., a compact disc, a digital video disc, a Blu-ray disc, etc.). Exemplary volatile storage media include, but are not limited to, RAM (e.g., dynamic RAM).

FIG. 1C illustrates an example computing device 50 that may be specifically configured to perform one or more of the processes described herein. Any of the systems, microservices, computing devices, and/or other components described herein may be implemented by computing device 50.

As shown in FIG. 1C, computing device 50 may include a communication interface 52, a processor 54, a storage device 56, and an input/output ("I/O") module 58 communicatively connected one to another via a communication infrastructure 60. While an exemplary computing device 50 is shown in FIG. 1C, the components illustrated in FIG. 1C are not intended to be limiting. Additional or alternative components may be used in other embodiments. Components of computing device 50 shown in FIG. 1C will now be described in additional detail.

Communication interface 52 may be configured to communicate with one or more computing devices. Examples of communication interface 52 include, without limitation, a wired network interface (such as a network interface card), a wireless network interface (such as a wireless network interface card), a modem, an audio/video connection, and any other suitable interface.

Processor 54 generally represents any type or form of processing unit capable of processing data and/or interpreting, executing, and/or directing execution of one or more of the instructions, processes, and/or operations described herein. Processor 54 may perform operations by executing computer-executable instructions 62 (e.g., an application, software, code, and/or other executable data instance) stored in storage device 56.

Storage device 56 may include one or more data storage media, devices, or configurations and may employ any type, form, and combination of data storage media and/or device. For example, storage device 56 may include, but is not limited to, any combination of the non-volatile media and/or volatile media described herein. Electronic data, including data described herein, may be temporarily and/or permanently stored in storage device 56. For example, data representative of computer-executable instructions 62 configured to direct processor 54 to perform any of the operations described herein may be stored within storage device 56. In some examples, data may be arranged in one or more databases residing within storage device 56.

I/O module 58 may include one or more I/O modules configured to receive user input and provide user output. I/O module 58 may include any hardware, firmware, software, or combination thereof supportive of input and output capabilities. For example, I/O module 58 may include hardware and/or software for capturing user input, including, but not

limited to, a keyboard or keypad, a touchscreen component (e.g., touchscreen display), a receiver (e.g., an RF or infrared receiver), motion sensors, and/or one or more input buttons.

I/O module **58** may include one or more devices for presenting output to a user, including, but not limited to, a graphics engine, a display (e.g., a display screen), one or more output drivers (e.g., display drivers), one or more audio speakers, and one or more audio drivers. In certain embodiments, I/O module **58** is configured to provide graphical data to a display for presentation to a user. The graphical data may be representative of one or more graphical user interfaces and/or any other graphical content as may serve a particular implementation.

FIG. **1D** illustrates an example implementation **100** of configuration **10**. As such, one or more components shown in FIG. **1D** may implement one or more components shown in FIG. **1A** and/or FIG. **1B**. In particular, implementation **100** illustrates an environment in which activities that occur within datacenters are modeled using data platform **12**. Using techniques described herein, a baseline of datacenter activity can be modeled, and deviations from that baseline can be identified as anomalous. Anomaly detection can be beneficial in a security context, a compliance context, an asset management context, a DevOps context, and/or any other data analytics context as may serve a particular implementation.

Two example datacenters (**104** and **106**) are shown in FIG. **1D**, and are associated with (e.g., belong to) entities named entity A and entity B, respectively. A datacenter may include dedicated equipment (e.g., owned and operated by entity A, or owned/leased by entity A and operated exclusively on entity A's behalf by a third party). A datacenter can also include cloud-based resources, such as infrastructure as a service (IaaS), platform as a service (PaaS), and/or software as a service (SaaS) elements. The techniques described herein can be used in conjunction with multiple types of datacenters, including ones wholly using dedicated equipment, ones that are entirely cloud-based, and ones that use a mixture of both dedicated equipment and cloud-based resources.

Both datacenter **104** and datacenter **106** include a plurality of nodes, depicted collectively as set of nodes **108** and set of nodes **110**, respectively, in FIG. **1D**. These nodes may implement compute assets **16**. Installed on each of the nodes are in-server/in-virtual-machine (VM)/embedded-in-IoT device agents (e.g., agent **112**), which are configured to collect data and report it to data platform **12** for analysis. As described herein, agents may be small, self-contained binaries that can be run on any appropriate platforms, including virtualized ones (and, as applicable, within containers). Agents may monitor the nodes on which they execute for a variety of different activities, including: connection, process, user, machine, and file activities. Agents can be executed in user space, and can use a variety of kernel modules (e.g., auditd, iptables, netfilter, pcap, etc.) to collect data. Agents can be implemented in any appropriate programming language, such as C or Golang, using applicable kernel APIs.

As described herein, agents can selectively report information to data platform **12** in varying amounts of detail and/or with variable frequency. As is also described herein, the data collected by agents may be used by data platform **12** to create polygraphs, which are graphs of logical entities, connected by behaviors. In some embodiments, agents report information directly to data platform **12**. In other embodiments, at least some agents provide information to a data aggregator, such as data aggregator **114**, which in turn provides information to data platform **12**. The functionality

of a data aggregator can be implemented as a separate binary or other application (distinct from an agent binary), and can also be implemented by having an agent execute in an "aggregator mode" in which the designated aggregator node acts as a Layer 7 proxy for other agents that do not have access to data platform **12**. Further, a chain of multiple aggregators can be used, if applicable (e.g., with agent **112** providing data to data aggregator **114**, which in turn provides data to another aggregator (not pictured) which provides data to data platform **12**). An example way to implement an aggregator is through a program written in an appropriate language, such as C or Golang.

Use of an aggregator can be beneficial in sensitive environments (e.g., involving financial or medical transactions) where various nodes are subject to regulatory or other architectural requirements (e.g., prohibiting a given node from communicating with systems outside of datacenter **104**). Use of an aggregator can also help to minimize security exposure more generally. As one example, by limiting communications with data platform **12** to data aggregator **114**, individual nodes in nodes **108** need not make external network connections (e.g., via Internet **124**), which can potentially expose them to compromise (e.g., by other external devices, such as device **118**, operated by a criminal). Similarly, data platform **12** can provide updates, configuration information, etc., to data aggregator **114** (which in turn distributes them to nodes **108**), rather than requiring nodes **108** to allow incoming connections from data platform **12** directly.

Another benefit of an aggregator model is that network congestion can be reduced (e.g., with a single connection being made at any given time between data aggregator **114** and data platform **12**, rather than potentially many different connections being open between various of nodes **108** and data platform **12**). Similarly, network consumption can also be reduced (e.g., with the aggregator applying compression techniques/bundling data received from multiple agents).

One example way that an agent (e.g., agent **112**, installed on node **116**) can provide information to data aggregator **114** is via a REST API, formatted using data serialization protocols such as Apache Avro. One example type of information sent by agent **112** to data aggregator **114** is status information. Status information may be sent by an agent periodically (e.g., once an hour or once any other predetermined amount of time). Alternatively, status information may be sent continuously or in response to occurrence of one or more events. The status information may include, but is not limited to, a. an amount of event backlog (in bytes) that has not yet been transmitted, b. configuration information, c. any data loss period for which data was dropped, d. a cumulative count of errors encountered since the agent started, e. version information for the agent binary, and/or f. cumulative statistics on data collection (e.g., number of network packets processed, new processes seen, etc.).

A second example type of information that may be sent by agent **112** to data aggregator **114** is event data (described in more detail herein), which may include a UTC timestamp for each event. As applicable, the agent can control the amount of data that it sends to the data aggregator in each call (e.g., a maximum of 10 MB) by adjusting the amount of data sent to manage the conflicting goals of transmitting data as soon as possible, and maximizing throughput. Data can also be compressed or uncompressed by the agent (as applicable) prior to sending the data.

Each data aggregator may run within a particular customer environment. A data aggregator (e.g., data aggregator **114**) may facilitate data routing from many different agents

11

(e.g., agents executing on nodes **108**) to data platform **12**. In various embodiments, data aggregator **114** may implement a SOCKS **5** caching proxy through which agents can connect to data platform **12**. As applicable, data aggregator **114** can encrypt (or otherwise obfuscate) sensitive information prior to transmitting it to data platform **12**, and can also distribute key material to agents which can encrypt the information (as applicable). Data aggregator **114** may include a local storage, to which agents can upload data (e.g., pcap packets). The storage may have a key-value interface. The local storage can also be omitted, and agents configured to upload data to a cloud storage or other storage area, as applicable. Data aggregator **114** can, in some embodiments, also cache locally and distribute software upgrades, patches, or configuration information (e.g., as received from data platform **12**).

Various examples associated with agent data collection and reporting will now be described.

In the following example, suppose that a user (e.g., a network administrator) at entity A (hereinafter “user A”) has decided to begin using the services of data platform **12**. In some embodiments, user A may access a web frontend (e.g., web app **120**) using a computer **126** and enrolls (on behalf of entity A) an account with data platform **12**. After enrollment is complete, user A may be presented with a set of installers, pre-built and customized for the environment of entity A, that user A can download from data platform **12** and deploy on nodes **108**. Examples of such installers include, but are not limited to, a Windows executable file, an IOS app, a Linux package (e.g., .deb or .rpm), a binary, or a container (e.g., a Docker container). When a user (e.g., a network administrator) at entity B (hereinafter “user B”) also signs up for the services of data platform **12**, user B may be similarly presented with a set of installers that are pre-built and customized for the environment of entity B.

User A deploys an appropriate installer on each of nodes **108** (e.g., with a Windows executable file deployed on a Windows-based platform or a Linux package deployed on a Linux platform, as applicable). As applicable, the agent can be deployed in a container. Agent deployment can also be performed using one or more appropriate automation tools, such as Chef, Puppet, Salt, and Ansible. Deployment can also be performed using managed/hosted container management/orchestration frameworks such as Kubernetes, Mesos, and/or Docker Swarm.

In various embodiments, the agent may be installed in the user space (i.e., is not a kernel module), and the same binary is executed on each node of the same type (e.g., all Windows-based platforms have the same Windows-based binary installed on them). An illustrative function of an agent, such as agent **112**, is to collect data (e.g., associated with node **116**) and report it (e.g., to data aggregator **114**). Other tasks that can be performed by agents include data configuration and upgrading.

One approach to collecting data as described herein is to collect virtually all information available about a node (and, e.g., the processes running on it). Alternatively, the agent may monitor for network connections, and then begin collecting information about processes associated with the network connections, using the presence of a network packet associated with a process as a trigger for collecting additional information about the process. As an example, if a user of node **116** executes an application, such as a calculator application, which does not typically interact with the network, no information about use of that application may be collected by agent **112** and/or sent to data aggregator **114**. If, however, the user of node **116** executes an ssh command

12

(e.g., to ssh from node **116** to node **122**), agent **112** may collect information about the process and provide associated information to data aggregator **114**. In various embodiments, the agent may always collect/report information about certain events, such as privilege escalation, irrespective of whether the event is associated with network activity.

An approach to collecting information (e.g., by an agent) is as follows, and described in conjunction with process **200** depicted in FIG. 2A. An agent (e.g., agent **112**) monitors its node (e.g., node **116**) for network activity. One example way that agent **112** can monitor node **116** for network activity is by using a network packet capture tool (e.g., listening using libpcap). As packets are received (**201**), the agent obtains and maintains (e.g., in an in-memory cache) connection information associated with the network activity (**202**). Examples of such information include DNS query/response, TCP, UDP, and IP information.

The agent may also determine a process associated with the network connection (**203**). One example approach is for the agent to use a kernel network diagnostic API (e.g., netlink_diag) to obtain inode/process information from the kernel. Another example approach is for the agent to scan using netstat (e.g., on /proc/net/tcp, /proc/net/tcp6, /proc/net/udp, and /proc/net/udp6) to obtain sockets and relate them to processes. Information such as socket state (e.g., whether a socket is connected, listening, etc.) can also be collected by the agent.

One way an agent can obtain a mapping between a given inode and a process identifier is to scan within the /proc/pid directory. For each of the processes currently running, the agent examines each of their file descriptors. If a file descriptor is a match for the inode, the agent can determine that the process associated with the file descriptor owns the inode. Once a mapping is determined between an inode and a process identifier, the mapping is cached. As additional packets are received for the connection, the cached process information is used (rather than a new search being performed).

In some cases, exhaustively scanning for an inode match across every file descriptor may not be feasible (e.g., due to CPU limitations). In various embodiments, searching through file descriptors is accordingly optimized. User filtering is one example of such an optimization. A given socket is owned by a user. Any processes associated with the socket will be owned by the same user as the socket. When matching an inode (identified as relating to a given socket) against processes, the agent can filter through the processes and only examine the file descriptors of processes sharing the same user owner as the socket. In various embodiments, processes owned by root are always searched against (e.g., even when user filtering is employed).

Another example of an optimization is to prioritize searching the file descriptors of certain processes over others. One such prioritization is to search through the subdirectories of /proc/ starting with the youngest process. One approximation of such a sort order is to search through /proc/ in reverse order (e.g., examining highest numbered processes first). Higher numbered processes are more likely to be newer (i.e., not long-standing processes), and thus more likely to be associated with new connections (i.e., ones for which inode-process mappings are not already cached). In some cases, the most recently created process may not have the highest process identifier (e.g., due to the kernel wrapping through process identifiers).

Another example prioritization is to query the kernel for an identification of the most recently created process and to search in a backward order through the directories in /proc/

(e.g., starting at the most recently created process and working backwards, then wrapping to the highest value (e.g., 32768) and continuing to work backward from there). An alternate approach is for the agent to keep track of the newest process that it has reported information on (e.g., to data aggregator **114**), and begin its search of /proc/ in a forward order starting from the PID of that process.

Another example prioritization is to maintain, for each user actively using node **116**, a list of the five (or any other number) most recently active processes. Those processes are more likely than other processes (less active, or passive) on node **116** to be involved with new connections, and can thus be searched first. For many processes, lower valued file descriptors tend to correspond to non-sockets (e.g., stdin, stdout, stderr). Yet another optimization is to preferentially search higher valued file descriptors (e.g., across processes) over lower valued file descriptors (that are less likely to yield matches).

In some cases, while attempting to locate a process identifier for a given inode, an agent may encounter a socket that does not correspond to the inode being matched against and is not already cached. The identity of that socket (and its corresponding inode) can be cached, once discovered, thus removing a future need to search for that pair.

In some cases, a connection may terminate before the agent is able to determine its associated process (e.g., due to a very short-lived connection, due to a backlog in agent processing, etc.). One approach to addressing such a situation is to asynchronously collect information about the connection using the audit kernel API, which streams information to user space. The information collected from the audit API (which can include PID/inode information) can be matched by the agent against pcap/inode information. In some embodiments, the audit API is always used, for all connections. However, due to CPU utilization considerations, use of the audit API can also be reserved for short/otherwise problematic connections (and/or omitted, as applicable).

Once the agent has determined which process is associated with the network connection (**203**), the agent can then collect additional information associated with the process (**204**). As will be described in more detail below, some of the collected information may include attributes of the process (e.g., a process parent hierarchy, and an identification of a binary associated with the process). As will also be described in more detail below, other of the collected information is derived (e.g., session summarization data and hash values).

The collected information is then transmitted (**205**), e.g., by an agent (e.g., agent **112**) to a data aggregator (e.g., data aggregator **114**), which in turn provides the information to data platform **12**. In some embodiments, all information collected by an agent may be transmitted (e.g., to a data aggregator and/or to data platform **12**). In other embodiments, the amount of data transmitted may be minimized (e.g., for efficiency reasons), using various techniques.

One approach to minimizing the amount of data flowing from agents (such as agents installed on nodes **108**) to data platform **12** is to use a technique of implicit references with unique keys. The keys can be explicitly used by data platform **12** to extract/derive relationships, as necessary, in a data set at a later time, without impacting performance.

As previously mentioned, some data collected about a process is constant and does not change over the lifetime of the process (e.g., attributes), and some data changes (e.g., statistical information and other variable information). Constant data can be transmitted (**205**) once, when the agent first

becomes aware of the process. And, if any changes to the constant data are detected (e.g., a process changes its parent), a refreshed version of the data can be transmitted (**205**) as applicable.

In some examples, an agent may collect variable data (e.g., data that may change over the lifetime of the process). In some examples, variable data can be transmitted (**205**) at periodic (or other) intervals. Alternatively, variable data may be transmitted in substantially real time as it is collected. In some examples, the variable data may indicate a thread count for a process, a total virtual memory used by the process, the total resident memory used by the process, the total time spent by the process executing in user space, and/or the total time spent by the process executing in kernel space. In some examples, the data may include a hash that may be used within data platform **12** to join process creation time attributes with runtime attributes to construct a full dataset.

Below are additional examples of data that an agent, such as agent **112**, can collect and provide to data platform **12**.

1. User Data

Core User Data: user name, UID (user ID), primary group, other groups, home directory. Failed Login Data: IP address, hostname, username, count.

User Login Data: user name, hostname, IP address, start time, TTY (terminal), UID (user ID), GID (group ID), process, end time.

2. Machine Data

Dropped Packet Data: source IP address, destination IP address, destination port, protocol, count.

Machine Data: hostname, domain name, architecture, kernel, kernel release, kernel version, OS, OS version, OS description, CPU, memory, model number, number of cores, last boot time, last boot reason, tags (e.g., Cloud provider tags such as AWS, GCP, or Azure tags), default router, interface name, interface hardware address, interface IP address and mask, promiscuous mode.

3. Network Data

Network Connection Data: source IP address, destination IP address, source port, destination port, protocol, start time, end time, incoming and outgoing bytes, source process, destination process, direction of connection, histograms of packet length, inter packet delay, session lengths, etc.

Listening Ports in Server: source IP address, port number, protocol, process.

Dropped Packet Data: source IP address, destination IP address, destination port, protocol, count.

Arp Data: source hardware address, source IP address, destination hardware address, destination IP address.

DNS Data: source IP address, response code, response string, question (request), packet length, final answer (response).

4. Application Data

Package Data: exe path, package name, architecture, version, package path, checksums (MD5, SHA-1, SHA-256), size, owner, owner ID.

Application Data: command line, PID (process ID), start time, UID (user ID), EUID (effective UID), PPID (parent process ID), PGID (process group ID), SID (session ID), exe path, username, container ID.

5. Container Data

Container Image Data: image creation time, parent ID, author, container type, repo, (AWS) tags, size, virtual size, image version.

Container Data: container start time, container type, container name, container ID, network mode, privileged, PID mode, IP addresses, listening ports, volume map, process ID.

6. File Data

File path, file data hash, symbolic links, file creation data, file change data, file metadata, file mode.

As mentioned above, an agent, such as agent **112**, can be deployed in a container (e.g., a Docker container), and can also be used to collect information about containers. Collection about a container can be performed by an agent irrespective of whether the agent is itself deployed in a container or not (as the agent can be deployed in a container running in a privileged mode that allows for monitoring).

Agents can discover containers (e.g., for monitoring) by listening for container create events (e.g., provided by Docker), and can also perform periodic ordered discovery scans to determine whether containers are running on a node. When a container is discovered, the agent can obtain attributes of the container, e.g., using standard Docker API calls (e.g., to obtain IP addresses associated with the container, whether there's a server running inside, what port it is listening on, associated PIDs, etc.). Information such as the parent process that started the container can also be collected, as can information about the image (which comes from the Docker repository).

In various embodiments, agents may use namespaces to determine whether a process is associated with a container. Namespaces are a feature of the Linux kernel that can be used to isolate resources of a collection of processes. Examples of namespaces include process ID (PID) namespaces, network namespaces, and user namespaces. Given a process, the agent can perform a fast lookup to determine whether the process is part of the namespace the container claims to be its namespace.

As mentioned, agents can be configured to report certain types of information (e.g., attribute information) once, when the agent first becomes aware of a process. In various embodiments, such static information is not reported again (or is reported once a day, every twelve hours, etc.), unless it changes (e.g., a process changes its parent, changes its owner, or a SHA-1 of the binary associated with the process changes).

In contrast to static/attribute information, certain types of data change constantly (e.g., network-related data). In various embodiments, agents are configured to report a list of current connections every minute (or other appropriate time interval). In that connection list will be connections that started in that minute interval, connections that ended in that minute interval, and connections that were ongoing throughout the minute interval (e.g., a one minute slice of a one hour connection).

In various embodiments, agents are configured to collect/compute statistical information about connections (e.g., at the one minute level of granularity and or at any other time interval). Examples of such information include, for the time interval, the number of bytes transferred, and in which direction. Another example of information collected by an agent about a connection is the length of time between packets. For connections that span multiple time intervals (e.g., a seven minute connection), statistics may be calculated for each minute of the connection. Such statistical information (for all connections) can be reported (e.g., to a data aggregator) once a minute.

In various embodiments, agents are also configured to maintain histogram data for a given network connection, and provide the histogram data (e.g., in the Apache Avro data exchange format) under the Connection event type data. Examples of such histograms include: 1. a packet length histogram (packet_len_hist), which characterizes network packet distribution; 2. a session length histogram (session_

len_hist), which characterizes a network session length; 3. a session time histogram (session_time_hist), which characterizes a network session time; and 4. a session switch time histogram (session_switch_time_hist), which characterizes network session switch time (i.e., incoming->outgoing and vice versa). For example, histogram data may include one or more of the following fields: 1. count, which provides a count of the elements in the sampling; 2. sum, which provides a sum of elements in the sampling; 3. max, which provides the highest value element in the sampling; 4. std_dev, which provides the standard deviation of elements in the sampling; and 5. buckets, which provides a discrete sample bucket distribution of sampling data (if applicable).

For some protocols (e.g., HTTP), typically, a connection is opened, a string is sent, a string is received, and the connection is closed. For other protocols (e.g., NFS), both sides of the connection engage in a constant chatter. Histograms allow data platform **12** to model application behavior (e.g., using machine learning techniques), for establishing baselines, and for detecting deviations. As one example, suppose that a given HTTP server typically sends/receives 1,000 bytes (in each direction) whenever a connection is made with it. If a connection generates 500 bytes of traffic, or 2,000 bytes of traffic, such connections would be considered within the typical usage pattern of the server. Suppose, however, that a connection is made that results in 10G of traffic. Such a connection is anomalous and can be flagged accordingly.

Returning to FIG. 1D, as previously mentioned, data aggregator **114** may be configured to provide information (e.g., collected from nodes **108** by agents) to data platform **12**. Data aggregator **128** may be similarly configured to provide information to data platform **12**. As shown in FIG. 1D, both aggregator **114** and aggregator **128** may connect to a load balancer **130**, which accepts connections from aggregators (and/or as applicable, agents), as well as other devices, such as computer **126** (e.g., when it communicates with web app **120**), and supports fair balancing. In various embodiments, load balancer **130** is a reverse proxy that load balances accepted connections internally to various microservices (described in more detail below), allowing for services provided by data platform **12** to scale up as more agents are added to the environment and/or as more entities subscribe to services provided by data platform **12**. Example ways to implement load balancer **130** include, but are not limited to, using HaProxy, using nginx, and using elastic load balancing (ELB) services made available by Amazon.

Agent service **132** is a microservice that is responsible for accepting data collected from agents (e.g., provided by aggregator **114**). In various embodiments, agent service **132** uses a standard secure protocol, such as HTTPS to communicate with aggregators (and, as applicable, agents), and receives data in an appropriate format such as Apache Avro. When agent service **132** receives an incoming connection, it can perform a variety of checks, such as to see whether the data is being provided by a current customer, and whether the data is being provided in an appropriate format. If the data is not appropriately formatted (and/or is not provided by a current customer), it may be rejected.

If the data is appropriately formatted, agent service **132** may facilitate copying the received data to a streaming data stable storage using a streaming service (e.g., Amazon Kinesis and/or any other suitable streaming service). Once the ingesting into the streaming service is complete, agent service **132** may send an acknowledgement to the data provider (e.g., data aggregator **114**). If the agent does not receive such an acknowledgement, it is configured to retry

17

sending the data to data platform **12**. One way to implement agent service **132** is as a REST API server framework (e.g., Java DropWizard), configured to communicate with Kinesis (e.g., using a Kinesis library).

In various embodiments, data platform **12** uses one or more streams (e.g., Kinesis streams) for all incoming customer data (e.g., including data provided by data aggregator **114** and data aggregator **128**), and the data is sharded based on the node (also referred to herein as a “machine”) that originated the data (e.g., node **116** vs. node **122**), with each node having a globally unique identifier within data platform **12**. Multiple instances of agent service **132** can write to multiple shards.

Kinesis is a streaming service with a limited period (e.g., 1-7 days). To persist data longer than a day, the data may be copied to long term storage **42** (e.g., S3). Data loader **136** is a microservice that is responsible for picking up data from a data stream (e.g., a Kinesis stream) and persisting it in long term storage **42**. In one example embodiment, files collected by data loader **136** from the Kinesis stream are placed into one or more buckets, and segmented using a combination of a customer identifier and time slice. Given a particular time segment, and a given customer identifier, the corresponding file (stored in long term storage) contains five minutes (or another appropriate time slice) of data collected at that specific customer from all of the customer’s nodes. Data loader **136** can be implemented in any appropriate programming language, such as Java or C, and can be configured to use a Kinesis library to interface with Kinesis. In various embodiments, data loader **136** uses the Amazon Simple Queue Service (SQS) (e.g., to alert DB loader **140** that there is work for it to do).

DB loader **140** is a microservice that is responsible for loading data into an appropriate data store **30**, such as SnowflakeDB or Amazon Redshift, using individual per-customer databases. In particular, DB loader **140** is configured to periodically load data into a set of raw tables from files created by data loader **136** as per above. DB loader **140** manages throughput, errors, etc., to make sure that data is loaded consistently and continuously. Further, DB loader **140** can read incoming data and load into data store **30** data that is not already present in tables of data store **30** (also referred to herein as a database). DB loader **140** can be implemented in any appropriate programming language, such as Java or C, and an SQL framework such as jOOQ (e.g., to manage SQLs for insertion of data), and SQL/JDBC libraries. In some examples, DB loader **140** may use Amazon S3 and Amazon Simple Queue Service (SQS) to manage files being transferred to and from data store **30**.

Customer data included in data store **30** can be augmented with data from additional data sources, such as AWS CloudTrail and/or other types of external tracking services. To this end, data platform may include a tracking service analyzer **144**, which is another microservice. Tracking service analyzer **144** may pull data from an external tracking service (e.g., Amazon CloudTrail) for each applicable customer account, as soon as the data is available. Tracking service analyzer **144** may normalize the tracking data as applicable, so that it can be inserted into data store **30** for later querying/analysis. Tracking service analyzer **144** can be written in any appropriate programming language, such as Java or C. Tracking service analyzer **144** also makes use of SQL/JDBC libraries to interact with data store **30** to insert/query data.

As described herein, data platform **12** can model activities that occur within datacenters, such as datacenters **104** and **106**. The model may be stable over time, and differences,

18

even subtle ones (e.g., between a current state of the datacenter and the model) can be surfaced. The ability to surface such anomalies can be particularly beneficial in datacenter environments where rogue employees and/or external attackers may operate slowly (e.g., over a period of months), hoping that the elastic nature of typical resource use (e.g., virtualized servers) will help conceal their nefarious activities.

Using techniques described herein, data platform **12** can automatically discover entities (which may implement compute assets **16**) deployed in a given datacenter. Examples of entities include workloads, applications, processes, machines, virtual machines, containers, files, IP addresses, domain names, and users. The entities may be grouped together logically (into analysis groups) based on behaviors, and temporal behavior baselines can be established. In particular, using techniques described herein, periodic graphs can be constructed (also referred to herein as poly-graphs), in which the nodes are applicable logical entities, and the edges represent behavioral relationships between the logical entities in the graph. Baselines can be created for every node and edge.

Communication (e.g., between applications/nodes) is one example of a behavior. A model of communications between processes is an example of a behavioral model. As another example, the launching of applications is another example of a behavior that can be modeled. The baselines may be periodically updated (e.g., hourly) for every entity. Additionally or alternatively, the baselines may be continuously updated in substantially real-time as data is collected by agents. Deviations from the expected normal behavior can then be detected and automatically reported (e.g., as anomalies or threats detected). Such deviations may be due to a desired change, a misconfiguration, or malicious activity. As applicable, data platform **12** can score the detected deviations (e.g., based on severity and threat posed). Additional examples of analysis groups include models of machine communications, models of privilege changes, and models of insider behaviors (monitoring the interactive behavior of human users as they operate within the datacenter).

Two example types of information collected by agents are network level information and process level information. As previously mentioned, agents may collect information about every connection involving their respective nodes. And, for each connection, information about both the server and the client may be collected (e.g., using the connection-to-process identification techniques described above). DNS queries and responses may also be collected. The DNS query information can be used in logical entity graphing (e.g., collapsing many different IP addresses to a single service—e.g., s3.amazonaws.com). Examples of process level information collected by agents include attributes (user ID, effective user ID, and command line). Information such as what user/application is responsible for launching a given process and the binary being executed (and its SHA-256 values) may also be provided by agents.

The dataset collected by agents across a datacenter can be very large, and many resources (e.g., virtual machines, IP addresses, etc.) are recycled very quickly. For example, an IP address and port number used at a first point in time by a first process on a first virtual machine may very rapidly be used (e.g., an hour later) by a different process/virtual machine.

A dataset (and elements within it) can be considered at both a physical level, and a logical level, as illustrated in FIG. 2B. In particular, FIG. 2B illustrates an example 5-tuple of data **210** collected by an agent, represented

19

physically (216) and logically (217). The 5-tuple includes a source address 211, a source port 212, a destination address 213, a destination port 214, and a protocol 215. In some cases, port numbers (e.g., 212, 214) may be indicative of the nature of a connection (e.g., with certain port usage standardized). However, in many cases, and in particular in datacenters, port usage is ephemeral. For example, a Docker container can listen on an ephemeral port, which is unrelated to the service it will run. When another Docker container starts (for the same service), the port may well be different. Similarly, particularly in a virtualized environment, IP addresses may be recycled frequently (and are thus also potentially ephemeral) or could be NATed, which makes identification difficult.

A physical representation of the 5-tuple is depicted in region 216. A process 218 (executing on machine 219) has opened a connection to machine 220. In particular, process 218 is in communication with process 221. Information such as the number of packets exchanged between the two machines over the respective ports can be recorded.

As previously mentioned, in a datacenter environment, portions of the 5-tuple may change-potentially frequently—but still be associated with the same behavior. Namely, one application (e.g., Apache) may frequently be in communication with another application (e.g., Oracle), using ephemeral datacenter resources. Further, either/both of Apache and Oracle may be multi-homed. This can lead to potentially thousands of 5-tuples (or more) that all correspond to Apache communicating with Oracle within a datacenter. For example, Apache could be executed on a single machine, and could also be executed across fifty machines, which are variously spun up and down (with different IP addresses each time). An alternate representation of the 5-tuple of data 210 is depicted in region 217, and is logical. The logical representation of the 5-tuple aggregates the 5-tuple (along with other connections between Apache and Oracle having other 5-tuples) as logically representing the same connection. By aggregating data from raw physical connection information into logical connection information, using techniques described herein, a size reduction of six orders of magnitude in the data set can be achieved.

FIG. 2C depicts a portion of a logical polygraph. Suppose a datacenter has seven instances of the application `update_engine` 225, executing as seven different processes on seven different machines, having seven different IP addresses, and using seven different ports. The instances of `update_engine` variously communicate with `update.core-os.net` 226, which may have a single IP address or many IP addresses itself, over the one hour time period represented in the polygraph. In the example shown in FIG. 2C, `update_engine` is a client, connecting to the server `update.core-os.net`, as indicated by arrow 228.

Behaviors of the seven processes are clustered together, into a single summary. As indicated in region 227, statistical information about the connections is also maintained (e.g., number of connections, histogram information, etc.). A polygraph such as is depicted in FIG. 2C can be used to establish a baseline of behavior (e.g., at the one-hour level), allowing for the future detection of deviations from that baseline. As one example, suppose that statistically an `update_engine` instance transmits data at 11 bytes per second. If an instance were instead to transmit data at 1000 bytes per second, such behavior would represent a deviation from the baseline and could be flagged accordingly. Similarly, changes that are within the baseline (e.g., an eighth instance of `update_engine` appears, but otherwise behaves as the other instances; or one of the seven instances disappears)

20

are not flagged as anomalous. Further, datacenter events, such as failover, autobalancing, and A-B refresh are unlikely to trigger false alarms in a polygraph, as at the logical level, the behaviors remain the same.

In various embodiments, polygraph data is maintained for every application in a datacenter, and such polygraph data can be combined to make a single datacenter view across all such applications. FIG. 2D illustrates a portion of a polygraph for a service that evidences more complex behaviors than are depicted in FIG. 2C. In particular, FIG. 2D illustrates the behaviors of S3 as a service (as used by a particular customer datacenter). Clients within the datacenter variously connect to the S3 service using one of five fully qualified domains (listed in region 230). Contact with any of the domains is aggregated as contact with S3 (as indicated in region 231). Depicted in region 232 are various containers which (as clients) connect with S3. Other containers (which do not connect with S3) are not included. As with the polygraph portion depicted in FIG. 2C, statistical information about the connections is known and summarized, such as the number of bytes transferred, histogram information, etc.

FIG. 2E illustrates a communication polygraph for a datacenter. In particular, the polygraph indicates a one hour summary of approximately 500 virtual machines, which collectively run one million processes, and make 100 million connections in that hour. As illustrated in FIG. 2E, a polygraph represents a drastic reduction in size (e.g., from tracking information on 100 million connections in an hour, to a few hundred nodes and a few hundred edges). Further, as a datacenter scales up (e.g., from using 10 virtual machines to 100 virtual machines as the datacenter uses more workers to support existing applications), the polygraph for the datacenter will tend to stay the same size (with the 100 virtual machines clustering into the same nodes that the 10 virtual machines previously clustered into). As new applications are added into the datacenter, the polygraph may automatically scale to include behaviors involving those applications.

In the particular polygraph shown in FIG. 2E, nodes generally correspond to workers, and edges correspond to communications the workers engage in (with connection activity being the behavior modeled in polygraph 235). Another example polygraph could model other behavior, such as application launching. The communications graphed in FIG. 2E include traffic entering the datacenter, traffic exiting the datacenter, and traffic that stays wholly within the datacenter (e.g., traffic between workers). One example of a node included in polygraph 235 is the `sshd` application, depicted as node 236. As indicated in FIG. 2E, 421 instances of `sshd` were executing during the one hour time period of data represented in polygraph 235. As indicated in region 237, nodes within the datacenter communicated with a total of 1349 IP addresses outside of the datacenter (and not otherwise accounted for, e.g., as belonging to a service such as Amazon AWS 238 or Slack 239).

In the following examples, suppose that user B, an administrator of datacenter 106, is interacting with data platform 12 to view visualizations of polygraphs in a web browser (e.g., as served to user B via web app 120). One type of polygraph user B can view is an application-communication polygraph, which indicates, for a given one hour window (or any other suitable time interval), which applications communicated with which other applications. Another type of polygraph user B can view is an application launch polygraph. User B can also view graphs related to user behavior, such as an insider behavior graph which

21

tracks user connections (e.g., to internal and external applications, including chains of such behavior), a privilege change graph which tracks how privileges change between processes, and a user login graph, which tracks which (logical) machines a user logs into.

FIG. 2F illustrates an example of an application-communication polygraph for a datacenter (e.g., datacenter 106) for the one hour period of 9 am-10 am on June 5. The time slice currently being viewed is indicated in region 240. If user B clicks his mouse in region 241, user B will be shown a representation of the application-communication polygraph as generated for the following hour (10 am-11 am on June 5).

FIG. 2G depicts what is shown in user B's browser after he has clicked on region 241, and has further clicked on region 242. The selection in region 242 turns on and off the ability to compare two time intervals to one another. User B can select from a variety of options when comparing the 9 am-10 am and 10 am-11 am time intervals. By clicking region 248, user B will be shown the union of both graphs (i.e., any connections that were present in either time interval). By clicking region 249, user B will be shown the intersection of both graphs (i.e., only those connections that were present in both time intervals).

As shown in FIG. 2G, user B has elected to click on region 250, which depicts connections that are only present in the 9 am-10 am polygraph in a first color 251, and depicts connections that are only present in the 10 am-11 am polygraph in a second color 252. Connections present in both polygraphs are omitted from display. As one example, in the 9 am-10 am polygraph (corresponding to connections made during the 9 am-10 am time period at datacenter 106), a connection was made by a server to sshd (253) and also to systemd (254). Both of those connections ended prior to 10 am and are thus depicted in the first color. As another example, in the 10 am-11 am polygraph (corresponding to connections made during the 10 am-11 am time period at datacenter 106), a connection was made from a known bad external IP to nginx (255). The connection was not present during the 9 am-10 am time slice and thus is depicted in the second color. As yet another example, two different connections were made to a Slack service between 9 am and 11 am. However, the first was made by a first client during the 9 am-10 am time slice (256) and the second was made by a different client during the 10 am-11 am slice (257), and so the two connections are depicted respectively in the first and second colors and blue.

Returning to the polygraph depicted in FIG. 2F, suppose user B enters "etcd" into the search box located in region 244. User B will then be presented with the interface illustrated in FIG. 2H. As shown in FIG. 2H, three applications containing the term "etcd" were engaged in communications during the 9 am-10 am window. One application is etcdctl, a command line client for etcd. As shown in FIG. 2H, a total of three different etcdctl processes were executed during the 9 am-10 am window, and were clustered together (260). FIG. 2H also depicts two different clusters that are both named etcd2. The first cluster includes (for the 9 am-10 am window) five members (261) and the second cluster includes (for the same window) eight members (262). The reason for these two distinct clusters is that the two groups of applications behave differently (e.g., they exhibit two distinct sets of communication patterns). Specifically, the instances of etcd2 in cluster 261 only communicate with locksmithctl (263) and other etcd2 instances (in both clusters 261 and 262). The instances of etcd2 in cluster 262 communicate with additional entities, such as etcdctl and Docker

22

containers. As desired, user B can click on one of the clusters (e.g., cluster 261) and be presented with summary information about the applications included in the cluster, as is shown in FIG. 2I (e.g., in region 265). User B can also double click on a given cluster (e.g., cluster 261) to see details on each of the individual members of the cluster broken out.

Suppose user B now clicks on region 245 of the interface shown in FIG. 2F. User B will then be shown an application launch polygraph. Launching an application is another example of a behavior. The launch polygraph models how applications are launched by other applications. FIG. 2J illustrates an example of a portion of a launch polygraph. In particular, user B has typed "find" into region 266, to see how the "find" application is being launched. As shown in FIG. 2J, in the launch polygraph for the 10 am-11 am time period, find applications (267) are always launched by bash (268), which is in turn always launched by systemd (269). If find is launched by a different application, this would be anomalous behavior.

FIG. 2K illustrates another example of a portion of an application launch polygraph. In FIG. 2K, user B has searched (270) for "python ma" to see how "python marathon_lb" (271) is launched. As shown in FIG. 2K, in each case (during the one hour time slice of 10 am-11 am), python marathon_lb is launched as a result of a chain of the same seven applications each time. If python marathon_lb is ever launched in a different manner, this indicates anomalous behavior. The behavior could be indicative of malicious activities, but could also be due to other reasons, such as a misconfiguration, a performance-related issue, and/or a failure, etc.

Suppose user B now clicks on region 246 of the interface shown in FIG. 2F. User B will then be shown an insider behavior graph. The insider behavior graph tracks information about behaviors such as processes started by a user interactively using protocols such as ssh or telnet, and any processes started by those processes. As one example, suppose an administrator logs into a first virtual machine in datacenter 106 (e.g., using sshd via an external connection he makes from a hotel), using a first set of credentials (e.g., first.last@example.com and an appropriate password). From the first virtual machine, the administrator connects to a second virtual machine (e.g., using the same credentials), then uses the sudo command to change identities to those of another user, and then launches a program. Graphs built by data platform 12 can be used to associate the administrator with each of his actions, including launching the program using the identity of another user.

FIG. 2L illustrates an example of a portion of an insider behavior graph. In particular, in FIG. 2L, user B is viewing a graph that corresponds to the time slice of 3 pm-4 pm on June 1. FIG. 2L illustrates the internal/external applications that users connected to during the one hour time slice. If a user typically communicates with particular applications, that information will become part of a baseline. If the user deviates from his baseline behavior (e.g., using new applications, or changing privilege in anomalous ways), such anomalies can be surfaced.

FIG. 2M illustrates an example of a portion of a privilege change graph, which identifies how privileges are changed between processes. Typically, when a user launches a process (e.g., "ls"), the process inherits the same privileges that the user has. And, while a process can have fewer privileges than the user (i.e., go down in privilege), it is rare (and generally undesirable) for a user to escalate in privilege. Information included in the privilege change graph can be

determined by examining the parent of each running process, and determining whether there is a match in privilege between the parent and the child. If the privileges are different, a privilege change has occurred (whether a change up or a change down). The application `ntpd` is one rare example of a scenario in which a process escalates (272) to root, and then returns back (273). The `sudo` command is another example (e.g., used by an administrator to temporarily have a higher privilege). As with the other examples, `ntpd`'s privilege change actions, and the legitimate actions of various administrators (e.g., using `sudo`) will be incorporated into a baseline model by data platform 12. When deviations occur, such as where a new application that is not `ntpd` escalates privilege, or where an individual that has not previously/does not routinely use `sudo` does so, such behaviors can be identified as anomalous.

FIG. 2N illustrates an example of a portion of a user login graph, which identifies which users log into which logical nodes. Physical nodes (whether bare metal or virtualized) are clustered into a logical machine cluster, for example, using yet another graph, a machine-server graph, an example of which is shown in FIG. 2O. For each machine, a determination is made as to what type of machine it is, based on what kind(s) of workflows it runs. As one example, some machines run as master nodes (having a typical set of workflows they run, as master nodes) and can thus be clustered as master nodes. Worker nodes are different from master nodes, for example, because they run Docker containers, and frequently change as containers move around. Worker nodes can similarly be clustered.

As previously mentioned, the polygraph depicted in FIG. 2E corresponds to activities in a datacenter in which, in a given hour, approximately 500 virtual machines collectively run once million processes, and make 100 million connections in that hour. The polygraph represents a drastic reduction in size (e.g., from tracking information on 100 million connections in an hour, to a few hundred nodes and a few hundred edges). Using techniques described herein, such a polygraph can be constructed (e.g., using commercially available computing infrastructure) in less than an hour (e.g., within a few minutes). Thus, ongoing hourly snapshots of a datacenter can be created within a two hour moving window (i.e., collecting data for the time period 8 am-9 am, while also generating a snapshot for the time previous time period 7 am-8 am). The following describes various example infrastructure that can be used in polygraph construction, and also describes various techniques that can be used to construct polygraphs.

Returning to FIG. 1D, embodiments of data platform 12 may be built using any suitable infrastructure as a service (IaaS) (e.g., AWS). For example, data platform 12 can use Simple Storage Service (S3) for data storage, Key Management Service (KMS) for managing secrets, Simple Queue Service (SQS) for managing messaging between applications, Simple Email Service (SES) for sending emails, and Route 53 for managing DNS. Other infrastructure tools can also be used. Examples include: orchestration tools (e.g., Kubernetes or Mesos/Marathon), service discovery tools (e.g., Mesos-DNS), service load balancing tools (e.g., marathon-LB), container tools (e.g., Docker or rkt), log/metric tools (e.g., collectd, fluentd, kibana, etc.), big data processing systems (e.g., Spark, Hadoop, AWS Redshift, Snowflake etc.), and distributed key value stores (e.g., Apache Zookeeper or etcd2).

As previously mentioned, in various embodiments, data platform 12 may make use of a collection of microservices. Each microservice can have multiple instances, and may be

configured to recover from failure, scale, and distribute work amongst various such instances, as applicable. For example, microservices are auto-balancing for new instances, and can distribute workload if new instances are started or existing instances are terminated. In various embodiments, microservices may be deployed as self-contained Docker containers. A Mesos-Marathon or Spark framework can be used to deploy the microservices (e.g., with Marathon monitoring and restarting failed instances of microservices as needed). The service `etcd2` can be used by microservice instances to discover how many peer instances are running, and used for calculating a hash-based scheme for workload distribution. Microservices may be configured to publish various health/status metrics to either an SQS queue, or `etcd2`, as applicable. In some examples, Amazon DynamoDB can be used for state management.

Additional information on various microservices used in embodiments of data platform 12 is provided below.

Graph generator 146 is a microservice that may be responsible for generating raw behavior graphs on a per customer basis periodically (e.g., once an hour). In particular, graph generator 146 may generate graphs of entities (as the nodes in the graph) and activities between entities (as the edges). In various embodiments, graph generator 146 also performs other functions, such as aggregation, enrichment (e.g., geolocation and threat), reverse DNS resolution, TF-IDF based command line analysis for command type extraction, parent process tracking, etc.

Graph generator 146 may perform joins on data collected by the agents, so that both sides of a behavior are linked. For example, suppose a first process on a first virtual machine (e.g., having a first IP address) communicates with a second process on a second virtual machine (e.g., having a second IP address). Respective agents on the first and second virtual machines may each report information on their view of the communication (e.g., the PID of their respective processes, the amount of data exchanged and in which direction, etc.). When graph generator performs a join on the data provided by both agents, the graph will include a node for each of the processes, and an edge indicating communication between them (as well as other information, such as the directionality of the communication—i.e., which process acted as the server and which as the client in the communication).

In some cases, connections are process to process (e.g., from a process on one virtual machine within the cloud environment associated with entity A to another process on a virtual machine within the cloud environment associated with entity A). In other cases, a process may be in communication with a node (e.g., outside of entity A) which does not have an agent deployed upon it. As one example, a node within entity A might be in communication with node 172, outside of entity A. In such a scenario, communications with node 172 are modeled (e.g., by graph generator 146) using the IP address of node 172. Similarly, where a node within entity A does not have an agent deployed upon it, the IP address of the node can be used by graph generator in modeling.

Graphs created by graph generator 146 may be written to data store 30 and cached for further processing. A graph may be a summary of all activity that happened in a particular time interval. As each graph corresponds to a distinct period of time, different rows can be aggregated to find summary information over a larger timestamp. In some examples, picking two different graphs from two different timestamps can be used to compare different periods. If necessary, graph generator 146 can parallelize its workload (e.g., where its backlog cannot otherwise be handled within a particular

25

time period, such as an hour, or if is required to process a graph spanning a long time period).

Graph generator **146** can be implemented in any appropriate programming language, such as Java or C, and machine learning libraries, such as Spark's MLLib. Example ways that graph generator computations can be implemented include using SQL or Map-R, using Spark or Hadoop.

SSH tracker **148** is a microservice that may be responsible for following ssh connections and process parent hierarchies to determine trails of user ssh activity. Identified ssh trails are placed by the SSH tracker **148** into data store **30** and cached for further processing.

SSH tracker **148** can be implemented in any appropriate programming language, such as Java or C, and machine learning libraries, such as Spark's MLLib. Example ways that SSH tracker computations can be implemented include using SQL or Map-R, using Spark or Hadoop.

Threat aggregator **150** is a microservice that may be responsible for obtaining third party threat information from various applicable sources, and making it available to other micro-services. Examples of such information include reverse DNS information, GeoIP information, lists of known bad domains/IP addresses, lists of known bad files, etc. As applicable, the threat information is normalized before insertion into data store **30**. Threat aggregator **150** can be implemented in any appropriate programming language, such as Java or C, using SQL/JDBC libraries to interact with data store **30** (e.g., for insertions and queries).

Scheduler **152** is a microservice that may act as a scheduler and that may run arbitrary jobs organized as a directed graph. In some examples, scheduler **152** ensures that all jobs for all customers are able to run during a given time interval (e.g., every hour). Scheduler **152** may handle errors and retrying for failed jobs, track dependencies, manage appropriate resource levels, and/or scale jobs as needed. Scheduler **152** can be implemented in any appropriate programming language, such as Java or C. A variety of components can also be used, such as open source scheduler frameworks (e.g., Airflow), or AWS services (e.g., the AWS Data pipeline) which can be used for managing schedules.

Graph Behavior Modeler (GBM) **154** is a microservice that may compute polygraphs. In particular, GBM **154** can be used to find clusters of nodes in a graph that should be considered similar based on some set of their properties and relationships to other nodes. As described herein, the clusters and their relationships can be used to provide visibility into a datacenter environment without requiring user specified labels. GBM **154** may track such clusters over time persistently, allowing for changes to be detected and alerts to be generated.

GBM **154** may take as input a raw graph (e.g., as generated by graph generator **146**). Nodes are actors of a behavior, and edges are the behavior relationship itself. For example, in the case of communication, example actors include processes, which communicate with other processes. The GBM **154** clusters the raw graph based on behaviors of actors and produces a summary (the polygraph). The polygraph summarizes behavior at a datacenter level. The GBM **154** also produces "observations" that represent changes detected in the datacenter. Such observations may be based on differences in cumulative behavior (e.g., the baseline) of the datacenter with its current behavior. The GBM **154** can be implemented in any appropriate programming language, such as Java, C, or Golang, using appropriate libraries (as applicable) to handle distributed graph computations (handling large amounts of data analysis in a short amount of time). Apache Spark is another example tool that can be used

26

to compute polygraphs. The GBM **154** can also take feedback from users and adjust the model according to that feedback. For example, if a given user is interested in relearning behavior for a particular entity, the GBM **154** can be instructed to "forget" the implicated part of the polygraph.

GBM runner **156** is a microservice that may be responsible for interfacing with GBM **154** and providing GBM **154** with raw graphs (e.g., using a query language, such as SQL, to push any computations it can to data store **30**). GBM runner **156** may also insert polygraph output from GBM **154** to data store **30**. GBM runner **156** can be implemented in any appropriate programming language, such as Java or C, using SQL/JDBC libraries to interact with data store **30** to insert and query data.

Alert generator **158** is a microservice that may be responsible for generating alerts. Alert generator **158** may examine observations (e.g., produced by GBM **154**) in aggregate, deduplicate them, and score them. Alerts may be generated for observations with a score exceeding a threshold. Alert generator **158** may also compute (or retrieve, as applicable) data that a customer (e.g., user A or user B) might need when reviewing the alert. Examples of events that can be detected by data platform **12** (and alerted on by alert generator **158**) include, but are not limited to the following:

- new user: This event may be created the first time a user (e.g., of node **116**) is first observed by an agent within a datacenter.

- user launched new binary: This event may be generated when an interactive user launches an application for the first time.

- new privilege escalation: This event may be generated when user privileges are escalated and a new application is run.

- new application or container: This event may be generated when an application or container is seen for the first time.

- new external connection: This event may be generated when a connection to an external IP/domain is made from a new application.

- new external host or IP: This event may be generated when a new external host or IP is involved in a connection with a datacenter.

- new internal connection: This event may be generated when a connection between internal-only applications is seen for the first time.

- new external client: This event may be generated when a new external connection is seen for an application which typically does not have external connections.

- new parent: This event may be generated when an application is launched by a different parent.

- connection to known bad IP/domain: Data platform **12** maintains (or can otherwise access) one or more reputation feeds. If an environment makes a connection to a known bad IP or domain, an event will be generated.

- login from a known bad IP/domain: An event may be generated when a successful connection to a datacenter from a known bad IP is observed by data platform **12**.

Alert generator **158** can be implemented in any appropriate programming language, such as Java or C, using SQL/JDBC libraries to interact with data store **30** to insert and query data. In various embodiments, alert generator **158** also uses one or more machine learning libraries, such as Spark's MLLib (e.g., to compute scoring of various observations). Alert generator **158** can also take feedback from users about which kinds of events are of interest and which to suppress.

QsJobServer **160** is a microservice that may look at all the data produced by data platform **12** for an hour, and compile a materialized view (MV) out of the data to make queries faster. The MV helps make sure that the queries customers most frequently run, and data that they search for, can be easily queried and answered. QsJobServer **160** may also precompute and cache a variety of different metrics so that they can quickly be provided as answers at query time. QsJobServer **160** can be implemented using any appropriate programming language, such as Java or C, using SQL/JDBC libraries. In some examples, QsJobServer **160** is able to compute an MV efficiently at scale, where there could be a large number of joins. An SQL engine, such as Oracle, can be used to efficiently execute the SQL, as applicable.

Alert notifier **162** is a microservice that may take alerts produced by alert generator **158** and send them to customers' integrated Security Information and Event Management (SIEM) products (e.g., Splunk, Slack, etc.). Alert notifier **162** can be implemented using any appropriate programming language, such as Java or C. Alert notifier **162** can be configured to use an email service (e.g., AWS SES or pagerduty) to send emails. Alert notifier **162** may also provide templating support (e.g., Velocity or Moustache) to manage templates and structured notifications to SIEM products.

Reporting module **164** is a microservice that may be responsible for creating reports out of customer data (e.g., daily summaries of events, etc.) and providing those reports to customers (e.g., via email). Reporting module **164** can be implemented using any appropriate programming language, such as Java or C. Reporting module **164** can be configured to use an email service (e.g., AWS SES or pagerduty) to send emails. Reporting module **164** may also provide templating support (e.g., Velocity or Moustache) to manage templates (e.g., for constructing HTML-based email).

Web app **120** is a microservice that provides a user interface to data collected and processed on data platform **12**. Web app **120** may provide login, authentication, query, data visualization, etc. features. Web app **120** may, in some embodiments, include both client and server elements. Example ways the server elements can be implemented are using Java DropWizard or Node.js to serve business logic, and a combination of JSON/HTTP to manage the service. Example ways the client elements can be implemented are using frameworks such as React, Angular, or Backbone. JSON, jQuery, and JavaScript libraries (e.g., underscore) can also be used.

Query service **166** is a microservice that may manage all database access for web app **120**. Query service **166** abstracts out data obtained from data store **30** and provides a JSON-based REST API service to web app **120**. Query service **166** may generate SQL queries for the REST APIs that it receives at run time. Query service **166** can be implemented using any appropriate programming language, such as Java or C and SQL/JDBC libraries, or an SQL framework such as jOOQ. Query service **166** can internally make use of a variety of types of databases, including a relational database engine **168** (e.g., AWS Aurora) and/or data store **30** to manage data for clients. Examples of tables that query service **166** manages are OLTP tables and data warehousing tables.

Cache **170** may be implemented by Redis and/or any other service that provides a key-value store. Data platform **12** can use cache **170** to keep information for frontend services about users. Examples of such information include valid tokens for a customer, valid cookies of customers, the last time a customer tried to login, etc.

FIG. 3A illustrates an example of a process for detecting anomalies in a network environment. In various embodiments, process **300** is performed by data platform **12**. The process begins at **301** when data associated with activities occurring in a network environment (such as entity A's datacenter) is received. One example of such data that can be received at **301** is agent-collected data described above (e.g., in conjunction with process **200**).

At **302**, a logical graph model is generated, using at least a portion of the monitored activities. A variety of approaches can be used to generate such logical graph models, and a variety of logical graphs can be generated (whether using the same, or different approaches). The following is one example of how data received at **301** can be used to generate and maintain a model.

During bootstrap, data platform **12** creates an aggregate graph of physical connections (also referred to herein as an aggregated physical graph) by matching connections that occurred in the first hour into communication pairs. Clustering is then performed on the communication pairs. Examples of such clustering, described in more detail below, include performing Matching Neighbor clustering and similarity (e.g., SimRank) clustering. Additional processing can also be performed (and is described in more detail below), such as by splitting clusters based on application type, and annotating nodes with DNS query information. The resulting graph (also referred to herein as a base graph or common graph) can be used to generate a variety of models, where a subset of node and edge types (described in more detail below) and their properties are considered in a given model. One example of a model is a UID to UID model (also referred to herein as a Uid2Uid model) which clusters together processes that share a username and show similar privilege change behavior. Another example of a model is a CType model, which clusters together processes that share command line similarity. Yet another example of a model is a PType model, which clusters together processes that share behaviors over time.

Each hour (or any other predetermined time interval) after bootstrap, a new snapshot is taken (i.e., data collected about a datacenter in the last hour is processed) and information from the new snapshot is merged with existing data to create and (as additional data is collected/processed) maintain a cumulative graph. The cumulative graph (also referred to herein as a cumulative PType graph and a polygraph) is a running model of how processes behave over time. Nodes in the cumulative graph are PType nodes, and provide information such as a list of all active processes and PIDs in the last hour, the number of historic total processes, the average number of active processes per hour, the application type of the process (e.g., the CType of the PType), and historic CType information/frequency. Edges in the cumulative graph can represent connectivity and provide information such as connectivity frequency. The edges can be weighted (e.g., based on number of connections, number of bytes exchanged, etc.). Edges in the cumulative graph (and snapshots) can also represent transitions.

One approach to merging a snapshot of the activity of the last hour into a cumulative graph is as follows. An aggregate graph of physical connections is made for the connections included in the snapshot (as was previously done for the original snapshot used during bootstrap). And, clustering/splitting is similarly performed on the snapshot's aggregate graph. Next, PType clusters in the snapshot's graph are compared against PType clusters in the cumulative graph to identify commonality.

One approach to determining commonality is, for any two nodes that are members of a given CmdType (described in more detail below), comparing internal neighbors and calculating a set membership Jaccard distance. The pairs of nodes are then ordered by decreasing similarity (i.e., with the most similar sets first). For nodes with a threshold amount of commonality (e.g., at least 66% members in common), any new nodes (i.e., appearing in the snapshot's graph but not the cumulative graph) are assigned the same PType identifier as is assigned to the corresponding node in the cumulative graph. For each node that is not classified (i.e., has not been assigned a PType identifier), a network signature is generated (i.e., indicative of the kinds of network connections the node makes, who the node communicates with, etc.). The following processing is then performed until convergence. If a match of the network signature is found in the cumulative graph, the unclassified node is assigned the PType identifier of the corresponding node in the cumulative graph. Any nodes which remain unclassified after convergence are new PTypes and are assigned new identifiers and added to the cumulative graph as new. As applicable, the detection of a new PType can be used to generate an alert. If the new PType has a new CmdType, a severity of the alert can be increased. If any surviving nodes (i.e., present in both the cumulative graph and the snapshot graph) change PTypes, such change is noted as a transition, and an alert can be generated. Further, if a surviving node changes PType and also changes CmdType, a severity of the alert can be increased.

Changes to the cumulative graph (e.g., a new PType or a new edge between two PTypes) can be used (e.g., at **303**) to detect anomalies (described in more detail below). Two example kinds of anomalies that can be detected by data platform **12** include security anomalies (e.g., a user or process behaving in an unexpected manner) and devops/root cause anomalies (e.g., network congestion, application failure, etc.). Detected anomalies can be recorded and surfaced (e.g., to administrators, auditors, etc.), such as through alerts which are generated at **304** based on anomaly detection.

Additional detail regarding processing performed, by various components depicted in FIG. **1D** (whether performed individually or in combination), in conjunction with model/polygraph construction (e.g., as performed at **302**) are provided below.

As explained above, an aggregated physical graph can be generated on a per customer basis periodically (e.g., once an hour) from raw physical graph information, by matching connections (e.g., between two processes on two virtual machines). In various embodiments, a deterministic fixed approach is used to cluster nodes in the aggregated physical graph (e.g., representing processes and their communications). As one example, Matching Neighbors Clustering (MNC) can be performed on the aggregated physical graph to determine which entities exhibit identical behavior and cluster such entities together.

FIG. **3B** depicts a set of example processes (p1, p2, p3, and p4) communicating with other processes (p10 and p11). FIG. **3B** is a graphical representation of a small portion of an aggregated physical graph showing (for a given time period, such as an hour) which processes in a datacenter communicate with which other processes. Using MNC, processes p1, p2, and p3 will be clustered together (**305**), as they exhibit identical behavior (they communicate with p10 and only p10). Process p4, which communicates with both p10 and p11, will be clustered separately.

In MNC, only those processes exhibiting identical (communication) behavior will be clustered. In various embodi-

ments, an alternate clustering approach can also/instead be used, which uses a similarity measure (e.g., constrained by a threshold value, such as a 60% similarity) to cluster items. In some embodiments, the output of MNC is used as input to SimRank, in other embodiments, MNC is omitted.

FIG. **3C** depicts a set of example processes (p4, p5, p6) communicating with other processes (p7, p8, p9). As illustrated, most of nodes p4, p5, and p6 communicate with most of nodes p7, p8, and p9 (as indicated in FIG. **3C** with solid connection lines). As one example, process p4 communicates with process p7 (**310**), process p8 (**311**), and process p9 (**312**). An exception is process p6, which communicates with processes p7 and p8, but does not communicate with process p9 (as indicated by dashed line **313**). If MNC were applied to the nodes depicted in FIG. **3C**, nodes p4 and p5 would be clustered (and node p6 would not be included in their cluster).

One approach to similarity clustering is to use SimRank. In an embodiment of the SimRank approach, for a given node v in a directed graph, $I(v)$ and $O(v)$ denote the respective set of in-neighbors and out-neighbors of v . Individual in-neighbors are denoted as $I_i(v)$, for $1 \leq i \leq |I(v)|$, and individual out-neighbors are denoted as $O_i(v)$, for $1 \leq i \leq |O(v)|$. The similarity between two objects a and b can be denoted by $s(a,b) \in [1,0]$. A recursive equation (hereinafter "the SimRank equation") can be written for $s(a,b)$, where, if $a=b$, then $s(a,b)$ is defined as 1, otherwise,

$$s(a, b) = \frac{C}{|I(a)||I(b)|} \sum_{i=1}^{|I(a)|} \sum_{j=1}^{|I(b)|} s(I_i(a), I_j(b))$$

where C is a constant between 0 and 1. One example value for the decay factor C is 0.8 (and a fixed number of iterations such as five). Another example value for the decay factor C is 0.6 (and/or a different number of iterations). In the event that a or b has no in-neighbors, similarity is set to $s(a,b)=0$, so the summation is defined to be 0 when $I(a)=\emptyset$ or $I(b)=\emptyset$.

The SimRank equations for a graph G can be solved by iteration to a fixed point. Suppose n is the number of nodes in G . For each iteration k , n^2 entries $s_k(*,*)$ are kept, where $s_k(a,b)$ gives the score between a and b on iteration k . Successive computations of $s_{k+1}(*,*)$ are made based on $s_k(*,*)$. Starting with $s_0(*,*)$, where each $s_0(a,b)$ is a lower bound on the actual SimRank score

$$s(a, b): s_0(a, b) = \begin{cases} 1, & \text{if } a = b, \\ 0, & \text{if } a \neq b. \end{cases}$$

The SimRank equation can be used to compute $s_{k+1}(a, b)$ from $s_k(*,*)$ with

$$s_{k+1}(a, b) = \frac{C}{|I(a)||I(b)|} \sum_{i=1}^{|I(a)|} \sum_{j=1}^{|I(b)|} s(I_i(a), I_j(b))$$

for $a \neq b$, and $s_{k+1}(a, b)=1$ for $a=b$. On each iteration $k+1$, the similarity of (a,b) is updated using the similarity scores of the neighbors of (a,b) from the previous iteration k according to the SimRank equation. The values $s_k(*,*)$ are nondecreasing as k increases.

Returning to FIG. **3C**, while MNC would cluster nodes p4 and p5 together (and not include node p6 in their cluster),

application of SimRank would cluster nodes p4-p6 into one cluster (314) and also cluster nodes p7-p9 into another cluster (315).

FIG. 3D depicts a set of processes, and in particular server processes s1 and s2, and client processes c1, c2, c3, c4, c5, and c6. Suppose only nodes s1, s2, c1, and c2 are present in the graph depicted in FIG. 3D (and the other nodes depicted are omitted from consideration). Using MNC, nodes s1 and s2 would be clustered together, as would nodes c1 and c2. Performing SimRank clustering as described above would also result in those two clusters (s1 and s2, and c1 and c2). As previously mentioned, in MNC, identical behavior is required. Thus, if node c3 were now also present in the graph, MNC would not include c3 in a cluster with c2 and c1 because node c3 only communicates with node s2 and not node s1. In contrast, a SimRank clustering of a graph that includes nodes s1, s2, c1, c2, and c3 would result (based, e.g., on an applicable selected decay value and number of iterations) in a first cluster comprising nodes s1 and s2, and a second cluster of c1, c2, and c3. As an increasing number of nodes which communicate with server process s2, and do not also communicate with server process s1, are included in the graph (e.g., as c4, c5, and c6 are added), under SimRank, nodes s1 and s2 will become decreasingly similar (i.e., their intersection is reduced).

In various embodiments, SimRank is modified (from what is described above) to accommodate differences between the asymmetry of client and server connections. As one example, SimRank can be modified to use different thresholds for client communications (e.g., an 80% match among nodes c1-c6) and for server communications (e.g., a 60% match among nodes s1 and s2). Such modification can also help achieve convergence in situations such as where a server process dies on one node and restarts on another node.

The application of MNC/SimRank to an aggregated physical graph results in a smaller graph, in which processes which are determined to be sufficiently similar are clustered together. Typically, clusters generated as output of MNC will be underinclusive. For example, for the nodes depicted in FIG. 3C, process p6 will not be included in a cluster with processes p4 and p5, despite substantial similarity in their communication behaviors. The application of SimRank (e.g., to the output of MNC) helps mitigate the underinclusiveness of MNC, but can result in overly inclusive clusters. As one example, suppose (returning to the nodes depicted in FIG. 3B) that as a result of applying SimRank to the depicted nodes, nodes p1-p4 are all included in a single cluster. Both MNC and SimRank operate agnostically of which application a given process belongs to. Suppose processes p1-p3 each correspond to a first application (e.g., an update engine), and process p4 corresponds to a second application (e.g., sshd). Further suppose process p10 corresponds to contact with AWS. Clustering all four of the processes together (e.g., as a result of SimRank) could be problematic, particularly in a security context (e.g., where granular information useful in detecting threats would be lost).

As previously mentioned, data platform 12 may maintain a mapping between processes and the applications to which they belong. In various embodiments, the output of SimRank (e.g., SimRank clusters) is split based on the applications to which cluster members belong (such a split is also referred to herein as a "CmdType split"). If all cluster members share a common application, the cluster remains. If different cluster members originate from different applications, the cluster members are split along application-type (CmdType) lines. Using the nodes depicted in FIG. 3D as an example, suppose that nodes c1, c2, c3, and c5 all share

"update engine" as the type of application to which they belong (sharing a CmdType). Suppose that node c4 belongs to "ssh," and suppose that node c6 belongs to "bash." As a result of SimRank, all six nodes (c1-c6) might be clustered into a single cluster. After a CmdType split is performed on the cluster, however, the single cluster will be broken into three clusters (c1, c2, c3, c5; c4; and c6). Specifically, the resulting clusters comprise processes associated with the same type of application, which exhibit similar behaviors (e.g., communication behaviors). Each of the three clusters resulting from the CmdType split represents, respectively, a node (also referred to herein as a PType) of a particular CmdType. Each PType is given a persistent identifier and stored persistently as a cumulative graph.

A variety of approaches can be used to determine a CmdType for a given process. As one example, for some applications (e.g., sshd), a one-to-one mapping exists between the CmdType and the application/binary name. Thus, processes corresponding to the execution of sshd will be classified using a CmdType of sshd. In various embodiments, a list of common application/binary names (e.g., sshd, apache, etc.) is maintained by data platform 12 and manually curated as applicable. Other types of applications (e.g., Java, Python, and Ruby) are multi-homed, meaning that several very different applications may all execute using the binary name, "java." For these types of applications, information such as command line/execution path information can be used in determining a CmdType. In particular, the subapplication can be used as the CmdType of the application, and/or term frequency analysis (e.g., TF/IDF) can be used on command line information to group, for example, any marathon related applications together (e.g., as a python.marathon CmdType) and separately from other Python applications (e.g., as a python.airflow CmdType).

In various embodiments, machine learning techniques are used to determine a CmdType. The CmdType model is constrained such that the execution path for each CmdType is unique. One example approach to making a CmdType model is a random forest based approach. An initial CmdType model is bootstrapped using process parameters (e.g., available within one minute of process startup) obtained using one hour of information for a given customer (e.g., entity A). Examples of such parameters include the command line of the process, the command line of the process's parent(s) (if applicable), the uptime of the process, UID/EUID and any change information, TTY and any change information, listening ports, and children (if any). Another approach is to perform term frequency clustering over command line information to convert command lines into cluster identifiers.

The random forest model can be used (e.g., in subsequent hours) to predict a CmdType for a process (e.g., based on features of the process). If a match is found, the process can be assigned the matching CmdType. If a match is not found, a comparison between features of the process and its nearest CmdType (e.g., as determined using a Levenshtein distance) can be performed. The existing CmdType can be expanded to include the process, or, as applicable, a new CmdType can be created (and other actions taken, such as generating an alert). Another approach to handling processes which do not match an existing CmdType is to designate such processes as unclassified, and once an hour, create a new random forest seeded with process information from a sampling of classified processes (e.g., 10 or 100 processes per CmdType) and the new processes. If a given new process winds up in an

existing set, the process is given the corresponding CmdType. If a new cluster is created, a new CmdType can be created.

Conceptually, a polygraph represents the smallest possible graph of clusters that preserve a set of rules (e.g., in which nodes included in the cluster must share a CmdType and behavior). As a result of performing MNC, SimRank, and cluster splitting (e.g., CmdType splitting) many processes are clustered together based on commonality of behavior (e.g., communication behavior) and commonality of application type. Such clustering represents a significant reduction in graph size (e.g., compared to the original raw physical graph).

Nonetheless, further clustering can be performed (e.g., by iterating on the graph data using the GBM to achieve such a polygraph). As more information within the graph is correlated, more nodes can be clustered together, reducing the size of the graph, until convergence is reached and no further clustering is possible.

FIG. 3E depicts two pairs of clusters. In particular, cluster 320 represents a set of client processes sharing the same CmdType ("a1"), communicating (collectively) with a server process having a CmdType ("a2"). Cluster 322 also represents a set of client processes having a CmdType a1 communicating with a server process having a CmdType a2. The nodes in clusters 320 and 322 (and similarly nodes in 321 and 323) remain separately clustered (as depicted) after MNC/SimRank/CmdType splitting-isolated islands. One reason this could occur is where server process 321 corresponds to processes executing on a first machine (having an IP address of 1.1.1.1). The machine fails and a new server process 323 starts, on a second machine (having an IP address of 2.2.2.2) and takes over for process 321.

Communications between a cluster of nodes (e.g., nodes of cluster 320) and the first IP address can be considered different behavior from communications between the same set of nodes and the second IP address, and thus communications 324 and 325 will not be combined by MNC/SimRank in various embodiments. Nonetheless, it could be desirable for nodes of clusters 320/322 to be combined (into cluster 326), and for nodes of clusters 321/323 to be combined (into cluster 327), as representing (collectively) communications between a1 and a2. One task that can be performed by data platform 12 is to use DNS query information to map IP addresses to logical entities. As will be described in more detail below, GBM 154 can make use of the DNS query information to determine that graph nodes of cluster 320 and graph nodes of cluster 322 both made DNS queries for "appserverabc.example.com," which first resolved to 1.1.1.1 and then to 2.2.2.2, and to combine nodes 320/322 and 321/323 together into a single pair of nodes (326 communicating with 327).

In various embodiments, GBM 154 operates in a batch manner in which it receives as input the nodes and edges of a graph for a particular time period along with its previous state, and generates as output clustered nodes, cluster membership edges, cluster-to-cluster edges, events, and its next state.

GBM 154 may not try to consider all types of entities and their relationships that may be available in a conceptual common graph all at once. Instead, GBM uses a concept of models where a subset of node and edge types and their properties are considered in a given model. Such an approach is helpful for scalability, and also to help preserve detailed information (of particular importance in a security context)—as clustering entities in a more complex and larger graph could result in less useful results. In particular,

such an approach allows for different types of relationships between entities to be preserved/more easily analyzed.

While GBM 154 can be used with different models corresponding to different subgraphs, core abstractions remain the same across types of models.

For example, each node type in a GBM model is considered to belong to a class. The class can be thought of as a way for the GBM to split nodes based on the criteria it uses for the model. The class for a node is represented as a string whose value is derived from the node's key and properties depending on the GBM Model. Note that different GBM models may create different class values for the same node. For each node type in a given GBM model, GBM 154 can generate clusters of nodes for that type. A GBM generated cluster for a given member node type cannot span more than one class for that node type. GBM 154 generates edges between clusters that have the same types as the edges between source and destination cluster node types.

Additionally or alternatively, the processes described herein as being used for a particular model can be used (can be the same) across models, and different models can also be configured with different settings.

Additionally or alternatively, the node types and the edge types may correspond to existing types in the common graph node and edge tables but this is not necessary. Even when there is a correspondence, the properties provided to GBM 154 are not limited to the properties that are stored in the corresponding graph table entries. They can be enriched with additional information before being passed to GBM 154.

Logically, the input for a GBM model can be characterized in a manner that is similar to other graphs. Edge triplets can be expressed, for example, as an array of source node type, edge type, and destination node type. And, each node type is associated with node properties, and each edge type is associated with edge properties. Other edge triplets can also be used (and/or edge triplets can be extended) in accordance with various embodiments.

Note that the physical input to the GBM model need not (and does not, in various embodiments) conform to the logical input. For example, the edges in the PtypeConn model correspond to edges between Matching Neighbors (MN) clusters, where each process node has an MN cluster identifier property. In the User ID to User ID model (also referred to herein as the Uid2Uid model), edges are not explicitly provided separately from nodes (as the cuid array in the node properties serves the same purpose). In both cases, however, the physical information provides the applicable information necessary for the logical input.

The state input for a particular GBM model can be stored in a file, a database, or other appropriate storage. The state file (from a previous run) is provided, along with graph data, except for when the first run for a given model is performed, or the model is reset. In some cases, no data may be available for a particular model in a given time period, and GBM may not be run for that time period. As data becomes available at a future time, GBM can run using the latest state file as input.

GBM 154 outputs cluster nodes, cluster membership edges, and inter-cluster relationship edges that are stored (in some embodiments) in the graph node tables: node_c, node_cm, and node_icr, respectively. The type names of nodes and edges may conform to the following rules:

A given node type can be used in multiple different GBM models. The type names of the cluster nodes generated by two such models for that node type will be different. For instance, process type nodes will appear in both

PtypeConn and Uid2Uid models, but their cluster nodes will have different type names.

The membership edge type name is "MemberOf."

The edge type names for cluster-to-cluster edges will be the same as the edge type names in the underlying node-to-node edges in the input.

The following are example events GBM 154 can generate: new class, new cluster, new edge from class to class, split class (the notion that GBM 154 considers all nodes of a given type and class to be in the same cluster initially and if GBM 154 splits them into multiple clusters, it is splitting a class), new edge from cluster and class, new edge between cluster and cluster, and/or new edge from class to cluster.

One underlying node or edge in the logical input can cause multiple types of events to be generated. Conversely, one event can correspond to multiple nodes or edges in the input. Not every model generates every event type.

Additional information regarding examples of data structures/models that can be used in conjunction with models used by data platform 12 is now provided.

In some examples, a PTypeConn Model clusters nodes of the same class that have similar connectivity relationships. For example, if two processes had similar incoming neighbors of the same class and outgoing neighbors of the same class, they could be clustered.

The node input to the PTypeConn model for a given time period includes non-interactive (i.e., not associated with tty) process nodes that had connections in the time period and the base graph nodes of other types (IP Service Endpoint (IPSep) comprising an IP address and a port, DNS Service Endpoint (DNSSep) and IPAddress) that have been involved in those connections. The base relationship is the connectivity relationship for the following type triplets:

Process, ConnectedTo, Process
Process, ConnectedTo, IP Service Endpoint (IPSep)
Process, ConnectedTo, DNS Service Endpoint (DNSSep)
IPAddress, ConnectedTo, ProcessProcess, DNS, ConnectedTo, Process

The edge inputs to this model are the ConnectedTo edges from the MN cluster, instead of individual node-to-node ConnectedTo edges from the base graph. The membership edges created by this model refer to the base graph node type provided in the input.

Class Values:

The class values of nodes are determined as follows depending on the node type (e.g., Process nodes, IPSep nodes, DNSSep nodes, and IP Address nodes).

Process nodes:

if exe_path contains java then "java <cmdline_term_1> . . ."
else if exe_path contains python then "python <cmdline_term_1> . . ."
else "last_part_of_exe_path"

IPSep nodes:

if IP_internal then "IntIPS"
else if severity=0 then "<IP_addr>: <protocol>: <port>"
else "<IP_addr>: <port>_BadIP"

DNSSep nodes:

if IP_internal=1 then "<hostname>"
else if severity=0 then "<hostname>: <protocol>: port"
else "<hostname>: <port>_BadIP"

IPAddress nodes (will appear only on client side):

if IP_internal=1 then "IPIntC"
else if severity=0 then "ExtIPC"
else "ExtBadIPC"

Events:

A new class event in this model for a process node is equivalent to seeing a new CType being involved in a connection for the first time. Note that this does not mean the CType was not seen before. It is possible that it was previously seen but did not make a connection at that time.

A new class event in this model for an IPSep node with IP_internal=0 is equivalent to seeing a connection to a new external IP address for the first time.

A new class event in this model for a DNSSep node is equivalent to seeing a connection to a new domain for the first time.

A new class event in this model for an IPAddress node with IP_internal=0 and severity=0 is equivalent to seeing a connection from any external IP address for the first time.

A new class event in this model for an IPAddress node with IP_internal=0 and severity >0 is equivalent to seeing a connection from any bad external IP address for the first time.

A new class to class to edge from a class for a process node to a class for a process node is equivalent to seeing a communication from the source CType making a connection to the destination CType for the first time.

A new class to class to edge from a class for a process node to a class for a DNSSep node is equivalent to seeing a communication from the source CType making a connection to the destination domain name for the first time.

An IntPConn Model may be similar to the PTypeConn Model, except that connection edges between parent/child processes and connections between processes where both sides are not interactive are filtered out.

A Uid2Uid Model may cluster processes with the same username that show similar privilege change behavior. For instance, if two processes with the same username had similar effective user values, launched processes with similar usernames, and were launched by processes with similar usernames, then they could be clustered.

An edge between a source cluster and destination cluster generated by this model means that all of the processes in the source cluster had a privilege change relationship to at least one process in the destination cluster.

The node input to this model for a given time period includes process nodes that are running in that period. The value of a class of process nodes is "<username>".

The base relationship that is used for clustering is privilege change, either by the process changing its effective user ID, or by launching a child process which runs with a different user.

The physical input for this model includes process nodes (only), with the caveat that the complete ancestor hierarchy of process nodes active (i.e., running) for a given time period is provided as input even if an ancestor is not active in that time period. Note that effective user IDs of a process are represented as an array in the process node properties, and launch relationships are available from ppid_hash fields in the properties as well.

A new class event in this model is equivalent to seeing a user for the first time.

A new class to class edge event is equivalent to seeing the source user making a privilege change to the destination user for the first time.

A Ct2Ct Model may cluster processes with the same CType that show similar launch behavior. For instance, if two processes with the same CType have launched processes with similar CTypes, then they could be clustered.

The node input to this model for a given time period includes process nodes that are running in that period. The

value class of process nodes is CType (similar to how it is created for the PtypeConn Model).

The base relationship that is used for clustering is a parent process with a given CType launching a child process with another given destination CType.

The physical input for this model includes process nodes (only) with the caveat that the complete ancestor hierarchy active process nodes (i.e., that are running) for a given time period is provided as input even if an ancestor is not active in that time period. Note that launch relationships are available from ppid_hash fields in the process node properties.

An edge between a source cluster and destination cluster generated by this model means that all of the processes in the source cluster launched at least one process in the destination cluster.

A new class event in this model is equivalent to seeing a CType for the first time. Note that the same type of event will be generated by the PtypeConn Model as well.

A new class to class edge event is equivalent to seeing the source CType launching the destination CType for the first time.

An MTypeConn Model may cluster nodes of the same class that have similar connectivity relationships. For example, if two machines had similar incoming neighbors of the same class and outgoing neighbors of the same class, they could be clustered.

A new class event in this model will be generated for external IP addresses or (as applicable) domain names seen for the first time. Note that a new class to class to edge Machine, class to class for an IPsep or DNSName node will also be generated at the same time.

The membership edges generated by this model will refer to Machine, IPAddress, DNSName, and IPsep nodes in the base graph. Though the nodes provided to this model are IPAddress nodes instead of IPsep nodes, the membership edges it generates will refer to IPsep type nodes. Alternatively, the base graph can generate edges between Machine and IPsep node types. Note that the Machine to IPAddress edges have tcp_dst_ports/udp_dst_ports properties that can be used for this purpose.

The node input to this model for a given time period includes machine nodes that had connections in the time period and the base graph nodes of other types (IPAddress and DNSName) that were involved in those connections.

The base relationship is the connectivity relationship for the following type triplets:

Machine, ConnectedTo, Machine
Machine, ConnectedTo, IPAddress
Machine, ConnectedTo, DNSName
IPAddress, ConnectedTo, Machine, DNS, ConnectedTo, Machine

The edge inputs to this model are the corresponding ConnectedTo edges in the base graph.

Class Values:

Machine:

The class value for all Machine nodes is "Machine."

The machine_terms property in the Machine nodes is used, in various embodiments, for labeling machines that are clustered together. If a majority of the machines clustered together share a term in the machine_terms, that term can be used for labeling the cluster.

IPsep:

The class value for IPsep nodes is determined as follows:

if IP_internal then "IntIPS"

else

5 if severity=0 then "<ip_addr>: <protocol>: <port>"

else "<IP_addr BadIP>"

IPAddress:

The class value for IPAddress nodes is determined as follows:

10 if IP_internal then "IntIPC"

else

if severity=0 then "ExtIPC"

else "ExtBadIPC"

15 DNSName:

The class value for DNSName nodes is determined as follows:

if severity=0 then "<hostname>"

else then "<hostname>BadIP"

20 An example structure for a New Class Event is now described.

The key field for this event type looks as follows (using the PtypeConn model as an example):

```
{
  "node": {
    "class": {
      "cid": "httpd"
    },
    "key": {
      "cid": "29654"
    },
    "type": "PtypeConn"
  }
}
```

It contains the class value and also the ID of the cluster where that class value is observed. Multiple clusters can be observed with the same value in a given time period. It contains the class value and also the ID of the cluster where that class value is observed. Multiple clusters can be observed with the same value in a given time period. Accordingly, in some embodiments, GBM 154 generates multiple events of this type for the same class value. The properties field looks as follows:

```
{
  "set_size": 5
}
```

The set_size indicates the size of the cluster referenced in the keys field.

50 Conditions:

For a given model and time period, multiple NewClass events can be generated if there is more than one cluster in that class. NewNode events will not be generated separately in this case.

Example New Class to Class Edge Event structure:

The key field for this event type looks as follows (using the PtypeConn model as an example):

```
"edge": {
  "dst_node": {
    "class": {
      "cid": "java war"
    },
    "key": {
      "cid": "27635"
    },
    "type": "PtypeConn"
  },
  "type": "PtypeConn"
}
```

39

```

"src_node": {
  "class": {
    "cid": "IntIPC"
  },
  "key": {
    "cid": "20881"
  },
  "type": "PtypeConn"
},
"type": "ConnectedTo"
}

```

The key field contains source and destination class values and also source and destination cluster identifiers (i.e., the src/dst_node: key.cid represents the src/dst cluster identifier).

In a given time period for a given model, an event of this type could involve multiple edges between different cluster pairs that have the same source and destination class values.

GBM 154 can generate multiple events in this case with different source and destination cluster identifiers.

The props fields look as follows for this event type:

```

{
  "dst_set_size": 2,
  "src_set_size": 1
}

```

The source and destination sizes represent the sizes of the clusters given in the keys field.

Conditions:

For a given model and time period, multiple NewClassTo-Class events can be generated if there are more than one pair of clusters in that class pair. NewNodeToNode events are not generated separately in this case.

Combining Events at the Class Level: for a given model and time period, the following example types of events can represent multiple changes in the underlying GBM cluster level graph in terms of multiple new clusters or multiple new edges between clusters:

```

NewClass
NewEdgeClassToClass
NewEdgeNodeToClass
NewEdgeClassToNode

```

Multiple NewClass events with the same model and class can be output if there are multiple clusters in that new class.

Multiple NewEdgeClassToClass events with the same model and class pair can be output if there are multiple new cluster edges within that class pair.

Multiple NewEdgeNodeToClass events with the same model and destination class can be output if there are multiple new edges from the source cluster to the destination clusters in that destination class (the first time seeing this class as a destination cluster class for the source cluster).

Multiple NewEdgeClassToNode events with the same model and source class can be output if there are multiple new edges from source clusters to the destination clusters in that source class (the first time seeing this class as a source cluster class for the destination cluster).

These events may be combined at the class level and treated as a single event when it is desirable to view changes at the class level, e.g., when one wants to know when there is a new CType.

In some examples, different models may have partial overlap in the types of nodes they use from the base graph. Therefore, they can generate NewClass type events for the same class. NewClass events can also be combined across models when it is desirable to view changes at the class level.

40

Using techniques herein, actions can be associated with processes and (e.g., by associating processes with users) actions can thus also be associated with extended user sessions. Such information can be used to track user behavior correctly, even where a malicious user attempts to hide his trail by changing user identities (e.g., through lateral movement). Extended user session tracking can also be useful in operational use cases without malicious intent, e.g., where users make original logins with distinct usernames (e.g., "charlie" or "dave") but then perform actions under a common username (e.g., "admin" or "support"). One such example is where multiple users with administrator privileges exist, and they need to gain superuser privilege to perform a particular type of maintenance. It may be desirable to know which operations are performed (as the superuser) by which original user when debugging issues. In the following examples describing extended user session tracking, reference is generally made to using the secure shell (ssh) protocol as implemented by openssh (on the server side) as the mechanism for logins. However, extended user session tracking is not limited to the ssh protocol or a particular limitation and the techniques described herein can be extended to other login mechanisms.

On any given machine, there will be a process that listens for and accepts ssh connections on a given port. This process can run the openssh server program running in daemon mode or it could be running another program (e.g., initd on a Linux system). In either case, a new process running openssh will be created for every new ssh login session and this process can be used to identify an ssh session on that machine. This process is called the "privileged" process in openssh.

After authentication of the ssh session, when an ssh client requests a shell or any other program to be run under that ssh session, a new process that runs that program will be created under (i.e., as a child of) the associated privileged process. If an ssh client requests port forwarding to be performed, the connections will be associated with the privileged process.

In modern operating systems such as Linux and Windows, each process has a parent process (except for the very first process) and when a new process is created the parent process is known. By tracking the parent-child hierarchy of processes, one can determine if a particular process is a descendant of a privileged openssh process and thus if it is associated with an ssh login session.

For user session tracking across machines (or on a single machine with multiple logins) in a distributed environment, it is established when two login sessions have a parent-child relationship. After that, the "original" login session, if any, for any given login session can be determined by following the parent relationship recursively.

FIG. 3F is a representation of a user logging into a first machine and then into a second machine from the first machine, as well as information associated with such actions. In the example of FIG. 3F, a user, Charlie, logs into Machine A (331) from a first IP address (332). As part of the login process, he provides a username (333). Once connected to Machine A, an openssh privileged process (334) is created to handle the connection for the user, and a terminal session is created and a bash process (335) is created as a child. Charlie launches an ssh client (336) from the shell, and uses it to connect (337) to Machine B (338). As with the connection he makes to Machine A, Charlie's connection to Machine B will have an associated incoming IP address (339), in this case, the IP address of Machine A. And, as part of the login process with Machine B, Charlie will provide a username (340) which need not be the same as username

41

333. An openssh privileged process (341) is created to handle the connection, and a terminal session and child bash process (342) will be created. From the command line of Machine B, Charlie launches a curl command (343), which opens an HTTP connection (344) to an external Machine C (345).

FIG. 3G is an alternate representation of actions occurring in FIG. 3F, where events occurring on Machine A are indicated along line 350, and events occurring on Machine B are indicated along line 351. As shown in FIG. 3G, an incoming ssh connection is received at Machine A (352). Charlie logs in (as user “x”) and an ssh privileged process is created to handle Charlie’s connection (353). A terminal session is created and a bash process is created (354) as a child of process 353. Charlie wants to ssh to Machine B, and so executes an ssh client on Machine A (355), providing credentials (as user “y”) at 356. Charlie logs into Machine B, and an sash privileged process is created to handle Charlie’s connection (357). A terminal session is created and a bash process is created (358) as a child of process 357. Charlie then executes curl (359) to download content from an external domain (via connection 360).

The external domain could be a malicious domain, or it could be benign. Suppose the external domain is malicious (and, e.g., Charlie has malicious intent). It would be advantageous (e.g., for security reasons) to be able to trace the contact with the external domain back to Machine A, and then back to Charlie’s IP address. Using techniques described herein (e.g., by correlating process information collected by various agents), such tracking of Charlie’s activities back to his original login (330) can be accomplished. In particular, an extended user session can be tracked that associates Charlie’s ssh processes together with a single original login and thus original user.

As described herein, software agents (such as agent 112) may run on machines (such as a machine that implements one of nodes 116) and detect new connections, processes, and/or logins. As also previously explained, such agents send associated records to data platform 12 which includes one or more datastores (e.g., data store 30) for persistently storing such data. Such data can be modeled using logical tables, also persisted in datastores (e.g., in a relational database that provides an SQL interface), allowing for querying of the data. Other datastores such as graph oriented databases and/or hybrid schemes can also be used.

The following identifiers are commonly used in the tables:

MID
PID_hash

An ssh login session can be identified uniquely by an (MID, PID_hash) tuple. The MID is a machine identifier that is unique to each machine, whether physical or virtual, across time and space. Operating systems use numbers called process identifiers (PIDs) to identify processes running at a given time. Over time processes may die and new processes may be started on a machine or the machine itself may restart. The PID is not necessarily unique across time in that the same PID value can be reused for different processes at different times. In order to track process descendants across time, one should therefore account for time as well. In order to be able to identify a process on a machine uniquely across time, another number called a PID_hash is generated for the process. In various embodiments, the PID_hash is generated using a collision-resistant hash function that takes the PID, start time, and (in various embodiments, as applicable) other properties of a process.

42

Input data collected by agents comprises the input data model and is represented by the following logical tables:

connections
processes
logins

A connections table may maintain records of TCP/IP connections observed on each machine. Example columns included in a connections table are as follows:

Column Name	Description
MID	Identifier of the machine that the connection was observed on.
start_time	Connection start time.
PID_hash	Identifier of the process that was associated with the connection.
src_IP_addr	Source IP address (the connection was initiated from this IP address).
src_port	Source port.
dst_IP_addr	Destination IP address (the connection was made to this IP address).
dst_port	Destination port.
Prot	Protocol (TCP or UDP).
Dir	Direction of the connection (incoming or outgoing) with respect to this machine.

The source fields (IP address and port) correspond to the side from which the connection was initiated. On the destination side, the agent associates an ssh connection with the privileged ssh process that is created for that connection.

For each connection in the system, there will be two records in the table, assuming that the machines on both sides of the connection capture the connection. These records can be matched based on equality of the tuple (src_IP_addr, src_port, dst_IP_addr, dst_port, Prot) and proximity of the start_time fields (e.g., with a one minute upper threshold between the start_time fields).

A processes table maintains records of processes observed on each machine. It may have the following columns:

Column Name	Description
MID	Identifier of the machine that the process was observed on.
PID_hash	Identifier of the process.
start_time	Start time of the process.
exe_path	The executable path of the process.
PPID_hash	Identifier of the parent process.

A logins table may maintain records of logins to machines. It may have the following columns:

Column Name	Description
MID	Identifier of the machine that the login was observed on.
sshd_PID_hash	Identifier of the sshd privileged process associated with login.
login_time	Time of login.
login_username	Username used in login.

Output data generated by session tracking is represented with the following logical tables:

login-local-descendant
login-connection
login-lineage

Using data in these tables, it is possible to determine descendant processes of a given ssh login session across the environment (i.e., spanning machines). Conversely, given a

process, it is possible to determine if it is an ssh login descendant as well as the original ssh login session for it if so.

A login-local-descendant table maintains the local (i.e., on the same machine) descendant processes of each ssh login session. It may have the following columns:

Column Name	Description
MID	Identifier of the machine that the login was observed on.
sshd_PID_hash	Identifier of the sshd privileged process associated with login.
login_time	Time of login.
login_username	Username used in login.

A login-connections table may maintain the connections associated with ssh logins. It may have the following columns:

Column Name	Description
MID	Identifier of the machine that the process was observed on.
sshd_PID_hash	Identifier of the sshd privileged process associated with the login.
login_time	Time of login.
login_username	The username used in the login.
src_IP_addr	Source IP address (connection was initiated from this IP address).
src_port	Source port.
dst_IP_addr	Destination IP address (connection was made to this IP address).
dst_port	Destination port.

A login-lineage table may maintain the lineage of ssh login sessions. It may have the following columns:

Column Name	Description
MID	Identifier of the machine that the ssh login was observed on.
sshd_PID_hash	Identifier of the sshd privileged process associated with the login.
parent_MID	Identifier of the machine that the parent ssh login was observed on.
parent_sshd_PID_hash	Identifier of the sshd privileged process associated with the parent login.
origin_MID	Identifier of the machine that the origin ssh login was observed on.
origin_sshd_PID_hash	Identifier of the sshd privileged process associated with the origin login.

The parent_MID and parent_sshd_PID_hash columns can be null if there is no parent ssh login. In that case, the (MID, sshd_PID_hash) tuple will be the same as the (origin_MID, origin_sshd_PID_hash) tuple.

FIG. 3H illustrates an example of a process for performing extended user tracking. In various embodiments, process 361 is performed by data platform 12. The process begins at 362 when data associated with activities occurring in a network environment (such as entity A's datacenter) is received. One example of such data that can be received at 362 is agent-collected data described above (e.g., in conjunction with process 200). At 363, the received network activity is used to identify user login activity. And, at 364, a logical graph that links the user login activity to at least one user and at least one process is generated (or updated, as applicable). Additional detail regarding process 361, and in particular, portions 363 and 364 of process 361 are described in more detail below (e.g., in conjunction with discussion of FIG. 3J).

FIG. 3I depicts a representation of a user logging into a first machine, then into a second machine from the first machine, and then making an external connection. The scenario depicted in FIG. 3I is used to describe an example of processing that can be performed on data collected by agents to generate extended user session tracking information. FIG. 3I is an alternate depiction of the information shown in FIGS. 3F and 3G.

At time t1 (365), a first ssh connection is made to Machine A (366) from an external source (367) by a user having a username of "X." In the following example, suppose the external source has an IP address of 1.1.1.10 and uses source port 10000 to connect to Machine A (which has an IP address of 2.2.2.20 and a destination port 22). External source 367 is considered an external source because its IP address is outside of the environment being monitored (e.g., is a node outside of entity A's datacenter, connecting to a node inside of entity A's datacenter).

A first ssh login session LS1 is created on machine A for user X. The privileged openssh process for this login is A1 (368). Under the login session LS1, the user creates a bash shell process with PID_hash A2 (369).

At time t2 (370), inside the bash shell process A2, the user runs an ssh program under a new process A3 (371) to log in to machine B (372) with a different username ("Y"). In particular, an ssh connection is made from source IP address 2.2.2.20 and source port 10001 (Machine A's source information) to destination IP address 2.2.2.21 and destination port 22 (Machine B's destination information).

A second ssh login session LS2 is created on machine B for user Y. The privileged openssh process for this login is B1 (373). Under the login session LS2, the user creates a bash shell process with PID_hash B2 (374).

At time t3 (376), inside the bash shell process B2, the user runs a curl command under a new process B3 (377) to

download a file from an external destination (378). In particular, an HTTPS connection is made from source IP address 2.2.2.21 and source port 10002 (Machine B's source information) to external destination IP address 3.3.3.30 and destination port 443 (the external destination's information).

Using techniques described herein, it is possible to determine the original user who initiated the connection to external destination 378, which in this example is a user having the username X on machine A (where the extended user session can be determined to start with ssh login session LS1).

Based on local descendant tracking, the following determinations can be on machine A and B without yet having performed additional processing (described in more detail below):

A3 is a descendant of A1 and thus associated with LS1.

The connection to the external domain from machine B is initiated by B3.

B3 is a descendant of B1 and is thus associated with LS2.

Connection to the external domain is thus associated with LS2.

An association between A3 and LS2 can be established based on the fact that LS2 was created based on an ssh connection initiated from A3. Accordingly, it can be determined that LS2 is a child of LS1.

To determine the user responsible for making the connection to the external destination (e.g., if it were a known bad destination), first, the process that made the connection would be traced, i.e., from B3 to LS2. Then LS2 would be traced to LS1 (i.e., LS1 is the origin login session for LS2). Thus the user for this connection is the user for LS1, i.e., X. As represented in FIG. 3I, one can visualize the tracing by following the links (in the reverse direction of arrows) from external destination 378 to A1 (368).

In the example scenario, it is assumed that both ssh connections occur in the same analysis period. However, the approaches described herein will also work for connections and processes that are created in different time periods.

FIG. 3J illustrates an example of a process for performing extended user tracking. In various embodiments, process 380 is performed periodically (e.g., once an hour in a batch fashion) by ssh tracker 148 to generate new output data. In general, batch processing allows for efficient analysis of large volumes of data. However, the approach can be adapted, as applicable, to process input data on a record-by-record fashion while maintaining the same logical data processing flow. As applicable the results of a given portion of process 380 are stored for use in a subsequent portion.

The process begins at 381 when new ssh connection records are identified. In particular, new ssh connections started during the current time period are identified by querying the connections table. The query uses filters on the start_time and dst_port columns. The values of the range filter on the start_time column are based on the current time period. The dst_port column is checked against ssh listening port(s). By default, the ssh listening port number is 22. However, as this could vary across environments, the port(s) that openssh servers are listening to in the environment can be determined by data collection agents dynamically and used as the filter value for the dst_port as applicable. In the scenario depicted in FIG. 3I, the query result will generate the records shown in FIG. 3K. Note that for the connection between machine A and B, the two machines are likely to report start_time values that are not exactly the same but close enough to be considered matching (e.g., within one minute or another appropriate amount of time). In the above table, they are shown to be the same for simplicity.

At 382, ssh connection records reported from source and destination sides of the same connection are matched. The ssh connection records (e.g., returned from the query at 381) are matched based on the following criteria:

The five tuples (src_IP, dst_IP, IP_prot, src_port, dst_port) of the connection records must match.

The delta between the start times of the connections must be within a limit that would account for the worst case clock difference expected between two machines in the environment and typical connection setup latency.

If there are multiple matches possible, then the match with the smallest time delta is chosen.

Note that record 390 from machine A for the incoming connection from the external source cannot be matched with another record as there is an agent only on the destination side for this connection. Example output of portion 382 of process 380 is shown in FIG. 3L. The values in the dst_PID_hash column (391) are that of the sshd privileged process associated with ssh logins.

At 383, new logins during the current time period are identified by querying the logins table. The query uses a range filter on the login_time column with values based on the current time period. In the example depicted in FIG. 3I, the query result will generate the records depicted in FIG. 3M.

At 384, matched ssh connection records created at 382 and new login records created at 383 are joined to create new records that will eventually be stored in the login-connection table. The join condition is that dst_MID of the matched connection record is equal to the MID of the login record and the dst_PID_hash of the matched connection record is equal to the sshd_PID_hash of the login record. In the example depicted in FIG. 3I, the processing performed at 384 will generate the records depicted in FIG. 3N.

At 385, login-local-descendant records in the lookback time period are identified. It is possible that a process that is created in a previous time period makes an ssh connection in the current analysis batch period. Although not depicted in the example illustrated in FIG. 3I, consider a case where bash process A2 does not create ssh process A3 right away but instead that the ssh connection A3 later makes to machine B is processed in a subsequent time period than the one where A2 was processed. While processing this subsequent time period in which processes A3 and B1 are seen, knowledge of A2 would be useful in establishing that B1 is associated with A3 (via ssh connection) which is associated with A2 (via process parentage) which in turn would be useful in establishing that the parent of the second ssh login is the first ssh login. The time period for which look back is performed can be limited to reduce the amount of historical data that is considered. However, this is not a requirement (and the amount of look back can be determined, e.g., based on available processing resources). The login local descendants in the lookback time period can be identified by querying the login-local-descendant table. The query uses a range filter on the login_time column where the range is from start_time_of_current_period-lookback_time to start_time_of_current_period. (No records as a result of performing 385 on the scenario depicted in FIG. 3I are obtained, as only a single time period is applicable in the example scenario.)

At 386, new processes that are started in the current time period are identified by querying the processes table. The query uses a range filter on the start_time column with values based on the current time period. In the example depicted in FIG. 3I, the processing performed at 386 will generate the records depicted in FIG. 3O.

At 387, new login-local-descendant records are identified. The purpose is to determine whether any of the new processes in the current time period are descendants of an ssh login process and if so to create records that will be stored in the login-local-descendant table for them. In order to do so, the parent-child relationships between the processes are recursively followed. Either a top down or bottom up approach can be used. In a top down approach, the ssh local descendants in the lookback period identified at 385, along with new ssh login processes in the current period identified at 384 are considered as possible ancestors for the new processes in the current period identified at 386.

Conceptually, the recursive approach can be considered to include multiple sub-steps where new processes that are identified to be ssh local descendants in the current sub-step are considered as ancestors for the next step. In the example scenario depicted in FIG. 3I, the following descendancy relationships will be established in two sub-steps:

Sub-Step 1:

Process A2 is a local descendant of LS1 (i.e., MID=A, sshd_PID_hash=A1) because it is a child of process A1 which is the login process for LS1.

Process B2 is a local descendant of LS2 (i.e., MID=B, sshd_PID_hash=B1) because it is a child of process B1 which is the login process for LS2.

Sub-Step 2:

Process A3 is a local descendant of LS1 because it is a child of process A2 which is associated to LS1 in sub-step 1.

Process B3 is a local descendant of LS2 because it is a child of process B1 which is associated to LS2 in sub-step 1.

Implementation portion 387 can use a datastore that supports recursive query capabilities, or, queries can be constructed to process multiple conceptual sub-steps at once. In the example depicted in FIG. 3I, the processing performed at 387 will generate the records depicted in FIG. 3P. Note that the ssh privileged processes associated with the logins are also included as they are part of the login session.

At 388, the lineage of new ssh logins created in the current time period is determined by associating their ssh connections to source processes that may be descendants of other ssh logins (which may have been created in the current period or previous time periods). In order to do so, first an attempt is made to join the new ssh login connections in the current period (identified at 384) with the combination of the login local descendants in the lookback period (identified at 385) and the login local descendants in the current time period (identified at 386). This will create adjacency relationships between child and parent logins. In the example depicted in FIG. 3I, the second ssh login connection will be associated with process A3 and an adjacency relationship between the two login sessions will be created (as illustrated in FIG. 3Q).

Next, the adjacency relationships are used to find the original login sessions. While not shown in the sample scenario, there could be multiple ssh logins in a chain in the current time period, in which case a recursive approach (as in 387) could be used. At the conclusion of portion 388, the login lineage records depicted in FIG. 3R will be generated.

Finally, at 389, output data is generated. In particular, the new login-connection, login-local-descendant, and login-lineage records generated at 384, 387, and 388 are inserted into their respective output tables (e.g., in a transaction manner).

An alternate approach to matching TCP connections between machines running an agent is for the client to generate a connection GUID and send it in the connection request (e.g., the SYN packet) it sends and for the server to extract the GUID from the request. If two connection records from two machines have the same GUID, they are for the same connection. Both the client and server will store the GUID (if it exists) in the connection records they maintain and report. On the client side, the agent can configure the network stack (e.g., using IP tables functionality on Linux) to intercept an outgoing TCP SYN packet and modify it to add the generated GUID as a TCP option. On the server side, the agent already extracts TCP SYN packets and thus can look for this option and extract the GUID if it exists.

Example graph-based user tracking and threat detection embodiments associated with data platform 12 will now be described. Administrators and other users of network environments (e.g., entity A's datacenter 104) often change roles to perform tasks. As one example, suppose that at the start

of a workday, an administrator (hereinafter "Joe Smith") logs in to a console, using an individualized account (e.g., username=joe.smith). Joe performs various tasks as himself (e.g., answering emails, generating status reports, writing code, etc.). For other tasks (e.g., performing updates), Joe may require different/additional permission than his individual account has (e.g., root privileges). One way Joe can gain access to such permissions is by using sudo, which will allow Joe to run a single command with root privileges. Another way Joe can gain access to such permissions is by su or otherwise logging into a shell as root. After gaining root privileges, another thing that Joe can do is switch identities. As one example, to perform administrative tasks, Joe may use "su help" or "su database-admin" to become (respectively) the help user or the database-admin user on a system. He may also connect from one machine to another, potentially changing identities along the way (e.g., logging in as joe.smith at a first console, and connecting to a database server as database-admin). When he's completed various administrative tasks, Joe can relinquish his root privileges by closing out of any additional shells created, reverting back to a shell created for user joe.smith.

While there are many legitimate reasons for Joe to change his identity throughout the day, such changes may also correspond to nefarious activity. Joe himself may be nefarious, or Joe's account (joe.smith) may have been compromised by a third party (whether an "outsider" outside of entity A's network, or an "insider"). Using techniques described herein, the behavior of users of the environment can be tracked (including across multiple accounts and/or multiple machines) and modeled (e.g., using various graphs described herein). Such models can be used to generate alerts (e.g., to anomalous user behavior). Such models can also be used forensically, e.g., helping an investigator visualize various aspects of a network and activities that have occurred, and to attribute particular types of actions (e.g., network connections or file accesses) to specific users.

In a typical day in a datacenter, a user (e.g., Joe Smith) will log in, run various processes, and (optionally) log out. The user will typically log in from the same set of IP addresses, from IP addresses within the same geographical area (e.g., city or country), or from historically known IP addresses/geographical areas (i.e., ones the user has previously/occasionally used). A deviation from the user's typical (or historical) behavior indicates a change in login behavior. However, it does not necessarily mean that a breach has occurred. Once logged into a datacenter, a user may take a variety of actions. As a first example, a user might execute a binary/script. Such binary/script might communicate with other nodes in the datacenter, or outside of the datacenter, and transfer data to the user (e.g., executing "curl" to obtain data from a service external to the datacenter). As a second example, the user can similarly transfer data (e.g., out of the datacenter), such as by using POST. As a third example, a user might change privilege (one or more times), at which point the user can send/receive data as per above. As a fourth example, a user might connect to a different machine within the datacenter (one or more times), at which point the user can send/receive data as per the above.

In various embodiments, the above information associated with user behavior is broken into four tiers. The tiers represent example types of information that data platform 12 can use in modeling user behavior:

1. The user's entry point (e.g., domains, IP addresses, and/or geolocation information such as country/city) from which a user logs in.
2. The login user and machine class.
3. Binaries, executables, processes, etc. a user launches.

4. Internal servers with which the user (or any of the user's processes, child processes, etc.) communicates, and external contacts (e.g., domains, IP addresses, and/or geo-location information such as country/city) with which the user communicates (i.e., transfers data).

In the event of a security breach, being able to concretely answer questions about such information can be very important. And, collectively, such information is useful in providing an end-to-end path (e.g., for performing investigations).

In the following example, suppose a user ("UserA") logs into a machine ("Machine01") from a first IP address ("IP01"). Machine01 is inside a datacenter. UserA then launches a script ("runnable.sh") on Machine01. From Machine01, UserA next logs into a second machine ("Machine02") via ssh, also as UserA, also within the datacenter. On Machine02, UserA again launches a script ("new_runnable.sh"). On Machine02, UserA then changes privilege, becoming root on Machine02. From Machine02, UserA (now as root) logs into a third machine ("Machine03") in the datacenter via ssh, as root on Machine03. As root on Machine03, the user executes a script ("collect_data.sh") on Machine03. The script internally communicates (as root) to a MySQL-based service internal to the datacenter, and downloads data from the MySQL-based service. Finally, as root on Machine03, the user externally communicates with a server outside the datacenter ("External01"), using a POST command. To summarize what has occurred, in this example, the source/entry point is IPO1. Data is transferred to an external server External01. The machine performing the transfer to External01 is Machine03. The user transferring the data is "root" (on Machine03), while the actual user (hiding behind root) is UserA.

In the above scenario, the "original user" (ultimately responsible for transmitting data to External01) is UserA, who logged in from IP01. Each of the processes ultimately started by UserA, whether started at the command line (tty) such as "runnable.sh" or started after an ssh connection such as "new_runnable.sh," and whether as UserA, or as a subsequent identity, are all examples of child processes which can be arranged into a process hierarchy.

As previously mentioned, machines can be clustered together logically into machine clusters. One approach to clustering is to classify machines based on information such as the types of services they provide/binaries they have installed upon them/processes they execute. Machines sharing a given machine class (as they share common binaries/services/etc.) will behave similarly to one another. Each machine in a datacenter can be assigned to a machine cluster, and each machine cluster can be assigned an identifier (also referred to herein as a machine class). One or more tags can also be assigned to a given machine class (e.g., database_servers_west or prod_web_frontend). One approach to assigning a tag to a machine class is to apply term frequency analysis (e.g., TF/IDF) to the applications run by a given machine class, selecting as tags those most unique to the class. Data platform 12 can use behavioral baselines taken for a class of machines to identify deviations from the baseline (e.g., by a particular machine in the class).

FIG. 3S illustrates an example of a process for detecting anomalies. In various embodiments, process 392 is performed by data platform 12. As explained above, a given session will have an original user. And, each action taken by the original user can be tied back to the original user, despite privilege changes and/or lateral movement throughout a datacenter. Process 392 begins at 393 when log data associated with a user session (and thus an original user) is

received. At 394, a logical graph is generated, using at least a portion of the collected data. When an anomaly is detected (395), it can be recorded, and as applicable, an alert is generated (396). The following are examples of graphs that can be generated (e.g., at 394), with corresponding examples of anomalies that can be detected (e.g., at 395) and alerted upon (e.g., at 396).

FIG. 4A illustrates a representation of an embodiment of an insider behavior graph. In the example of FIG. 4A, each node in the graph can be: (1) a cluster of users; (2) a cluster of launched processes; (3) a cluster of processes/servers running on a machine class; (4) a cluster of external IP addresses (of incoming clients); or (5) a cluster of external servers based on DNS/IP/etc. As depicted in FIG. 4A, graph data is vertically tiered into four tiers. Tier 0 (400) corresponds to entry point information (e.g., domains, IP addresses, and/or geolocation information) associated with a client entering the datacenter from an external entry point. Entry points are clustered together based on such information. Tier 1 (401) corresponds to a user on a machine class, with a given user on a given machine class represented as a node. Tier 2 (402) corresponds to launched processes, child processes, and/or interactive processes. Processes for a given user and having similar connectivity (e.g., sharing the processes they launch and the machines with which they communicate) are grouped into nodes. Finally, Tier 3 (403) corresponds to the services/servers/domains/IP addresses with which processes communicate. A relationship between the tiers can be stated as follows: Tier 0 nodes log in to tier 1 nodes. Tier 1 nodes launch tier 2 nodes. Tier 2 nodes connect to tier 3 nodes.

The inclusion of an original user in both Tier 1 and Tier 2 allows for horizontal tiering. Such horizontal tiering ensures that there is no overlap between any two users in Tier 1 and Tier 2. Such lack of overlap provides for faster searching of an end-to-end path (e.g., one starting with a Tier 0 node and terminating at a Tier 3 node). Horizontal tiering also helps in establishing baseline insider behavior. For example, by building an hourly insider behavior graph, new edges/changes in edges between nodes in Tier 1 and Tier 2 can be identified. Any such changes correspond to a change associated with the original user. And, any such changes can be surfaced as anomalous and alerts can be generated.

As explained above, Tier 1 corresponds to a user (e.g., user "U") logging into a machine having a particular machine class (e.g., machine class "M"). Tier 2 is a cluster of processes having command line similarity (e.g., CType "C"), having an original user "U," and running as a particular effective user (e.g., user "U1"). The value of U1 may be the same as U (e.g., joe.smith in both cases), or the value of U1 may be different (e.g., U=joe.smith and U1=root). Thus, while an edge may be present from a Tier 1 node to a Tier 2 node, the effective user in the Tier 2 node may or may not match the original user (while the original user in the Tier 2 node will match the original user in the Tier 1 node).

A change from a user U into a user U1 can take place in a variety of ways. Examples include where U becomes U1 on the same machine (e.g., via su), and also where U sshes to other machine(s). In both situations, U can perform multiple changes, and can combine approaches. For example, U can become U1 on a first machine, ssh to a second machine (as U1), become U2 on the second machine, and ssh to a third machine (whether as user U2 or user U3). In various embodiments, the complexity of how user U ultimately becomes U3 (or U5, etc.) is hidden from a viewer of an insider behavior graph, and only an original user (e.g., U) and the effective user of a given node (e.g., U5) are

51

depicted. As applicable (e.g., if desired by a viewer of the insider behavior graph), additional detail about the path (e.g., an end-to-end path of edges from user U to user U5) can be surfaced (e.g., via user interactions with nodes).

FIG. 4B illustrates an example of a portion of an insider behavior graph (e.g., as rendered in a web browser). In the example shown, node 405 (the external IP address, 52.32.40.231) is an example of a Tier 0 node, and represents an entry point into a datacenter. As indicated by directional arrows 406 and 407, two users, “user1_prod” and “user2_prod,” both made use of the source IP 52.32.40.231 when logging in between 5 pm and 6 pm on Sunday July 30 (408). Nodes 409 and 410 are examples of Tier 1 nodes, having user1_prod and user2_prod as associated respective original users. As previously mentioned, Tier 1 nodes correspond to a combination of a user and a machine class. In the example depicted in FIG. 4B, the machine class associated with nodes 409 and 410 is hidden from view to simplify visualization, but can be surfaced to a viewer of interface 404 (e.g., when the user clicks on node 409 or 410).

Nodes 414-423 are examples of Tier 2 nodes-processes that are launched by users in Tier 1 and their child, grandchild, etc. processes. Note that also depicted in FIG. 4B is a Tier 1 node 411 that corresponds to a user, “root,” that logged in to a machine cluster from within the datacenter (i.e., has an entry point within the datacenter). Nodes 425-1 and 425-2 are examples of Tier 3 nodes-internal/external IP addresses, servers, etc., with which Tier 2 nodes communicate.

In the example shown in FIG. 4B, a viewer of interface 404 has clicked on node 423. As indicated in region 426, the user running the marathon container is “root.” However, by following the directional arrows in the graph backwards from node 423 (i.e., from right to left), the viewer can determine that the original user, responsible for node 423, is “user1_prod,” who logged into the datacenter from IP 52.32.40.231.

The following are examples of changes that can be tracked using an insider behavior graph model:

- A user logs in from a new IP address.
- A user logs in from a geolocation not previously used by that user.
- A user logs into a new machine class.
- A user launches a process not previously used by that user.
- A user connects to an internal server to which the user has not previously connected.
- An original user communicates with an external server (or external server known to be malicious) with which that user has not previously communicated.
- A user communicates with an external server which has a geolocation not previously used by that user.

Such changes can be surfaced as alerts, e.g., to help an administrator determine when/what anomalous behavior occurs within a datacenter. Further, the behavior graph model can be used (e.g., during forensic analysis) to answer questions helpful during an investigation. Examples of such questions include:

- Was there any new login activity (Tier 0) in the timeframe being investigated? As one example, has a user logged in from an IP address with unknown geolocation information? Similarly, has a user started communicating externally with a new Tier 3 node (e.g., one with unknown geolocation information).
- Has there been any suspicious login activity (Tier 0) in the timeframe being investigated? As one example, has a user logged in from an IP address that corresponds to

52

a known bad IP address as maintained by Threat aggregator 150? Similarly, has there been any suspicious Tier 3 activity?

Were any anomalous connections made within the datacenter during the timeframe being investigated? As one example, suppose a given user (“Frank”) typically enters a datacenter from a particular IP address (or range of IP addresses), and then connects to a first machine type (e.g., bastion), and then to a second machine type (e.g., database_prod). If Frank has directly connected to database_prod (instead of first going through bastion) during the timeframe, this can be surfaced using the insider graph.

Who is (the original user) responsible for running a particular process?

An example of an insider behavior graph being used in an investigation is depicted in FIGS. 4C and 4D. FIG. 4C depicts a baseline of behavior for a user, “Bill.” As shown in FIG. 4C, Bill typically logs into a datacenter from the IP address, 71.198.44.40 (427). He typically makes use of ssh (428), and sudo (429), makes use of a set of typical applications (430) and connects (as root) with the external service, api.lacework.net (431).

Suppose Bill’s credentials are compromised by a nefarious outsider (“Eddie”). FIG. 4D depicts an embodiment of how the graph depicted in FIG. 4C would appear once Eddie begins exfiltrating data from the datacenter. Eddie logs into the datacenter (using Bill’s credentials) from 52.5.66.8 (432). As Bill, Eddie escalates her privilege to root (e.g., via su), and then becomes a different user, Alex (e.g., via su alex). As Alex, Eddie executes a script, “sneak.sh” (433), which launches another script, “post.sh” (434), which contacts external server 435 which has an IP address of 52.5.66.7, and transmits data to it. Edges 436-439 each represent changes in Bill’s behavior. As previously mentioned, such changes can be detected as anomalies and associated alerts can be generated. As a first example, Bill logging in from an IP address he has not previously logged in from (436) can generate an alert. As a second example, while Bill does typically make use of sudo (429), he has not previously executed sneak.sh (433) or post.sh (434) and the execution of those scripts can generate alerts as well. As a third example, Bill has not previously communicated with server 435, and an alert can be generated when he does so (439). Considered individually, each of edges 436-439 may indicate nefarious behavior, or may be benign. As an example of a benign edge, suppose Bill begins working from a home office two days a week. The first time he logs in from his home office (i.e., from an IP address that is not 71.198.44.40), an alert can be generated that he has logged in from a new location. Over time, however, as Bill continues to log in from his home office but otherwise engages in typical activities, Bill’s graph will evolve to include logins from both 71.198.44.40 and his home office as baseline behavior. Similarly, if Bill begins using a new tool in his job, an alert can be generated the first time he executes the tool, but over time will become part of his baseline.

In some cases, a single edge can indicate a serious threat. For example, if server 432 (or 435) is included in a known bad IP listing, edge 436 (or 439) indicates compromise. An alert that includes an appropriate severity level (e.g., “threat level high”) can be generated. In other cases, a combination of edges could indicate a threat (where a single edge might otherwise result in a lesser warning). In the example shown in FIG. 4D, the presence of multiple new edges is indicative of a serious threat. Of note, even though “sneak.sh” and “post.sh” were executed by Alex, because data platform 12

53

also keeps track of an original user, the compromise of user B's account will be discovered.

FIG. 4E illustrates a representation of an embodiment of a user login graph. In the example of FIG. 4E, tier 0 (440) clusters source IP addresses as belonging to a particular country (including an "unknown" country) or as a known bad IP. Tier 1 (441) clusters user logins, and tier 2 (442) clusters type of machine class into which a user is logging in. The user login graph tracks the typical login behavior of users. By interacting with a representation of the graph, answers to questions such as the following can be obtained:

Where is a user logging in from?

Have any users logged in from a known bad address?

Have any non-developer users accessed development machines?

Which machines does a particular user access?

Examples of alerts that can be generated using the user login graph include:

A user logs in from a known bad IP address.

A user logs in from a new country for the first time.

A new user logs into the datacenter for the first time.

A user accesses a machine class that the user has not previously accessed.

One way to track privilege changes in a datacenter is by monitoring a process hierarchy of processes. To help filter out noisy commands/processes such as "su-u," the hierarchy of processes can be constrained to those associated with network activity. In a *nix system, each process has two identifiers assigned to it, a process identifier (PID) and a parent process identifier (PPID). When such a system starts, the initial process is assigned a PID 0. Each user process has a corresponding parent process.

Using techniques described herein, a graph can be constructed (also referred to herein as a privilege change graph) which models privilege changes. In particular, a graph can be constructed which identifies where a process P1 launches a process P2, where P1 and P2 each have an associated user U1 and U2, with U1 being an original user, and U2 being an effective user. In the graph, each node is a cluster of processes (sharing a CType) executed by a particular (original) user. As all the processes in the cluster belong to the same user, a label that can be used for the cluster is the user's username. An edge in the graph, from a first node to a second node, indicates that a user of the first node changed its privilege to the user of the second node.

FIG. 4F illustrates an example of a privilege change graph. In the example shown in FIG. 4F, each node (e.g., nodes 444 and 445) represents a user. Privilege changes are indicated by edges, such as edge 446.

As with other graphs, anomalies in graph 443 can be used to generate alerts. Three examples of such alerts are as follows:

New user entering the datacenter. Any time a new user enters the datacenter and runs a process, the graph will show a new node, with a new CType. This indicates a new user has been detected within the datacenter. FIG. 4F is a representation of an example of an interface that depicts such an alert. Specifically, as indicated in region 447, an alert for the time period 1 pm-2 pm on June 8 was generated. The alert identifies that a new user, Bill (448) executed a process.

Privilege change. As explained above, a new edge, from a first node (user A) to a second node (user B) indicates that user A has changed privilege to user B.

Privilege escalation. Privilege escalation is a particular case of privilege change, in which the first user becomes root.

54

An example of an anomalous privilege change and an example of an anomalous privilege escalation are each depicted in graph 450 of FIG. 4G. In particular, as indicated in region 451, two alerts for the time period 2 pm-3 pm on June 8 were generated (corresponding to the detection of the two anomalous events). In region 452, root has changed privilege to the user "daemon," which root has not previously done. This anomaly is indicated to the user by highlighting the daemon node (e.g., outlining it in a particular color, e.g., red). As indicated by edge 453, Bill has escalated his privilege to the user root (which can similarly be highlighted in region 454). This action by Bill represents a privilege escalation.

An Extensible query interface for dynamic data compositions and filter applications will now be described.

As described herein, datacenters are highly dynamic environments. And, different customers of data platform 12 (e.g., entity A vs. entity B) may have different/disperate needs/requirements of data platform 12, e.g., due to having different types of assets, different applications, etc. Further, as time progresses, new software tools will be developed, new types of anomalous behavior will be possible (and should be detectable), etc. In various embodiments, data platform 12 makes use of predefined relational schema (including by having different predefined relational schema for different customers). However, the complexity and cost of maintaining/updating such predefined relational schema can rapidly become problematic-particularly where the schema includes a mix of relational, nested, and hierarchical (graph) datasets. In other embodiments, the data models and filtering applications used by data platform 12 are extensible. As will be described in more detail below, in various embodiments, data platform 12 supports dynamic query generation by automatic discovery of join relations via static or dynamic filtering key specifications among composable data sets. This allows a user of data platform 12 to be agnostic to modifications made to existing data sets as well as creation of new data sets. The extensible query interface also provides a declarative and configurable specification for optimizing internal data generation and derivations.

As will also be described in more detail below, data platform 12 is configured to dynamically translate user interactions (e.g., received via web app 120) into SQL queries (and without the user needing to know how to write queries). Such queries can then be performed (e.g., by query service 166) against any compatible backend (e.g., data store 30).

FIG. 4H illustrates an example of a user interacting with a portion of an interface. When a user visits data platform 12 (e.g., via web app 120 using a browser), data is extracted from data store 30 as needed (e.g., by query service 166), to provide the user with information, such as the visualizations depicted variously herein. As the user continues to interact with such visualizations (e.g., clicking on graph nodes, entering text into search boxes, navigating between tabs (e.g., tab 455 vs. 465)), such interactions act as triggers that cause query service 166 to continue to obtain information from data store 30 as needed (and as described in more detail below).

In the example shown in FIG. 4H, user A is viewing a dashboard that provides various information about entity A users (455), during the time period March 2 at midnight-March 25 at 7 pm (which she selected by interacting with region 456). Various statistical information is presented to user A in region 457. Region 458 presents a timeline of events that occurred during the selected time period. User A has opted to list only the critical, high, and medium events

during the time period by clicking on the associated boxes (459-461). A total of 55 low severity, and 155 info-only events also occurred during the time period. Each time user A interacts with an element in FIG. 4H (e.g., clicks on box 461, clicks on link 464-1, or clicks on tab 465), her actions are translated/formalized into filters on the data set and used to dynamically generate SQL queries. The SQL queries are generated transparently to user A (and also to a designer of the user interface shown in FIG. 4H).

User A notes in the timeline (462) that a user, UserA, connected to a known bad server (examplebad.com) using wget, an event that has a critical severity level. User A can click on region 463 to expand details about the event inline (which will display, for example, the text “External connection made to known bad host examplebad.com at port 80 from application ‘wget’ running on host devl.lacework.internal as user userA”) directly below timeline 462. User A can also click on link 464-1, which will take her to a dossier for the event (depicted in FIG. 4I). As will be described in more detail below, a dossier is a template for a collection of visualizations.

As shown in interface 466, the event of UserA using wget to contact examplebad.com on March 16 was assigned an event ID of 9291 by data platform 12 (467). For convenience to user A, the event is also added to her dashboard in region 476 as a bookmark (468). A summary of the event is depicted in region 469. By interacting with boxes shown in region 470, user A can see a timeline of related events. In this case, user A has indicated that she would like to see other events involving the wget application (by clicking box 471). Events of critical and medium security involving wget occurred during the one hour window selected in region 472.

Region 473 automatically provides user A with answers to questions that may be helpful to have answers to while investigating event 9291. If user A clicks on any of the links in the event description (474), she will be taken to a corresponding dossier for the link. As one example, suppose user A clicks on link 475. She will then be presented with interface 477 shown in FIG. 4J.

Interface 477 is an embodiment of a dossier for a domain. In this example, the domain is “examplebad.com,” as shown in region 478. Suppose user A would like to track down more information about interactions entity A resources have made with examplebad.com between January 1 and March 20. She selects the appropriate time period in region 479 and information in the other portions of interface 477 automatically update to provide various information corresponding to the selected time frame. As one example, user A can see that contact was made with examplebad.com a total of 17 times during the time period (480), as well as a list of each contact (481). Various statistical information is also included in the dossier for the time period (482). If she scrolls down in interface 477, user A will be able to view various polygraphs associated with examplebad.com, such as an application-communication polygraph (483).

Data stored in data store 30 can be internally organized as an activity graph. In the activity graph, nodes are also referred to as Entities. Activities generated by Entities are modeled as directional edges between nodes. Thus, each edge is an activity between two Entities. One example of an Activity is a “login” Activity, in which a user Entity logs into a machine Entity (with a directed edge from the user to the machine). A second example of an Activity is a “launch” Activity, in which a parent process launches a child process (with a directed edge from the parent to the child). A third example of an Activity is a “DNS query” Activity, in which either a process or a machine performs a query (with a

directed edge from the requestor to the answer, e.g., an edge from a process to www.example.com). A fourth example of an Activity is a network “connected to” Activity, in which processes, IP addresses, and listen ports can connect to each other (with a directed edge from the initiator to the server).

As will be described in more detail below, query service 166 provides either relational views or graph views on top of data stored in data store 30. Typically, a user will want to see data filtered using the activity graph. For example, if an entity was not involved in an activity in a given time period, that entity should be filtered out of query results. Thus, a request to show “all machines” in a given time frame will be interpreted as “show distinct machines that were active” during the time frame.

Query service 166 relies on three main data model elements: fields, entities, and filters. As used herein, a field is a collection of values with the same type (logical and physical). A field can be represented in a variety of ways, including: 1. a column of relations (table/view), 2. a return field from another entity, 3. an SQL aggregation (e.g., COUNT, SUM, etc.), 4. an SQL expression with the references of other fields specified, and 5. a nested field of a JSON object. As viewed by query service 166, an entity is a collection of fields that describe a data set. The data set can be composed in a variety of ways, including: 1. a relational table, 2. a parameterized SQL statement, 3. DynamicSQL created by a Java function, and 4. join/project/aggregate/subclass of other entities. Some fields are common for all entities. One example of such a field is a “first observed” timestamp (when first use of the entity was detected). A second example of such a field is the entity classification type (e.g., one of: 1. Machine (on which an agent is installed), 2. Process, 3. Binary, 4. UID, 5. IP, 6. DNS Information, 7. ListenPort, and 8. PType). A third example of such a field is a “last observed” timestamp.

A filter is an operator that: 1. takes an entity and field values as inputs, 2. a valid SQL expression with specific reference(s) of entity fields, or 3. is a conjunct/disjunct of filters. As will be described in more detail below, filters can be used to filter data in various ways, and limit data returned by query service 166 without changing the associated data set.

As mentioned above, a dossier is a template for a collection of visualizations. Each visualization (e.g., the box including chart 484) has a corresponding card, which identifies particular target information needed (e.g., from data store 30) to generate the visualization. In various embodiments, data platform 12 maintains a global set of dossiers/cards. Users of data platform 12 such as user A can build their own dashboard interfaces using preexisting dossiers/cards as components, and/or they can make use of a default dashboard (which incorporates various of such dossiers/cards).

A JSON file can be used to store multiple cards (e.g., as part of a query service catalog). A particular card is represented by a single JSON object with a unique name as a field name. Each card may be described by the following named fields:

TYPE: the type of the card. Example values include:

Entity (the default type)

SQL

Filters

DynamicSQL

graphFilter

graph

Function

Template

57

PARAMETERS: a JSON array object that contains an array of parameter objects with the following fields:

name (the name of the parameter)
required (a Boolean flag indicating whether the parameter is required or not)

default (a default value of the parameter)

props (a generic JSON object for properties of the parameter. Possible values are: "utype" (a user defined type), and "scope" (an optional property to configure a namespace of the parameter))

value (a value for the parameter-non-null to override the default value defined in nested source entities)

SOURCES: a JSON array object explicitly specifying references of input entities. Each source reference has the following attributes:

name (the card/entity name or fully-qualified Table name)
type (required for base Table entity)

alias (an alias to access this source entity in other fields (e.g., returns, filters, groups, etc))

RETURNS: a required JSON array object of a return field object. A return field object can be described by the following attributes:

field (a valid field name from a source entity)

expr (a valid SQL scalar expression. References to input fields of source entities are specified in the format of #{Entity}.Field}. Parameters can also be used in the expression in the format of \$ {ParameterName})

type (the type of field, which is required for return fields specified by expr. It is also required for all return fields of an Entity with an SQL type)

alias (the unique alias for return field)

aggr (possible aggregations are: COUNT, COUNT_DISTINCT, DISTINCT, MAX, MIN, AVG, SUM, FIRST_VALUE, LAST_VALUE)

case (JSON array object represents conditional expressions "when" and "expr")

fieldsFrom, and, except (specification for projections from a source entity with excluded fields)

props (general JSON object for properties of the return field. Possible properties include: "filterGroup," "title," "format," and "utype")

PROPS: generic JSON objects for other entity properties

SQL: a JSON array of string literals for SQL statements.

Each string literal can contain parameterized expressions \$ {ParameterName} and/or composable entity by #{Entity-Name} GRAPH: required for graph entity. Has the following required fields:

source (including "type," "props," and "keys")

target (including "type," "props," and "keys")

edge (including "type" and "props")

JOINS: a JSON array of join operators. Possible fields for a join operator include:

type (possible join types include: "loj"-Left Outer Join, "join"-Inner Join, "in"-Semi Join, "implicit"-Implicit Join)

left (a left hand side field of join)

right (a right hand side field of join)

keys (key columns for multi-way joins)

order (a join order of multi-way joins)

FKEYS: a JSON array of FilterKey(s). The fields for a FilterKey are:

type (type of FilterKey)

fieldRefs (reference(s) to return fields of an entity defined in the sources field)

alias (an alias of the FilterKey, used in implicit join specification)

58

FILTERS: a JSON array of filters (conjunct). Possible fields for a filter include:

type (types of filters, including: "eq"-equivalent to SQL=, "ne"-equivalent to SQL < >, "ge"-equivalent to SQL >=, "gt"-equivalent to SQL >, "l"-equivalent to SQL <=, "lt"-equivalent to SQL <, "like"-equivalent to SQL LIKE, "not_like"-equivalent to SQL NOT LIKE, "rlike"-equivalent to SQL RLIKE (Snowflake specific), "not_rlike"-equivalent to SQL NOT RLIKE (Snowflake specific), "in"-equivalent to SQL IN, "not_in"-equivalent to SQL NOT IN)

expr (generic SQL expression)

field (field name)

value (single value)

values (for both IN and NOT IN)

ORDERS: a JSON array of ORDER BY for returning fields. Possible attributes for the ORDER BY clause include:

field (field ordinal index (1 based) or field alias)

order (asc/desc, default is ascending order)

GROUPS: a JSON array of GROUP BY for returning fields. Field attributes are:

field (ordinal index (1 based) or alias from the return fields) LIMIT: a limit for the number of records to be returned

OFFSET: an offset of starting position of returned data. Used in combination with limit for pagination.

Suppose customers of data platform 12 (e.g., entity A and entity B) request new data transformations or a new aggregation of data from an existing data set (as well as a corresponding visualization for the newly defined data set). As mentioned above, the data models and filtering applications used by data platform 12 are extensible. Thus, two example scenarios of extensibility are (1) extending the filter data set, and (2) extending a FilterKey in the filter data set.

Data platform 12 includes a query service catalog that enumerates cards available to users of data platform 12. New cards can be included for use in data platform 12 by being added to the query service catalog (e.g., by an operator of data platform 12). For reusability and maintainability, a single external-facing card (e.g., available for use in a dossier) can be composed of multiple (nested) internal cards. Each newly added card (whether external or internal) will also have associated FilterKey(s) defined. A user interface (UI) developer can then develop a visualization for the new data set in one or more dossier templates. The same external card can be used in multiple dossier templates, and a given external card can be used multiple times in the same dossier (e.g., after customization). Examples of external card customization include customization via parameters, ordering, and/or various mappings of external data fields (columns).

As mentioned above, a second extensibility scenario is one in which a FilterKey in the filter data set is extended (i.e., existing template functions are used to define a new data set). As also mentioned above, data sets used by data platform 12 are composable/reusable/extensible, irrespective of whether the data sets are relational or graph data sets. One example data set is the User Tracking polygraph, which is generated as a graph data set (comprising nodes and edges). Like other polygraphs, User Tracking is an external data set that can be visualized both as a graph (via the nodes and edges) and can also be used as a filter data set for other cards, via the cluster identifier (CID) field.

As mentioned above, as users such as user A navigate through/interact with interfaces provided by data platform 12 (e.g., as shown in FIG. 4H), such interactions trigger query service 166 to generate and perform queries against data store 30. Dynamic composition of filter datasets can be

implemented using FilterKeys and FilterKey Types. A FilterKey can be defined as a list of columns and/or fields in a nested structure (e.g., JSON). Instances of the same FilterKey Type can be formed as an Implicit Join Group. The same instance of a FilterKey can participate in different Implicit Join Groups. A list of relationships among all possible Implicit Join Groups is represented as a Join graph for the entire search space to create a final data filter set by traversing edges and producing Join Path(s).

Each card (e.g., as stored in the query service catalog and used in a dossier) can be introspected by a/card/describe/CardID REST request.

At runtime (e.g., whenever it receives a request from web app 120), query service 166 parses the list of implicit joins and creates a Join graph to manifest relationships of FilterKeys among Entities. A Join graph (an example of which is depicted in FIG. 4K) comprises a list of Join Link(s). A Join Link represents each implicit join group by the same FilterKey type. A Join Link maintains a reverse map (Entity-to-FilterKey) of FilterKeys and their Entities. As previously mentioned, Entities can have more than one FilterKey defined. The reverse map guarantees one FilterKey per Entity can be used for each JoinLink. Each JoinLink also maintains a list of entities for the priority order of joins. Each JoinLink is also responsible for creating and adding directional edge(s) to graphs. An edge represents a possible join between two Entities.

At runtime, each Implicit Join uses the Join graph to find all possible join paths. The search of possible join paths starts with the outer FilterKey of an implicit join. One approach is to use a shortest path approach, with breadth first traversal and subject to the following criteria:

- Use the priority order list of Join Links for all entities in the same implicit join group.
- Stop when a node (Entity) is reached which has local filter(s).
- Include all join paths at the same level (depth).
- Exclude join paths based on the predefined rules (path of edges).

FIG. 4L illustrates an example of a process for dynamically generating and executing a query. In various embodiments, process 485 is performed by data platform 12. The process begins at 486 when a request is received to filter information associated with activities within a network environment. One example of such a request occurs in response to user A clicking on tab 465. Another example of such a request occurs in response to user A clicking on link 464-1. Yet another example of such a request occurs in response to user A clicking on link 464-2 and selecting (e.g., from a dropdown) an option to filter (e.g., include, exclude) based on specific criteria that she provides (e.g., an IP address, a username, a range of criteria, etc.).

At 487, a query is generated based on an implicit join. One example of processing that can be performed at 487 is as follows. As explained above, one way dynamic composition of filter datasets can be implemented is by using FilterKeys and FilterKey Types. And, instances of the same FilterKey Type can be formed as an Implicit Join Group. A Join graph for the entire search space can be constructed from a list of all relationships among all possible Join Groups. And, a final data filter set can be created by traversing edges and producing one or more Join Paths. Finally, the shortest path in the join paths is used to generate an SQL query string.

One approach to generating an SQL query string is to use a query building library (authored in an appropriate language such as Java). For example, a common interface

“sqlGen” may be used in conjunction with process 485 as follows. First, a card/entity is composed by a list of input cards/entities, where each input card recursively is composed by its own list of input cards. This nested structure can be visualized as a tree of query blocks (SELECT) in standard SQL constructs. SQL generation can be performed as the traversal of the tree from root to leaf entities (top-down), calling the sqlGen of each entity. Each entity can be treated as a subclass of the Java class (Entity). An implicit join filter (EntityFilter) is implemented as a subclass of Entity, similar to the right hand side of a SQL semi-join operator. Unlike the static SQL semi-join construct, it is conditionally and recursively generated even if it is specified in the input sources of the JSON specification. Another recursive interface can also be used in conjunction with process 485, preSQLGen, which is primarily the entry point for EntityFilter to run a search and generate nested implicit join filters. During preSQLGen recursive invocations, the applicability of implicit join filters is examined and pushed down to its input subquery list. Another top-down traversal, pullUp-Cachable, can be used to pull up common sub-query blocks, including those dynamically generated by preSQLGen, such that SELECT statements of those cacheable blocks are generated only once at top-level WITH clauses. A recursive interface, sqlWith, is used to generate nested subqueries inside WITH clauses. The recursive calls of a sqlWith function can generate nested WITH clauses as well. An sqlFrom function can be used to generate SQL FROM clauses by referencing those subquery blocks in the WITH clauses. It also produces INNER/OUTER join operators based on the joins in the specification. Another recursive interface, sqlWhere, can be used to generate conjuncts and disjuncts of local predicates and semi-join predicates based on implicit join transformations. Further, sqlProject, sqlGroupBy, sqlOrderBy, and sqlLimitOffset can respectively be used to translate the corresponding directives in JSON spec to SQL SELECT list, GROUP BY, ORDER BY, and LIMIT/OFFSET clauses.

Returning to process 485, at 488, the query (generated at 487) is used to respond to the request. As one example of the processing performed at 488, the generated query is used to query data store 30 and provide (e.g., to web app 120) fact data formatted in accordance with a schema (e.g., as associated with a card associated with the request received at 486).

Although the examples described herein largely relate to embodiments where data is collected from agents and ultimately stored in a data store such as those provided by Snowflake, in other embodiments data that is collected from agents and other sources may be stored in different ways. For example, data that is collected from agents and other sources may be stored in a data warehouse, data lake, data mart, and/or any other data store.

A data warehouse may be embodied as an analytic database (e.g., a relational database) that is created from two or more data sources. Such a data warehouse may be leveraged to store historical data, often on the scale of petabytes. Data warehouses may have compute and memory resources for running complicated queries and generating reports. Data warehouses may be the data sources for business intelligence (‘BI’) systems, machine learning applications, and/or other applications. By leveraging a data warehouse, data that has been copied into the data warehouse may be indexed for good analytic query performance, without affecting the write performance of a database (e.g., an Online Transaction Processing (‘OLTP’) database). Data warehouses also enable the joining data from multiple sources for analysis.

For example, a sales OLTP application probably has no need to know about the weather at various sales locations, but sales predictions could take advantage of that data. By adding historical weather data to a data warehouse, it would be possible to factor it into models of historical sales data.

Data lakes, which store files of data in their native format, may be considered as “schema on read” resources. As such, any application that reads data from the lake may impose its own types and relationships on the data. Data warehouses, on the other hand, are “schema on write,” meaning that data types, indexes, and relationships are imposed on the data as it is stored in the EDW. “Schema on read” resources may be beneficial for data that may be used in several contexts and poses little risk of losing data. “Schema on write” resources may be beneficial for data that has a specific purpose, and good for data that must relate properly to data from other sources. Such data stores may include data that is encrypted using homomorphic encryption, data encrypted using privacy-preserving encryption, smart contracts, non-fungible tokens, decentralized finance, and other techniques.

Data marts may contain data oriented towards a specific business line whereas data warehouses contain enterprise-wide data. Data marts may be dependent on a data warehouse, independent of the data warehouse (e.g., drawn from an operational database or external source), or a hybrid of the two. In embodiments described herein, different types of data stores (including combinations thereof) may be leveraged. Such data stores may be proprietary or may be embodied as vendor provided products or services such as, for example, Google BigQuery, Druid, Amazon Redshift, IBM Db2, Dremio, Databricks Lakehouse Platform, Cloud-
era, Azure Synapse Analytics, and others.

The deployments (e.g., a customer’s cloud deployment) that are analyzed, monitored, evaluated, or otherwise observed by the systems described herein (e.g., systems that include components such as the platform 12 of FIG. 1D, the data collection agents described herein, and/or other components) may be provisioned, deployed, and/or managed using infrastructure as code (‘IaC’). IaC involves the managing and/or provisioning of infrastructure through code instead of through manual processes. With IaC, configuration files may be created that include infrastructure specifications. IaC can be beneficial as configurations may be edited and distributed, while also ensuring that environments are provisioned in a consistent manner. IaC approaches may be enabled in a variety of ways including, for example, using IaC software tools such as Terraform by HashiCorp. Through the usage of such tools, users may define and provide data center infrastructure using JavaScript Object Notation (‘JSON’), YAML, proprietary formats, or some other format. In some embodiments, the configuration files may be used to emulate a cloud deployment for the purposes of analyzing the emulated cloud deployment using the systems described herein. Likewise, the configuration files themselves may be used as inputs to the systems described herein, such that the configuration files may be inspected to identify vulnerabilities, misconfigurations, violations of regulatory requirements, or other issues. In fact, configuration files for multiple cloud deployments may even be used by the systems described herein to identify best practices, to identify configuration files that deviate from typical configuration files, to identify configuration files with similarities in deployments that have been determined to be deficient in some way, or the configuration files may be leveraged in some other ways to detect vulnerabilities, misconfigurations, violations of regulatory requirements, or other issues prior to deploying an infrastructure

that is described in the configuration files. In some embodiments the techniques described herein may be use in multi-cloud, multi-tenant, cross-cloud, cross-tenant, cross-user, industry cloud, digital platform, and other scenarios depending on specific need or situation.

In some embodiments, the deployments that are analyzed, monitored, evaluated, or otherwise observed by the systems described herein (e.g., systems that include components such as the platform 12 of FIG. 1D, the data collection agents described herein, and/or other components) may be monitored to determine the extent to which a particular component has experienced “drift” relative to its associated IaC configuration. Discrepancies between how cloud resources were defined in an IaC configuration file and how they are currently configured in runtime may be identified and remediation workflows may be initiated to generate an alert, reconfigure the deployment, or take some other action. Such discrepancies may occur for a variety of reasons. Such discrepancies may occur, for example, due to maintenance operations being performed, due to incident response tasks being carried out, or for some other reason. Readers will appreciate that while IaC helps avoid initial misconfigurations of a deployment by codifying and enforcing resource creation, resource configuration, security policies, and so on, the systems described herein may prevent unwanted drift from occurring during runtime and after a deployment has been created in accordance with an IaC configuration.

In some embodiments, the deployments (e.g., a customer’s cloud deployment) that are analyzed, monitored, evaluated, or otherwise observed by the systems described herein (e.g., systems that include components such as the platform 12 of FIG. 1D, the data collection agents described herein, and/or other components) may also be provisioned, deployed, and/or managed using security as code (‘SaC’). SaC extends IaC concepts by defining cybersecurity policies and/or standards programmatically, so that the policies and/or standards can be referenced automatically in the configuration scripts used to provision cloud deployments. Stated differently, SaC can automate policy implementation and cloud deployments may even be compared with the policies to prevent “drift.” For example, if a policy is created where all personally identifiable information (‘PII’) or personal health information (‘PHI’) must be encrypted when it is stored, that policy is translated into a process that is automatically launched whenever a developer submits code, and code that violates the policy may be automatically rejected.

In some embodiments, SaC may be implemented by initially classifying workloads (e.g., by sensitivity, by criticality, by deployment model, by segment). Policies that can be instantiated as code may subsequently be designed. For example, compute-related policies may be designed, access-related policies may be designed, application-related policies may be designed, network-related policies may be designed, data-related policies may be designed, and so on. Security as code may then be instantiated through architecture and automation, as successful implementation of SaC can benefit from making key architectural-design decisions and executing the right automation capabilities. Next, operating model protections may be built and supported. For example, an operating model may “shift left” to maximize self-service and achieve full-life-cycle security automation (e.g., by standardizing common development toolchains, CI/CD pipelines, and the like). In such an example, security policies and access controls may be part of the pipeline, automatic code review and bug/defect detection may be performed, automated build processes may be performed, vulnerability scanning may be performed, checks against a

risk-control framework may be made, and other tasks may be performed all before deploying an infrastructure or components thereof.

The systems described herein may be useful in analyzing, monitoring, evaluating, or otherwise observing a GitOps environment. In a GitOps environment, Git may be viewed as the one and only source of truth. As such, GitOps may require that the desired state of infrastructure (e.g., a customer's cloud deployment) be stored in version control such that the entire audit trail of changes to such infrastructure can be viewed or audited. In a GitOps environment, all changes to infrastructure are embodied as fully traceable commits that are associated with committer information, commit IDs, time stamps, and/or other information. In such an embodiment, both an application and the infrastructure (e.g., a customer's cloud deployment) that supports the execution of the application are therefore versioned artifacts and can be audited using the gold standards of software development and delivery. Readers will appreciate that while the systems described herein are described as analyzing, monitoring, evaluating, or otherwise observing a GitOps environment, in other embodiments other source control mechanisms may be utilized for creating infrastructure, making changes to infrastructure, and so on. In these embodiments, the systems described herein may similarly be used for analyzing, monitoring, evaluating, or otherwise observing such environments.

As described in other portions of the present disclosure, the systems described herein may be used to analyze, monitor, evaluate, or otherwise observe a customer's cloud deployment. While securing traditional datacenters requires managing and securing an IP-based perimeter with networks and firewalls, hardware security modules ('HSMs'), security information and event management ('SIEM') technologies, and other physical access restrictions, such solutions are not particularly useful when applied to cloud deployments. As such, the systems described herein may be configured to interact with and even monitor other solutions that are appropriate for cloud deployments such as, for example, "zero trust" solutions.

A zero trust security model (a.k.a., zero trust architecture) describes an approach to the design and implementation of IT systems. A primary concept behind zero trust is that devices should not be trusted by default, even if they are connected to a managed corporate network such as the corporate LAN and even if they were previously verified. Zero trust security models help prevent successful breaches by eliminating the concept of trust from an organization's network architecture. Zero trust security models can include multiple forms of authentication and authorization (e.g., machine authentication and authorization, human/user authentication and authorization) and can also be used to control multiple types of accesses or interactions (e.g., machine-to-machine access, human-to-machine access).

In some embodiments, the systems described herein may be configured to interact with zero trust solutions in a variety of ways. For example, agents that collect input data for the systems described herein (or other components of such systems) may be configured to access various machines, applications, data sources, or other entity through a zero trust solution, especially where local instances of the systems described herein are deployed at edge locations. Likewise, given that zero trust solutions may be part of a customer's cloud deployment, the zero trust solution itself may be monitored to identify vulnerabilities, anomalies, and so on. For example, network traffic to and from the zero trust solution may be analyzed, the zero trust solution may be

monitored to detect unusual interactions, log files generated by the zero trust solution may be gathered and analyzed, and so on.

In some embodiments, the systems described herein may leverage various tools and mechanisms in the process of performing its primary tasks (e.g., monitoring a cloud deployment). For example, Linux eBPF is mechanism for writing code to be executed in the Linux kernel space. Through the usage of eBPF, user mode processes can hook into specific trace points in the kernel and access data structures and other information. For example, eBPF may be used to gather information that enables the systems described herein to attribute the utilization of networking resources or network traffic to specific processes. This may be useful in analyzing the behavior of a particular process, which may be important for observability/SIEM.

The systems described may be configured to collect security event logs (or any other type of log or similar record of activity) and telemetry in real time for threat detection, for analyzing compliance requirements, or for other purposes. In such embodiments, the systems described herein may analyze telemetry in real time (or near real time), as well as historical telemetry, to detect attacks or other activities of interest. The attacks or activities of interest may be analyzed to determine their potential severity and impact on an organization. In fact, the attacks or activities of interest may be reported, and relevant events, logs, or other information may be stored for subsequent examination.

In one embodiment, systems described herein may be configured to collect security event logs (or any other type of log or similar record of activity) and telemetry in real time to provide customers with a SIEM or SIEM-like solution. SIEM technology aggregates event data produced by security devices, network infrastructure, systems, applications, or other source. Centralizing all of the data that may be generated by a cloud deployment may be challenging for a traditional SIEM, however, as each component in a cloud deployment may generate log data or other forms of machine data, such that the collective amount of data that can be used to monitor the cloud deployment can grow to be quite large. A traditional SIEM architecture, where data is centralized and aggregated, can quickly result in large amounts of data that may be expensive to store, process, retain, and so on. As such, SIEM technologies may frequently be implemented such that silos are created to separate the data.

In some embodiments of the present disclosure, data that is ingested by the systems described herein may be stored in a cloud-based data warehouse such as those provided by Snowflake and others. Given that companies like Snowflake offer data analytics and other services to operate on data that is stored in their data warehouses, in some embodiments one or more of the components of the systems described herein may be deployed in or near Snowflake as part of a secure data lake architecture (a.k.a., a security data lake architecture, a security data lake/warehouse). In such an embodiment, components of the systems described herein may be deployed in or near Snowflake to collect data, transform data, analyze data for the purposes of detecting threats or vulnerabilities, initiate remediation workflows, generate alerts, or perform any of the other functions that can be performed by the systems described herein. In such embodiments, data may be received from a variety of sources (e.g., EDR or EDR-like tools that handle endpoint data, cloud access security broker ('CASB') or CASB-like tools that handle data describing interactions with cloud applications, Identity and Access Management ('IAM') or IAM-like

65

tools, and many others), normalized for storage in a data warehouse, and such normalized data may be used by the systems described herein. In fact, the systems described herein may actually implement the data sources (e.g., an EDR tool, a CASB tool, an IAM tool) described above.

In some embodiments one data source that is ingested by the systems described herein is log data, although other forms of data such as network telemetry data (flows and packets) and/or many other forms of data may also be utilized. In some embodiments, event data can be combined with contextual information about users, assets, threats, vulnerabilities, and so on, for the purposes of scoring, prioritization and expediting investigations. In some embodiments, input data may be normalized, so that events, data, contextual information, or other information from disparate sources can be analyzed more efficiently for specific purposes (e.g., network security event monitoring, user activity monitoring, compliance reporting). The embodiments described here offer real-time analysis of events for security monitoring, advanced analysis of user and entity behaviors, querying and long-range analytics for historical analysis, other support for incident investigation and management, reporting (for compliance requirements, for example), and other functionality.

In some embodiments, the systems described herein may be part of an application performance monitoring ('APM') solution. APM software and tools enable the observation of application behavior, observation of its infrastructure dependencies, observation of users and business key performance indicators ('KPIs') throughout the application's life cycle, and more. The applications being observed may be developed internally, as packaged applications, as software as a service ('SaaS'), or embodied in some other ways. In such embodiments, the systems described herein may provide one or more of the following capabilities:

The ability to operate as an analytics platform that ingests, analyzes, and builds context from traces, metrics, logs, and other sources.

Automated discovery and mapping of an application and its infrastructure components.

Observation of an application's complete transactional behavior, including interactions over a data communications network.

Monitoring of applications running on mobile (native and browser) and desktop devices.

Identification of probable root causes of an application's performance problems and their impact on business outcomes.

Integration capabilities with automation and service management tools.

Analysis of business KPIs and user journeys (for example, login to check-out).

Domain-agnostic analytics capabilities for integrating data from third-party sources.

Endpoint monitoring to understand the user experience and its impact on business outcomes.

Support for virtual desktop infrastructure ('VDI') monitoring.

In embodiments where the systems described herein are used for APM, some components of the system may be modified, other components may be added, some components may be removed, and other components may remain the same. In such an example, similar mechanisms as described elsewhere in this disclosure may be used to collect information from the applications, network resources used by the application, and so on. The graph based modelling

66

techniques may also be leveraged to perform some of the functions mentioned above, or other functions as needed.

In some embodiments, the systems described herein may be part of a solution for developing and/or managing artificial intelligence ('AI') or machine learning ('ML') applications. For example, the systems described herein may be part of an AutoML tool that automate the tasks associated with developing and deploying ML models. In such an example, the systems described herein may perform various functions as part of an AutoML tool such as, for example, monitoring the performance of a series of processes, micro-services, and so on that are used to collectively form the AutoML tool. In other embodiments, the systems described herein may perform other functions as part of an AutoML tool or may be used to monitor, analyze, or otherwise observe an environment that the AutoML tool is deployed within.

In some embodiments, the systems described herein may be used to manage, analyze, or otherwise observe deployments that include other forms of AI/ML tools. For example, the systems described herein may manage, analyze, or otherwise observe deployments that include AI services. AI services are, like other resources in an as-a-service model, ready-made models and AI applications that are consumable as services and made available through APIs. In such an example, rather than using their own data to build and train models for common activities, organizations may access pre-trained models that accomplish specific tasks. Whether an organization needs natural language processing ('NLP'), automatic speech recognition ('ASR'), image recognition, or some other capability, AI services simply plug-and-play into an application through an API. Likewise, the systems described herein may be used to manage, analyze, or otherwise observe deployments that include other forms of AI/ML tools such as Amazon Sagemaker (or other cloud machine-learning platform that enables developers to create, train, and deploy ML models) and related services such as Data Wrangler (a service to accelerate data prep for ML) and Pipelines (a CI/CD service for ML).

In some embodiments, the systems described herein may be used to manage, analyze, or otherwise observe deployments that include various data services. For example, data services may include secure data sharing services, data marketplace services, private data exchanges services, and others. Secure data sharing services can allow access to live data from its original location, where those who are granted access to the data simply reference the data in a controlled and secure manner, without latency or contention from concurrent users. Because changes to data are made to a single version, data remains up-to-date for all consumers, which ensures data models are always using the latest version of such data. Data marketplace services operate as a single location to access live, ready-to-query data (or data that is otherwise ready for some other use). A data marketplace can even include a "feature stores," which can allow data scientists to repurpose existing work. For example, once a data scientist has converted raw data into a metric (e.g., costs of goods sold), this universal metric can be found quickly and used by other data scientists for quick analysis against that data.

In some embodiments, the systems described herein may be used to manage, analyze, or otherwise observe deployments that include distributed training engines or similar mechanisms such as, for example, tools built on Dask. Dask is an open source library for parallel computing that is written in Python. Dask is designed to enable data scientists to improve model accuracy faster, as Dask enables data

67

scientists to do everything in Python end-to-end, which means that they no longer need to convert their code to execute in environments like Apache Spark. The result is reduced complexity and increased efficiency. The systems described herein may also be used to manage, analyze, or otherwise observe deployments that include technologies such as RAPIDS (an open source Python framework which is built on top of Dask). RAPIDS optimizes compute time and speed by providing data pipelines and executing data science code entirely on graphics processing units (GPUs) rather than CPUs. Multi-cluster, shared data architecture, DataFrames, Java user-defined functions (UDF) are supported to enable trained models to run within a data warehouse.

In some embodiments, the systems described herein may be leveraged for the specific use case of detecting and/or remediating ransomware attacks and/or other malicious action taken with respect to data, systems, and/or other resources associated with one or more entities.

Ransomware is a type of malware from cryptovirology that threatens to publish the victim's data or perpetually block access to such data unless a ransom is paid. In such embodiments, ransomware attacks may be carried out in a manner such that patterns (e.g., specific process-to-process communications, specific data access patterns, unusual amounts of encryption/re-encryption activities) emerge, where the systems described herein may monitor for such patterns. Alternatively, ransomware attacks may involve behavior that deviates from normal behavior of a cloud deployment that is not experiencing a ransomware attack, such that the mere presence of unusual activity may trigger the systems described herein to generate alerts or take some other action, even without explicit knowledge that the unusual activity is associated with a ransomware attack.

In some embodiments, the systems described herein may be configured to enforce such policies as part of an effort to thwart ransomware attacks. For example, particular network sharing protocols (e.g., Common Internet File System ('CIFS'), Network File System ('NFS')) may be avoided when implementing storage for backup data, policies that protect backup systems may be implemented and enforced to ensure that usable backups are always available, multi-factor authentication for particular accounts may be utilized and accounts may be configured with the minimum privilege required to function, isolated recovery environments may be created and isolation may be monitored and enforced to ensure the integrity of the recovery environment, and so on. As described in the present disclosure, the systems described herein may be configured to explicitly enforce such policies or may be configured to detect unusual activity that represents a violation of such policies, such that the mere presence of unusual activity may trigger the systems described herein to generate alerts or take some other action, even without explicit knowledge that the unusual activity is associated with a violation of a particular policy.

Readers will appreciate that ransomware attacks are often deployed as part of a larger attack that may involve, for example:

Penetration of the network through means such as, for example, stolen credentials and remote access malware.

Stealing of credentials for critical system accounts, including subverting critical administrative accounts that control systems such as backup, Active Directory ('AD'), DNS, storage admin consoles, and/or other key systems.

68

Attacks on a backup administration console to turn off or modify backup jobs, change retention policies, or even provide a roadmap to where sensitive application data is stored. Data theft attacks.

As a result of the many aspects that are part of a ransomware attack, embodiments of the present disclosure may be configured as follows:

The systems may include one or more components that detect malicious activity based on the behavior of a process.

The systems may include one or more components that store indicator of compromise ('IOC') or indicator of attack ('IOA') data for retrospective analysis.

The systems may include one or more components that detect and block fileless malware attacks.

The systems may include one or more components that remove malware automatically when detected.

The systems may include a cloud-based, SaaS-style, multi-tenant infrastructure.

The systems may include one or more components that identify changes made by malware and provide the recommended remediation steps or a rollback capability.

The systems may include one or more components that detect various application vulnerabilities and memory exploit techniques.

The systems may include one or more components that continue to collect suspicious event data even when a managed endpoint is outside of an organization's network.

The systems may include one or more components that perform static, on-demand malware detection scans of folders, drives, devices, or other entities.

The systems may include data loss prevention (DLP) functionality.

In some embodiments, the systems described herein may manage, analyze, or otherwise observe deployments that include deception technologies. Deception technologies allow for the use of decoys that may be generated based on scans of true network areas and data. Such decoys may be deployed as mock networks running on the same infrastructure as the real networks, but when an intruder attempts to enter the real network, they are directed to the false network and security is immediately notified. Such technologies may be useful for detecting and stopping various types of cyber threats such as, for example, Advanced Persistent Threats ('APTs'), malware, ransomware, credential dumping, lateral movement and malicious insiders. To continue to outsmart increasingly sophisticated attackers, these solutions may continuously deploy, support, refresh and respond to deception alerts.

In some embodiments, the systems described herein may manage, analyze, or otherwise observe deployments that include various authentication technologies, such as multi-factor authentication and role-based authentication. In fact, the authentication technologies may be included in the set of resources that are managed, analyzed, or otherwise observed as interactions with the authentication technologies may be monitored. Likewise, log files or other information retained by the authentication technologies may be gathered by one or more agents and used as input to the systems described herein.

In some embodiments, the systems described herein may be leveraged for the specific use case of detecting supply chain attacks. More specifically, the systems described herein may be used to monitor a deployment that includes software components, virtualized hardware components, and other components of an organization's supply chain such that interactions with an outside partner or provider with access to an organization's systems and data can be

monitored. In such embodiments, supply chain attacks may be carried out in a manner such that patterns (e.g., specific interactions between internal and external systems) emerge, where the systems described herein may monitor for such patterns. Alternatively, supply chain attacks may involve behavior that deviates from normal behavior of a cloud deployment that is not experiencing a supply chain attack, such that the mere presence of unusual activity may trigger the systems described herein to generate alerts or take some other action, even without explicit knowledge that the unusual activity is associated with a supply chain attack.

In some embodiments, the systems described herein may be leveraged for other specific use cases such as, for example, detecting the presence of (or preventing infiltration from) cryptocurrency miners (e.g., bitcoin miners), token miners, hashing activity, non-fungible token activity, other viruses, other malware, and so on. As described in the present disclosure, the systems described herein may monitor for such threats using known patterns or by detecting unusual activity, such that the mere presence of unusual activity may trigger the systems described herein to generate alerts or take some other action, even without explicit knowledge that the unusual activity is associated with a particular type of threat, intrusion, vulnerability, and so on.

In some embodiments, the various forms of malicious code, malicious actors, or other malicious entities may be generated using traditional programming methodologies where software is developed by programmers. In other embodiments, such software may be generated by AI tools such as, for example, Chat Generative Pre-trained Transformer ("ChatGPT"). As such, the embodiments described herein may be configured to evaluate various entities for signatures that are indicative of AI generated code, such as the inclusion of libraries typically used by AI code generating tools, programming styles that are common in code that is generated by AI code generating tools, or any other marker that some piece of software was generated by AI code generating tools. In such a way, software that is generated by AI code generating tools (which may be used for the rapid development of malicious code) may be identified and subjected to a higher level of scrutiny as code that is generated in more traditional ways.

The systems described above may include a variety of different user interfaces that may be used to conduct investigations, access alerts, set policies, or otherwise used to facilitate any of the functionality described above. In addition to those interfaces expressly described already, the systems described herein may leverage natural language interfaces to conduct investigations, access alerts, set policies, or facilitate any of the functionality described herein. Such natural language interfaces can include speech-to-text interfaces, chatbots such as ChatGPT, Natural-language user interfaces (LUI or NLUI), or some other interface that includes natural language processing capabilities. In fact, the systems described herein may even leverage such technologies for alert processing, event processing, and so on. In these embodiments, alerts that are generated may be sent to, for example, a chatbot (e.g., ChatGPT) that can be used to process the alert, including capturing information describing the assets involved, information describing the potential impact of a threat or breach, and so on. Such a chatbot may even generate a detailed explanation of how to remediate the issue, generate code to remediate the issue, or even executed code to remediate the issue in a fully automated embodiment.

The systems described herein may also be leveraged for endpoint protection, such the systems described herein form

all of or part of an endpoint protection platform. In such an embodiment, agents, sensors, or similar mechanisms may be deployed on or near managed endpoints such as computers, servers, virtualized hardware, internet of things ("IoT") devices, mobile devices, phones, tablets, watches, other personal digital devices, storage devices, thumb drives, secure data storage cards, or some other entity. In such an example, the endpoint protection platform may provide functionality such as:

Prevention and protection against security threats including malware that uses file-based and fileless exploits.

The ability to apply control (allow/block) to access of software, scripts, processes, microservices, and so on.

The ability to detect and prevent threats using behavioral analysis of device activity, application activity, user activity, and/or other data.

The ability for facilities to investigate incidents further and/or obtain guidance for remediation when exploits evade protection controls.

The ability to collect and report on inventory, configuration and policy management of the endpoints.

The ability to manage and report on operating system security control status for the monitored endpoints.

The ability to scan systems for vulnerabilities and report/manage the installation of security patches.

The ability to report on internet, network and/or application activity to derive additional indications of potentially malicious activity.

Example embodiments are described in which policy enforcement, threat detection, or some other function is carried out by the systems described herein by detecting unusual activity, such that the mere presence of unusual activity may trigger the systems described herein to generate alerts or take some other action, even without explicit knowledge that the unusual activity is associated with a particular type of threat, intrusion, vulnerability, and so on. Although these examples are largely described in terms of identifying unusual activity, in these examples the systems described herein may be configured to learn what constitutes "normal activity"—where "normal activity" is activity observed, modeled, or otherwise identified in the absence of a particular type of threat, intrusion, vulnerability, and so on. As such, detecting "unusual activity" may alternatively be viewed as detecting a deviation from "normal activity" such that "unusual activity" does not need to be identified and sought out. Instead, deviations from "normal activity" may be assumed to be "unusual activity".

Readers will appreciate that while specific examples of the functionality that the systems described herein can provide are included in the present disclosure, such examples are not to be interpreted as limitations as to the functionality that the systems described herein can provide. Other functionality may be provided by the systems described herein, all of which are within the scope of the present disclosure. For the purposes of illustration and not as a limitation, additional examples can include governance, risk, and compliance ("GRC"), threat detection and incident response, identity and access management, network and infrastructure security, data protection and privacy, identity and access management ("IAM"), and many others.

In order to provide the functionality described above, the systems described herein or the deployments that are monitored by such systems may implement a variety of techniques. For example, the systems described herein or the deployments that are monitored by such systems may tag data and logs to provide meaning or context, persistent monitoring techniques may be used to monitor a deployment

at all times and in real time, custom alerts may be generated based on rules, tags, and/or known baselines from one or more polygraphs, and so on.

Although examples are described above where data may be collected from one or more agents, in some embodiments other methods and mechanisms for obtaining data may be utilized. For example, some embodiments may utilize agent-less deployments where no agent (or similar mechanism) is deployed on one or more customer devices, deployed within a customer's cloud deployment, or deployed at another location that is external to the data platform. In such embodiments, the data platform may acquire data through one or more APIs such as the APIs that are available through various cloud services. For example, one or more APIs that enable a user to access data captured by Amazon CloudTrail may be utilized by the data platform to obtain data from a customer's cloud deployment without the use of an agent that is deployed on the customer's resources. In some embodiments, agents may be deployed as part of a data acquisition service or tool that does not utilize a customer's resources or environment. In some embodiments, agents (deployed on a customer's resources or elsewhere) and mechanisms in the data platform that can be used to obtain data from through one or more APIs such as the APIs that are available through various cloud services may be utilized. In some embodiments, one or more cloud services themselves may be configured to push data to some entity (deployed anywhere), which may or may not be an agent. In some embodiments, other data acquisition techniques may be utilized, including combinations and variations of the techniques described above, each of which is within the scope of the present disclosure.

Readers will appreciate that while specific examples of the cloud deployments that may be monitored, analyzed, or otherwise observed by the systems described herein have been provided, such examples are not to be interpreted as limitations as to the types of deployments that may be monitored, analyzed, or otherwise observed by the systems described herein. Other deployments may be monitored, analyzed, or otherwise observed by the systems described herein, all of which are within the scope of the present disclosure. For the purposes of illustration and not as a limitation, additional examples can include multi-cloud deployments, on-premises environments, hybrid cloud environments, sovereign cloud environments, heterogeneous environments, DevOps environments, DevSecOps environments, GitOps environments, quantum computing environments, data fabrics, composable applications, composable networks, decentralized applications, and many others.

Readers will appreciate that while specific examples of the types of data that may be collected, transformed, stored, and/or analyzed by the systems described herein have been provided, such examples are not to be interpreted as limitations as to the types of data that may be collected, transformed, stored, and/or analyzed by the systems described herein. Other types of data can include, for example, data collected from different tools (e.g., DevOps tools, DevSecOps, GitOps tools), different forms of network data (e.g., routing data, network translation data, message payload data, Wifi data, Bluetooth data, personal area networking data, payment device data, near field communication data, metadata describing interactions carried out over a network, and many others), data describing processes executing in a container, lambda, EC2 instance, virtual machine, or other execution environment, data associated with a virtualization platform (e.g., VMWare vSphere, VMware vCenter servers, vSphere plug-ins, etc.), data asso-

ciated with a virtual machine monitor (e.g., hypervisors, ESXi hosts, etc.), information describing the execution environment itself, and many other types of data. In some embodiments, various backup images may also be collected, transformed, stored, and/or analyzed by the systems described herein for the purposes of identifying anomalies. Such backup images can include backup images of an entire cloud deployment, backup images of some subset of a cloud deployment, backup images of some other system or device(s), and so on. In such a way, backup images may serve as a separate data source that can be analyzed for detecting various anomalies.

For further explanation, FIG. 5 sets forth a flowchart illustrating an example method of configuring cloud deployments based on learnings obtained by monitoring other cloud deployments in accordance with some embodiments of the present disclosure. The cloud deployments 508, 514 may be similar to the cloud deployments described above, where a particular cloud deployment can include a variety of components 510, 512 such as one or more applications, one or more data sources, networking resources, processing resources, and other resources. Such components 510, 512 may, in some embodiments, be deployed in the cloud deployments 508, 514 using one or more as-a-service models where software, infrastructure, platforms, databases, and other components as delivered as services. Configuring cloud deployments 508, 514 based on learnings obtained by monitoring other cloud deployments may be carried out using the systems described above. As such, one or more of the steps depicted in FIG. 5 may be performed by the systems described above.

The example method depicted in FIG. 5 includes determining 502 normal behavior for one or more components 510 in a first cloud deployment 508. Determining 502 normal behavior for one or more components 510 in a first cloud deployment 508 may be carried out, for example, as described in greater detail above (at times described as identifying 'normal activity') by the systems described above (also referred to herein as a 'data platform').

The example method depicted in FIG. 5 also includes determining 504 normal behavior for one or more components 512 in one or more other cloud deployments 514. Determining 504 normal behavior for one or more components 512 in one or more other cloud deployments 514 may also be carried out, for example, as described in greater detail above (at times described as identifying 'normal activity') by the systems described above.

In some embodiments, a customer-specific data platform may be used to analyze, monitor, or otherwise observe a particular customer's cloud deployment (or some other deployment). Within such a cloud deployment, various clusters may exist. For example, a collection of microservices may form a cluster by virtue of those microservices communicating only (or mostly) with each other. Likewise, one or more cloud computing instances (e.g., one or more EC2 instances) and a database may form a cluster by virtue of the EC2 instances accessing the database as the only source of data utilized by the EC2 instances. Using the techniques and mechanisms described above, such clusters may be identified. Although clusters may be identified and characteristics associated with the cluster may be learned, limited insights may be gained if only a particular customer's cloud deployment is analyzed, monitored, or otherwise observed. In accordance with embodiments of the present disclosure, cross-customer analysis may be leveraged to

gain deeper insights than would be gained if only a single customer's cloud deployment is analyzed, monitored, or otherwise observed.

In some embodiments, cross-customer analysis may be carried out by gathering information related to cloud deployments (or some other deployments) for multiple customers and comparing such information. Using the example described above, information describing clusters identified in a first customer's cloud deployment may be compared to information describing clusters identified in a second customer's cloud deployment for the purposes of identifying similar or identical clusters in each customer's cloud deployment.

Consider an example in which each customer's deployment included a web server that was deployed in one or more EC2 instances. In such an example, a particular cluster that represents the web server may be identified in each customer's deployment. For example, a first cluster in the first customer's deployment may represent a first web server and a second cluster in the second customer's deployment may represent a second web server. Because the first cluster and the second cluster would have similar characteristics (e.g., each cluster receives data communications using HTTP or HTTPS or any other suitable communication protocol, each cluster communicates with a web browser, each cluster requires similar computing resources, and so on), the first cluster and the second cluster may be identified as being identical clusters by comparing the characteristics of each cluster that each cluster. This process may be repeated across the cloud deployments for many customers such that a collection of 'web server' clusters (in this example) may be identified.

Readers will appreciate that although the example described above relates to an embodiment where a collection of 'web server' clusters are identified in different customer's cloud deployments, identifying the nature or type (e.g., a web server) of the clusters is not required. In fact, by comparing the characteristics of different clusters to each other, similar or identical clusters may be identified even if the exact nature/type of those clusters is not known. For example, a comparison of the characteristics of multiple clusters may only reveal that the clusters are identical, even if such a comparison does not reveal that clusters are 'web server' clusters. Multiple clusters that have been identified as being identical (or sufficiently similar as measured by a threshold) will be referred to throughout the remainder of this document as a "cluster set," where the clusters that are members of the cluster set may be deployed across multiple customer's cloud deployments.

In some embodiments, information describing each member of the cluster set may be utilized to identify distributions across the cluster set. Distributions may be identified for traditional resource consumption metrics such as, for example, CPU usage, memory usage, network bandwidth usage, and others. A distribution may reveal, for example, that all members of the cluster set utilize between 10-60 Mb/s of network bandwidth, with the vast majority of members of the cluster set utilize between 45-60 Mb/s of network bandwidth. Readers will appreciate that distributions may also be identified for other quantifiable characteristics of each cluster. Such quantifiable characteristics can include, for example, the failure rate of a cluster or particular components thereof, an identification of communication protocols used by a cluster or particular components thereof, an identification of the types of communications endpoints that a cluster or particular components thereof communicate with (e.g., endpoints that reside on the public internet v.

endpoints that are in a private network), characteristics that can be classified by a binary value (e.g., does any component in the cluster perform privileged operations), information describing the various privileges that are given to a particular cluster, and many more.

In some embodiments, the distributions may be used to identify 'normal' behavior for a particular cluster. Consider an example in which the cluster set is identified, where each member of the cluster set represents a payroll system deployed in a particular customer's cloud deployment. In such an example, a distribution may be identified which indicates that all members of the cluster communicate with (and has privileged access to) the same set of cloud services, including: 1) a cloud database service (e.g., Amazon Aurora, Microsoft Azure SQL Database, Amazon Relational Database Service, Google Cloud SQL, Amazon DynamoDB), 2) a vendor provided SaaS offering that provides bill payment services, and 3) a vendor provided SaaS offering that provides accounting services. In such an example, by looking at each member of the cluster set and identifying that each member communicates with the same set of cloud services, a baseline may be established that identifies 'normal' behavior for each member of the cluster set, at least with respect to the specific characteristic (i.e., what cloud services are utilized by members) that the distribution is based on. As such, if monitoring a particular cluster revealed that some member of the cluster set accessed a source code repository cloud service (e.g., GitHub Enterprise on AWS), this sort of access would be outside of the typical distribution for this cluster set and could serve as the basis for raising an alert, denying access to the service, or initiating some other alerting/remediation workflow. Readers will appreciate that many distributions may be created for each cluster set, where each distribution is based on one or more characteristics of the members of the cluster set.

The example method depicted in FIG. 5 also includes recommending **506**, based on the normal behavior for one or more components **512** in one or more other cloud deployments **514**, a change to the first cloud deployment **508**. Recommending **506** a change to the first cloud deployment **508** may be carried out, for example, in response to determining that the normal behavior in one or more other cloud deployments **514** differs from the normal behavior for one or more components **510** in a first cloud deployment **508**. In such an example, changes to the first cloud deployment **508** may be recommended that (if implemented) would cause the first cloud deployment **508** to be more similar to the other cloud deployments **514**. For example, if the normal behavior in one or more other cloud deployments **514** indicates that all computing resources (e.g., virtual machines, container, serverless computing resources) communicate with each other using a particular secure data communications protocol and the normal behavior for one or more components **510** in a first cloud deployment **508** is for computing resources to communicate using some other data communications protocol, a change may be recommended that involves reconfiguring the computing resources to communicate using the particular secure data communications protocol.

Readers will appreciate that in some embodiments the mere fact that normal behavior in a first cloud deployment **508** deviates from normal behavior in one or more other cloud deployments **514** may be sufficient rationale for recommending **506** a change to the first cloud deployment **508**. That is, a departure from normality and standard practices alone may result in recommending **506** a change to the first cloud deployment **508**. In other embodiments,

75

recommending **506** a change to the first cloud deployment **508** may only occur where the normal behavior in one or more other cloud deployments **514** is determined to be superior to the normal behavior in the first cloud deployment **508**. For example, recommending **506** a change to the first cloud deployment **508** may only be carried where the normal behavior in one or more other cloud deployments **514** is representative of a stronger security posture than the normal behavior in the first cloud deployment **508**.

Consider an example in which a particular threat was detected (in part by detecting a deviation from normal behavior for one or more components **512**) in a particular customer's cloud deployment **514**, where the threat turned out to be a ransomware attack, which may in some embodiments include an encryption component and/or a data theft or leakage component. In such an example, if an identical (or sufficiently similar, following a general recognized pattern or 'fingerprint') threat is detected in the first customer's cloud deployment **508** (in part by detecting a similar deviation from normal behavior for one or more components **510**), information describing the remedial actions (e.g., disabling encryption, increasing the frequency of backups, locking down a backup system, blocking transmission of data externally, etc.) that were taken by the particular customer **514** may even recommended **506** as changes to be made to the first cloud deployment **508**. Furthermore, if many customers had experienced the same attack and the data platform could determine with sufficient certainty that the first cloud deployment **508** was experiencing the same attack, workflows may be automatically initiated to carry out various remedial actions.

For further explanation, FIG. 6 sets forth a flowchart illustrating an additional example method of configuring cloud deployments based on learnings obtained by monitoring other cloud deployments in accordance with some embodiments of the present disclosure. The example depicted in FIG. 6 is similar to the example depicted in FIG. 5, as the example depicted in FIG. 6 also includes determining **502** normal behavior for one or more components **510** in a first cloud deployment **508**, determining **504** normal behavior for one or more components **512** in one or more other cloud deployments **514**, and recommending **506** a change to the first cloud deployment **508** based on the normal behavior for one or more components **512** in one or more other cloud deployments **514**.

In some embodiments, customers (and their corresponding deployments) may be modeled into logical groups such that cross customer learning could be carried out only across customers in the same logical group, or other customers in the same logical group may be given a greater weighting for the purposes of cross customer learning. For example, a logical group may be tied to a customer's business function (e.g., the customer is a financial company, the customer is a health care company, the customer is a services company, the customer has inventory to manage, the customer sells to retail customers, and so on). Likewise, a logical group may be tied to a customer's cloud deployment (e.g., the deployment uses a particular combination of resources, the deployment uses a particular set of cloud-services, the deployment utilizes availability zones and regions in a particular way, and so on). In such an example, a particular customer may be associated with multiple logical groups and cross customer learning for the particular customer may only involve other customers that are in the same (or sufficiently similar) logical groups. As such, the particular customer's resources (e.g., people, cloud deployment) may only be included in

76

cluster sets with resources associated with other customers that are in the same (or sufficiently similar) logical groups.

To that end, the example method depicted in FIG. 6 also includes identifying **604**, from the other cloud deployments, cloud deployments that are similar to the first cloud deployment **508**. Identifying **604** cloud deployments that are similar to the first cloud deployment **508** may be carried out, for example, by taking an inventory of the various components in each cloud deployment **508**, **514** and identifying cloud deployments that have similar components, portions of the cloud deployment that have similar deployments, and so on. Alternatively, identifying **604** cloud deployments that are similar to the first cloud deployment **508** may be carried out, for example, by identifying cloud deployments that are associated with customers in the same industry as the customer associated with the first cloud deployment **508**, by identifying cloud deployments that are associated with a similar intended functionality as the first cloud deployment **508** (e.g., both cloud deployment are intended to provide an online store for an online retailer), and so on. In the examples in the preceding sentence, it may be assumed that two cloud deployments should be relatively similar even without inspecting the cloud deployments themselves. In such examples, similarity may be judged based on the application of rules, policies, heuristics, or similar mechanism. Alternatively, similarity may be detected via machine learning techniques as where information describing the activity of various components in many cloud deployments are fed as input to one or more machine learning models which subsequently identifies clusters across various cloud deployments that are similar.

In the example depicted in FIG. 6, recommending **506** the change to the first cloud deployment **508** may be based on the normal behavior for the cloud deployments that are similar to the first cloud deployment. Recommending **606** the change to the first cloud deployment **508** based on the normal behavior for the cloud deployments that are similar to the first cloud deployment **508** may be carried out, for example, by ignoring normal behavior for dissimilar cloud deployments and only taking into consideration those cloud deployments that have been identified **604** as being sufficiently similar to the first cloud deployment **508**.

The example method depicted in FIG. 6 also includes identifying **602**, from the other cloud deployments, one or more highly rated cloud deployments. A cloud deployment may be identified **602** as being 'highly rated' in the sense that fewer than a threshold number of vulnerabilities have been detected in the cloud deployment over some period of time, fewer than a threshold number of outages (or less than a threshold amount of downtime) has been detected in the cloud deployment over some period of time, fewer than a threshold number of regulatory violations (or attempted violations) have been detected in the cloud deployment over some period of time, or identified **602** based on some other standard. In fact, comparisons may be relative (rather than comparing some quantifiable aspect of the other deployments operation) such that, for example, the 15% of cloud deployments with the fewest detected vulnerabilities over some period of time as identified **602** as being 'highly rated'. In other embodiments, identifying **602** one or more highly rated cloud deployments may be carried out through the use of machine learning techniques where information describing the activity of various components in many cloud deployments are fed as input to one or more machine learning models which subsequently identifies deployments that have the best performance against a combination of one

or more metrics (e.g., vulnerabilities, availability, cost). In such a way, the best-of-breed cloud deployments may be identified **602**.

In the example depicted in FIG. 6, recommending **506** the change to the first cloud deployment **508** may be based on the normal behavior for the highly rated cloud deployments. Recommending **608** the change to the first cloud deployment **508** based on the normal behavior for the highly rated cloud deployments may be carried out, for example, by ignoring normal behavior for cloud deployments that are not highly rated and only taking into consideration those cloud deployments are highly rated.

Readers will appreciate that although the example in FIG. 6 is depicted where identifying **604** cloud deployments that are similar to the first cloud deployment **508** and identifying **602** one or more highly rated cloud deployments are alternatives to each other, in other embodiments both steps may be performed. In such embodiments, recommending **506** the change to the first cloud deployment **508** may be based on the normal behavior of other cloud deployments that are both highly rated and similar to the first cloud deployment **508**.

For further explanation, FIG. 7 sets forth a flowchart illustrating an additional example method of configuring cloud deployments based on learnings obtained by monitoring other cloud deployments in accordance with some embodiments of the present disclosure. The example depicted in FIG. 7 is similar to the examples depicted in FIG. 5 and FIG. 6, as the example depicted in FIG. 7 also includes determining **502** normal behavior for one or more components **510** in a first cloud deployment **508**, determining **504** normal behavior for one or more components **512** in one or more other cloud deployments **514**, and recommending **506** a change to the first cloud deployment **508** based on the normal behavior for one or more components **512** in one or more other cloud deployments **514**.

The example method depicted in FIG. 7 also includes ranking **702** the first cloud deployment **508** relative to the other cloud deployments **514**. Ranking **702** the first cloud deployment **508** relative to the other cloud deployments **514** may be carried out, for example, by identifying how well the cloud deployment **508** compares to other cloud deployments **514** with respect to one or more metrics. For example, cloud deployments may be ranked based on a weighted combination of multiple metrics (e.g., reliability, cost, regulatory compliance) such that the best-of-breed deployments may be identified. In such an example, the ranking of the first cloud deployment **508** relative to the other cloud deployments **514** may be presented to an administrator or other user associated with the first cloud deployment **508** for analysis by the administrator or other user.

The example method depicted in FIG. 7 also includes comparing **704** the trajectory of the first cloud deployment **508** to the trajectory of the other cloud deployments **514**. The trajectory of a particular cloud deployment **508**, **514** may be determined, for example, by evaluating the particular cloud deployment's **508**, **514** performance over time as measured by one or more metrics. For example, the performance of each cloud deployment may be periodically scored based on a weighted combination of multiple metrics (e.g., reliability, cost, regulatory compliance). In such an example, the trajectory for a particular cloud deployment may be determined by determining the extent to which the score associated with a particular cloud deployment is changing over time. For example, if the first cloud deployment's **508** scoring is improving over time while the other cloud deployments' **514** scores are remaining the same over time, the

trajectory of the first cloud deployment **508** may be determined to be better than the trajectory of the other cloud deployments **514** (in embodiments where a higher score is considered to be better).

Readers will appreciate that ranking **702** the first cloud deployment **508** relative to the other cloud deployments **514** and comparing **704** the trajectory of the first cloud deployment **508** to the trajectory of the other cloud deployments **514** are just two examples of the sort of analytics that can be performed by comparing the performance of multiple cloud deployments **508**, **514**. Readers will appreciate that other analytics may also be put in place due to the availability of information describing the performance of multiple cloud deployments **508**, **514**.

For further explanation, FIG. 8 sets forth a flowchart illustrating an additional example method of configuring cloud deployments based on learnings obtained by monitoring other cloud deployments in accordance with some embodiments of the present disclosure. The example depicted in FIG. 8 is similar to the examples depicted in FIGS. 5-7, as the example depicted in FIG. 8 also includes determining **502** normal behavior for one or more components **510** in a first cloud deployment **508**, determining **504** normal behavior for one or more components **512** in one or more other cloud deployments **514**, and recommending **506** a change to the first cloud deployment **508** based on the normal behavior for one or more components **512** in one or more other cloud deployments **514**.

The example method depicted in FIG. 8 also includes identifying **802** other cloud deployments **514** to exclude from consideration when recommending changes to the first cloud deployment **508**. Readers will appreciate that other cloud deployments **514** may be excluded from consideration when recommending changes to the first cloud deployment **508** for a variety of reasons. For example, another cloud deployment **514** may be so dissimilar to the first cloud deployment **508** that there may be very little to learn that is relevant to the first cloud deployment **508**, another cloud deployment **514** may be deficient for a variety of reasons such that there may be very little high quality takeaways from evaluating the other cloud deployment **514** that are relevant to the first cloud deployment **508**, another cloud deployment **514** may be a relatively recent deployment such that any takeaways from evaluating the other cloud deployment **514** may not be reliable and proven over time, or for some other reason. In such an example, identifying **802** other cloud deployments **514** to exclude from consideration when recommending changes to the first cloud deployment **508** may be carried out by filtering the set of other cloud deployments **514** according to some criteria, or carried out in some other way. In the example depicted in FIG. 8, recommending **506** the change to the first cloud deployment **508** may therefore include recommending **806** changes based on the other cloud deployments **514** that are not excluded from consideration.

The example method depicted in FIG. 8 also includes identifying **804** a response to a recommended change in one or more other cloud deployments **514**. Identifying **804** a response to a recommended change in one or more other cloud deployments **514** may be carried out, for example, by tracking the status of a recommended change that was recommended for another cloud deployment **514** to determine whether the recommended change was actually implemented. Consider an example in which a recommendation was made to have another cloud deployment **514** switch from using a first IaaS offering to a second IaaS offering in response to detecting some condition in the cloud deploy-

ment **514**. In such an example, identifying **804** a response to a recommended change in one or more other cloud deployments **514** may be carried out by determining whether the recommended change was actually implemented, and a switch was made to utilize the second IaaS offering as part of the cloud deployment **514**. Readers will appreciate that by repeatedly identifying **804** responses to recommended changes in one or more other cloud deployments **514** can be used by the systems described above as an indication as to the systems described above are making useful recommendations. For example, if identifying **804** a response to a recommended change in one or more other cloud deployments **514** reveals that a very small percentage of administrators actually implement the recommended change, this may indicate that the recommended change is not particularly valuable and may be used as feedback in determining whether to make the recommended change if the condition that triggered the recommendation is encountered again. Alternatively, if identifying **804** a response to a recommended change in one or more other cloud deployments **514** reveals that a very large percentage of administrators actually implement the recommended change, this may indicate that the recommended change is valuable and may be used as feedback in determining whether to make the recommended change if the condition that triggered the recommendation is encountered again. In such a way, the response from other customers may be used to drive recommendations that are made regarding the first cloud deployment **508**.

In the example depicted in FIG. 8, recommending **506** the change to the first cloud deployment **508** recommending **808** the change to the first cloud deployment **508** based on the response to the recommended change in one or more other cloud deployments **514**. Recommending **808** the change to the first cloud deployment **508** based on the response to the recommended change in one or more other cloud deployments **514** may be carried out, for example, only if a predetermined percentage of previous recipients of similar recommendations have actually implemented the recommended change, as described above.

Readers will appreciate that while the examples described in the preceding two paragraphs relate to an example in which the response to a recommended change in one or more other cloud deployments **514** is expressed in terms of whether the recommended change was or was not implemented, in other embodiments the response to a recommended change in one or more other cloud deployments **514** may be measured in other ways. For example, a response to a recommended change in one or more other cloud deployments **514** may represent whether the recommended change did or did not resolve the condition that caused the recommendation to be generated in the first place. In other embodiments, the response to a recommended change in one or more other cloud deployments **514** may be measured or determined using different criteria.

Although some of the figures described above depict only a single other cloud deployment **514**, a single other cloud deployment **514** is included for case of illustration but in no way represents a limitation of the embodiments described herein. In fact, most embodiments will include multiple other cloud deployments **514** that may be monitored and learned from.

Readers will appreciate that while many of the embodiments described above relate to embodiments where cross-customer learnings are used to generate recommendations for changes to the first cloud deployment **508**, in other embodiments the cross-customer learning may be used to initiate, based on the normal behavior for the one or more

other cloud deployments, a change to the first cloud deployment. Initiating such changes may be carried out, for example, by initiating one or more remediation workflows that implement the changes. As was the case with recommendations, initiating a change to the first cloud environment may be based on the normal behavior for similar cloud deployments, based on the normal behavior for the highly rated cloud deployments, based on the cloud deployments to exclude from consideration when recommending changes to the first cloud deployment, and so on.

For further explanation, FIG. 9 sets forth a flowchart illustrating an example method of learning from similar cloud deployments in accordance with some embodiments of the present disclosure. The example method depicted in FIG. 9 may be carried out by the systems described above (also referred to as a 'data platform' above). As such, the systems described above may include computer program instructions executing on computer hardware, virtualized hardware, or some other execution environment (e.g., one or more containers, one or more serverless compute instances), where the computer program instructions carry out the steps described in FIG. 9 when the computer program instructions are executed.

The example depicted in FIG. 9 includes a plurality of cloud deployments, including a first cloud deployment **910** and one or more additional cloud deployments **912**, which in this illustration includes three cloud deployments **914a**, **914b**, **914n**, although more or fewer cloud deployments may be included in the set of additional cloud deployments **912** in other embodiments. Such cloud deployments may be embodied as described above and may include a variety of components such as, for example, software applications, storage resources, computing resources, networking resources, and other resources. In each cloud deployment, each of the resources may be delivered as services provided by a public cloud, private cloud, hybrid cloud, and so on.

The example method depicted in FIG. 9 includes identifying **902**, for at least a portion of a first cloud deployment **910**, one or more additional cloud deployments **912** to utilize for cross-customer learning. Cross-customer learning, as the phrase is used here, can generally be described as the process of learning about the cloud deployments of different customers, different organizations, or some other entity for the benefit of tailoring another entity's cloud deployment. For example, the cloud deployments of Customer A, Customer B, Customer C, Customer D, and Customer F may be evaluated, and the information gathered about their deployments may be leveraged to help shape the cloud deployment of Customer F. In such an example, lessons learned by other customers may be used to help guide a particular customer's design, deployment, and management of their cloud deployments, which may be particularly useful for organizations that are pivoting to the cloud from on-premises based deployments.

Identifying **902** one or more additional cloud deployments **912** to utilize for cross-customer learning may be carried out, for example, by identifying cloud deployments for other customers that are in similar industries as the customer associated with the first cloud deployment **910**, by identifying highly rated (as described above) cloud deployments, by identifying cloud deployments (or portions thereof) that are similar to the first cloud deployment **910** (or some portion thereof), or in some other way. In some embodiments, identifying **902** one or more additional cloud deployments **912** to utilize for cross-customer learning may be carried out using machine learning techniques as one or more machine learning models may be used to identify

81

cloud deployments that are similar to the first cloud deployment 910, to identify best-of-breed cloud deployments, and so on. In such a way, identifying 902 one or more additional cloud deployments 912 to utilize for cross-customer learning may result in a subset of curated, relevant, and/or exemplary cloud deployments being utilized for cross-customer learning rather than using the entire set of available cloud deployments being utilized for cross-customer learning (although some embodiments could utilize all cloud deployments for cross-customer learning).

The example method depicted in FIG. 9 also includes receiving 904 information 916 describing one or more actions associated with the additional cloud deployments 912. The one or more actions associated with the additional cloud deployments 912 may be embodied as actions taken by components within the additional cloud deployments 912. For example, a particular software service accessing a data store may be an example of an action that is associated with the additional cloud deployments 912, especially where the software service and the data store are components of a particular additional cloud deployment 914a. Likewise, a message, request, or other form of data communications that are exchanged between the additional cloud deployments 912 and actors that are external to the additional cloud deployments 912 may be an example of an action that is associated with the additional cloud deployments 912. For example, a server that is external to any of the additional cloud deployments 912 may send requests to access some software service within a particular additional cloud deployment 914a.

In other embodiments, the one or more actions associated with the additional cloud deployments 912 may include one or more user interactions with the additional cloud deployments 912. Such user interactions can include a description of which resources are accessed and/or utilized by particular users, particular business units (e.g., the finance department of a business, the engineering department of a business), particular personas (e.g., a system administrator, a software developer, a human resource manager), and so on. The one or more actions associated with the additional cloud deployments 912 may also include, for example, one or more interactions involving external applications, external computing devices, or similar external actor and the additional cloud deployments 912. Such user interactions can include, for example, a description of which resources within the additional cloud deployments 912 are accessed and/or utilized by particular external application, what IP address is associated with an external entity that is communicating with the additional cloud deployments 912, a description of the specific requests that an external actor is issuing to the resources within the additional cloud deployments 912, and so on.

In the example method depicted in FIG. 9, receiving 904 information 916 describing one or more actions associated with the additional cloud deployments 912 may be carried out by receiving (directly or indirectly) such information 916 from the additional cloud deployments 912 themselves. In fact, the additional cloud deployments 912 may be queried for such information. In other embodiments, the information 916 describing one or more actions associated with the additional cloud deployments 912 may be retained by the systems described above as part of monitoring the additional cloud deployments 912. In such an example, receiving 904 information 916 describing one or more actions associated with the additional cloud deployments 912 may be carried out by querying a data repository (e.g., a data warehouse) that contains such information that was

82

gathered while monitoring the additional cloud deployments 912. Readers will appreciate that information 916 may be received 904 describing other types of actions associated with the additional cloud deployments 912.

The example method depicted in FIG. 9 also includes receiving 906 information 918 describing configurations associated with the additional cloud deployments 912. The information 918 describing configurations associated with the additional cloud deployments 912 may be embodied, for example, as information describing how components within a cloud deployment are organized (including which other components they may communicate with), information describing what permissions are granted to various users, information describing the manner in which internal (i.e., occurring entirely within the cloud deployment) data communications and/or external (i.e., occurring at least partially external to the cloud deployment) data communications are carried out, and so on. Such information 918 may be received 906 in a similar manner as described above with respect to step 904.

The example method depicted in FIG. 9 also includes identifying 908, based on the configurations and the one or more actions, one or more configurations to adopt for the first cloud deployment 910. Readers will appreciate that by identifying 908 one or more configurations to adopt for the first cloud deployment 910 in such a way, the configuration of the first cloud deployment 910 may be influenced by the monitoring of the additional cloud deployments 912. In such an example, once the configurations that the first cloud deployment 910 should adopt have been identified 908, the first cloud deployment 910 (or associated entities) may be reconfigured to implement such configurations.

Identifying 908 one or more configurations to adopt for the first cloud deployment 910 may be carried out, for example, by identifying configurations in the additional cloud deployments 912 that were effective in dealing with various threats, securing vulnerabilities, or otherwise contributed to a healthy cloud deployment. Some configurations associated with the additional cloud deployments 912 may have been put in place in response to some action. For example, a configuration to always backup data stored in a first storage service (e.g., AWS S3) to a second, distinct storage service (e.g., AWS Glacier) may have been put in place in response to some attack that was directed to a particular cloud deployment's 914a S3 buckets. In such an embodiment, if the first cloud deployment 910 also leverages S3 as its object store, it may be desirable for the first cloud deployment 910 to adopt a configuration setting that would result in its S3 buckets being backed up to Glacier so that the first cloud deployment 910 can be ready to survive a similar attack that was experienced by the particular cloud deployment 914a.

For further explanation, FIG. 10 sets forth a flowchart illustrating an example method of learning from similar cloud deployments in accordance with some embodiments of the present disclosure. The example method depicted in FIG. 10 is similar to the example depicted in FIG. 9, as the example depicted in FIG. 10 also includes identifying 902 one or more additional cloud deployments 912 to utilize for cross-customer learning, receiving 904 information 916 describing one or more actions associated with the additional cloud deployments 912, receiving 906 information 918 describing configurations associated with the additional cloud deployments 912, and identifying 908 one or more configurations to adopt for the first cloud deployment 910.

In the example method depicted in FIG. 10, receiving 904 information 916 describing one or more actions associated

with the additional cloud deployments **912** can include receiving **1002** information describing a security threat to one or more of the additional cloud deployments **912**. The security threat may be embodied, for example, as a ransomware attack that was directed to one or more of the additional cloud deployments **912**, as a denial of service attack that was directed to one or more of the additional cloud deployments **912**, as an SQL injection attack that was directed to one or more of the additional cloud deployments **912**, or as some other security threat. The information describing a security threat to one or more of the additional cloud deployments **912** can include, for example, information describing where an attack originated from, information describing the components within the cloud deployment that were targeted by an attack, information describing data access patterns that were associated with an attack, or some other information that would be useful in detecting subsequent security threats.

In the example method depicted in FIG. **10**, receiving **906** information **918** describing configurations associated with the additional cloud deployments **912** can include receiving **1004** information describing configuration settings used to combat the security threat. The configuration settings used to combat the security threat may be embodied, for example, as configuration settings that caused the additional cloud deployments **912** to blacklist certain IP addresses or network domains, as configuration settings that caused the additional cloud deployments **912** to close vulnerabilities that allowed an attack to succeed, or some other configuration that would be useful in detecting/preventing/mitigating a security threat.

In the example method depicted in FIG. **10**, receiving **904** information **916** describing one or more actions associated with the additional cloud deployments **912** can also include receiving **1006** information describing a detected vulnerability associated with one or more of the additional cloud deployments **912**. The detected vulnerability may be embodied, for example, as a vulnerability to some known security threat, as a vulnerability to some data loss event, as a data breach vulnerability, or as some other vulnerability. The information describing a detected vulnerability of one or more of the additional cloud deployments **912** can include, for example, information describing vulnerable components, information describing data communications protocols, endpoints, or other data communications components that expose vulnerabilities, or some other information that would be useful in detecting a vulnerability.

In the example method depicted in FIG. **10**, receiving **906** information **918** describing configurations associated with the additional cloud deployments **912** can also include receiving **1008** information describing configuration settings used to address the vulnerability. The configuration settings used to address the vulnerability may be embodied, for example, as configuration settings that caused the additional cloud deployments **912** to blacklist certain IP addresses or network domains, as configuration settings that caused certain components within the additional cloud deployments **912** to not connect to external data communications networks, as configuration settings that caused certain components to be accessible only by using enhanced authentication protocols, or as some other configuration that would be useful in detecting, preventing, or otherwise mitigating a vulnerability.

For further explanation, FIG. **11** sets forth a flowchart illustrating an example method of learning from similar cloud deployments in accordance with some embodiments of the present disclosure. The example method depicted in FIG. **11** is similar to the examples depicted in FIG. **9** and

FIG. **10**, as the example depicted in FIG. **11** also includes identifying **902** one or more additional cloud deployments **912** to utilize for cross-customer learning, receiving **906** information **918** describing configurations associated with the additional cloud deployments **912**, and identifying **908** one or more configurations to adopt for the first cloud deployment **910**.

In the example method depicted in FIG. **11**, receiving **906** information **918** describing configurations associated with the additional cloud deployments **912** can include receiving **1102** information describing permissions for one or more users of the additional cloud deployments **912**. The one or more users of the additional cloud deployments **912** may be embodied, for example, as individual users (e.g., Bob Smith), as a collection of users (e.g., Finance Group, Marketing Group), and so on. The one or more users of the additional cloud deployments **912** may also be embodied as different personas within an organization, where each persona is associated with a certain role. For example, there may be ‘software developer’ personas, ‘database administrator’ personas, ‘customer support’ personas, and many others. Receiving **1102** information describing permissions for one or more users of the additional cloud deployments **912** may be carried out, for example, by receiving (directly or indirectly) information from one or more of the additional cloud deployments **912** that described what resources a particular user/persona/group can access, what level of privileges they have with respect to a particular resource, and any other information describing the user/persona/group’s privileges. In fact, such information may be retained in a data warehouse during monitoring of the additional cloud deployments **912** and received **1102** by querying such a data warehouse.

The example method depicted in FIG. **11** also includes determining **1104**, based on the information describing permissions for one or more users of the additional cloud deployments, that one or more users of the first cloud deployment **910** are over-permissioned. A particular user may be over-permissioned in the sense that the user has privileges or permissions beyond those that should be associated with the user. Consider the example of two distinct personas within an organization: 1) a software engineer, and 2) an accounts payable administrator. In such an example, the software engineer might have sufficient privileges to access a code repository, a dev/test environment, a code documentation tool, and so on. The accounts payable administrator, however, might have sufficient privileges to access bill paying software, a financial ledger, and so. The software developer having access to bill paying software or the accounts payable administrator having access to a code repository may be examples of the users being over-permissioned, as the user has access to resources that they should not be able to access. Such situations can create vulnerabilities as an over-permissioned user (or a malicious actor using the over-permissioned user’s credentials) may intentionally or unintentionally perform some harmful act that they would not be capable of performing if they weren’t over-permissioned.

Determining **1104** that one or more users of the first cloud deployment **910** are over-permissioned may be carried out, for example, by examining the information describing permissions for one or more users of the additional cloud deployments **912** to determine whether similar users in the additional cloud deployments **912** have different/fewer privileges. As such, a user in a first cloud deployment **910** may be determined to be over-permissioned if other cloud

85

deployments 912 are giving similar users fewer permissions than such a user has in the first cloud deployment 910.

In the example method depicted in FIG. 11, identifying 908 one or more configurations to adopt for the first cloud deployment 910 can include identifying 1106 a reduced privilege level to give to the one or more users. Identifying 1106 a reduced privilege level to give to the one or more users may be carried out, for example, by giving an over-privileged user of the first cloud deployment 910 privileges that are similar to those held by similar users in the additional cloud deployments 912.

For further explanation, FIG. 12 sets forth a flowchart illustrating an example method of learning from similar cloud deployments in accordance with some embodiments of the present disclosure. The example method depicted in FIG. 12 is similar to the examples depicted in FIGS. 9-11, as the example depicted in FIG. 12 also includes identifying 902 one or more additional cloud deployments 912 to utilize for cross-customer learning, receiving 906 information 918 describing configurations associated with the additional cloud deployments 912, and identifying 908 one or more configurations to adopt for the first cloud deployment 910.

The example method depicted in FIG. 12 also includes receiving 1202 information 1204 describing one or more deployment processes associated with the additional cloud deployments 912. The information 1204 describing one or more deployment processes associated with the additional cloud deployments 912 can include, for example, information describing how a particular component within an additional cloud deployment was created (e.g., who requested that the component be created, when was the request to create the component issued). In addition, in some embodiments such information 1204 can include information describing the software development processes that were used to develop software that will be deployed in a particular cloud deployment (e.g., was a code repository used, who has access to the repository, who committed the last change to some code before deployment). In such a way, the information 1204 describing one or more deployment processes associated with the additional cloud deployments 912 can describe the processes that are being implemented in other cloud deployments. Such information 1204 may be used, for example, to determine whether the development and deployment processes associated with the first cloud deployment 910 are abnormal, to use as the basis for making recommended changes to the development and deployment processes associated with the first cloud deployment 910, to initiate remediation workflows to back out changes to the first cloud deployment 910 that deviated from normal activity or violated some policy, and so on.

The example method depicted in FIG. 12 also includes identifying 1206 abnormally configured components in the first cloud deployment 910. Abnormally configured components in the first cloud deployment 910 may be embodied as components that are configured in a way that deviates from typical configurations observed by monitoring the additional cloud deployments 912. Monitoring the additional cloud deployments 912 may reveal, for example, that a connection to a server that is contained in another cloud deployment 914a should always go through an authentication server that is contained in another cloud deployment 914a. In such an example, if an examination of the first cloud deployment 910 reveals that an authentication server has been bypassed by a connection to a server, this may be an indication of a breach or misconfiguration regardless of customer's network topology. In such an example, the component (i.e., the server) may be identified 1206 as being abnormally configured

86

based on its deviation from typical configurations observed in the additional cloud deployments 912.

Although the examples described above relate to embodiments where one or more configurations to adopt for the first cloud deployment 910 are identified 908 using the information described above, in other embodiments such information may be used for other purposes. For example, information gained by observing additional cloud deployments 912 may be leveraged to identify best-of-breed deployments, similar deployments, and so on.

In some embodiments, the distributions that identify 'normal' behavior for a particular cluster set may be used for a variety of purposes. For example, anomaly detection may be performed by identifying members of the cluster set that are operating outside of a particular typical distribution. Likewise, best practices may be identified using distributions and members of the cluster set that are not adhering to best practices may be identified if they are operating outside of a particular typical distribution. Vulnerabilities may also be identified using distributions, for example, by identifying members of the cluster set that have over-privileged users or components that may be able access things that they should not be able to access. Readers will appreciate that that deviation from an established baseline of normal behavior, normal activity, normal configuration, or other form of normal operation may be indicative of many other things, all of which may be detected through the usage of the distributions described above paired with the monitoring of particular clusters.

In some embodiments, the distributions that identify 'normal' behavior for a particular cluster set may be used to not only detect threats, vulnerabilities, compromise, and things of that nature, but the distributions that identify 'normal' behavior for a particular cluster set may be used to optimize the operation or configuration of a particular cluster. For example, cloud deployments that utilize fewer resources may be examined and characteristics that are common across such the cloud deployments may be identified as being high efficiency characteristics. Likewise, cloud deployments that are the subject of fewer alerts may be examined and characteristics that are common across such the cloud deployments may be identified as being high efficiency characteristics.

In some embodiments, historical information may also be retained and utilized to show the trajectory of a particular one or more clusters. For example, historical information may be used to compare the current state of a particular cluster (as measured by one or more quantifiable characteristics associated with the cluster) with a previous state of the cluster such that trends and/or trajectories may be identified. Consider an example in which a particular characteristic associated with a cluster identifies how many users are accessing the cluster. Historical information associated with the cluster may be compared to current (or most recent) information associated with the cluster to determine, for example, that more or less users are accessing the cluster, that the number of users that are accessing the cluster is increasing or decreasing at a certain rate, and so on. In some situations, some changes may be acceptable (e.g., if the cluster represents an interface to an online store, and increase in the number of users accessing the online store may be perfectly acceptable given an expansion in the organization's customer base) whereas in other situations changes may be troubling (e.g., if the cluster represents a source code repository and more users are accessing the repository in spite of a contraction in the number of devel-

opers that are employed by an organization) and my require alerts, further investigation, or some other remediation workflow.

In some embodiments, historical information may also be retained and utilized to show the trajectory of a particular cluster relative to other members of a cluster set. For example, historical information may be used to compare the failure rate of particular cluster (as measured by one or more quantifiable characteristics associated with the cluster) over time with the failure rate over time of other clusters in the cluster set. Such a comparison may reveal, for example, that a particular cluster was previously failing at a rate that was in line with the failure rate of other members of the cluster set, but that the particular cluster has more recently been failing more/less than other members of the cluster set. As such, through the use of such techniques a determination may be made as to whether a particular cluster is becoming more/less healthy than other members of the cluster set, more/less secure than other members of the cluster set, more/less efficient than other members of the cluster set, more/less reliable than other members of the cluster set, more/less compliant with relevant regulations than other members of the cluster set, slower/faster than other members of the cluster set, and so on. Such an analysis may be part of identifying best-of-breed deployments, best practices, providing remediation actions, providing recommendations, or utilized for a variety of other purposes.

Although the examples described above relate to embodiments where components within a cloud deployment are analyzed, monitored, or otherwise observed, in other embodiments the techniques described herein may be applied to other entities. For example, the techniques described above may be applied to analyze, monitor, or otherwise observe different personas with an organization, different users with an organization, different user groups with an organization, and so on. Using such techniques, 'normal' behaviors for a particular persona (e.g., a database administrator, a network administrator) can be identified, 'normal' behaviors for a particular user group (e.g., users that are part of an organization's finance department, users that are part of an organization's engineering department) can be identified, and so on.

In some embodiments, by identifying 'normal' behaviors (i.e., those behaviors that are consistent with standard distributions for particular characteristics associated with an entity) with different personas, different users, different user groups, or other entities, abnormal behavior may also be identified. Consider an example in which a group of users is identified as a cluster by virtue of those users accessing the same set of applications (e.g., all of the users access a set of applications that are finance-related applications). Further assume that similar clusters are identified for other customers, such that a cluster set may be formed. In such an example, a distribution may be identified for the set of applications that members of the cluster typically access. For example, each member of the cluster set may typically access an accounting application, a spreadsheet application, a payroll application, and so on. If one member of the cluster set consists of one or more users that also access an organization's source code repository, such behavior may be determined to be outside of the typical distribution for members of the cluster set. In response to making some determination, alerts may be generated, access to the source code repository may be blocked for the users in the cluster, a remediation workflow may be initiated, or some other action may be taken.

In some embodiments, the techniques described above may be particularly useful for identifying over-privileged users, user groups, personas, or other entity. Identifying over-privileged users, user groups, personas may be carried out, at least in part, by identifying what set of privileges is 'normal' for a particular entity to have based on evaluating what privileges are given to similar or identical members of a cluster set. That is, an evaluation may be made as to what privileges are given to users, user groups, personas, or other entity by one or more other customers. If the set of privileges given to a particular customer's users, user groups, personas, or other entity are not consistent with (e.g., the set of privileges are much greater than) the privileges given to similar or identical users, user groups, personas, or other entity of another customer (i.e., the other members of the cluster set), a determination may be made that the particular customer's users, user groups, personas, or other entities are over-privileged. In some embodiments, an evaluation as to whether a user, user group, persona, or other entity is over-privileged may also include identifying privileges that the user has and comparing that with the privileges that the user actually utilizes. If some privileges are never used, this may be taken as an indication that the user is over-privileged.

In some embodiments, the cross-customer techniques described above may be used to provide additional context to issues identified in a particular customer's cloud deployment, to provide recommendations to a particular customer, or even to drive remediation actions. Consider an example in which a particular threat was detected in a first customer's cloud deployment, where the threat turned out to be a ransomware attack, which may in some embodiments include an encryption component and/or a data theft or leakage component. In such an example, if an identical (or sufficiently similar, following a general recognized pattern or 'fingerprint') threat is detected in a second customer's cloud deployment, additional context may be provided by including information in an alert that is delivered to the second customer that indicates that the threat matches the profile of a ransomware attack that was detected in the first customer's cloud deployment. In fact, information describing the remedial actions (e.g., disabling encryption, increasing the frequency of backups, locking down a backup system, blocking transmission of data externally, etc.) that were taken by the first customer may even be included in the alert to the second customer or otherwise recommended to the second customer. Furthermore, if many customers had experienced the same attack and the data platform could determine with sufficient certainty that the second customer's cloud deployment was experiencing the same attack, workflows may be automatically initiated to carry out various remedial actions.

In some embodiments, the way other customers investigated or responded to a particular alert may also be used when presenting alerts to a particular customer. Consider an example in which a particular threat was detected in ten different customer's cloud deployments. In such an example, assume that 9 of the 10 customers ignored the alert. In such an example, the conditions that are indicative of the particular threat are detected in a particular customer's cloud deployment, when raising the alert to the particular customer, information may be included in the alert indicating that most other customers ignored the alert. In such an embodiment, it is the behavior of users of the data platform—not components in a cloud deployment—that is being monitored and evaluated for the benefit of other customers. In other embodiments, information indicating

that customers do or do not ignore a particular alert may be used when scoring or ranking the alert. For example, alerts that are ignored by most customers may be ranked as less critical than alerts that are acted upon by most customers. In addition, the particular security stance of a particular customer may be utilized when determining the extent to which their usage of the data platform should be utilized when guiding other customers. If a first customer has relatively poor security practices, for example, the first customer's usage of the data platform may be ignored (or given less weight) for the purposes of guiding other customers. If a second customer has relatively good security practices, however, the second customer's usage of the data platform may be taken into consideration (or given more weight) for the purposes of guiding other customers. Likewise, if users that ignored the alert experienced a security breach while users that did not ignore the alert did not experience a security breach, such an outcome could be taken into consideration when ranking the alert, determining what information to include in the alert, and so on.

In addition to using other customers interactions with the data platform to score, rank, suppress, or provide context for the alerts provided to other customers, the remediation actions taken by other customers may also be used for guiding a particular customer. Consider an example in which a particular threat was detected in ten different customer's cloud deployments. In such an example, assume that 9 of the 10 customers investigated the threat by accessing a particular customer support page. In this example, if the conditions that are indicative of the threat are detected in a particular customer's cloud deployment, information may be included in an alert indicating that most other customers that received the alert investigated the alert by accessing the particular customer support page. Likewise, if other customers successfully resolved the issue, information may be included in an alert indicating the solution that was implemented by other customers.

In some embodiments, the way customers investigated or responded to a particular alert may be used with other information to determine whether the customer's responses were actually correct. Consider an example in which an alert is sent to 10 customers identifying a vulnerability that would allow crypto miners to use the customer's resources for solving complex computational problems for the purposes of acquiring cryptocurrency. In such an example, assume that 9 of 10 customers ignored the alert. Further assume that in this example, the 9 customers that ignored the alert subsequently had their cloud resources utilized by hackers for crypto mining, whereas the 1 customer that did not ignore the alert did not suffer such an attack. In such an example, the data platform described above should not cease issuing alerts when detecting this vulnerability by virtue of 9 of 10 customers ignoring the alert. Instead, by taking the ultimate outcome for each customer into consideration, rather than suppressing the alert, the data platforms should respond by taking actions that would make it more likely that customers would not ignore these alerts. For example, the severity level of the alert may be raised, a user of the data platform may be required to confirm receipt of the alert, contextual information could be included in the alert indicating that recipients that do not respond to the alert end up having their systems hijacked by crypto miners, or some other action may be taken.

In some embodiments, a customer's interactions may be analyzed to improve how alerts are delivered to a customer. For example, if an evaluation of customer interactions with the data platform indicates that most customer ignore alerts

issued between 11:00 PM-7:00 AM whereas most customers take action in response to alerts issued between 7:01 AM-10:59 PM, then alerts that are issued between 11:00 PM-7:00 AM may be reissued between 7:01 AM-10:59 PM. Likewise, if a particular class of alerts (e.g., those related to vulnerability threats) are largely acted upon whereas another class of alerts (e.g., those related to a lack of compliance with regulatory requirements) are largely ignored, the manner in which alerts are generated may be altered. For example, alerts that are related to a lack of compliance with regulatory requirements may be issued to additional users, such as some user that is designated as having a compliance persona within a customer's organization. In other embodiments, other aspects of the customer's interactions with the data platform may be used to improve the manner in which the data platform interacts with customers (e.g., what type of devices received alerts, were alerts delivered in a primary or secondary window, do the parties that the alerts were sent to have other obligations at the time of alert as determined from an inspection of their calendar, and so on). While the example described above related to embodiments where the interactions of multiple customers with the data platform is analyzed, in other embodiments a single customer's interaction with the data platform may serve as the basis for altering the way that the data platform interacts with the customer.

In some embodiments, the same or additional cross customer learning techniques may be applied to earlier stages of a software development pipeline and even before an actual cloud deployment is in place for a particular customer. Stated differently, the same or additional cross customer learning techniques may be applied to things other than a deployed system. For example, cross customer learning may be applied to development processes, testing processes, deployment processes, and so on.

Readers will appreciate that developing, testing, and deploying software in cloud environment comes with a few requirements that were not always present in software development. In particular, software is always expected to be running but the software is also expected to continue to be updated with fixes, new features, or other improvements. As such, it is not desirable to cease running a software application, install a new version, and begin running the new version. As a result of these changes to the software development paradigm, the software development processes, software testing and validation processes, and deployment processes tend to be fairly independent of each other. In some embodiments, each of these processes may be analyzed, monitored, or otherwise observed by the data platforms described herein. Because each of these processes are analyzed, monitored, or otherwise observed by the data platforms described herein, an opportunity exists to leverage cross customer learning to these processes.

In some embodiments, the development processes for a particular customer may be analyzed, monitored, or otherwise observed by the data platforms described herein. For example, interactions with a code repository may be monitored, permissions granted to each user of the code repository may be monitored, the number of people that are checking in and checking out code may be monitored, the extent to which code revisions are documented may be monitored, and so on. In such an example, the development processes for a particular customer may be learned and compared to normal behavior for other customers. Through such comparisons, inefficiencies may be identified, vulnerabilities may be identified, and other shortcomings may be identified. In response to the identification of inefficiencies,

vulnerabilities, or other shortcoming, alerts may be issued, remediation workflows may be initiated, or some other action may be taken.

In some embodiments, the testing processes for a particular customer may be analyzed, monitored, or otherwise observed by the data platforms described herein. For example, the number of tests that are run may be monitored, the type of tests that are run may be monitored, the number of people that are running tests and evaluating test results may be monitored, the processes through which the results of testing are communicated with developers may be monitored, and so on. In such an example, the testing processes for a particular customer may be learned and compared to normal behavior for other customers. Through such comparisons, inefficiencies may be identified, vulnerabilities may be identified, and other shortcomings may be identified. In response to the identification of inefficiencies, vulnerabilities, or other shortcoming, alerts may be issued, remediation workflows may be initiated, or some other action may be taken.

In some embodiments, the results of testing may be carried forward and utilized when an application is deployed. Consider an example in which testing reveals that the code performs some action that appears to create a possible vulnerability, but further testing and validation reveals that a vulnerability is not created. When this piece of code is subsequently deployed, it presumably will perform the same action (i.e., the action that was flagged during testing) that appears to create a possible vulnerability. Rather than raising an alert, initiating some investigative action, initiating a remedial action, or responding in a similar way, by carrying forward the knowledge gained during testing (i.e., that a vulnerability was not, in fact, created) the possible vulnerability may be ignored based on the conclusions reached during testing.

In some embodiments, the deployment processes for a particular customer may be analyzed, monitored, or otherwise observed by the data platforms described herein. In such an example, the deployment processes for a particular customer may be learned and compared to normal behavior for other customers. Through such comparisons, inefficiencies may be identified, vulnerabilities may be identified, and other shortcomings may be identified. In response to the identification of inefficiencies, vulnerabilities, or other shortcoming, alerts may be issued, remediation workflows may be initiated, or some other action may be taken.

In some embodiments, some forms of static analysis may be used (in conjunction with other features of the data platform) to detect anomalies, vulnerabilities, threats, misconfigurations, violations of regulatory requirements, and many other things. Consider an example in which a cloud deployment is deployed using IaC. In such an example, one or more configuration files may be examined, and the state of the cloud deployment may be monitored to identify situations in which the state of the cloud deployment drifts from the configuration of the cloud deployment that was described in the configuration file. As such, the intended configuration of the cloud deployment (at least as expressed in one or more configuration files) may be used as a baseline to measure the current cloud deployment, such that alerts may be issued or other remediation workflows may be initiated when a customer's cloud deployment deviates from its codified state.

In some embodiments, other forms of code other than an IaC configuration may be examined to detect anomalies, vulnerabilities, threats, misconfigurations, violations of regulatory requirements, and many other things. For

example, the source code that has been deployed in a customer's environment may be examined to determine all the things that the code could do. This information may be compared to a polygraph for the customer's cloud deployment, which identifies all things that a customer's cloud deployment does do, as learned by monitoring and observing the customer's cloud deployment. Consider an example in which a polygraph for a customer's cloud deployment indicates that a first microservice in their cloud deployment only communicates with other internal microservices. In such an example, however, assume that the source code for the first microservice includes a messaging library that it uses to communicate with the other internal microservices. Further assume in this example, however, that examining the source code for the first microservice reveals that the messaging library also includes functions that enable a user of the library to send messages to recipients on an external network using standard internet protocols (e.g., TCP/IP, HTTPS, and so on). In such an example, although the polygraph for the customer's cloud deployment reveals that the first microservice only communicates with other internal microservices, the presence of functions that could be used for communications with external services, machines, and other entities may be undesirable. As such and in accordance with some embodiments of the present disclosure, the data platform may be configured to alert a customer or initiate some other workflow upon detecting that source code includes features that, if executed, would rise to the level of anomalous or otherwise unusual activity. For example, the data platform may prompt the customer to delete or disable the library functions described above that enable undesirable data communications.

In another example, assume that some source code has all the necessary capabilities (e.g., hashing functions, mathematical calculations, etc.) to mine cryptocurrency, as the mining process may only require performing standard computations. During a crypto miner attack, however, the source code may be subverted such that the code runs in a loop to perform the standard computations required for mining cryptocurrency. An evaluation as to what functions the source code can perform may therefore not reveal anything concerning but monitoring the actual operation of the source code may reveal that the source code is operating in an undesirable way. Likewise, static analysis may reveal things like a non-incrementing counter or other mechanism that would result in an infinite loop or similar operation. As such and in accordance with some embodiments of the present disclosure, the data platform may be configured to alert a customer or initiate some other workflow upon detecting that source code includes features that, if or when executed, rise to the level of anomalous or otherwise unusual activity.

In another example, assume that some source code has all the necessary capabilities to carry out the steps required for a ransomware attack, as the ransomware attack may only require performing standard operations like reading data, encrypting data, sending data, communicating externally, and so on. During a ransomware attack, however, the source code may be subverted such that the code does far more encryption than would be expected in a typical code module. An evaluation as to what functions the source code can perform may therefore not reveal anything concerning but monitoring the actual operation of the source code may reveal that the source code is operating in an undesirable way. Likewise, static analysis may reveal that all paths through the source result in data being encrypted, or that all paths through the source code results in some atypical pattern (e.g., equal amounts of reading data, encrypting data,

and writing data, or unusual external data flows), which may rise to the level of being unusual or concerning behavior. As such and in accordance with some embodiments of the present disclosure, the data platform may be configured to alert a customer or initiate some other workflow upon detecting that source code includes features that, if or when executed, rise to the level of anomalous or otherwise unusual activity.

In some embodiments, the data platforms described above may be used to identify discriminates between two entities that may otherwise appear to be similar or identical. In the examples described herein, ‘discriminates’ may be embodied as characteristics of entities that, when not similar or identical, prevent the entities from being similar or identical. For example, many customers may have many Java processes. If the data platform were to identify the set of things that each Java process does for each customer and identify that set of things as the ‘normal’ or acceptable set of things that any Java process was allowed to do, this set would be far too large. As such, the data platform may be configured to look at things like command line arguments and know that one or more Java processes with one set of jar files and command line arguments is actually a separate program from one or more Java processes with another set of jar files and command line arguments. In other words, the data platform may identify discriminates (in the example, a first discriminate being distinct command line arguments and a second discriminate may be that the jar files for each process are distinct) between two sets of Java processes to determine that the Java processes are not actually similar or identical entities. As such, any attempt to engage in cross customer learning based on a first set of Java processes in a first customer’s environment and a second set of Java processes in a second customer’s environment may result in undesirable outcomes. Through the use of discriminates, however, a decision can be reached that these two sets of Java processes are not related and attempts to learn through an examination of these distinct entities would be undesirable.

In some embodiments, the usage of discriminates described above may be extended to other entities. For example, if a first user in a first customer’s environment is a database administrator and a second user in a second customer’s environment is also a database administrator, these users may initially be determined to be similar enough such that cross customer learning can take place. In such an example, however, if an examination of their activities, privileges, or something else reveals that they are not actually occupying the same roles, cross customer learning with respect to these two users may be disabled. In such embodiments, the data platform described above, and models leveraged by such a data platform may be used to identify discriminates that may be used to decouple multiple users, applications, microservices, devices, or other entities that would otherwise be candidates for cross customer learning.

In some embodiments, the data platforms may be configured to analyze, monitor, or otherwise observe environments other than cloud deployments. In fact, the data platforms described here could apply the principles and techniques described herein to any environment such as, for example, an on-premises environment, a hybrid cloud environment, or some special purpose environment. As one example of a special purpose environment, consider an example in which the data platforms described herein are used to analyze, monitor, or otherwise observe a container orchestration environment such as a Kubernetes cluster (which may be deployed on-premises, in a public cloud, or in some other way). In such an example, the data platform may be con-

figured to ingest Kubernetes audit logs via one or more agents or in some other way. Through the ingestion and subsequent analysis of such audit logs, the data platform may model normal behaviors of a Kubernetes cluster, normal behavior of a cluster administrator, and so on. As described above, any deviations from such normal behaviors may result in an alert being generated or some other remediation workflow being initiated. For example, if the ingestion and subsequent analysis of audit logs revealed that workloads are deployed and deleted according to some pattern, a customer deploying or deleting a workload in a manner that is inconsistent with the identified pattern may result in an alert being generated or some other remediation workflow being initiated. In fact, by evaluating audit logs from multiple customer’s Kubernetes clusters, cross customer learning can be carried out to help define normal behavior for a Kubernetes cluster, a Kubernetes administrator, or some other entity associated with a Kubernetes cluster. For example, evaluating audit logs from the Kubernetes deployment of multiple customers may reveal that one entity (presumed to be a Kubernetes administrator, or related group of such administrators) is generally responsible for creating and deleting nodes from the cluster whereas another entity (perhaps a developer) is responsible for deploying new versions of the code that is executing in a container that is supported by the cluster. In such an example, if a single entity (or related group of entities) is observed creating nodes and modifying the code that is executing on a node by deploying a new container, an alert may be generated, or some other remediation workflow may be initiated as a consequence of observing this atypical pattern.

For further explanation, FIG. 13 sets forth a flowchart of an example method for a guided anomaly detection framework according to some embodiments of the present disclosure. The method of FIG. 13 may be performed, for example, in a data platform (also referred to as an anomaly detection framework) as described above. The anomaly detection framework includes one or more functions or services used to detect, in the cloud deployment, anomalies, threats, and the like as are described above. The anomaly detection framework also includes particular interfaces (e.g., user interfaces, APIs, database interfaces, a natural language interface, and the like) to access data monitored or generated by such anomaly detection functions or services.

As an example, the anomaly detection framework may be accessed or interacted with using a natural language interface. Readers will appreciate that the natural language interface for an anomaly detection framework may be embodied, for example, as one or more modules of computer program instructions executing on computer hardware (including virtualized computer hardware) that can receive natural language inputs such as text, text generated using speech-to-text technologies, or other forms of natural language. The natural language interface may be configured to parse the natural language that it receives, process that input, and ultimately generate some input data that can be acted upon by the anomaly detection framework, as will be described in greater detail below. Alternatively, the process of translating natural language inputs to some input data that can be acted upon by the anomaly detection framework may be performed (at least in part) by modules that are external to the natural language interface. In such a way, users may interact with the anomaly detection framework using natural language instead of needing to understand more technical query languages, programming languages, or the like. Such a user may interact with the anomaly detection framework, for example, to conduct investigations into anomalies that

the anomaly detection framework has identified, where the anomalies are related to a cloud deployment that is being monitored by the anomaly detection framework (or where the anomaly detection framework is configured for monitoring the cloud deployment).

In some embodiments, the natural language interface accepts, as input, natural language inputs including text encodings of structured natural language. For example, in some embodiments, the natural language inputs may include inquiries (expressed in natural language) related to the cloud deployment or assets therein. In some embodiments, the natural language interface may provide such natural language inputs to the anomaly detection framework where the natural language inputs are converted into queries for data related to the cloud deployment. Such queries may include database queries, API calls, or other queries as can be appreciated that retrieve information necessary to respond to the natural language input. In other words, queries may include a programmatic or executable conversion or transformation of received natural language inputs. Accordingly, the queries include one or more functions or operations to retrieve or determine information corresponding to an inquiry or investigation expressed by the natural language input. A response to a natural language input may be generated based on a response to the corresponding query and provided to the natural language interface for rendering or display. The response may include, for example, a natural language formatting or presentation of data included in the response to the query.

In some embodiments, the natural language interface may be implemented at least partially on a user device. For example, in some embodiments, the natural language interface may include a binary or command line interface (CLI) on the user device. The CLI may be used to accept natural language inputs from a user and provide those natural language inputs to the anomaly detection framework. The CLI may also be used to display responses to natural language inputs as received from the anomaly detection framework. The CLI may further be used to display prompts or other information as described below.

The method of FIG. 13 includes gathering **1302** data describing activity associated with an anomaly detection framework that is monitoring a cloud deployment. In some embodiments, gathering **1302** the data describing activity associated with the anomaly detection framework may be performed in response to detecting some alert, anomaly, threat, or other event in order to facilitate an investigation of the event as will be described in further detail below. In some embodiments, gathering **1302** the data describing activity associated with the anomaly detection framework may be performed in response to a user accessing or establishing a session with the anomaly detection framework, such as using a natural language interface. For example, in response to starting execution of a binary or process for the natural language interface on the user device, or in response to logging in or authenticating with the anomaly detection framework via the natural language interface, a signal or command from the natural language interface may be sent that causes gathering **1302** of the data describing activity associated with the anomaly detection framework. In some embodiments, gathering **1302** the data describing activity associated with the anomaly detection framework may be performed as part of a background or continually executing process, performed at a predefined interval, and the like. Moreover, in some embodiments, combinations of approaches for gathering **1302** the data describing activity associated with the anomaly detection

framework may be used. For example, certain portions of data may be gathered **1302** in response to detecting a particular event, other portions gathered as part of a separate process independent of any particular alert or event, and further portions of data may be gathered **1302** in response to a particular user accessing the anomaly detection framework using the natural language interface.

The method of FIG. 13 also includes generating **1304**, based on the data describing activity associated with an anomaly detection framework, a prompt describing one or more natural language inputs for a security workflow. Each of the one or more natural language inputs may correspond to a query for information related to a particular cloud deployment that is being monitored by the anomaly detection framework. As described herein, the one or more natural language inputs described by the prompt may each correspond to a distinct query in that the one or more natural language inputs, if received via the natural language interface, cause a query for information to be generated. Furthermore, each of the natural language inputs ultimately result in a response that is based on that queried information provided via the natural language interface.

In some embodiments, the prompt describes the one or more natural language inputs in that the prompt suggests, to a user of the natural language interface, that any of the one or more natural language inputs could be provided as input to the natural language interface. For example, the prompt may state “Consider asking ‘Which of my virtual machines have recently failed?’” Where the prompt describes multiple natural language inputs, the prompt may state, for example, “Consider asking ‘Which of my virtual machines have recently failed?’ or ‘Which of my virtual machines have high memory utilization?’” In some embodiments, the prompt may indicate a particular event or alert that may serve as a basis for recommending the one or more natural language inputs. For example, the prompt may state, prior to describing the one or more natural language inputs, “An anomaly has been detected. Many virtual machines have recently failed.” In other words, the prompt serves to guide the user toward possible inquiries to submit via the natural language interface.

As is set forth above, the generated prompt describes one or more natural language inputs for a security workflow. A security workflow can include one or more related interactions (e.g., queries, natural language inputs, user interface inputs, and the like) for requesting information related to the cloud deployment, particularly with respect to security events such as anomalies, threats, and the like. The one or more natural language inputs may therefore correspond to a particular security workflow.

In some embodiments, the natural language inputs that are described in the prompt may include a predefined security workflow as described above. Approaches for selecting the predefined security workflow will be described in further detail below. In some embodiments, the particular security workflow may be dynamically generated by virtue of multiple interactions with the natural language interface by the user. For example, an initial selection of one or more natural language inputs may be performed for description in the prompt. Based on the particular natural language input received via the natural language interface, responses to queries for the received natural language input, and the like, a next selection of natural language inputs may be selected for inclusion in a subsequent prompt. Thus, the security workflow is effectively dynamically generated based on the received natural language inputs, responses to correspond-

ing queries for the received natural language inputs, and potentially other data as will be described in further detail below.

The method of FIG. 13 also includes providing 1306 a selected natural language input to a natural language interface of the anomaly detection framework. In this example, a user such as a system administrator, a member of the security team for an organization, or some other user may make a selection from the prompt that described one or more natural language inputs for a security workflow. In such a way, a particular natural language input may be selected, such that the selected natural language input may be provided 1306 to the anomaly detection framework via a natural language interface of the anomaly detection framework. As described in more detail elsewhere, by providing the selected natural language input to a natural language interface of the anomaly detection framework, a query may be generated and ultimately executed by the anomaly detection framework. For example, a text encoding of the selected natural language input may be provided to a user device executing the natural language interface for rendering or display via a CLI, user interface, or another interface as can be appreciated.

Readers will appreciate that, as described above and expanded upon below, the process of gathering data describing activity associated with an anomaly detection framework that is monitoring a cloud deployment, generating a prompt describing one or more natural language inputs for a security workflow, and providing a selected natural language input to a natural language interface of the anomaly detection framework may allow the anomaly detection framework to effectively guide a user through a security investigation. Additional details will be provided below, but the guidance that is provided to the user may be based on the actions of domain experts (i.e., experts in investigating potential security issues, compliance issues, governance issues, or other issues associated with a cloud deployment), the guidance may be based on insights derived by the anomaly detection framework, the guidance may be based on investigations from other customers, or the guidance may be generated in some other way so as to enable a relatively new or unskilled user to leverage the knowledge of more sophisticated entities.

As referenced above, the example method of FIG. 13 includes gathering 1302 data describing activity associated with an anomaly detection framework that is monitoring a cloud deployment. In some embodiments, the data describing activity associated with the anomaly detection framework may include data describing one or more events that occurred with respect to the cloud deployment. Such events may include events detected by the anomaly detection framework while monitoring activity associated with the cloud deployment. For example, in some embodiments, the one or more events may include one or more identified anomalies in the cloud deployment. Such anomalies may include, for example, deviations from normal user behavior, deviations from normal activity for particular resources, or other anomalies as can be appreciated. Accordingly, in some embodiments, data describing the one or more identified anomalies may include data describing particular alerts raised in response to detecting particular anomalies. In some embodiments, the data describing activity associated with the anomaly detection framework may also include data describing events or activity that have not been identified as anomalous but are monitored by the anomaly detection framework.

In some embodiments, the one or more events may include one or more detected security threats. In some embodiments, the one or more detected security threats may include detected anomalies or other detected events that have been classified as or escalated to the level of being a security threat. For example, one or more detected events may satisfy a pattern of activity indicative of a particular attack, breach, or other security threat as can be appreciated. As another example, activity with respect to particular assets of the cloud deployment (e.g., virtual machines, containers, storage resources, and the like) may indicate that the particular asset has been compromised by a malicious user, malware, and the like. For example, network activity of an asset may indicate that the asset is communicating with a known command-and-control server for a ransomware attack. As a further example, some data payload associated with a known exploit may be detected in network activity of some asset.

In some embodiments, the data describing activity associated with the anomaly detection framework may include data describing a state of one or more assets of the cloud deployment. For example, the state of one or more assets of the cloud deployment may include whether particular assets are active, suspended, in a failure state, and the like. As another example, the state of one or more assets of the cloud deployment may include configurations of particular assets, including resources allocated to the particular asset, software installed on the particular asset, permissions associated with the particular asset, or other configuration parameters as can be appreciated. In some embodiments, the state of one or more assets of the cloud deployment may include one or more vulnerabilities of particular assets. Such vulnerabilities may be identified, for example, based on the configuration of the assets described above, or by other approaches. In some embodiments, the state of one or more assets may include a current workload or a current amount of resources being used by the asset, including processing resources, memory resources, bandwidth resources, storage resources, and other resources as can be appreciated.

In some embodiments, the data describing activity associated with the anomaly detection framework may include data indicating one or more user interactions with the anomaly detection framework. Such interactions may be performed with respect to various interfaces of the anomaly detection framework, with respect to various exposed APIs or services of the anomaly detection framework, and the like. For example, in some embodiments, the one or more interactions may include one or more previous queries (e.g., queries for data related to the cloud deployment) to the anomaly detection framework. In some embodiments, the one or more previous queries may have been generated or provided by a user currently accessing the natural language interface. In some embodiments, the one or more previous queries may include one or more queries provided by a domain expert. A domain expert may be, for example, a designated user of the anomaly detection framework identified as having some particular relevance, knowledge, expertise, or specialty in security. For example, a domain expert may include a member of a security team, a manager or supervisor of a security team, a user identified as having particular credentials or certifications, a user identified as having completed some form of training or other process so as to be designated a domain expert, or other another user as can be appreciated. For example, such previous queries may have been generated or provided as input to some other interface of the anomaly detection framework.

In some embodiments, the one or more interactions may include one or more previous interactions with a user interface of the anomaly detection framework, such as a graphical user interface (GUI). The one or more previous user interface interactions may include interactions performed by a particular user (e.g., a user currently accessing the anomaly detection framework via the natural language interface), or interactions performed by other users such as domain experts. For example, such interactions may include selections of particular user interface elements to present certain types of information. Such interactions may also be correlated with particular queries generated or issued in response to the corresponding interactions.

In some embodiments, the one or more interactions may include one or more previously provided natural language inputs. In some embodiments, the one or more previously provided natural language inputs may have been provided by a user currently accessing the anomaly detection framework via a natural language interface. Such natural language inputs may have been provided, for example, via a natural language interface as described above. In some embodiments, the one or more previously provided natural language inputs may have been provided by other users such as domain experts. In some embodiments, such interactions may be correlated with data describing a state of the cloud deployment at the time they were performed or issued. For example, data describing one or more interactions may be correlated with data describing contemporaneous alerts, data describing a contemporaneous state of one or more assets, and the like.

In some embodiments, multiple interactions may be related together (e.g., as a sequence of interactions, as a non-linear directed or non-directed graph or other taxonomy of interactions, as an unordered collection of interactions, and the like) as security workflows. A security workflow may be embodied, for example, as a group of related interactions performed to investigate a particular anomaly, threat, or other event by requesting particular information. In some embodiments, security workflows may be manually defined. For example, a particular sequence or other grouping of interactions may be defined for particular events, for particular events with respect to a particular context such as a state of the cloud deployment or assets therein, and the like. Such groupings of interactions may be defined or curated by domain experts or other users as preferred or standardized security workflows for particular events. In some embodiments, such groupings of interactions may be dynamically determined. For example, multiple instances of particular interactions may be detected across multiple instances of a similar event, thereby indicating that such interactions should be included in a security workflow for that event.

As referenced above, the example method of FIG. 13 includes generating **1304**, based on the data describing activity associated with an anomaly detection framework, a prompt describing one or more natural language inputs for a security workflow. In some embodiments, the natural language inputs described in the prompt are generated **1304** based on the gathered **1302** data described above. For example, in some embodiments the data describing activity contemporaneous to a particular event (e.g., a particular anomaly, threat, and the like) may be used to select a particular predefined workflow from which the one or more natural language inputs are generated so as to involve the particular predefined workflow. In other embodiments, the natural language inputs that are described in the prompt may be generated **1304** independent of any particular predefined

workflow. Activity may be deemed to be contemporaneous to a particular event based on occurring within some time window before and/or after the event. Such activity may also include the event itself. For example, in some embodiments, the one or more natural language inputs may be determined by determining a particular predefined security workflow associated with an event most similar to the particular event, potentially based on similarities with respect to the context of the particular event and selecting the one or more natural language inputs from that predefined security workflow.

In some embodiments, generating the one or more natural language inputs may include providing input to a trained model configured to output an indication of a predefined security workflow from which the one or more natural language inputs are selected, to output a security workflow generated by the model itself, or to output a particular one or more natural language inputs independent for progressively dynamically generating a security workflow via subsequent user interactions. The trained model may be trained based on at least a portion of the gathered **1302** data described above. In some embodiments, some portion of the gathered **1302** data describing historical activity may be used to train the model. For example, data describing particular past events, data describing the state of various assets of the cloud deployment contemporaneous to such events, and/or other data may be correlated with particular interactions, particular security workflows, and the like. Where such correlated interactions or security workflows are not encoded or defined as natural language inputs, in some embodiments, such correlated interactions may also be further associated with particular natural language inputs, natural language keywords or templates, and the like. The model may accept, as input, gathered **1302** data describing activity contemporaneous to some recent event, including an indication of the particular event, data describing a current state of one or more assets of the cloud deployment, some portion of data describing historical activity, and potentially other data as can be appreciated.

The approaches set forth above describe an approach for a guided anomaly detection framework, particularly using a natural language interface for the anomaly detection framework. In response to an event and some user accessing the anomaly detection framework via the natural language interface, the user is prompted with a suggestion of various investigative inquiries that may be performed using natural language inputs. As the user continues to interact with the natural language interface, the user may be presented with other prompts for natural language inputs, effectively guiding or teaching a user how to perform an investigation for some event using the natural language interface.

Accordingly, in some embodiments, the approaches set forth herein with respect to generating **1304** prompts and providing **1306** the selected natural language input may be performed for a limited set of users accessing the anomaly detection framework using the natural language interface. For example, such users may include users that have provided some input or parameter to the natural language interface indicating that they wish to have a guided experience while using the natural language interface. As another example, such users may include users that have a limited amount of experience with the natural language interface by virtue of time, number of natural language inputs submitted, number of events investigated, and the like. As a further example, such users may include users having some tag or parameter of their user account set (e.g., by an administrator or other entity) indicating that the user should have a guided experience while using the natural language interface. Thus,

101

users may eventually migrate away from the guided experience of the natural language interface as their expertise in using the natural language interface grows.

For further explanation, FIG. 14 sets forth a flowchart of another example method for providing a guided anomaly detection framework according to some embodiments of the present disclosure. The method of FIG. 14 is similar to the method from FIG. 13 in that the method of FIG. 14 also includes gathering 1302 data describing activity associated with an anomaly detection framework that is monitoring a cloud deployment; generating 1304, based on the data, a prompt describing one or more natural language inputs for a security workflow; and providing 1306 the selected natural language input to a natural language interface.

The method of FIG. 14 differs from FIG. 13 in that the method of FIG. 14 also includes receiving 1402 a natural language input from the natural language interface. For example, a user may provide a natural language input to the natural language interface after having been provided the prompt. The natural language input is then sent from the user device to the anomaly detection framework. In some embodiments, the natural language input may include one of the one or more natural language inputs described in the prompt. In some embodiments, the natural language input may include an input different than the one or more natural language inputs described in the prompt.

The method of FIG. 14 also includes providing 1404, based on a corresponding query for the selected natural language input, a response to the selected natural language input. The corresponding query is a query for information related to the cloud deployment that is generated based on the received natural language input. For example, the corresponding query may be generated from the received natural language input using machine learning approaches, rules-based approaches, and other approaches as can be appreciated.

The query may include, for example, a database query encoded in a query language, an API call, a function call, or other query as can be appreciated. For example, in some embodiments, the query may be issued to a hypervisor or other software that manages configuration of the cloud deployment to retrieve data describing the state of the cloud deployment or one or more assets of the cloud deployment. As another example, in some embodiments, the query may be issued to a database or data warehouse storing event data associated with the cloud deployment (e.g., data received from agents as described above). In some embodiments, a response to the received natural language input may be generated by parsing or otherwise transforming the response to the query. For example, the response to the query may be transformed, parsed, and/or formatted into a readable text format. Accordingly, in some embodiments, providing 1404 the response to the selected natural language input may include providing the response to a user device for processing and presentation via the natural language interface.

The method of FIG. 14 also includes generating 1406, based on the received natural language input, another prompt describing another one or more other natural language inputs for the security workflow. For example, in embodiments where the one or more natural language inputs described in the initially generated 1304 prompt correspond to a predefined security workflow a dynamically generated security workflow provided as output by one or more models, the one or more natural language inputs in the other prompt may correspond to next natural language inputs in a

102

sequence or other ordering of natural language inputs for the security workflow in which the received 1402 natural language input is included.

In some embodiments, the one or more other natural language inputs for the other prompt may be generated according to similar approaches as are described above, including machine learning approaches using a trained model. In such embodiments, the natural language input and/or a response to the received natural language input (e.g., including a response to the query corresponding to the received natural language input) may be provided as input to such a model. Thus, both the received natural language input as well as data received in response to the natural language input may also be used to determine the next natural language inputs for inclusion in the other prompt. In such embodiments, other data including portions of the gathered 1302 data may also be provided as input to the model. The model may provide, as output, the one or more other natural language inputs for inclusion in the other prompt, or an updated security workflow from which the one or more other natural language inputs may be selected for inclusion in the other prompt. In such an example, additional selections may be made so as to continue the security investigation. In some embodiments, the method may return to receiving 1402 a natural language input, which may correspond to one of the other natural language inputs included in the generated 1406 prompt or a different natural language input. Thus, as natural language inputs are received, new prompts are generated and provided to the natural language interface. Such a process may continue as the user performs their desired investigation.

For further explanation, FIG. 15 sets forth a flowchart of another example method for providing a guided anomaly detection framework according to some embodiments of the present disclosure. The method of FIG. 15 is similar to the method from FIG. 13 in that the method of FIG. 15 also includes gathering 1302 data describing activity associated with an anomaly detection framework that is monitoring a cloud deployment; generating 1304, based on the data, a prompt describing one or more natural language inputs for a security workflow; and providing 1306 the selected natural language input to a natural language interface.

The method of FIG. 15 differs from FIG. 13 in that the method of FIG. 15 also includes gathering 1502 data associated with one or more other cloud deployments of one or more other customers. In some embodiments, the data platform and the anomaly detection framework may service multiple customers. Each customer may be associated with their own respective cloud deployments separately monitored by the anomaly detection framework. Accordingly, where the cloud deployment corresponds to a particular customer, data describing activity associated cloud deployments of other customers may also be gathered and used according to similar approaches as described herein with respect to the gathered data associated with the particular customer. For example, data associated with other customers may be used to train models for selecting natural language inputs for inclusion in a prompt, for generating or deriving security workflows for previously occurred events, and the like.

For further explanation, FIG. 16 sets forth a flowchart of another example method for providing a guided anomaly detection framework according to some embodiments of the present disclosure. The method of FIG. 16 is similar to the method from FIG. 13 in that the method of FIG. 16 also includes gathering 1302 data describing activity associated with an anomaly detection framework that is monitoring a

cloud deployment; generating **1304**, based on the data, a prompt describing one or more natural language inputs for a security workflow; and providing **1306** the selected natural language input to a natural language interface.

The method of FIG. **16** differs from FIG. **13** in that the method of FIG. **16** also includes providing **1602**, to the natural language interface, data describing how the prompt was generated. The data describing how the prompt was generated may be encoded as text displayed via the natural language interface. The data describing how the prompt was generated provides insight to a user as to why particular natural language inputs were selected for inclusion in the prompt, further training the user on methodologies of security investigations.

In some embodiments, the data describing how the prompt was generated includes data describing a particular alert. For example, the particular alert may correspond to a recently generated alert being investigated by the user. The data describing the particular alert may describe, for example, particular detected events that caused the alert to be generated. The data describing the particular alert may also include other detected activity, states of assets, and the like that caused the alert to be generated.

In some embodiments, the data describing how the prompt was generated includes data describing how the one or more natural language inputs were selected for inclusion in the prompt. For example, the data may describe an association between a particular alert or event and a particular security workflow from which the one or more natural language inputs were selected. As another example, the data may describe particular inputs to a model that contributed to a decision to select the one or more natural language inputs for the security workflow. As a further example, where the prompt is provided after some natural language input has been received (e.g., as the second or other subsequent prompt of multiple prompts) the data may describe relations between responses to previously submitted natural language inputs and the natural language inputs included in a prompt. In other words, the data may describe that, due to the response to the previous natural language input including some information, these natural language inputs were selected to follow up on that information.

In some embodiments, other data may also be provided to the natural language interface to inform a user. For example, data describing particular motivations or best practices that drove selection of particular natural language inputs in the prompt. In other words, the data may describe why particular natural language inputs are useful in investigating particular alerts or security events.

In some embodiments, providing **1602** the data describing how the prompt was generated may be provided in-line or with the prompt itself. In some embodiments, providing **1602** the data describing how the prompt was generated may be provided in response to receiving some other input via the natural language interface. For example, in response receiving to a natural language input of "Why?" or some other input after providing the prompt, the data describing how the prompt was generated may be provided **1602** to the natural language interface.

For further explanation, FIG. **17** sets for an example method of leveraging generative artificial intelligence (AI) for securing a monitored deployment **1702** in accordance with some embodiments of the present disclosure. The monitored deployment **1702** depicted in FIG. **17** may be embodied, for example, as a cloud deployment as described above (including a public cloud deployment, a private cloud deployment, or as a combination thereof), as an on-premises

deployment, or as some other deployment that can include compute resources, storage resources, networking resources, other infrastructure resources, software resources, cloud services, software execution resources (e.g., containers, AWS EC2 instances and the like, AWS Lambda instances and the like), other resources, or combinations of such resources. Such a deployment is a 'monitored' deployment in the sense that a monitoring tool **1710** such as the systems described above can be utilized to collect data (e.g., via one or more agents), analyze that data, generate alerts, make recommendations, perform remediation workflows, and perform many of the other functions described above.

The example method depicted in FIG. **17** includes receiving **1706** natural language input **1704** associated with the monitored deployment **1702** that is monitored by a monitoring tool **1710**. The natural language input **1704** may be received **1706**, for example, via an interface to the monitoring tool **1710**, via an interface that is communicatively coupled to the monitoring tool **1710**, via an interface to the generative AI application **1712**, via an interface that is communicatively coupled to the generative AI application **1712**, or in some other way. In this example, the natural language input **1704** can be associated with the monitored deployment **1702** in the sense that the natural language input **1704** may include a question about the monitored deployment **1702** (e.g., "how many EC2 instances are running in my deployment?"), the natural language input **1704** may include a request for clarification about some alert that the monitoring tool **1710** has generated about the monitored deployment **1702** (e.g., "what does this alert mean?"), the natural language input **1704** may include a request that is part of an investigation about the monitored deployment **1702** (e.g., "which containers in my deployment access the public internet?"), the natural language input **1704** may include a request to create a query that is executed by the monitoring tool **1710** to gather information about the monitored deployment **1702** (e.g., "create a query to determine which users have root access to some resource?"), or the natural language input **1704** may otherwise represent a request for information about (or otherwise associated with) the monitored deployment **1702**.

The example method depicted in FIG. **17** also includes receiving **1708**, from a generative AI application **1712**, a response **1714** to the natural language input **1704**. The generative AI application **1712** may be embodied, for example, as a chatbot, text-to-image application, text-to-video application, media input, audio or video input, or other application that leverages one or more generative AI models in interactions with users of the application (including other applications). For example, the generative AI application **1712** may leverage: 1) text generation models that can create coherent paragraphs of text, write poetry, generate code, compose stories, or even answer questions in a human-like manner; 2) image generation models that can produce realistic images of objects, scenes, people, and so on; 3) style transfer models can apply various styles to input data; 4) audio or video synthesis models that can generate audio or videos, 5) chatbots and conversational agents; and/or 6) other models.

The generative AI application **1712** can be embodied as a type of artificial intelligence system that is designed to generate new content, such as text, images, music, videos, or other forms of media, based on patterns and examples it has learned from existing data. Generative AI models may use techniques like deep learning to understand and replicate the patterns present in the training data, allowing them to produce novel and creative outputs that resemble the input

data. Whereas traditional AI algorithms may be used to identify patterns within a training data set and make predictions, generative AI applications may leverage machine learning algorithms to create outputs based on a training data set (also referred to herein as a ‘knowledge base’ or other collection of information that the generative AI application has access to). Such generative AI applications can produce outputs in the same medium in which it is prompted (e.g., text-to-text) or in a different medium from the given prompt (e.g., text-to-image or image-to-video). Examples of generative AI applications can include ChatGPT, Bard, DALL-E, Midjourney, DeepMind, and others, including variations of those applications that are tailored for some specific user group such as a particular business organization.

The generative AI application 1712 of FIG. 17 may be configured to access publicly available information as well as data sources associated with the monitoring tool 1710. The publicly available information may be information that can be found, for example, on the public internet or in some other source. The data sources associated with the monitoring tool 1710 may be embodied, for example, as user manuals or other user documentation associated with the monitoring tool 1710, help pages associated with the monitoring tool 1710, user communities or forums that are associated with the monitoring tool 1710, and so on. In such a way, the generative AI application 1712 may not only have access to publicly available information, but the generative AI application 1712 may also have access to information that may be specifically useful for utilizing the monitoring tool 1710, navigating the monitoring tool 1710, configuring the monitoring tool 1710, troubleshooting the monitoring tool 1710, or performing some other task that is specific to the monitoring tool 1710.

In some embodiments, the data sources associated with the monitoring tool 1710 can include data describing how to use the monitoring tool 1710. The data describing how to use the monitoring tool 1710 can include, for example, user manuals for the monitoring tool 1710, help pages for the monitoring tool 1710, information pulled from user communities associated with the monitoring tool 1710, and so on. In fact, the data describing how to use the monitoring tool 1710 can be obtained from sources that are not originally text based. For example, tutorial videos, video demonstrations, or other video data may be taken as input directly into a generative AI application 1712 that includes video-to-text models, or such video data may be run through one or more video-to-text translators that are external to the generative AI application 1712 and subsequently fed into the generative AI application 1712 as textual information that can be leveraged by the generative AI application 1712. In other embodiments, images data may be treated similarly, audio data may be treated similarly, or other forms of information that are not originally in textual form may be used.

In some embodiments, the data sources associated with the monitoring tool 1710 can include information contained in a user community for the monitoring tool 1710. In some embodiments, including those in which information pulled from user communities associated with the monitoring tool 1710 is utilized by the generative AI application 1712, information may be curated such that all information pulled from a particular source is not treated equally. For example, information entered into the user community by higher rated users may be given more weight than information entered into the user community by lower rated users, information entered into the user community that was verified by other users as having resolved some issue/question may be given

more weight than information entered into the user community that has not been verified by other users as having resolved some issue/question, information entered into the user community by credentialed users may be given more weight than information entered into the user community by non-credentialed users, and so on.

In the example depicted in FIG. 17, the response 1714 may be generated based at least in part on information contained in the data sources associated with the monitoring tool 1710. Readers will appreciate that the generative AI application 1712 may generate a response 1714 through the usage of machine learning techniques (e.g., deep learning) to generate new and creative content that resembles the patterns present in a training data set. The generative AI application 1712 can use neural networks or some other mechanism for machine learning, where a large dataset of examples that represent the type of content that the generative AI application 1712 is meant to generate are initially collected and preprocessed to ensure consistency and quality. The generative AI application 1712 may include models that are then trained, where the models may leverage specific neural network architectures designed for generative tasks such as, for example, a Generative Adversarial Network (‘GAN’). In GANs, a generator and discriminator may be trained such that the generator improves its ability to create realistic content by trying to fool the discriminator, while the discriminator improves its ability to differentiate real from fake content. Once the generator is trained, it may be able to take random noise or other inputs as seeds and generate new content that resembles the training data. For example, in text generation, the generator might produce coherent paragraphs of text based on a given prompt.

Consider an example in which the monitoring tool generates an alert which states that “External connection made to 11.22.33.44 at TCP port HTTPS (123) from host IP 100-101-102-103.us-west02.compute.internal.” In such an example, assume that a user of the monitoring tool 1710 provided the following natural language input 1704: “why does this alert matter?” In such an example, the user may be presented with a response 1714, directly or indirectly via the generative AI application 1712, stating (as an example of a conversational English explanation of the alert) that “this alert matters because it indicates that a connection has been made to an external IP address that your host has not contacted in the past 90 days. Raw IP address raise suspicions in cloud environments, as they are uncommon due to frequent changes caused by load balancers, proxies, and more. Attackers often use raw IP addresses to evade traceability and conceal their identities. It’s important to investigate further to determine the validity of the alert.” In this example, some information (e.g., the description of raw IP addresses and how attackers leverage raw IP addresses) may be pulled from public sources whereas other information is pulled from data sources associated with the monitoring tool 1710. For example, the data sources associated with the monitoring tool 1710 may include a policy indicating that an alert should be generated when a host in the monitored deployment 1702 attempts to connect to an external IP address that is has not connected to in the past 90 days.

For further explanation, FIG. 18 sets for an example method of leveraging generative artificial intelligence (‘AI’) for securing a monitored deployment 1702 in accordance with some embodiments of the present disclosure. The example depicted in FIG. 18 is similar to the example depicted in FIG. 17 as it also includes receiving 1706 natural language input 1704 associated with the monitored deploy-

107

ment 1702 and receiving 1708, from a generative AI application 1712, a response 1714 to the natural language input 1704.

In the example depicted in FIG. 18, the generative AI application 1712 also accesses data 1808 describing the monitored deployment 1702 that is gathered by the monitoring tool 1710. As described in greater detail elsewhere in this disclosure, the monitoring tool 1710 may gather a wide variety of data 1808 describing the monitored deployment 1702 including, for example, data related to the usage, performance, security, and overall health of cloud-based infrastructure and applications that are part of the monitored deployment 1702. For example, the monitoring tool 1710 may gather data describing how cloud resources are being utilized, data related to the performance of cloud services and applications, data related to security events and incidents, including intrusion attempts, vulnerabilities, access control violations, and other security-related activities, and many other types of data that are associated with the monitored deployment 1702. Furthermore, the monitoring tool 1710 may parse data sources such as logs that are generated by cloud services and applications, configuration data describing the configuration settings of cloud resources, network traffic data (e.g., describing network traffic patterns, data flows, communication between resources, network topology), user and identity data (e.g., describing users accessing cloud services, their authentication and authorization activities, the roles and permissions assigned to them), data related to regulatory compliance, data governance, and audit trails, and many other types of data. In such an example, the generative AI application 1712 may leverage this data to formulate a response 1714, to include within a response 1714, or in some other way.

In the example depicted in FIG. 18, the response 1714 is generated based at least in part on the data 1808 describing the monitored deployment 1702 (in addition to being generated based at least in part on data 1804 describing how to use the monitoring tool 1710 and information 1806 contained in a user community for the monitoring tool 1710). The data 1808 describing the monitored deployment 1702 may include, for example, information gathered by one or more agents that are part of the monitoring tool 1710 that describes some aspect of the monitored deployment 1702, information obtained from a cloud services provider that supports at least a portion of the monitored deployment 1702 that describes some aspect of the monitored deployment 1702, information gathered from IaC templates for the monitored deployment 1702, information gathered from log data associated with the monitored deployment 1702, and from other sources. In such an example, the data 1808 describing the monitored deployment 1702 may be utilized by the generative AI application 1712 during training of the generative AI application 1712, the data 1808 describing the monitored deployment 1702 may be utilized by the generative AI application 1712 during execution to generate or augment responses 1714, or used in some other way in the formation of a response 1714.

In some embodiments, the data 1808 describing the monitored deployment 1702 includes an alert 1810 that is generated by the monitoring tool 1710. The alert 1810 depicted in FIG. 18 may be embodied as any of the alerts described above. In some embodiments, the alert 1810 may be a notification, warning, or similar message that is triggered when the cloud monitoring tool 1710 detects an event, behavior, or condition that could potentially indicate a security threat or breach within the monitored environment 1702. Such an alert 1810 may be issued to inform admin-

108

istrators or security personnel about suspicious or unauthorized activities so that appropriate actions can be taken to mitigate risks and prevent potential security incidents. Examples of such an alert 1810 can include an unauthorized access alert if an attempt is made to access some resource by an unauthorized accessor, an alert indicating that malware or malicious software is detected within the monitored environment 1702, a data exfiltration alert indicating that an attempt has been made for unusual or unauthorized movement of sensitive data from the monitored environment 1702 to external sources, an alert indicating that anomalous network traffic has been detected, an alert indicating that some other anomalous activity has been detected, and so on. In some embodiments, the monitoring tool 1710 may generate such an alert 1810, which may be sent to administrators through various means, such as email, SMS, or integrated security management platforms, or delivered in some other way (e.g., via an interface). Information associated with the alert 1810 may be retained and made accessible to the generative AI application 1712 for training, generating responses, or for other purposes.

In some embodiments, the response 1714 that is generated by the generative AI application 1712 can include a natural language explanation of the alert 1810. The natural language explanation of the alert 1810 may be embodied, for example, as a description or clarification of the alert 1810 using human language in a way that is easy for people to understand. In such a way, complex technical information contained in (or otherwise associated with) the alert 1810 may be expressed in simple, relatable terms, making it understandable to individuals (potentially including those without specialized knowledge in the subject matter). In some embodiments, the natural language explanation of the alert 1810 may be augmented with actual data from the alert 1810, the associated condition that is the focus of the alert 1810, some other information associated with the monitored deployment 1702, or other information. In such a way, the natural language explanation of the alert 1810 may help bridge the gap between a user's knowledge and the level of specialized knowledge that may be required to most effectively utilize the findings of the monitoring tool 1710.

In some embodiments, the response 1714 that is generated by the generative AI application 1712 can include a description of a workflow designed to resolve a condition identified in the alert 1810. A workflow may include a series of steps that are designed, for example, to investigate, address, and mitigate the condition identified in the alert 1810. The workflow may include a series of actions that can be taken automatically, with assistance from a user such as a security administrator, or initiated/performed in some other way. The workflow can include steps such as, for example:

- a. Triaging the alert 1810 by reviewing the details of the alert 1810, including the nature of the event, the affected resources, and any contextual information available.
- b. Assessing the severity of the alert to prioritize response efforts.
- c. Gathering additional information, for example, by reviewing logs, analyzing network traffic, assessing the potential impact of the condition identified in the alert 1810, and so on.
- d. Performing a root cause analysis, for example, by attempting to identify the source of unauthorized access, the origin of malicious activity, misconfigurations, or other issues.
- e. Taking actions designed to contain the condition such as, for example, isolating affected resources within the

monitored deployment **1702**, blocking malicious IP addresses, disabling compromised accounts, and so on. f. Taking actions to eradicate the condition identified in the alert **1810** such as, for example, removing malware, patching vulnerabilities, correcting misconfigurations, or performing some other actions.

Readers will appreciate that in some embodiments the workflows may be preconfigured and executed automatically, especially where the alert **1810** identifies conditions that have been seen previously (either in the monitored deployment **1702** or in some other deployment). For example, where an attempt to access a known malicious IP address is detected, some workflow may automatically be executed given that attempting to access a known malicious IP address is (at least in relative terms) a somewhat common security threat. In fact, the systems described above may be configured to access a playbook of preconfigured workflows designed to remediate (at least in part) commonly detected conditions.

In some embodiments, the monitoring tool **1710** may be configured to support multiple personas. Each persona supported by the monitoring tool may be a representation of a typical user, customer group, industry, company size, company location, geography, type of company, type of user (e.g., a particular role within an organization), or other user or collection of users. Each persona may be a representation of different types of users, administrators, or stakeholders who interact with the monitoring tool **1710** to accomplish specific tasks. Personas that are supported by the monitoring tool **1710** can include, for example, a security analyst persona which represents a user that is typically responsible for monitoring security alerts, analyzing threats, and responding to incidents, a Chief Information Security Officer (CISO) persona which represents a user that is typically responsible for providing strategic direction for the organization's security initiatives, including developing and implementing security strategies, a developer persona which represents a user that is typically responsible for creating software applications (including those that execute on the monitored deployment **1702**), a network administrator persona that represents a user that is typically responsible for managing network infrastructure, an IT administrator persona which represents a user that is typically responsible for managing an organization's IT operations, and many other personas. Readers will appreciate that because each persona represents a user that has different goals, responsibilities, and preferences related to the monitoring tool **1710**, the information that is ultimately presented to a user that is associated with a first persona may need to be different than the information that is ultimately presented to a user that is associated with a second persona.

In some embodiments, the natural language input **1704** may be associated with a particular persona. The natural language input **1704** may be associated with a particular persona, for example, by tagging the natural language input **1704** with metadata that describes the persona of the user that generated the natural language input **1704**, by deriving the persona from information such as a username (where the user initially selects or is assigned a persona when their profile is created with the monitoring tool), or in some other way. In such an example, the natural language input **1704** itself may therefore include information that can be used to determine the persona of the user that generated the natural language input **1704**, although in other embodiments other techniques may be used. For example, a user may be requested to provide a persona, the user may be asked one or more questions to determine their persona, the user may

have a privilege level that is associated with some persona, the user's actions may be used to determine their persona, and so on.

In some embodiments, the response **1714** may be generated based at least in part on the particular persona. The generative AI application **1712** may generate (at least in part) the response **1714** based on the particular persona, for example, by having the persona provided to the generative AI application **1712** as additional input (along with the natural language input **1704**), by having distinct versions of the generative AI application **1712** that are specific to distinct personas in the sense that they are trained on and/or pull from different knowledge bases that are specific to each persona, or in some other way.

In some embodiments, the monitored deployment **1702** may be deployed in a cloud computing environment provided by a particular cloud services provider. A cloud services provider (CSP) may be embodied, for example, as a company or organization that offers a range of cloud computing services and solutions to individuals, businesses, and other entities. Cloud services providers may operate data centers and manage the infrastructure, platforms, applications, and other resources that are made available to customers over the internet. These services allow customers to access and use computing resources without the need to own and maintain physical hardware or software. Examples of such cloud services providers can include, for example, Amazon AWS, Microsoft Azure, Google Cloud (also known as 'GCP'), IBM Cloud, Oracle Cloud, Salesforce, and many others.

In some embodiments, the publicly available information can include information describing how various actions can be taken using the particular cloud services providers interfaces. The information describing how various actions can be taken using the particular cloud services providers interfaces can include, for example, user pages that are provided by the cloud services provider, user manuals generated by the cloud services providers, information provided by credentialed users of the cloud services provider, information pulled from a user community associated with the cloud services provider, and many other sources.

In some embodiments, the response **1714** is generated based at least in part on the information describing how various actions can be taken using the particular cloud services providers interfaces. Consider an example in which an alert is generated indicating that an AWS S3 bucket that is utilized as part of a monitored deployment **1702** is configured as a public bucket. In such an example, the response **1714** may include information describing the vulnerabilities associated with a public bucket and the response **1714** may even include an identification of a more secure privilege level (e.g., "private") that was identified based on AWS generated literature, as well as information describing a series of steps that may be taken in AWS to change the privilege level from public to private, where the series of steps are identified by generative AI application **1712** examining AWS generated literature, information from one or more AWS user communities, or from some other information describing how various actions can be taken using the particular cloud services providers (AWS in this example) interfaces.

In some embodiments, the response **1714** includes one or more suggested actions to be taken. The one or more suggested actions to be taken may be identified by the generative AI application **1712**, for example, by providing the generative AI application **1712** with access to an issue tracking system (e.g., a Jira deployment) that is utilized by

111

the owner of the monitoring tool **1710**, integrated into the monitoring tool **1710**, integrated into the cloud services provider's systems, or otherwise deployed such that issues that are identified by the monitoring tool **1710** can be tracked along with information describing how those issues were resolved. Stated differently, the generative AI application **1712** may have access to (during training, during execution) information that associates discovered issues with successful workflows that were used to remediate the issue. In such a way, when an issue is discovered by the monitoring tool **1710** and investigated by a user (e.g., a security analyst) through natural language inputs, the generative AI application **1712** may be configured to provide responses **1714** that include information (gathered from the sources described above) describing how those issues may be resolved, where such information is presented as a recommendation. In some embodiments, other or supplemental sources may be utilized to generate recommendations. The other or supplemental sources can include, for example, product literature, help pages, information gathered from previous interactions with support teams, and so on.

In some embodiments, the response **1714** includes a demonstration of performing some action in a user interface that is associated with the monitoring tool **1710**. The demonstration of performing some action in a user interface that is associated with the monitoring tool **1710** may be embodied, for example, as a pre-recorded video that illustrates what aspects of user interface that a user would click/select to perform some workflow or some portion of the workflow (e.g., a screen captured video of a user using the GUI and along with an explanation of the actions and features as the user navigates the GUI), a guided tour that includes a scripted walkthrough of the GUI where user interactions are simulated and explained, a pre-recorded scenario-based demonstration that is associated with some workflow or some portion of the workflow, and so on. In such a way, whatever actions are user is being recommended to take may be demonstrated rather than relying on a textual description of the actions that the user can take.

In some embodiments, the response **1714** includes a demonstration of performing some action in a user interface that is associated with a cloud services provider. The demonstration of performing some action in a user interface that is associated with a cloud services provider may be embodied, for example, as a pre-recorded video that illustrates what aspects of user interface that a user would click/select to perform some workflow or some portion of the workflow (e.g., a screen captured video of a user using the GUI and along with an explanation of the actions and features as the user navigates the GUI), a guided tour that includes a scripted walkthrough of the GUI where user interactions are simulated and explained, a pre-recorded scenario-based demonstration that is associated with some workflow or some portion of the workflow, and so on. In such a way, whatever actions a user is being recommended to take may be demonstrated (e.g., visually, via a 'walkthrough', or in some other way) rather than relying on a textual description of the actions that the user can take.

In some embodiments, the response **1714** includes a link to a portion of a user interface to take an action that is referenced in the response **1714**. The link to a portion of a user interface to take an action that is referenced in the response **1714** may be embodied, for example, as a hyperlink, icon, or other visual component that (when selected) can navigate a user to a portion of user interface to take an action that is referenced in the response **1714**. Consider an example in which the response **1714** indicates that some user

112

is overprivileged (e.g., a developer has access to sensitive financial documents, an accountant has access to a production code base) and that the user's privileges should be reduced. In such an example, the response **1714** may include a hyperlink, suggestion button, or other visual element that, when selected, opens a user interface that a user may utilize to alter user privileges. Readers will appreciate that depending on the action that is to be taken, different user interfaces may be opened, where the user interfaces may not even be part of the same tool. For example, one action may require that a web browser is opened to a location within that allows a user to change some aspects of a cloud deployment within AWS (e.g., a browser may be opened that allows a user to change whether an S3 bucket in AWS is public or private), another action may require that an interface to the monitoring tool **1710** is opened to perform some particular action (e.g., adding an IP address to a list of known malicious IP addresses so that any attempts to access the IP addresses by the monitored deployment's **1702** users or services are blocked automatically), another action may require that an interface associated with some other infrastructure component that is included in the monitored deployment be opened to perform some action (e.g., if the monitored deployment includes a K8s cluster, a Kubernetes management tool may be opened to modify the K8s cluster), and so on. Although the examples described above refer to a "link" to a portion of a user interface to take an action that is referenced in the response **1714**, in other embodiments some other mechanism may be used (e.g., a CLI command may be presented that can be copied or otherwise executed, a "take me there button" may be presented along with the response, and so on).

In some embodiments, the natural language input **1704** references a polygraph that is associated with the monitored deployment **1702**. As described in greater detail above, polygraphs may be generated to describe aspects of the monitored deployment **1702**. In the event that a user wants clarification, additional information, or has some other question about the polygraph, the user may generate a natural language input **1704** that references a polygraph that is associated with the monitored deployment **1702**. For example, the user may natural language input **1704** that states "why is node X connected to node Y?" Readers will appreciate that many other natural language inputs **1704** that reference a polygraph that is associated with the monitored deployment **1702** may be generated.

In some embodiments, the response **1714** includes information describing the polygraph that is associated with the monitored deployment **1702**. Because the response **1714** includes information describing the polygraph that is associated with the monitored deployment **1702**, the embodiments described here may represent an alternative way to interact with the polygraph (e.g., via prompt instead of UI/graph). As such, instead of (or in addition to) interacting with the polygraph graphically, users could learn things from what a polygraph contains via prompt. A user may ask (e.g., via a natural language input), for example, various questions (e.g., "what are my anomalous behaviors in the last one hour?", "what are the new workloads in my environment?", "who are all the people that have logged into my production environment in the last hour?") that are responded to textually, by highlighting the information in a graphical depiction of the polygraph, via audio input, gesture input, or in some other way.

In some embodiments, a user is presented with a proposed natural language input. The proposed natural language input may be embodied, for example, as a natural language input

113

selected from a list of predetermined natural language inputs, as a natural language input that other users have determined (e.g., by a poll or survey, by other explicit feedback, by implicit feedback (e.g., not asking a rephrased version of the question as the initial version apparently did not yield good results), or in some other way) to be of relatively high value, based on a recommendation generated by the generative AI application 1712, or in some other way. In this embodiment, the proposed natural language input may be used to essentially guide a user to ask questions, generate queries, investigate certain things, or other push the user in a direction that the is believed to be useful. In fact, in situations in which the monitoring tool 1710 has detected some condition that rises to the level of a relatively severe alert, the monitoring tool 1710 may (in conjunction with the generative AI application 1712) present a user with the proposed natural language input as an impetus to get the user initiate an investigation and/or potential remediation.

In some embodiments, the monitoring tool 1710 and the generative AI application 1712 may be used to enhance a client's understanding of the alerts (or other information) generated by the monitoring tool 1710. For example, a user of the monitoring tool may have a question regarding what a particular alert means or what they should do upon receipt of an alert. In some embodiments, the generative AI application 1712 may be fed documentation associated with the monitoring tool 1710 to allow the generative AI application 1712 to learn about the monitoring tool 1710 and to allow the generative AI application 1712 to provide enhanced data about the alerts. In these embodiments, when an alert is received, a user of the monitoring tool 1710 may ask a variety of questions about the alert (e.g., "why do I care about this alert?", "how could an attacker compromise my system with this alert?", "how can I fix this alert?", "can you write terraform to fix this alert?").

In some embodiments, customers may use alert channels for alerts that are generated by the monitoring tool 1710 in order to notify other teams about an issue. One of the most common alert channels is Slack, another is Microsoft Teams, another is Google Chat, and there are many other alternative communication channels. In these situations, when an alert comes in (from the monitoring tool 1710) that meets certain criteria, that alert is sent to a Slack channel for a team to review. But that alert may die in Slack and the customer is often left coming back to the monitoring tool 1710 to get more information. In DevOps, it is a best practice to meet teams where they are in the tools that they work in. As such, in some embodiments the monitoring tool 1710 may include (or otherwise be communicatively coupled to) a Slackbot (or bot for any other communication channel or tool) that can interact with a user after an alert is triggered. Such a Slackbot may also be communicatively coupled to the generative AI application 1712 to send natural language prompts to the generative AI application 1712 and receive responses from the generative AI application 1712. As an example flow, assume that the monitoring tool 1710 generates a new alert and as part of generating the alert, a slack message may be sent to a DevOps team. The DevOps team may subsequently ask questions (e.g., "how should I fix this issue?", "why do I care about this issue?") in the Slackbot (which may interact with the generative AI application 1712) and receive responses via the Slackbot in order to save the DevOps team from having to come into the monitoring tool 1710—instead allowing the DevOps team to work in the tools that they typically use.

In some embodiments, the generative AI application 1712 could enrich the context that is shown to the user or Slack

114

channel and propose actions again based on context (e.g., if Terraform is used, a quick-reply button could be shown to generate terraform code to remediate the issue. If CloudFormation is the preferred method, the quick reply would propose "create CloudFormation". If scripted remediations are installed, a quick reply could be shown to initiate the components). In these embodiments, users would be able to interact with the generative AI application 1712 via the Slack prompt and the quick replies. The Slackbot may keep the context of the conversation so users could improve the remediation until they would be able to either create a ticket (based on the configured ticketing system), a pull request or to just copy the result with one click to finalize the response workflow.

In some embodiments, to aid support teams and user communities associated with the monitoring tool 1710, the generative AI application 1712 may have access to systems used by support teams (e.g., a Zendesk database of tickets, Confluence articles, Vivun platform). Using this data, the generative AI application 1712 may be used to generate responses to questions asked by customers, prospects, or other users of the monitoring tool 1710.

In some embodiments, the natural language input 1704 includes a request to learn about new features associated with the monitoring tool 1710. A feature may be "new," for example, if the feature has not been previously used by a particular user, if a feature has been made available within a predetermined amount of time (e.g., within the last week, within the last month), if a feature has been upgraded since the user last used the feature, if a feature has been upgraded within a predetermined period of time, and so on.

In some embodiments, the response 1714 includes information describing new features associated with at least one of the monitoring tool 1710. The information describing new features associated with at least one of the monitoring tool 1710 may be embodied, for example, as a textual description of the new feature, as a pre-recorded video of how to use the feature, as a link to the interface that enables a user to utilize the new feature, or in some other way.

In some embodiments, the response 1714 can include information describing a policy that can be deployed by the monitoring tool 1710. The policy that can be deployed by the monitoring tool 1710 may be embodied, for example, as a rule, heuristic, or similar construct that is used by the monitoring tool 1710 as it monitors the monitored deployment 1702. The policy may state, for example, that an alert should be generated when some resource within the monitored deployment 1702 attempts to access a known malicious IP address, that an alert should be generated when a new user unsuccessfully attempts to access some resource within the monitored deployment 1702, that data communications should be blocked when initiated from certain countries or geographic regions, and so on. The policies that can be deployed by the monitoring tool 1710 may be identified, for example, by the generative AI application 1712 accessing a repository of policies that are deployed by other customers, by the generative AI application 1712 accessing a repository of predefined policies that are associated with some objective (e.g., a desired security posture, a desired security outcome), and in other ways. In such an example, the policies may be presented to a user of the monitoring tool 1710 as recommended policies that the user should adopt or at least may want to consider adopting.

In some embodiments, the natural language input 1704 may include a request for information describing how to use the monitoring tool 1710. For example, a user may enter a natural language input 1704 that states "how do I see the

115

polygraph?” In this example, the user may be presented with a response **1714** that indicates what menu selections a user should make to view the polygraph, the user may be presented with an icon or link that takes them to the polygraph, or the user may (optionally) be presented with a demonstration of navigating the user interface to access the polygraph. In this example, the response **1714** generated (at least in part) because the generative AI application **1712** may have access to the data sources associated with the monitoring tool **1710**, as described above.

Readers will appreciate that the process described above, where a natural language input **1704** is received and a response **1714** is generated may be an iterative process. For example, a user may provide a first natural language input **1704** that causes a first response **1714** to be generated, the user may then provide a second natural language input **1704** that causes a second response **1714** to be generated, the user may then provide a third natural language input **1704** that causes a third response **1714** to be generated, and so on. In this process, a subsequent natural language input **1704** may be intended to drill down on information presented in a previous response **1714**. Alternatively, a subsequent natural language input **1704** may be intended to shift directions, introduce different phrasing, or otherwise pivot the direction of the conversation (e.g., if a response **1714** does not adequately address an issue raised by some natural language input **1704**).

Consider an example in which the monitoring tool generates an alert which states that “External connection made to 11.22.33.44 at TCP port HTTPS (**123**) from host IP 100-101-102-103.us-west02.compute.internal.” In such an example, assume that a user of the monitoring tool **1710** provided the following natural language input **1704**: “what does this alert mean?” In such an example, the user may be presented with a response **1714**, directly or indirectly via the generative AI application **1712**, stating that “this alert means that a connection was made from your cloud deployment to the IP address 11.22.33.44 on TCP port HTTPS (**123**). The host name of the connecting machine is 100-101-102-103.us-west02.compute.internal. Would you like me to help you investigate this alert further or provide guidance on how to take action?” In this example, the user may provide a second natural language input **1704** stating “I want to know if this is malicious or not. Help me investigate.” In this example, the user may be presented with a response **1714**, directly or indirectly via the generative AI application **1712**, stating that “to determine if this connection is malicious, we can investigate further. Please follow the following steps: 1) Click on the IP address in the alert to access the network dossier. 2) Perform a reverse DNS lookup to gather more information about the IP address. 3) . . . ,” and so on. Alternative responses as described above could also be leveraged.

For further explanation, FIG. **19** sets forth a flowchart of an example method of an architecture for a generative artificial intelligence (AI)-enabled assistant in accordance with some embodiments of the present disclosure. The method of FIG. **19** may be performed, for example, in a data platform such as a data platform **12** as is described above. For example, the method of FIG. **19** may be performed by a data platform or other security framework monitoring a cloud deployment as is described above to perform functions such as anomaly detection, threat detection, and the like. Although the approaches set forth herein are described with respect to a cloud deployment, readers will appreciate that the approaches set forth herein may also be applied to other types of deployments or computing environments as can be

116

appreciated, such as locally implemented computing environments in a data center, server, and the like.

The method of FIG. **19** includes receiving **1902**, via a natural language interface for a security framework monitoring a cloud deployment, a natural language input. The natural language interface for the security framework allows a user to interact with the security framework using natural language inputs. For example, a user may enter a workflow to perform various tasks using natural language inputs and responses, such as investigation of events or alerts, monitoring the status of various compute assets, and the like. In this example, the natural language interface may include an interface for interacting with a generative artificial intelligence (AI) model that may perform or assist in performing various actions with respect to the security framework. In other words, one or more generative AI models may serve as an interactive assistant for a user using natural language inputs and outputs.

As referred to herein, generative AI uses models such as neural networks, including large language models (LLMs), large multimodal models (LMMs), and the like to generate content, such as text, code, graphics, animations, video, audiovisual representations, audio, speech, etc., in response to prompts. The generative AI models are trained using a corpus of training data content to learn the patterns and structure of that content. The generative AI model may then generate new content having the characteristics learned from the training data. Prompts may include text, code, audio, graphic, video, and representations in any other media. Such prompts may be provided to the generative AI model as a natural language input. For example, the approaches set forth herein may interact with a generative AI model using predefined prompts, dynamically generated prompts, prompts that include some portion of dynamically generated content (e.g., through the use of templates and dynamically populated variables), and the like.

In some embodiments, the natural language interface for the security framework may include or be encoded similar to a chat interface. In some embodiments, the natural language interface may be included in a particular user interface or dashboard for the security framework. In some embodiments, the natural language interface may be included in a user interface element similar to a collapsible or minimizable chat window that is anchored to the user interface for the security framework. For example, the natural language interface may include an input field into which a user may type or otherwise provide natural language inputs for processing. The user-provided inputs, responses to those user-provided natural language inputs, and/or other output from the natural language interface, may be presented or displayed in a message history or other message log in the natural language interface. Thus, in some embodiments, interactions with the natural language interface may mimic chat or messaging sessions between the user and another entity.

The natural language input may include a natural language encoding or description of a variety of actions that a user may wish the security framework to perform. For example, the natural language input may include a request for some information related to activity in the cloud deployment, detected events in the cloud deployment, alerts, and the like. As another example, the natural language input may include requests for suggestions as to remedial actions to perform, actions to perform in a security investigation, and the like. As a further example, the natural language input may include a request for some action to be performed with respect to the cloud deployment. For example, the natural

117

language input may include a request to modify some compute asset in the cloud deployment. One skilled in the art will appreciate that these example natural language inputs are merely illustrative and that other natural language inputs are also contemplated within the scope of the present disclosure, with the particular capabilities of the natural language interface may vary depending on various design and engineering considerations.

The method of FIG. 19 also includes selecting **1904**, by a delegator generative AI model and based on the natural language input, a particular generative AI model from a plurality of selectable generative AI models. Although the following discussion describes the use of a delegator generative AI model to select the particular generative AI model, in some embodiments a different type of AI model or machine learning model may also be used as a delegator instead of a generative AI model.

A delegator generative AI model is trained to select, based on the received **1902** natural language input, a particular generative AI model for ultimately servicing or processing the natural language input (e.g., to generate some response to the natural language input). Put differently, the delegator generative AI model is trained to delegate processing or servicing of the natural language input to some other generative AI model. For example, in some embodiments, the delegator generative AI model may perform an intent resolution to determine an intent of a user as expressed in the natural language input. Based on the determined intent, the delegator generative AI model may select a particular generative AI model for processing natural language inputs of that intent.

In some embodiments, selecting **1904** the particular generative AI model may include providing the natural language input as part of a prompt to the delegator generative AI model. For example, the natural language input may be inserted into a prompt template by populating a variable or placeholder for the natural language input. In some embodiments, the prompt template may indicate, to the delegator generative AI model, that the delegator generative AI model should select some generative AI model from the plurality of selectable generative AI models. In embodiments where the delegator generative AI model is one of a plurality of delegator generative AI models, to be described in further detail below such as where different delegator generative AI models correspond to different areas of functionality in the security framework, the prompt template may indicate, to the delegator generative AI model, the particular area of security framework functionality that the delegator generative AI model serves to assist.

In some embodiments, each of the plurality of selectable generative AI models are trained to perform different functions with respect to the security framework. In some embodiments, the plurality of selectable generative AI models may each be related to the same area of functionality of the security framework but are trained to perform different functions within that same area of functionality. Put differently, the plurality of selectable generative AI models may each encapsulate the necessary functionality to perform some task in the security framework. For example, the plurality of selectable generative AI models may each serve different functions related to alerts in the security framework, such as explaining alerts, looking up alerts, graphing entities related to alerts, and the like.

The method of FIG. 19 also includes prompting **1906** the particular generative AI model based on the natural language input. In some embodiments, the delegator generative AI model may be trained to generate the prompt for the par-

118

ticular generative AI model. In some embodiments, another process or service may generate the prompt for the particular generative AI model after selection of the particular generative AI model by the delegator generative AI model. In some embodiments, the delegator generative AI model may be configured to directly prompt the particular generative AI model. In some embodiments, another process or service may prompt the particular generative AI model (e.g., using a prompt generated by the delegator generative AI model or some other process or service).

In some embodiments, the prompt to the particular generative AI model may include the received natural language input. For example, the natural language input may be inserted into a prompt template. The prompt and/or the prompt template may indicate, to the particular generative AI model, the functionality that the particular generative AI model is trained to perform. In some embodiments, the prompt may include one or more portions of contextual information that may facilitate processing of the prompt (e.g., of the natural language input) by the particular generative AI model. Such contextual information may describe recent interactions by a user with the security framework, such as user interface interactions, issued queries or commands, natural language inputs, and the like. In some embodiments, the prompt may include various data points such as identifiers relevant to the natural language input. Such data points may be inserted into the prompt by the delegator generative AI model, by the natural language interface, or by another process or service. For example, where a user requests “Explain this alert,” the prompt may include a dynamically populated alert identifier for a particular alert as the natural language input does not explicitly identify the alert that the user is referencing. The alert identifier may be determined or aggregated based on contextual information, such as a most recently or currently viewed alert in a user interface, based on previous interactions with the natural language interface, and the like.

After prompting **1906** the particular generative AI model, the particular generative AI model processes the natural language input based on the prompt. In some embodiments, each of the selectable generative AI models may be configured to directly perform various actions or cause such actions to be performed in order to process the natural language input. Such actions may include, for example, the generation and issuance of queries such as database queries, the generation and issuance of API or service calls, and the like. In some embodiments, the selectable generative AI models may generate a response to its prompt based on responses to such actions (e.g., responses to API calls, queries, and the like). For example, the selectable generative AI models may transform received data, interpret or summarize received data, generate some content based on the received data, and the like.

In some embodiments, the delegator generative AI model may determine that multiple selectable generative AI models are required to process the received natural language input. In such embodiments, the delegator generative AI model may select multiple generative AI models from the plurality of selectable generative AI models. Each of the selected multiple generative AI models may be prompted individually (e.g., each of the selected multiple generative AI models operate independently) or at least partially sequentially (e.g., where the output from one of the selected multiple generative AI models may be used to generate an input prompt to another of the selected multiple generative AI models). In other words, in some embodiments, the selectable generative

AI models may be effectively chained together such that the output from one serves as an input to another.

For example, assume that a user requests “Summarize all recent alerts similar to this one” given some currently investigated alert. The delegator generative AI model may determine that, to process this request, a generative AI model for looking up alerts should be used to identify those alerts similar to the currently investigated alert and another generative AI model for summarizing alerts should be used to summarize each of the identified alerts. Accordingly, the generative AI model for looking up alerts may be prompted with an identifier for the currently investigated alert and provide, as output, identifiers or other data describing the similar alerts. This data describing the similar alerts may then be included in a prompt to the generative AI model for summarizing alerts that provides, as output, summaries for each of the identified similar alerts.

The method of FIG. 19 also includes providing 1908, via the natural language interface and in response to the natural language input, a response from the particular generative AI model. For example, the response from the particular generative AI model may be presented inline in a message history (e.g., a message or chat log) for the natural language interface. Thus, a user is presented, via the natural language interface, a response to their natural language input as generated by the generative AI model selected by the delegator generative AI model. In some embodiments, the response from the particular generative AI model may be transformed, modified, formatted, and the like before being provided 1908 to the user.

In some existing implementations, a natural language interface may be used to receive natural language inputs from a user and provide those natural language inputs to a generative AI model that generates some response. In other words, the natural language interface serves to interface the user with the generative AI model. Such approaches rely on a single generative AI model to perform all the functionality required to service requests from the user. This approach presents several drawbacks. To begin, as the required functionality of the generative AI model expands over time, it makes the generative AI model more difficult to train and the code for the generative AI model more difficult to maintain. Moreover, reliance on this single model may introduce drift or deviations in the ability for the model to perform some skills as training data relevant to certain skills may affect the ability for the model to perform other skills.

In contrast, the approaches set forth above encapsulate particular skills into different selectable generative AI models. This allows for the natural language interface to service a variety of different functionalities by using different selectable generative AI models. Moreover, it makes it simple to add or remove functionality by adding or removing models from the group of selectable models. A delegator model then needs only to be trained on selecting the newly added models or removing from consideration the removed models.

For further explanation, FIG. 20 sets forth a flowchart of another example method of an architecture for a generative artificial intelligence (AI)-enabled assistant in accordance with some embodiments of the present disclosure. The method of FIG. 20 is similar to FIG. 19 in that the method of FIG. 20 also includes: receiving 1902, via a natural language interface for a security framework monitoring a cloud deployment, a natural language input; selecting 1904, by a delegator generative artificial intelligence (AI) model and based on the natural language input, a particular generative AI model from a plurality of selectable generative AI

models; prompting 1906 the particular generative AI model based on the natural language input; and providing 1908, via the natural language interface and in response to the natural language input, a response from the particular generative AI model.

The method of FIG. 20 differs from FIG. 19 in that selecting 1904, by a delegator generative artificial intelligence (AI) model and based on the natural language input, a particular generative AI model from a plurality of selectable generative AI models includes determining 2002, by the delegator generative AI model, that additional information is needed to select the particular generative AI model. In some embodiments, the delegator generative AI model may be unable to select a particular generative AI model based on the natural language input. For example, the delegator generative AI model may not assign to any of the selectable generative AI models a confidence score exceeding some threshold or deviation relative to other confidence scores. As another example, the delegator generative AI model may assign to multiple of the selectable generative AI models a confidence score exceeding some threshold. Accordingly, the delegator generative AI model may determine 2002 that additional information is required in order to select a particular generative AI model from the plurality of selectable generative AI models with some degree of confidence.

FIG. 20 further differs from FIG. 19 in that selecting 1904, by a delegator generative artificial intelligence (AI) model and based on the natural language input, a particular generative AI model from a plurality of selectable generative AI models includes soliciting 2004 the additional information via the natural language interface. Soliciting 2004 the additional information may include presenting or displaying, via the natural language interface, a natural language request for additional information. For example, in some embodiments, the request for additional information may include a request to rephrase or expound upon the previously received natural language input (e.g., “Can you tell me more about what you would like to do?”). As another example, in some embodiments, the request for additional information may include a request to confirm some particular task or action before the delegator generative AI model selects the generative AI model for that task or action (e.g., “Did you mean that you want to do X?”). As a further example, in some embodiments, the request for additional information may include a request to select from multiple candidate tasks or actions corresponding to different selectable generative AI models (e.g., “Did you want to do X or Y?”). One skilled in the art will appreciate that these requests for additional information are merely illustrative and that other requests for additional information are also contemplated within the scope of the present disclosure.

Moreover, one skilled in the art will appreciate that the particular request for information may depend on why the delegator generative AI model needs additional information. For example, where two selectable generative AI models have relatively high confidence scores, the request for information may include a request to select between the actions of those two generative AI models. As another example, where no selectable generative AI model is assigned a high enough confidence score, the request for additional information may include a more general request to provide more detail from which a decision can be made.

In some embodiments, the delegator generative AI model may be able to select 1904 the particular generative AI model based on a response to the solicitation for additional information (e.g., another natural language input). In some embodiments, the delegator generative AI model may not be

121

able to select **1904** the particular generative AI model based on a response to the solicitation for additional information. In some of such embodiments, the delegator generative AI model may repeatedly solicit **2004** additional information via the natural language interface until a particular generative AI model can be selected **1904** or some other condition is satisfied (e.g., cancellation or refusal by a user).

For further explanation, FIG. **21** sets forth a flowchart of another example method of an architecture for a generative artificial intelligence (AI)-enabled assistant in accordance with some embodiments of the present disclosure. The method of FIG. **21** is similar to FIG. **19** in that the method of FIG. **21** also includes: receiving **1902**, via a natural language interface for a security framework monitoring a cloud deployment, a natural language input; selecting **1904**, by a delegator generative artificial intelligence (AI) model and based on the natural language input, a particular generative AI model from a plurality of selectable generative AI models; prompting **1906** the particular generative AI model based on the natural language input; and providing **1908**, via the natural language interface and in response to the natural language input, a response from the particular generative AI model.

The method of FIG. **21** differs from FIG. **19** in that the method of FIG. **21** also includes selecting **2102** the delegator generative AI model from a plurality of delegator generative AI models based on a currently active portion of a user interface. In some embodiments, the delegator generative AI model may be one of a plurality of delegator generative AI models. For example, each of the delegator generative AI models may correspond to different areas of functionality in the security framework (e.g., policies, alerts, queries, and the like). Each of the delegator generative AI models may have a corresponding set of selectable generative AI models from which a particular generative AI model may be selected. In other words, each delegator generative AI model may delegate processing of a natural language input to a particular set of generative AI models each encapsulating a particular task or action in the security framework. In some embodiments, each set of selectable generative AI models is mutually exclusive. In some embodiments, one or more selectable generative AI models may be included in multiple sets of selectable generative AI models (e.g., the sets of selectable generative AI models are at least partially overlapping).

In embodiments where there are multiple delegator generative AI models, one of the delegator generative AI models must be selected for use with the natural language interface. In this example, the delegator generative AI model is selected based on a currently active portion of a user interface of the security framework. The user interface may include, for example, a graphical user interface (GUI) through which a user may navigate to different screens, dashboards, visualizations, or other sections as can be appreciated. The currently active portion of the user interface may include a currently accessed or used portion of the user interface. For example, in some embodiments, various portions of the user interface may have their own natural language interface (e.g., an alerts section has its own natural language interface, a policy section has its own natural language interface, and the like). Accordingly, by using the natural language interface of a particular section of the user interface, the corresponding delegator generative AI model may be selected such that natural language inputs to that natural language interface are provided to the corresponding delegator generative AI model. In other words, each of the delegation generative AI models is accessible via a different corresponding natural language interface.

122

In some embodiments, the natural language interface may persist across different sections of the user interface. Such a natural language interface is hereinafter referred to as a “shared natural language interface” in that different sections of the user interface may utilize or access the same natural language interface. For example, the natural language interface may be embodied as a window or other user interface element that persists as the user navigates to different sections of the user interface. Such a window may be anchored to some section of the user interface, may be collapsed and expanded, and the like. In such embodiments, the delegator generative AI model may be selected based on which portion of the user interface is currently used by a user. For example, in response to a user expanding the window for the natural language interface, the delegator generative AI model for the currently active portion of the user interface may be selected such that inputs to the natural language input are provided to that delegator generative AI model.

In some embodiments, selecting **2102** the delegator generative AI model may be performed based on other data instead of or in addition to a currently active portion of the user interface. In some embodiments, selecting **2102** the delegator generative AI model may be performed by another generative AI model or another AI or machine learning model as can be appreciated. In some embodiments, selecting **2102** the delegator generative AI model may be performed using a rules-based approach. Other approaches for selecting **2102** the delegator generative AI model are also contemplated within the scope of the present disclosure.

For further explanation, FIG. **22** sets forth a flowchart of another example method of an architecture for a generative artificial intelligence (AI)-enabled assistant in accordance with some embodiments of the present disclosure. The method of FIG. **22** is similar to FIG. **21** in that the method of FIG. **22** also includes: selecting **2102** the delegator generative AI model from a plurality of delegator generative AI models based on a currently active portion of a user interface; receiving **1902**, via a natural language interface for a security framework monitoring a cloud deployment, a natural language input; selecting **1904**, by a delegator generative artificial intelligence (AI) model and based on the natural language input, a particular generative AI model from a plurality of selectable generative AI models; prompting **1906** the particular generative AI model based on the natural language input; and providing **1908**, via the natural language interface and in response to the natural language input, a response from the particular generative AI model.

The method of FIG. **22** differs from FIG. **21** in that the method of FIG. **22** also includes detecting **2202** a change in the currently active portion of the user interface. A change in the currently active portion of the user interface may include navigating to a different screen, section, dashboard, submenu, or other portion of the user interface of the security framework. In some embodiments, where different portions of the user interface correspond to different areas of functionality, the change in the currently active portion of the user interface may include a change to a different portion of the user interface corresponding to a different area of functionality (e.g., a change from a policies section to an alert section). Thus, in some embodiments, only changes across different areas of functionality in the user interface need be detected, not changes in the user interface within the same area of functionality.

The method of FIG. **22** also includes requesting **2204**, via a shared natural language interface, an indication to use a different delegator generative AI model. Here, the natural

123

language interface is a shared natural language interface that persists across different sections of the user interface. For example, the natural language interface may include an anchored window or other user interface element that persists as the user navigates the user interface. In response to detecting **2202** the change in the active portion of the user interface, a user is prompted to indicate whether a different delegator generative AI model should be used. Particularly, in some embodiments, the user may be prompted to indicate whether the shared natural language interface should use the delegator generative AI model corresponding to the currently active portion of the user interface (e.g., after the change).

As an example, each delegator generative AI model and their corresponding set of selectable generative AI models may be considered an “assistant” for some area of functionality. Assume that a user browsing the alert section of the user interface enters some natural language input to the shared natural language interface. The delegator generative AI model for alerts is selected and is used to select some generative AI model for processing the natural language input. The natural language inputs from the user and the received responses are presented via the message history of the shared natural language interface.

Further assume that the user then navigates to a policy section of the user interface. In response to detecting this change, a message may be presented via the shared natural language interface such as “Do you want to use the policy assistant?” If the user indicates that they do (e.g., a “yes” response), the delegator generative AI model for policies is selected such that subsequently received natural language inputs to the natural language interface are provided to the delegator generative AI model for policies. If the user indicates that they do not (e.g., a “no” response), inputs to the natural language interface will continue to be processed by the delegator generative AI model for alerts.

For further explanation, FIG. **23** sets forth a flowchart of another example method of an architecture for a generative artificial intelligence (AI)-enabled assistant in accordance with some embodiments of the present disclosure. The method of FIG. **23** is similar to FIG. **22** in that the method of FIG. **23** also includes: selecting **2102** the delegator generative AI model from a plurality of delegator generative AI models based on a currently active portion of a user interface; receiving **1902**, via a natural language interface for a security framework monitoring a cloud deployment, a natural language input; selecting **1904**, by a delegator generative artificial intelligence (AI) model and based on the natural language input, a particular generative AI model from a plurality of selectable generative AI models; prompting **1906** the particular generative AI model based on the natural language input; providing **1908**, via the natural language interface and in response to the natural language input, a response from the particular generative AI model; detecting **2202** a change in the currently active portion of the user interface; and requesting **2204**, via a shared natural language interface, an indication to use a different delegator generative AI model.

The method of FIG. **23** differs from FIG. **22** in that the method of FIG. **23** also includes saving **2302**, in response to receiving the indication to use the different delegator generative AI model, a message history associated with the delegator generative AI model (e.g., the previously used delegator generative AI model). The message history may include a history of natural language inputs to the natural language interface and the received responses as presented

124

via the natural language interface. In some embodiments, the message history may include additional metadata.

In some embodiments, saving **2302** the message history may include clearing a presentation of the message history in the natural language interface. Thus, interactions using a previous delegator generative AI model are cleared from view. In some embodiments, where a user switches back to using a previously used delegator generative AI model (e.g., by navigating back to a previously used portion of the user interface), the message history for that previously used delegator generative AI model may be loaded in order to populate the message history portion of the natural language interface. Thus, a user may resume a previous session with a particular assistant (e.g., a particular delegator generative AI model).

For further explanation, FIG. **24** sets forth a flowchart of another example method of an architecture for a generative artificial intelligence (AI)-enabled assistant in accordance with some embodiments of the present disclosure. The method of FIG. **24** is similar to FIG. **19** in that the method of FIG. **24** also includes: receiving **1902**, via a natural language interface for a security framework monitoring a cloud deployment, a natural language input; selecting **1904**, by a delegator generative artificial intelligence (AI) model and based on the natural language input, a particular generative AI model from a plurality of selectable generative AI models; prompting **1906** the particular generative AI model based on the natural language input; and providing **1908**, via the natural language interface and in response to the natural language input, a response from the particular generative AI model.

The method of FIG. **24** differs from FIG. **19** in that providing **1908**, via the natural language interface and in response to the natural language input, a response from the particular generative AI model includes providing **2402** the response based on an agreement between the intent resolution performed by the delegator generative AI model and another intent resolution performed by another delegator generative AI model. As is set forth above, a delegator generative AI model may select a particular generative AI model by performing an intent resolution on the natural language input. In some embodiments, different delegator generative AI models may be used to perform an intent resolution on the same natural language input, and therefore perform their own selections from the same set of selectable generative AI models. For example, the different delegator generative AI models may correspond to different versions of a base model.

In some embodiments, each of the different delegator generative AI models may perform their intent resolution (e.g., their selection of a generative AI model) in parallel. Accordingly, in some embodiments, a response is only provided via the natural language input where the intent resolutions of each delegator generative AI models match. Consider an example where a first delegator generative AI model (“model A”) is faster than a second delegator generative AI model (“model B”). However, model A may be less accurate or more prone to errors than model B. In this example, models A and B may perform their intent resolution in parallel. After model A completes its intent resolution, the natural language input to the generative AI model selected by model A. Once model B completes its intent resolution, the intent resolutions of model A and B may be compared to see if they match. If they do, the output from the selected generative AI model may be provided in response to the natural language input. In some embodiments, if they do not match, another response may be

125

provided via the natural language input, such as an error message or a request for additional information. In some embodiments, if they do not match and model B selected a different generative AI model, the natural language input may be provided to that generative AI model and its output provided as a response to the natural language input via the natural language interface.

As an example, assume that a user is accessing an assistant for alerts using a natural language interface. The user requests "Explain how to do my laundry." Model A may determine that this is a request to explain an alert as it is a request for some explanation. Accordingly, model A may select a generative AI model for explaining alerts and provide it the natural language input for processing. In parallel, the slower but more accurate model B may determine that this natural language input has nothing to do with alerts or security and that no generative AI model should be selected. Instead, model B may provide an error message as output. As models A and B disagree, the output from the generative AI model selected by model A will not be provided to the user.

As another example, assume that the user instead requests "Explain this alert." Model A may correctly determine that this is a request to explain an alert, causing selection and prompting of the correct generative AI model. Model B may finish its intent resolution while the generative AI model selected by model A is still processing the natural language output, also determining that this is a request to explain an alert. As models A and B agree, the output from the selected generative AI model may be provided as a response to the natural language input. This allows the greater accuracy of model B to be leveraged while requiring less time than having model B select the generative AI model and process the natural language input based on that selection.

For further explanation, FIG. 25 sets forth a flowchart of an example method of guiding query creation for a generative artificial intelligence (AI)-enabled assistant in accordance with some embodiments of the present disclosure. The method of FIG. 25 may be performed, for example, in a data platform such as a data platform 12 as is described above. For example, the method of FIG. 19 may be performed by a data platform or other security framework monitoring a cloud deployment as is described above to perform functions such as anomaly detection, threat detection, and the like. Although the approaches set forth herein are described with respect to a cloud deployment, readers will appreciate that the approaches set forth herein may also be applied to other types of deployments or computing environments as can be appreciated, such as locally implemented computing environments in a data center, server, and the like. Particularly, the method of FIG. 25 may be performed by an intermediary between a natural language interface for the security framework and one or more generative AI models used to process natural language inputs to the natural language interface of the security framework.

The method of FIG. 25 includes receiving 2502, via a natural language interface for a security framework monitoring a cloud deployment, a natural language input comprising one or more entity identifiers. The natural language interface for the security framework allows a user to interact with the security framework using natural language inputs (e.g., a text input structured in a human-understandable language). For example, a user may enter a workflow to perform various tasks using natural language inputs and responses, such as investigation of events or alerts, monitoring the status of various compute assets, and the like. In this example, the natural language interface may include an

126

interface for interacting with a generative artificial intelligence (AI) model that may perform or assist in performing various actions with respect to the security framework. In other words, one or more generative AI models may serve as an interactive assistant for a user using natural language inputs and outputs.

In some embodiments, the natural language interface for the security framework may include or be encoded similar to a chat interface. For example, the natural language interface may include an input field into which a user may type or otherwise provide natural language inputs for processing. The user-provided inputs, responses to those user-provided natural language inputs, and/or other output from the natural language interface, may be presented or displayed in a message history or other message log in the natural language interface. The natural language input may include a natural language encoding or description of a variety of actions that a user may wish the security framework to perform. For example, the natural language input may include a request for some information related to activity in the cloud deployment, actions to perform in a security investigation, and the like.

As is set forth above, the natural language input includes one or more entity identifiers. An entity identifier serves to identify or indicate one or more entities associated with the monitored deployment. An entity may include a variety of identifiable information objects as can be appreciated. For example, an entity may include a monitored resource such as a virtual machine, container, storage resource, and the like. As another example, an entity may include a user or grouping of users. As a further example, an entity may include particular alerts, events, policies, and the like. As yet another example, an entity may include some identifiable portion of data, such as a particular file or data object. Readers will appreciate that these example entities are merely illustrative and that other entities are also contemplated within the scope of the present disclosure.

In some embodiments, the entity identifier may include an entity-specific identifier that serves to identify a single, particular entity. In other words, the entity identifier may be unique to a particular entity. For example, the entity identifier may include an identifier of a particular policy, a particular machine identifier, and the like. In some embodiments, the entity identifier may include an identifier for a group of entities. For example, the entity identifier may include an identifier for a particular grouping or class of policies, for a particular classification of alerts, and the like.

Consider an example natural language input of "For {alert-id: 1234}, what are the remediation steps for {policy-id: 5678}?" Here, the natural language input includes two entity identifiers, "alert-id: 1234," an identifier for a particular alert, and "policy-id: 5678," an identifier for a particular policy. In this example, the entity identifiers are encoded in the natural language input as field-value pairs using the fields "policy-id" and "alert-id" and numeric identifiers separated by a colon character. Moreover, the entity identifiers are identified in the natural language input by being bound by delimiter characters "{ }." Other approaches for encoding or indicating entity identifiers in natural language inputs may also be used. As a further example, consider an example natural language input of "Is there a policy in {global-policies} that addresses exposure of containers to external networks?" Here, the entity identifier {global-policies} serves to identify a grouping of policies.

The method of FIG. 25 also includes gathering 2504 information based on the one or more entity identifiers. In other words, the one or more entity identifiers are used to

127

gather information that may assist a generative AI model in processing the natural language input. For example, in some embodiments, one or more databases or other data sources may be queried using the one or more entity identifiers. Accordingly, in some embodiments, gathering **2504** the information based on the one or more entity identifiers may include issuing **2506** a database query comprising the one or more entity identifiers. In some embodiments, a single database query may be issued that includes the one or more entity identifiers. In some embodiments, multiple database queries may be issued that may each include one or more of the entity identifiers. Readers will appreciate that the number and structure of the issued database queries may vary according to the particular entity identifiers referenced in the natural language input as well as other design or engineering considerations.

In some embodiments, gathering **2504** the information based on the one or more entity identifiers may be based on permissions of a user of the natural language interface. For example, particular users or groups of users may be limited in the particular entities for which they may access information. Accordingly, in some embodiments, one or more API keys or authentication credentials may be included in an issued database query or to establish a session with one or more databases or data sources.

In some embodiments, the data sources or databases from which the information is gathered **2504** may include databases maintained by the data platform for storing event information or other data describing monitored activity in the cloud deployment. In some embodiments, the sources or databases from which the information is gathered may include data sources describing particular policies implemented or that may be implemented in the cloud deployment. In some embodiments, the data sources or databases from which the information is gathered **2504** may include data sources describing particular vulnerabilities such as Common Vulnerabilities and Exposures (CVEs), describing security assessment frameworks, best practices, or industry standards such as MITRE ATT&CK (Adversarial Tactics, Techniques and Common Knowledge), or other data sources as can be appreciated. As an example, where an alert is generated for a detected vulnerability, assume a natural language input of “What are the risks of {alert-id: 2345}?” In this example, a database query may be issued that selects alert information for the included alert. The database query, or additional queries, may select information for the CVE of the vulnerability, MITRE information describing potential attack paths for this vulnerability, and the like. Readers will appreciate that, in some embodiments, the result of a particular query may cause additional queries to be generated using portions of information included in the result of the particular query.

In order to gather **2504** the information based on the one or more entity identifiers, including to issue **2506** the database query, the one or more entity identifiers may be extracted or identified from the natural language input. In some embodiments, as will be described in further detail below, the one or more entity identifiers may be identified using one or more delimiters defining the bounds of the one or more entity identifiers. In some embodiments, regular expressions or other parsing or string matching techniques may be applied to find entity identifiers matching a particular format or structure.

In some embodiments, gathering **2504** the information based on the one or more entity identifiers may include validating the one or more entity identifiers. In some embodiments, validating the one or more entity identifiers

128

may include determining that the entity identifier is a valid type of entity identifier. In some embodiments, where an entity is expressed as a field-value pair or is expressed in another format that indicates the type of entity being identified, validating the entity identifier may include determining that the field portion of the entity identifier corresponds to a known or defined type of entity. For example, assuming that alerts are valid types of entities, the entity identifier {alert-id: **1234**} may be validated as “alert-id” is a valid entity type. As another example, assuming that widgets are not a valid entity, the entity identifier {widget-id: **1234**} would not be valid. In some embodiments, where an entity identifier fails validation, a message may be presented (e.g., via the natural language interface) indicating that the particular entity identifier failed validation. This may allow a user to edit their previously submitted natural language input or create and provide a new natural language input for processing.

In some embodiments, an error may occur when gathering **2504** the information based on the one or more entity identifiers that prevents acquisition of some portion of information. For example, a user of the natural language interface may not have adequate permissions to access information related to particular entities. As another example, an entity identifier may not correspond to any particular entity (e.g., for {alert-id: 1234} there may be no alerts with the identifier of “1234”). Accordingly, issuing a database statement may cause an error to be generated and an error message provided in response. In some embodiments, a message or notification may be provided to a user (e.g., via the natural language interface) describing the error, thereby allowing a user to modify their previously submitted natural language input to prevent subsequent errors. In some embodiments, data describing the error may be included in the gathered **2504** information which may then be provided to the generative AI model as is described in further detail below.

The method of FIG. **25** also includes providing **2508**, to a generative artificial intelligence (AI) model, a prompt based on the natural language input and comprising the gathered information. A prompt to the generative AI model is a request for the generative AI model to perform some function, such as perform some analysis, answer some question, and the like. In some embodiments, a prompt may include additional information that assists the generative AI model in performing the requested function. Here, the gathered information may be provided to the generative AI model in order to satisfy a request described by the natural language input.

In some embodiments, the prompt to the generative AI model may include the natural language input itself. In some embodiments, the prompt to the generative AI model may include another natural language expression based on the natural language input. In some embodiments, the natural language input may be reformatted or otherwise transformed for inclusion in the prompt. For example, in some embodiments, delimiters designating entity identifiers may be removed or the entity identifiers may be reformatted into another format or encoding understood by the generative AI model.

In some embodiments, the prompt to the generative AI model may include additional information beyond the gathered **2504** information. For example, the natural language interface and/or another interface for the security framework may maintain contextual information. Such contextual information may describe, for example, current or recently detected events or alerts and/or user interactions with the

security framework. Such contextual information may also describe a state of a user interface for the security framework, such as an active screen, a currently investigated or browsed alert, policy, or other entity, and the like.

In some embodiments, the prompt to the generative AI model may be based on a prompt template. A prompt template is a defined structure for a prompt that may include one or more placeholder values or variables that are populated in order to generate the provided prompt. As an example, a prompt template may include a placeholder value or variable for a natural language input, a placeholder value or variable designating one or more portions of additional information for the generative AI model, and the like.

The method of FIG. 25 also includes receiving 2510, from the generative AI model, a response to the prompt. The response to the prompt is based on a processing of the prompt (e.g., of the natural language input and the additional gathered information) by the generative AI model. The response to the prompt may include, for example, another natural language expression or other content as can be appreciated. The method of FIG. 25 also includes presenting 2512 the response to the prompt via the natural language interface. For example, the response to the prompt may be rendered or displayed via a message log or message history of the natural language interface.

Existing solutions for using generative AI models in a natural language interface may include some amount of contextual information with a received natural language input as a prompt to the generative AI model. The particular contextual information provided to the generative AI model may be outside the control of the user of the natural language interface. That is, the natural language interface may automatically select or define the contextual information to be provided to the generative AI model independent of the particular natural language input from the user.

Such approaches limit the ability of the generative AI model to responding to natural language inputs to a scope of knowledge defined by the provided contextual knowledge and any other data sources that the generative AI model may be able to access. Moreover, as generative AI models may have limits to the amount of additional information that can be included in a prompt, merely expanding the amount of information provided to the generative AI model may be inadequate.

In contrast, the approaches set forth above allow for a user to indicate particular entities that may be relevant to their natural language input. Information related to these particular entities may be selectively gathered prior to prompting the generative AI model, improving the scope of knowledge of the generative AI model servicing the received prompt. This allows for a user to query the generative AI model for information that may not necessarily be relevant to any automatically maintained contextual information.

As an example, assume that a user opens a natural language interface while browsing information related to a recently generated alert for violating a particular policy. Here, contextual information may include information about the generated alert and/or the particular policy by virtue of the user browsing information related to this alert. Accordingly, were a user to ask, via the natural language interface, “Explain this alert,” the generative AI model may be able to generate an explanation for the alert given the provided contextual information and without the need for the user to specifically identify the alert.

In contrast, assume that the user instead asked “How is {policy-id: 1234} different than {policy-id: 5678}?” In this example and using solutions other than those presented

herein, the contextual information may be inadequate for the generative AI model to answer the question as the contextual information may lack information related to these policies, instead being focused on the currently browsed alert. Using the approaches set forth herein, information related to these policies may be accessed and included in the prompt to the generative AI model, thereby allowing the generative AI model to compare these policies and provide a response to the user of the natural language interface. Put differently, by accessing information related to specifically identified entities before prompting the generative AI model, a user is able to query the generative AI model using a scope of knowledge beyond that of automatically maintained contextual information, expanding the overall capabilities of the generative AI model and the natural language interface.

In some embodiments, the approaches set forth herein may be performed by an intermediary between the natural language interface and the generative AI model, such as an API server. As an example, the natural language interface may communicate with the API server using one or more exposed APIs, thereby allowing the natural language interface to provide natural language inputs to the API server and/or receive responses from the generative AI model via the API server. The API server may then access databases or other data sources using the entity identifiers in the natural language input and generate a prompt for the generative AI model. The API server may then use an API or other interface of the generative AI model to prompt and receive responses from the generative AI model.

For further explanation, FIG. 26 sets forth a flowchart of another example method of guiding query creation for a generative artificial intelligence (AI)-enabled assistant in accordance with some embodiments of the present disclosure. The method of FIG. 26 is similar to FIG. 25 in that the method of FIG. 26 also includes: receiving 2502, via a natural language interface for a security framework monitoring a cloud deployment, a natural language input comprising one or more entity identifiers; gathering 2504 information based on the one or more entity identifiers, including: issuing 2506 a database query comprising the one or more entity identifiers; providing 2508, to a generative artificial intelligence (AI) model, a prompt based on the natural language input and comprising the gathered information; receiving 2510, from the generative AI model, a response to the prompt; and present 2512 the response to the prompt via the natural language interface.

The method of FIG. 26 differs from FIG. 25 in that gathering 2504 information based on the one or more entity identifiers includes identifying 2602, in the natural language input, the one or more entity identifiers based on the one or more predefined delimiters. As is set forth above, in some embodiments, entity identifiers may be designated or identified in the natural language input using one or more delimiters. A delimiter is a character or combination of characters that define the boundaries between particular regions of the natural language input. Here, one or more delimiters may be used to define the boundaries of an entity identifier in the natural language input for subsequent identification or extraction.

In some embodiments, an entity identifier may be bound by predefined delimiters at the beginning and ending of the entity identifier. Such predefined delimiters may include defined pairs of delimiters such as parenthesis, brackets, or braces. As an example, in the phrase “Explain the alert {alert-id: 1234},” the entity identifier for an alert identifier is bound by the delimiters “{ }.” Moreover, in this example, a colon delimiter is used to separate a field and value pair in

the entity identifier. In some embodiments, an entity identifier may be bound using a predefined delimiter at the beginning of the entity identifier, such as by using special characters (e.g., “\$,” “@,” “#,” and the like). In some embodiments, the entity identifier may be bound at the end using any of a combination of characters such as a space, a punctuation mark, or an end of line. Consider the example phrase “Explain the alerts #alert-id: 1234 and #alert-id: 5678.” Here, the entity identifier for alert-id: 1234 is bound at the beginning by a “#” character and at the end by a space, while the entity identifier for alert-id: 5678 is bound at the beginning by a “#” character and at the end by a period. Readers will appreciate that these example delimiters are merely illustrative and that other delimiters are also contemplated within the scope of the present disclosure.

Accordingly, identifying **2602**, in the natural language input, the one or more entity identifiers based on the one or more predefined delimiters may include identifying, in the natural language expression, one or more substrings bound by the one or more predefined delimiters. These substrings will then serve as the identified **26023** one or more entity identifiers that may be used to gather **2504** information as described above. For example, these substrings may be inserted or included in one or more database statements issued **2506** in order to gather **2504** the information.

For further explanation, FIG. 27 sets forth a flowchart of another example method of guiding query creation for a generative artificial intelligence (AI)-enabled assistant in accordance with some embodiments of the present disclosure. The method of FIG. 27 is similar to FIG. 25 in that the method of FIG. 27 also includes: receiving **2502**, via a natural language interface for a security framework monitoring a cloud deployment, a natural language input comprising one or more entity identifiers; gathering **2504** information based on the one or more entity identifiers, including: issuing **2506** a database query comprising the one or more entity identifiers; providing **2508**, to a generative artificial intelligence (AI) model, a prompt based on the natural language input and comprising the gathered information; receiving **2510**, from the generative AI model, a response to the prompt; and present **2512** the response to the prompt via the natural language interface.

The method of FIG. 27 differs from FIG. 25 in that the method of FIG. 27 also includes detecting **2702** input of a predefined delimiter to the natural language interface. Input of the predefined delimiter may be detected as a user is providing input to an input field of the natural language interface (e.g., during typing or creation of a natural language input to be submitted via the natural language interface).

As will be described in further detail below, input of this predefined delimiter may cause a user to be presented with a listing of selectable entity identifiers. Accordingly, in some embodiments, the detected **2702** predefined delimiter may be different than other predefined delimiters used to indicate manually entered entity identifiers. As an example, in some embodiments, manually entered entity identifiers may be bound by pairs of delimiters such as braces or brackets while the detected **2702** predefined delimiter may include a special character or combination of characters.

The method of FIG. 27 also includes presenting **2704**, via the natural language interface, one or more selectable entity identifiers. The one or more selectable entity identifiers may be presented **2704** via a pop-up menu or other user interface element that allows for a user of the natural language interface to select one of the selectable entity identifiers. The particular selectable entity identifiers to be included in a

presented grouping of selectable entity identifiers may be determined according to a variety of approaches.

As an example, in some embodiments, the grouping of selectable entity identifiers may include a grouping of recently or frequently used (e.g., recently or frequently input by the user) entity identifiers. As another example, in some embodiments, the grouping of selectable entity identifiers may be narrowed in scope based on one or more keywords already present in the natural language input. For example, assume that the user has input the partial natural language input “Explain the alert #,” with “#” being the detected delimiter for presenting **2704** the selectable entity identifiers. As the partial natural language input includes the keyword “alert” before the detected delimiter, the grouping of selectable entity identifiers may be automatically narrowed to only include alert identifiers.

In some embodiments, the grouping of selectable entity identifiers may be based on a context of the natural language interface. The context of the natural language interface describes the current or recent interactions with the security framework by the user, including interactions via the natural language interface and/or other user interface elements. As an example, the context of the natural language interface may describe an active portion of a user interface over which the natural language interface is overlaid or presented. As another example, the context of the natural language interface may describe currently browsed or investigated information, such as an alert, event, policy, and the like currently browsed via a user interface of the security framework.

Accordingly, the grouping of selectable entity identifiers may include entity identifiers for one or more entities associated with the context. As an example, assume that a user is browsing information related to a particular alert and then accesses the natural language interface to further investigate the alert. Further assume that the user inputs the partial natural language input of “Tell me more about #,” with “#” being the detected delimiter. Here, a grouping of selectable entity identifiers may be generated and presented based on this particular alert. Such a grouping may include the alert identifier, identifiers for affected or relevant hosts, identifiers for violated policies, identifiers for associated users, and the like.

The method of FIG. 27 also includes populating **2706** an input field of the natural language interface with a selected one of the one or more selectable entity identifiers. In some embodiments, a user may select a one of the entity identifiers by clicking or otherwise providing an input directed to one of the selectable entity identifiers. In some embodiments, a user may select a one of the entity identifiers by inputting or partially inputting the text of one of the entity identifiers into the input field of the natural language interface. Populating the input field of the natural language interface causes the selected entity identifier to be included in a natural language input (e.g., a partial natural language input) being input by a user. In some embodiments, the selected entity identifier may be appended to the detected delimiter. In some embodiments, the selected entity identifier may replace the detected delimiter.

In some embodiments, the selected entity identifier may be appended as editable text such that the selected entity identifier may be modified or edited via the natural language interface prior to providing the natural language input. In some embodiments, the selected entity identifier may be included in the input field as a user interface element such as a button or bubble. In such embodiments, the user interface element may be removed from the input field by selecting a button or other input field of the user interface

element. In some embodiments, though the selected entity identifier is added to the input field as a user interface element, submission of the input to the input field may cause the text encoding of the selected entity identifier to be provided.

The approaches set forth above provide for a guided creation of queries (e.g., of natural language inputs) for the natural language interface by presenting a user with various selectable entity identifiers for inclusion in the natural language input. This improves the user experience and facilitates the creation of relevant natural language inputs for processing and investigation of security events.

One or more embodiments may be described herein with the aid of method steps illustrating the performance of specified functions and relationships thereof. The boundaries and sequence of these functional building blocks and method steps have been arbitrarily defined herein for convenience of description. Alternate boundaries and sequences can be defined so long as the specified functions and relationships are appropriately performed. Any such alternate boundaries or sequences are thus within the scope and spirit of the claims. Further, the boundaries of these functional building blocks have been arbitrarily defined for convenience of description. Alternate boundaries could be defined as long as the certain significant functions are appropriately performed. Similarly, flow diagram blocks may also have been arbitrarily defined herein to illustrate certain significant functionality.

To the extent used, the flow diagram block boundaries and sequence could have been defined otherwise and still perform the certain significant functionality. Such alternate definitions of both functional building blocks and flow diagram blocks and sequences are thus within the scope and spirit of the claims. One of average skill in the art will also recognize that the functional building blocks, and other illustrative blocks, modules and components herein, can be implemented as illustrated or by discrete components, application specific integrated circuits, processors executing appropriate software and the like or any combination thereof.

While particular combinations of various functions and features of the one or more embodiments are expressly described herein, other combinations of these features and functions are likewise possible. The present disclosure is not limited by the particular examples disclosed herein and expressly incorporates these other combinations.

Advantages and features of the present disclosure can be further described by the following statements:

1. A method for guiding query creation for a generative artificial intelligence (AI)-enabled assistant, the method comprising: receiving, via a natural language interface for a security framework monitoring a cloud deployment, a natural language input comprising one or more entity identifiers; gathering information based on the one or more entity identifiers; providing, to a generative artificial intelligence (AI) model, a prompt based on the natural language input and comprising the gathered information; and receiving, from the generative AI model, a response to the prompt.

2. The method of statement 1, further comprising presenting the response to the prompt via the natural language interface.

3. The method of statements 1 or 2, wherein gathering the information based on the one or more entity identifiers comprises issuing a database query comprising the one or more entity identifiers.

4. The method of any of statements 1-3, wherein the one or more entity identifiers are bound by one or more predefined delimiters in the natural language input.

5. The method of any of statements 1-4, wherein gathering the information based on the one or more entity identifiers comprises identifying, in the natural language input, the one or more entity identifiers based on the one or more predefined delimiters.

6. The method of any of statements 1-5, wherein at least one of the one or more entity identifiers comprises an entity-specific identifier.

7. The method of any of statements 1-6, wherein at least one of the one or more entity identifiers comprises an entity group identifier.

8. The method of any of statements 1-7, further comprising: detecting input of a predefined delimiter to the natural language interface; presenting, via the natural language interface, one or more selectable entity identifiers; and populating an input field of the natural language interface with a selected one of the one or more selectable entity identifiers.

9. The method of any of statements 1-8, wherein presenting the one or more selectable entity identifiers comprises identifying the one or more selectable entity identifiers based on a context of the natural language interface.

10. The method of any of statements 1-9, wherein the selected one of the selected one or more selectable entity identifiers are populated at a location in the input field corresponding to the predefined delimiter.

11. A non-transitory computer readable storage medium storing instructions for guiding query creation for a generative artificial intelligence (AI)-enabled assistant which, when executed, cause a processing device to: receive, via a natural language interface for a security framework monitoring a cloud deployment, a natural language input comprising one or more entity identifiers; gather information based on the one or more entity identifiers; provide, to a generative artificial intelligence (AI) model, a prompt based on the natural language input and comprising the gathered information; and receive, from the generative AI model, a response to the prompt.

12. The non-transitory computer readable storage medium of statement 11, wherein the instructions, when executed, further cause the processing device to present the response to the prompt via the natural language interface.

13. The non-transitory computer readable storage medium of statements 11 or 12, wherein, to gather the information based on the one or more entity identifiers, the instructions, when executed, further cause the processing device to issue a database query comprising the one or more entity identifiers.

14. The non-transitory computer readable storage medium of any of statements 11-13, wherein the one or more entity identifiers are bound by one or more predefined delimiters in the natural language input.

15. The non-transitory computer readable storage medium of any of statements 11-14, wherein, to gather the information based on the one or more entity identifiers, the instructions, when executed, further cause the processing device to identify, in the natural language input, the one or more entity identifiers based on the one or more predefined delimiters.

16. The non-transitory computer readable storage medium of any of statements 11-15, wherein at least one of the one or more entity identifiers comprises an entity-specific identifier.

135

17. The non-transitory computer readable storage medium of any of statements 11-16, wherein at least one of the one or more entity identifiers comprises an entity group identifier.

18. The non-transitory computer readable storage medium of any of statements 11-17, wherein the instructions, when executed, further cause the processing device to: detect input of a predefined delimiter to the natural language interface; present, via the natural language interface, one or more selectable entity identifiers; and populate an input field of the natural language interface with a selected one of the one or more selectable entity identifiers.

19. The non-transitory computer readable storage medium of any of statements 11-18, wherein, to present the one or more selectable entity identifiers, the instructions, when executed, further cause the processing device to identify the one or more selectable entity identifiers based on a context of the natural language interface.

20. The non-transitory computer readable storage medium of any of statements 11-19, wherein the selected one of the selected one or more selectable entity identifiers are populated at a location in the input field corresponding to the predefined delimiter.

What is claimed is:

1. A method for guiding query creation for a generative artificial intelligence (AI)-enabled assistant, the method comprising:

receiving, via a natural language interface for a security framework monitoring a cloud deployment, a natural language input comprising one or more entity identifiers;

gathering information based on the one or more entity identifiers;

providing, to a generative artificial intelligence (AI) model, a prompt based on the natural language input and comprising the gathered information; and

receiving, from the generative AI model, a response to the prompt.

2. The method of claim 1, wherein the one or more entity identifiers are bound by one or more predefined delimiters in the natural language input.

3. The method of claim 2, wherein gathering the information based on the one or more entity identifiers comprises identifying, in the natural language input, the one or more entity identifiers based on the one or more predefined delimiters.

4. The method of claim 1, further comprising: detecting input of a predefined delimiter to the natural language interface;

presenting, via the natural language interface, one or more selectable entity identifiers; and

populating an input field of the natural language interface with a selected one of the one or more selectable entity identifiers.

5. The method of claim 4, wherein presenting the one or more selectable entity identifiers comprises identifying the one or more selectable entity identifiers based on a context of the natural language interface.

6. The method of claim 4, wherein the selected one of the selected one or more selectable entity identifiers are populated at a location in the input field corresponding to the predefined delimiter.

7. The method of claim 1, further comprising presenting the response to the prompt via the natural language interface.

136

8. The method of claim 1, wherein gathering the information based on the one or more entity identifiers comprises issuing a database query comprising the one or more entity identifiers.

9. The method of claim 1, wherein at least one of the one or more entity identifiers comprises an entity-specific identifier.

10. The method of claim 1, wherein at least one of the one or more entity identifiers comprises an entity group identifier.

11. A non-transitory computer readable storage medium storing instructions for guiding query creation for a generative artificial intelligence (AI)-enabled assistant which, when executed, cause a processing device to:

receive, via a natural language interface for a security framework monitoring a cloud deployment, a natural language input comprising one or more entity identifiers;

gather information based on the one or more entity identifiers;

provide, to a generative artificial intelligence (AI) model, a prompt based on the natural language input and comprising the gathered information; and

receive, from the generative AI model, a response to the prompt.

12. The non-transitory computer readable storage medium of claim 11, wherein the one or more entity identifiers are bound by one or more predefined delimiters in the natural language input.

13. The non-transitory computer readable storage medium of claim 12, wherein, to gather the information based on the one or more entity identifiers, the instructions, when executed, further cause the processing device to identify, in the natural language input, the one or more entity identifiers based on the one or more predefined delimiters.

14. The non-transitory computer readable storage medium of claim 11, wherein the instructions, when executed, further cause the processing device to:

detect input of a predefined delimiter to the natural language interface;

present, via the natural language interface, one or more selectable entity identifiers; and

populate an input field of the natural language interface with a selected one of the one or more selectable entity identifiers.

15. The non-transitory computer readable storage medium of claim 14, wherein, to present the one or more selectable entity identifiers, the instructions, when executed, further cause the processing device to identify the one or more selectable entity identifiers based on a context of the natural language interface.

16. The non-transitory computer readable storage medium of claim 14, wherein the selected one of the selected one or more selectable entity identifiers are populated at a location in the input field corresponding to the predefined delimiter.

17. The non-transitory computer readable storage medium of claim 11, wherein the instructions, when executed, further cause the processing device to present the response to the prompt via the natural language interface.

18. The non-transitory computer readable storage medium of claim 11, wherein, to gather the information based on the one or more entity identifiers, the instructions, when executed, further cause the processing device to issue a database query comprising the one or more entity identifiers.

19. The non-transitory computer readable storage medium of claim 11, wherein at least one of the one or more entity identifiers comprises an entity-specific identifier.

20. The non-transitory computer readable storage medium of claim 11, wherein at least one of the one or more entity identifiers comprises an entity group identifier.

* * * * *